

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO BÀI TẬP LỚN

**KIẾN TRÚC THIẾT KẾ PHẦN MỀM
ĐỀ TÀI: XÂY DỰNG HỆ THỐNG ECOMMERCE**

Sinh viên thực hiện:	Vũ Trọng Khôi
Mã Sinh viên:	B22DCCN468
Lớp:	D22CNPM06
Nhóm BTL:	02
Giảng viên hướng dẫn:	PGS.TS Trần Đình Quế

Hà Nội, 2026

MỤC LỤC

CHƯƠNG 1. TỪ MONOLITHIC ĐẾN MICROSERVICES VÀ DDD 1

1.1 Giới thiệu Monolithic Architecture.....	1
1.1.1. Khái niệm.....	1
1.1.2. Cấu trúc điển hình.....	1
1.1.3. Ví dụ thực tế	2
1.1.4. Nhược điểm chi tiết	4
1.1.5. Khi nào nên dùng Monolithic	5
1.2 Microservices Architecture	5
1.2.1. Khái niệm.....	5
1.2.2. Đặc điểm	5
1.2.3. Kiến trúc Microservices trong Bigtech.....	7
1.2.4. So sánh Monolithic vs Microservices.....	8
1.2.5. Nhược điểm	12
1.2.6. Nguyên tắc thiết kế	14
1.3 Domain Driven Design (DDD)	15
1.3.1. Mục tiêu	16
1.3.2. Các khái niệm cốt lõi	16
1.3.3. Context Map	18
1.3.4. DDD trong Microservices.....	19
1.4 Case Study: Healthcare (Luyện Decomposition).....	20
1.4.1. Mô tả bài toán	20
1.4.2. Bước 1: Xác định Domain	21
1.4.3. Bước 2: Xác định Bounded Context	21
1.4.4. Bước 3: Phân rã thành Microservices.....	22
1.4.5. Bước 4: Xác định quan hệ	22
1.4.6. Cài đặt với Django	23
1.4.7. Ví dụ API.....	25
1.5 Kết luận	26

CHƯƠNG 2. PHÁT TRIỂN HỆ E-COMMERCE MICROSERVICES 28

2.1 Xác định yêu cầu.....	28
2.1.1 Functional Requirements	28
2.1.2 Non-functional Requirements.....	33
2.2 Phân rã hệ thống theo DDD	34

2.2.1. Bounded Context	34
2.2.2. Nguyên tắc	36
2.2.3. Thiết kế class diagram tổng quát	37
2.2.4. Biểu đồ thiết kế lớp.....	41
2.3 Thiết kế Product Service (Django).....	43
2.3.1 Phân loại sản phẩm	43
2.3.2. Model tổng quát	43
2.3.3. Chi tiết theo Domain.....	44
2.3.4. API	47
2.4 Thiết kế User Service (Django)	48
2.4.1. Phân loại người dùng	48
2.4.2. Model	48
2.4.3. Phân quyền.....	50
2.4.5. Workflow	51
2.4.5. API	53
2.5 Thiết kế Cart Service.....	53
2.5.1 Model	53
2.5.2 Logic xử lý	55
2.5.3 API (Endpoints)	56
2.6 Thiết kế Order Service	56
2.6.1. Model	56
2.6.2. Workflow	59
2.6.3. API	61
2.7 Thiết kế Payment Service	63
2.7.1. Model	63
2.7.2. Trạng thái	64
2.7.3. Workflow	64
2.7.4. API	65
2.8 Thiết kế Shipping Service	66
2.8.1 Model	66
2.8.2 Giải thích các trạng thái	67
2.8.4 API	69
2.9 Thiết kế Interaction Service	69
2.9.1 Model	69

2.9.2 Workflow	70
2.9.3 API (Endpoints)	71
2.10 Thiết kế Comment Rate Service	71
2.10.1. Model	71
2.10.2. Workflow	72
2.10.3. API	73
2.11 Bài tập thực hành	73
2.11.1. Biểu đồ lớp, Datamodel	74
2.11.2. Mapping Database	83
2.10.3. So sánh MySQL, PostgreSQL	91
2.11 Kết luận	91
CHƯƠNG 3. AI SERVICE CHO TƯ VẤN SẢN PHẨM.....	93
3.1 Mục tiêu	93
3.1.1 Đầu vào của hệ thống AI (Input & Context)	93
3.1.2 Đầu ra của hệ thống AI (Output)	94
3.2 Kiến trúc AI Service.....	94
3.2.1 Đầu vào (Input).....	94
3.2.2 Quy trình xử lý (Processing).....	95
3.2.3 Đầu ra (Output).....	96
3.2.4 Pipeline của AI Service	96
3.3 Thu thập dữ liệu	98
3.3.1 User Behavior Data (Dữ liệu hành vi người dùng)	98
3.3.2. Bộ dữ liệu RetailRocket (E-commerce Dataset)	98
3.3.3. Bộ dữ liệu eCommerce Events	101
3.3.4. Bộ dữ liệu eCommerce Behavior (Multi Category Store).....	102
3.3.5 Ví dụ dataset	105
3.4 Mô hình AI (Sequence Modeling)	105
3.4.1. LSTM.....	105
3.4.2. biLSTM.....	108
3.4.3. RNN	112
3.4.4. Chiến lược huấn luyện, kiểm thử.....	114
3.4.5. Đánh giá mô hình.....	117
3.5 Knowledge Graph với Neo4j	122
3.5.1. Knowledge Graph.....	122

3.5.2. Mô hình đồ thị	123
3.5.3. Cypher.....	126
3.5.4. Truy vấn gợi ý.....	126
3.6 RAG (Retrieval-Augmented Generation)	127
3.6.1. Khái niệm.....	127
3.6.2. Pipeline của RAG	128
3.6.2 Vector Database (ChromaDB).....	129
3.6.3 Ví dụ truy vấn tìm kiếm ngữ nghĩa (Semantic Search)	129
3.7 Kết hợp Hybrid Model	130
3.7.1 Sức mạnh kiềng ba chân (The Triple-Power).....	130
3.7.2 Pipeline Kết hợp (Hybrid Flow)	130
3.8 Hai dạng AI Service	131
3.8.1 Dạng 1: Danh sách gợi ý tự động (Recommendation List)	131
3.8.2 Dạng 2: Trợ lý tư vấn thông minh (Chatbot Consultant)	132
3.9 Triển khai AI Service	133
3.9.1 Tech Stack (Hệ sinh thái công nghệ).....	133
3.9.2 Kiến trúc tích hợp (Architecture).....	133
3.10 Bài tập	134
CHƯƠNG 4. XÂY DỰNG HỆ THỐNG HOÀN CHỈNH.....	136
4.1 Kiến trúc tổng thể.....	136
4.1.1. Mô hình hệ thống.....	136
4.1.2. Nguyên tắc	137
4.2 System Architecture	138
4.2.1. Overview.....	138
4.2.2 Microservice Architecture	138
4.2.3 API Gateway.....	139
4.2.4 Service Communication.....	139
4.2.5. Containerization and Deployment (Container hóa và Triển khai)	139
4.2.6. System Structure	139
4.3 API Gateway (Django).....	140
4.3.1 Vai trò của API Gateway	140
4.3.2 Cấu hình định tuyến (Routing Configuration).....	141
4.4 Authentication (JWT).....	141
4.4.1 Cài đặt thư viện.....	142

4.4.2 Cấu hình hệ thống (Cấu hình mẫu trong settings.py)	142
4.4.3 Luồng xác thực hệ thống (Authentication Flow).....	142
4.5 Giao tiếp giữa các Service.....	142
4.5.1 Đồng bộ qua REST API (Synchronous Communication)	143
4.5.2 Các nguyên tắc Best Practice được áp dụng.....	143
4.6 Docker hóa hệ thống	143
4.6.1 Dockerfile mẫu cho các Django Services.....	143
4.6.2 Docker-compose (Điều phối toàn hệ thống).....	144
4.7 Luồng hệ thống (End-to-End)	145
4.7.1 Mô tả chi tiết Use Case: Mua hàng.....	145
4.7.2. Sequence Logic	146
4.8 Triển khai Kubernetes (Optional)	147
4.8.1 Cấu hình Deployment mẫu (Cho user-service)	147
4.8.2 Cấu hình Service mẫu.....	148
4.9 Logging và Monitoring	148
4.9.1 Tập trung hóa Log với ELK Stack.....	148
4.9.2 Giám sát hiệu năng với Prometheus + Grafana	149
4.10 Đánh giá hệ thống	150
4.10.1 Đánh giá hiệu năng (Performance)	150
4.10.2 Khả năng mở rộng (Scalability).....	150
4.10.3 Ưu điểm	151
4.10.4 Nhược điểm và Thách thức.....	151
4.11. Kết quả thực tế	151
4.11.1. Giao diện khách hàng	151
4.11.2. Giao diện nhân viên	158

DANH MỤC HÌNH ẢNH

Hình 1.1. Kiến trúc monolithics	1
Hình 1.2. Mô hình Scale Cube	Error! Bookmark not defined.
Hình 1.3. Kiến trúc Microservice cho hệ Ecommerce	6
Hình 1.4. So sánh kiến trúc Mono và Micro trong hệ Ecommere.....	9
Hình 1.5. Lỗi trace id trong microservice.....	13
Hình 1.6. Nguyên tắc Single Responsibility trong hệ thống thực tế	14
Hình 1.7. Bounded Context trong hệ Ecommerce.....	18
Hình 1.8. Context Map trong hệ Ecommerce.....	18
Hình 1.9. Cấu trúc thư mục Healthcare Microservice.....	25
Hình 2.1. Biểu đồ Usecase dành cho khách hàng	29
Hình 2.2. Biểu đồ Use Case dành cho nhân viên	30
Hình 2.3. Biểu đồ Use Case hệ thống AI	31
Hình 2.4. Biểu đồ UseCase chức năng mua hàng	31
Hình 2.5. Biểu đồ Use Case chức năng xem gợi ý sản phẩm.....	32
Hình 2.6. Biểu đồ Usecase chatbot.....	32
Hình 2.7. Biểu đồ Usecase nhân viên quản lý đơn hàng.....	33
Hình 2.8. Bounded Context xác định cho hệ Ecommerce.....	35
Hình 2.9. Nguyên tắc cốt lõi thiết kế Microservcie.....	36
Hình 2.10. Biểu đồ Lớp tổng quát cho hệ Ecommerce	38
Hình 2.11. Biểu đồ thiết kế lớp tổng quát cho hệ Ecommerce.....	41
Hình 2.12. Biểu đồ tuần tự chức năng cập nhật sản phẩm	46
Hình 2.13. Biểu đồ hoạt động chức năng cập nhật sản phẩm	47
Hình 2.14. Biểu đồ tuần tự chức năng đăng ký người dùng.....	51
Hình 2.15. Biểu đồ hoạt động chức năng đăng ký người dùng.....	52
Hình 2.16. Biểu đồ tuần tự chức năng thêm giỏ hàng.....	55
Hình 2.17. Biểu đồ hoạt động chức năng thêm giỏ hàng	56
Hình 2.18. Biểu đồ tuần tự chức năng mua hàng	59
Hình 2.19. Biểu đồ hoạt động chức năng mua hàng	61
Hình 2.20. Biểu đồ tuần tự luồng xử lý thanh toán	64
Hình 2.21. Biểu đồ hoạt động luồng xử lý thanh toán	65
Hình 2.22. Biểu đồ tuần tự luồng khởi tạo vận đơn	68
Hình 2.23. Biểu đồ hoạt động luồng khởi tạo vận đơn	69
Hình 2.24. Biểu đồ tuần tự luồng ghi nhận tương tác	70
Hình 2.25. Biểu đồ hoạt động luồng ghi nhận tương tác	71
Hình 2.26. Biểu đồ tuần tự luồng đánh giá.....	72
Hình 2.27. Biểu đồ hoạt động luồng đánh giá.....	73
Hình 2.28. Biểu đồ lớp UserService.....	74
Hình 2.29. Biểu đồ thiết kế lớp UserService.....	74
Hình 2.30. Biểu đồ Datamodel UserService	75
Hình 2.31. Biểu đồ lớp ProductService.....	75
Hình 2.32. Biểu đồ thiết kế lớp Product Service.....	76
Hình 2.33. Biểu đồ Datamodel ProductService	76
Hình 2.34. Biểu đồ lớp CartService	77
Hình 2.35. Biểu đồ Datamodel CartService	77

Hình 2.36. Biểu đồ lớp OrderService	78
Hình 2.37. Biểu đồ thiết kế lớp OrderService	78
Hình 2.38. . Biểu đồ Datamodel OrderService.....	79
Hình 2.39. Biểu đồ lớp PayService	79
Hình 2.40. Biểu đồ thiết kế lớp PayService	80
Hình 2.41. Biểu đồ Datamodel PayService	80
Hình 2.42. Biểu đồ lớp ShipService	80
Hình 2.43. Biểu đồ thiết kế lớp ShipService	81
Hình 2.44. Biểu đồ Datamodel ShipService.....	81
Hình 2.45. Biểu đồ lớp CommentService.....	82
Hình 2.46. Biểu đồ thiết kế lớp ReviewService	82
Hình 2.47. Biểu đồ Datamodel ReviewService.....	83
Hình 3.1. Quy trình xử lý AI Service	95
Hình 3.2. Pipeline Chi tiết AI Service	97
Hình 3.3. Biểu đồ so sánh top Accuracy 3 model	117
Hình 3.4. Biểu đồ training loss	118
Hình 3.5. Confusion Matrix model BiLSTM	119
Hình 3.6. Confusion Matrix model LSTM.....	119
Hình 3.7. Confusion Matrix model RNN	120
Hình 3.8. Trực quan hóa quan hệ User-Product với Neo4j.....	124
Hình 3.9. Trực quan hóa quan hệ User-User với Neo4j.....	126
Hình 3.10. Biểu đồ tuần tự luồng lấy danh sách gợi ý	132
Hình 3.11. Biểu đồ tuần tự chức năng chatbot	133
Hình 4.1. Kiến trúc dịch vụ Microservice với hệ Ecommerce	136
Hình 4.2. Kiến trúc triển khai với Django	137
Hình 4.3. System Structure.....	140
Hình 4.4. Biểu đồ tuần tự chi tiết luồng khách hàng mua hàng	146
Hình 4.5. Giao diện trang chủ & tư vấn người dùng.....	152
Hình 4.6. Giao diện sản phẩm yêu thích	152
Hình 4.7. Giao diện đổi voucher	153
Hình 4.8. Giao diện cá nhân khách hàng.....	153
Hình 4.9. Giao diện tìm sản phẩm.....	154
Hình 4.10. Giao diện gợi ý theo từ khóa tìm kiếm.....	154
Hình 4.11. Giao diện chi tiết sản phẩm	155
Hình 4.12. Giao diện giỏ hàng.....	155
Hình 4.13. Giao diện thanh toán.....	156
Hình 4.14. Giao diện thanh toán thành công.....	156
Hình 4.15. Giao diện danh sách đơn hàng.....	157
Hình 4.16. Giao diện tư vấn dựa vào giỏ hàng.....	157
Hình 4.17. Giao diện chatbot.....	158
Hình 4.18. Giao diện quản lý sản phẩm	158
Hình 4.19. Giao diện quản lý đơn hàng.....	159
Hình 4.20. Giao diện quản lý Voucher.....	159
Hình 4.21. Giao diện quản lý vận đơn.....	160

CHƯƠNG 1. TỪ MONOLITHIC ĐẾN MICROSERVICES VÀ DDD

1.1 Giới thiệu Monolithic Architecture

1.1.1. Khái niệm

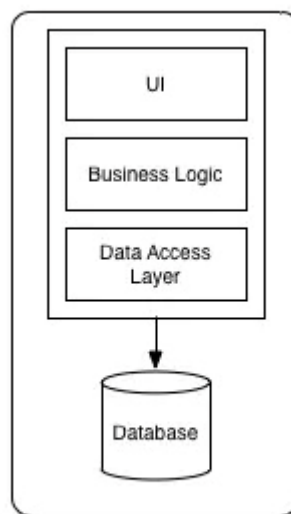
Trong kỹ thuật phần mềm, kiến trúc nguyên khối (Monolithic Architecture) là một mô hình thiết kế truyền thống, trong đó toàn bộ ứng dụng được xây dựng thành một khối thống nhất, tự chứa (self-contained) và hoàn toàn độc lập với các ứng dụng khác.

Ở mô hình này, tất cả các thành phần cần thiết để thực thi ứng dụng đều được tích hợp vào một cơ sở mã (codebase) duy nhất. Thay vì phân tách thành các dịch vụ rời rạc, mọi thứ từ giao diện, xử lý nghiệp vụ cho đến truy cập dữ liệu đều được biên dịch, đóng gói và thực thi cùng nhau, thường chia sẻ chung không gian bộ nhớ cũng như tài nguyên của máy chủ.

Mặc dù các kiến trúc phân tán như Microservices đang là xu hướng, Monolithic vẫn là một sự lựa chọn tuyệt vời và phổ biến cho các ứng dụng có độ phức tạp thấp, hoặc được phát triển bởi các nhóm có quy mô nhỏ.

1.1.2. Cấu trúc điển hình

Một ứng dụng Monolithic thường được tổ chức theo kiến trúc phân lớp (layered architecture), trong đó các module được sắp xếp theo hệ thống phân cấp nhưng lại chia sẻ chung hệ sinh thái tài nguyên.



Monolithic Architecture

Hình 1.1. Kiến trúc monolithics

Một cấu trúc Monolithic điển hình bao gồm ba lớp chính:

- **Presentation Layer (UI - Lớp giao diện người dùng):** Đây là thành phần phía máy khách (client-side) chịu trách nhiệm quản lý những gì được hiển thị trực tiếp cho người dùng. Lớp này xử lý các giao diện hình ảnh, văn bản, cũng như tiếp nhận các tương tác của người dùng thông qua trình duyệt hoặc thiết bị.
- **Business Logic Layer (Lớp logic nghiệp vụ):** Nằm ở phía máy chủ (server-side), lớp này chứa đựng toàn bộ các quy tắc, quy trình và thuật toán cốt lõi của phần mềm,. Các thành phần trong lớp này giao tiếp với nhau thông qua các lời gọi hàm trực tiếp (direct function calls) thay vì giao tiếp qua mạng, giúp quá trình xử lý diễn ra cực kỳ nhanh chóng,.
- **Data Access Layer (Lớp truy cập dữ liệu):** Lớp này chịu trách nhiệm giao tiếp với cơ sở dữ liệu – thường là một hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) – để tổ chức, lưu trữ và truy xuất dữ liệu dưới dạng các hàng và cột. Trong kiến trúc Monolithic, tất cả các module nghiệp vụ đều sẽ kết nối chung đến một cơ sở dữ liệu duy nhất này.

Sự tương tác giữa các lớp diễn ra theo một luồng có thứ tự và nghiêm ngặt. Khi người dùng thực hiện một thao tác trên Presentation Layer, một yêu cầu sẽ được gửi đến Business Logic Layer. Tại đây, các hàm xử lý sẽ được gọi.

- **Lời gọi hàm trực tiếp (In-memory calls):** Khác với Microservices phải giao tiếp qua mạng (HTTP/gRPC), các module trong Monolithic chia sẻ chung bộ nhớ nền tảng. Khi lớp giao diện cần dữ liệu, nó gọi trực tiếp đến hàm của lớp nghiệp vụ. Điều này triệt tiêu hoàn toàn độ trễ mạng (network latency), giúp tốc độ phản hồi nội bộ cực kỳ nhanh.
- **Chia sẻ chung tài nguyên (Shared Resources):** Tất cả các thành phần đều sử dụng chung cấu hình hệ thống, bộ nhớ đệm (cache) và các thư viện hỗ trợ.

Đặc trưng lớn nhất của Monolithic là việc tất cả các module nghiệp vụ đều "nhìn" vào một sơ đồ dữ liệu duy nhất.

- **Toàn vẹn dữ liệu dễ dàng:** Vì chỉ có một cơ sở dữ liệu, việc đảm bảo tính ACID (Atomicity, Consistency, Isolation, Durability) trở nên đơn giản. Bạn có thể thực hiện một Transaction (giao dịch) kéo dài qua nhiều module khác nhau mà không lo ngại về vấn đề đồng bộ hóa phức tạp giữa các dịch vụ.
- **Truy vấn kết hợp (Join):** Việc lấy dữ liệu từ nhiều thực thể khác nhau (ví dụ: lấy thông tin Đơn hàng cùng với thông tin Khách hàng) được thực hiện bằng các câu lệnh SQL JOIN đơn giản trên một DB duy nhất.

1.1.3. Ví dụ thực tế

Để làm rõ cách hoạt động của Monolithic, hãy lấy ví dụ về việc xây dựng một nền tảng thương mại điện tử (e-commerce). Một hệ thống e-commerce hoàn chỉnh sẽ bao gồm nhiều vùng nghiệp vụ khác nhau như: quản lý kho hàng (inventory), xử lý đơn hàng (order

fulfillment), dịch vụ chăm sóc khách hàng (customer service), và công thanh toán (payment processing).

Trong kiến trúc Monolithic, toàn bộ các chức năng kể trên được mã hóa và đóng gói chặt chẽ trong cùng một ứng dụng duy nhất. Khi một khách hàng truy cập website (Presentation Layer), hệ thống sẽ duyệt qua danh mục, áp dụng các mã khuyến mãi (Business Logic Layer) và ghi lại giao dịch vào một cơ sở dữ liệu duy nhất (Data Access Layer). Giả sử vào dịp Black Friday, lưu lượng người dùng truy cập vào tính năng "Thanh toán" tăng vọt. Thay vì chỉ mở rộng (scale) riêng module thanh toán, hệ thống Monolithic bắt buộc bạn phải sao chép toàn bộ khối ứng dụng e-commerce khổng lồ (bao gồm cả các module đang rảnh rỗi như quản lý kho) ra nhiều máy chủ mới để đáp ứng tải,.

Mọi thay đổi được ghi vào một cơ sở dữ liệu (Database) trung tâm. Thường thì các bảng Orders, Products, Users,... đều nằm chung một Schema. Điều này giúp việc duy trì tính nhất quán của dữ liệu (ACID Transactions) rất dễ dàng: Nếu thanh toán lỗi, hệ thống chỉ cần Rollback một transaction duy nhất là xong.

Dưới đây là ví dụ cụ thể về xây dựng nền tảng e-commerce monolithic với công nghệ Django:

```
bookstore_monolith/
├── manage.py
├── bookstore_project/      # Cấu hình chính (settings.py, urls.py)
├── apps/
│   ├── users/             # Quản lý người dùng & Profile
│   ├── products/          # Danh mục sách, tác giả, giá
│   ├── orders/            # Giỏ hàng, thanh toán, hóa đơn
│   ├── inventory/         # Kho hàng, số lượng tồn
│   └── shipping/          # Thông tin giao hàng
├── static/                # CSS, JS, Images dùng chung
└── templates/             # Toàn bộ giao diện (HTML) của hệ thống
```

Hãy xem cách kiến trúc này xử lý một yêu cầu đặt hàng:

1. Routing: *bookstore_project/urls.py* đóng vai trò "người điều phối", nhận request và chuyển hướng nó về *apps/orders/urls.py*.
2. Processing: View trong *orders* sẽ gọi Model của *users* để check thông tin người dùng, gọi Model của *inventory* để check hàng.
3. Database: Toàn bộ các thao tác đọc/ghi diễn ra trên cùng một Database. Nếu có lỗi, Django dùng *transaction.atomic* để đảm bảo hoặc là toàn bộ thao tác thành công, hoặc là không có gì thay đổi (tính nhất quán cao).
4. Response: View chọn một file HTML từ thư mục *templates/*, kết hợp dữ liệu và trả về cho khách hàng.

Trong mô hình này, toàn bộ các chức năng kinh doanh (Users, Products, Orders,...) đều sống chung dưới một mái nhà.

- Một Codebase duy nhất: Tất cả mã nguồn được lưu trữ trong cùng một kho lưu trữ (repository). Điều này giúp việc tìm kiếm code, refactor (tối ưu hóa mã) và quản lý các dependency (thư viện) trở nên nhất quán.
- Chia sẻ tài nguyên: Các folder như static/ và templates/ được dùng chung cho toàn bộ hệ thống. Bạn không cần phải thiết lập cấu hình giao diện hay CSS riêng biệt cho từng module; mọi thứ đều có thể kế thừa từ một khung xương (base template) duy nhất.

Tuy nhiên, kiến trúc này chỉ phù hợp với các dự án vừa và nhỏ, các sản phẩm đang trong giai đoạn khởi nghiệp (MVP) hoặc những hệ thống có quy mô đội ngũ phát triển tập trung, nhưng khi hệ thống bắt đầu phát triển quá lớn và phức tạp, nó sẽ bộc lộ rất nhiều hạn chế

1.1.4. Nhược điểm chi tiết

Dù mang lại sự đơn giản trong giai đoạn đầu, kiến trúc Monolithic sẽ bộc lộ nhiều điểm yếu chí mạng khi hệ thống phình to:

- **Khó khăn trong việc mở rộng (Kém linh hoạt khi Scale):** Như ví dụ e-commerce ở trên, Monolithic yêu cầu "mở rộng phụ thuộc" (dependent scaling). Khi một phần nhỏ của ứng dụng quá tải, bạn buộc phải nhân bản toàn bộ hệ thống, dẫn đến việc lãng phí tài nguyên máy chủ, bộ nhớ và chi phí vận hành một cách không cần thiết.
- **Độ tin cậy và khả năng chịu lỗi thấp (Poor Fault Tolerance):** Vì các thành phần liên kết quá chặt chẽ (tight coupling) và chia sẻ chung bộ nhớ, một lỗi nhỏ ở bất kỳ module nào có thể làm treo và sập toàn bộ ứng dụng.
- **Rủi ro khi Deploy:** Mỗi lần cập nhật dù chỉ là sửa lỗi chính tả trên giao diện, bạn vẫn phải build lại toàn bộ gói mã nguồn khổng lồ. Quy trình CI/CD kéo dài (có khi mất hàng giờ để chạy hết bộ test) làm nản lòng đội ngũ phát triển.
- **Ràng buộc công nghệ (Technological Lock-in):** Trong khối Monolithic, mọi thành phần phải sử dụng chung một ngôn ngữ lập trình, một runtime và hệ sinh thái thư viện. Điều này triệt tiêu cơ hội áp dụng các công nghệ tiên tiến mang tính đặc thù (như Rust để xử lý hiệu năng cao, hay Python để tích hợp AI) cho những module cụ thể.
- **Tích tụ nợ kỹ thuật và cực kỳ phức tạp để bảo trì:** Khi cơ sở mã (codebase) ngày càng lớn, sự phụ thuộc chéo giữa các thành phần trở nên dày đặc. Sự phức tạp này biến việc tái cấu trúc (refactoring) thành một cơn ác mộng và khiến các lập trình viên mới gặp vô vàn khó khăn để hiểu được kiến trúc tổng thể.
- **Sự phức tạp trong quản lý nhóm:** Khi số lượng lập trình viên tăng lên, Monolithic trở thành một "bãi mìn". Nhiều người cùng sửa đổi trên một tệp tin cấu hình hoặc chung một schema database dẫn đến xung đột (merge conflict) liên tục. Trách nhiệm

giữa các nhóm bị chồng chéo, khó xác định ai chịu trách nhiệm khi có lỗi xảy ra.

1.1.5. Khi nào nên dùng Monolithic

Bất chấp các nhược điểm, Monolithic tuyệt đối không phải là kiến trúc "lỗi thời". Nó vẫn là sự lựa chọn tối ưu và mang lại hiệu quả cao nhất trong các kịch bản sau:

- Công ty khởi nghiệp (Startups) và Các dự án cơ bản: Các startup thường cần tối ưu hóa nguồn vốn và cần ra mắt sản phẩm thật nhanh (time-to-market)
- Xây dựng sản phẩm thử nghiệm (MVP): Khi dự án có quy mô nhỏ, phạm vi tính năng ít biến động và mục đích chính là nhanh chóng xác thực ý tưởng kinh doanh, Monolithic là cách tiếp cận thực dụng nhất,.
- Ứng dụng đòi hỏi hiệu suất cao và độ trễ cực thấp: Nhờ việc không có độ trễ mạng (network latency) giữa các thành phần (do giao tiếp nội bộ thông qua lời gọi hàm trực tiếp), Monolithic xử lý các tác vụ cực kỳ nhanh.
- Môi trường có yêu cầu nghiêm ngặt về kiểm toán: Đối với các lĩnh vực như y tế hoặc tài chính, việc quản lý mã nguồn tập trung và triển khai thành một khối duy nhất giúp các quy trình kiểm tra bảo mật, cấp chứng nhận tuân thủ và truy xuất nguồn gốc (traceability) diễn ra dễ dàng hơn.

1.2 Microservices Architecture

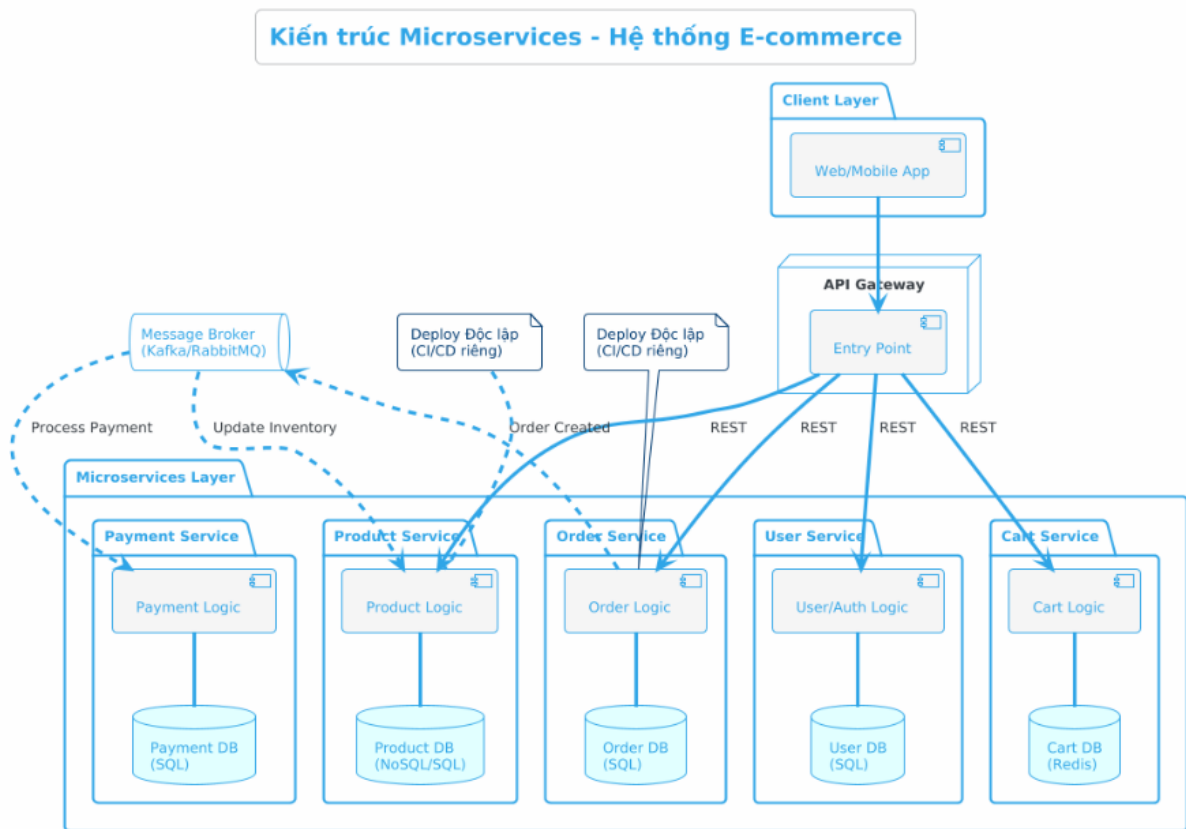
1.2.1. Khái niệm

Trong những năm gần đây, Microservices (Kiến trúc vi dịch vụ) đã trở thành một trong những xu hướng kiến trúc phần mềm thống trị nhằm giải quyết các giới hạn của kiến trúc Monolithic (nguyên khối) truyền thống. Theo định nghĩa của các chuyên gia tiên phong như Martin Fowler và James Lewis, Microservices là một phương pháp tiếp cận để phát triển một ứng dụng đơn lẻ như một **tập hợp các dịch vụ nhỏ** (suite of small services).

- Mỗi dịch vụ vận hành trong môi trường runtime riêng, đảm bảo rằng lỗi của dịch vụ này không gây sụp đổ toàn bộ hệ thống (fault isolation).
- Các dịch vụ tương tác thông qua các giao thức tối giản, phổ biến nhất là **HTTP/REST Resource API** hoặc các cơ chế truyền tin bất đồng bộ (Message Queues).

1.2.2. Đặc điểm

Kiến trúc Microservices sở hữu những đặc điểm cốt lõi giúp phân biệt nó với các phong cách kiến trúc khác (như SOA hay Monolith):



Hình 1.2. Kiến trúc Microservice cho hệ Ecommerce

- **Mỗi service có database riêng (Database per service):** Đây là một đặc điểm then chốt để đảm bảo sự liên kết lỏng lẻo (loose coupling). Thay vì chia sẻ chung một cơ sở dữ liệu khổng lồ (Global data model), mỗi microservice sở hữu và quản lý một cơ sở dữ liệu độc lập. Ở cấp độ thời gian chạy (runtime), điều này đảm bảo rằng một dịch vụ sẽ không bao giờ bị chặn (blocked) do một dịch vụ khác đang giữ khóa (lock) trên cơ sở dữ liệu. Ở cấp độ phát triển, các lập trình viên có thể thay đổi schema của database mà không cần phải phối hợp với các team khác.
Ví dụ: mỗi service có một DB riêng. Order Service - OrderDB, CartService - CartDB,...
- **Giao tiếp qua API (REST/gRPC) và Messaging:** Do các dịch vụ chạy trên các tiến trình riêng biệt, chúng không thể gọi hàm trực tiếp (method calls) như trong kiến trúc nguyên khối, mà phải giao tiếp qua mạng thông qua các giao thức liên tiến trình (IPC). Các giao thức này bao gồm giao tiếp đồng bộ như **HTTP-based REST** hoặc **gRPC**, và giao tiếp bất đồng bộ thông qua **Message Broker** (như Apache Kafka, RabbitMQ). Microservices ưu tiên các cơ chế giao tiếp "dumb pipes" (đường ống giao tiếp đơn giản) thay vì các hệ thống trung gian phức tạp (Smart pipes/ESB) như trong SOA.
- **Deploy độc lập (Independent deployment):** Tính độc lập là trái tim của Microservices. Mỗi dịch vụ là một thành phần phần mềm có thể triển khai độc lập

(independently deployable). Chúng sở hữu mã nguồn riêng và một pipeline triển khai tự động riêng biệt (CI/CD). Một team có thể cập nhật, kiểm thử và đưa một dịch vụ lên môi trường production mà không cần phải biên dịch hay triển khai lại toàn bộ hệ thống,.

1.2.3. Kiến trúc Microservices trong Bigtech.

Sự chuyển dịch từ Monolithic sang Microservices tại các Big Tech không phải là một sở thích công nghệ, mà là một sự tất yếu về kinh tế và vận hành.

- Sự bùng nổ về quy mô (Scalability): Khi lượng người dùng tăng từ hàng triệu lên hàng tỷ, việc mở rộng một khối Monolith duy nhất đòi hỏi tài nguyên phần cứng cực lớn và kém hiệu quả.
- Tốc độ đưa sản phẩm ra thị trường (Time-to-market): Trong mô hình Monolith, một thay đổi nhỏ ở giao diện cũng yêu cầu build và deploy lại toàn bộ hệ thống, gây nghẽn cổ chai tại bộ phận kiểm thử (QA).
- Sự cô lập lỗi (Fault Isolation): Các Big Tech không thể chấp nhận việc một lỗi nhỏ ở module gợi ý sản phẩm làm sập toàn bộ quy trình thanh toán của khách hàng.

* Amazon : Cuộc cách mạng về giao diện dịch vụ (API)

Vào đầu những năm 2000, trang web Amazon.com là một khối mã nguồn (Monolith) khổng lồ, khiến các kỹ sư mất hàng tuần để kiểm thử mỗi khi thay đổi một dòng code. Năm 2002, Jeff Bezos đưa ra sắc lệnh nổi tiếng: tất cả các team phải cung cấp dữ liệu và chức năng thông qua các giao diện dịch vụ (Service Interfaces) và chỉ giao tiếp qua mạng.

Amazon chuyển sang mô hình **"You build it, you run it"**. Mỗi nhóm nhỏ (Two-pizza teams) sở hữu trọn vòng đời của một service. Đến nay, hệ thống của Amazon bao gồm hàng chục nghìn microservices, cho phép họ thực hiện trung bình 1 bản deploy mỗi 11.7 giây. Họ đã đạt được những con số ấn tượng:

- Tốc độ triển khai: Sau khi chuyển đổi, Amazon thực hiện trung bình 11.7 giây/lần deploy (khoảng 50 triệu bản deploy mỗi năm).
- Số lượng Service: Hệ thống hiện nay vận hành hàng chục nghìn microservices độc lập.

* Netflix : Tiên phong trong kiến trúc chịu lỗi (Resilience)

Netflix đã định nghĩa lại cách thế giới nhìn nhận về Microservices khi chuyển toàn bộ từ Data Center vật lý lên AWS vào năm 2009 sau khi một lỗi database làm sập dịch vụ DVD trong 3 ngày. Netflix chia nhỏ hệ thống thành hơn 1.000 microservices. Mỗi khi bạn nhấn "Play", có khoảng 500 microservices hoạt động ngầm để kiểm tra gói cước, thiết bị, phụ đề và gợi ý phim.

Đóng góp: Họ tạo ra bộ công cụ Netflix OSS (như Spring Cloud Netflix), giới thiệu các khái niệm quan trọng như Circuit Breaker (ngắt mạch khi service lỗi) và Service Discovery. Họ chứng minh rằng trong hệ thống phân tán, lỗi là điều chắc chắn xảy ra, và thiết kế phải thích nghi với lỗi thay vì cố ngăn chặn nó.

Hiện nay Netflix hiện có hơn 1.000 dịch vụ chạy song song, xử lý hơn 2 nghìn tỷ (2 trillion) sự kiện mỗi ngày và chạy trên hơn 100.000 instance của Amazon EC2.

* Uber : Sự tiến hóa từ Micro sang Macro

Uber khởi đầu với kiến trúc Monolith đơn giản để kết nối tài xế và hành khách. Uber

là minh chứng cho việc Microservices có thể trở nên quá phức tạp nếu không được quản lý tốt.

Giai đoạn bùng nổ: Khi mở rộng ra hàng trăm thành phố, họ gặp khó khăn trong việc quản lý logic kinh doanh khác nhau. Họ đã tách ra hàng nghìn microservices. Với 6.000 service, việc theo dõi lỗi (tracing) trở nên cực kỳ khó khăn. Một request đặt xe có thể đi qua hàng trăm service.. Họ chuyển sang mô hình "Domain-oriented Microservices" – gom các service có liên quan chặt chẽ về nghiệp vụ thành các Domain để giảm bớt sự phức tạp trong giao tiếp mạng. Uber đã giới thiệu kiến trúc UFA (Failover Architecture) giúp giảm tài nguyên dự phòng từ 2.0 lần xuống còn 1.3 lần, tiết kiệm hàng triệu lõi CPU (khoảng 1 triệu CPU cores được cắt giảm từ nền tảng 4 triệu cores).

Tuy vậy, Các Big Tech cũng thừa nhận rằng Microservices đi kèm với "thuế phức tạp" (Complexity Tax):

- Trễ mạng (Network Latency): Việc gọi qua lại giữa các service làm tăng độ trễ so với gọi hàm trong bộ nhớ.
- Nhất quán dữ liệu: Việc duy trì dữ liệu đồng nhất giữa các database riêng biệt của từng service là bài toán cực khó (thường phải dùng Pattern Saga hoặc Event-driven).
- Chi phí vận hành: Đòi hỏi đội ngũ kỹ sư có trình độ cao về DevOps và hệ thống phân tán.

Quá trình phát triển của các Big Tech cho thấy Microservices không phải là "viên đạn bạc" giải quyết mọi vấn đề, nhưng nó là công cụ mạnh mẽ nhất để quản lý sự phức tạp ở quy mô cực lớn. Sự thành công của họ nằm ở chỗ biết khi nào cần chia nhỏ và khi nào cần tối ưu hóa các ranh giới dịch vụ để phục vụ mục tiêu kinh doanh.

1.2.4. So sánh Monolithic vs Microservices

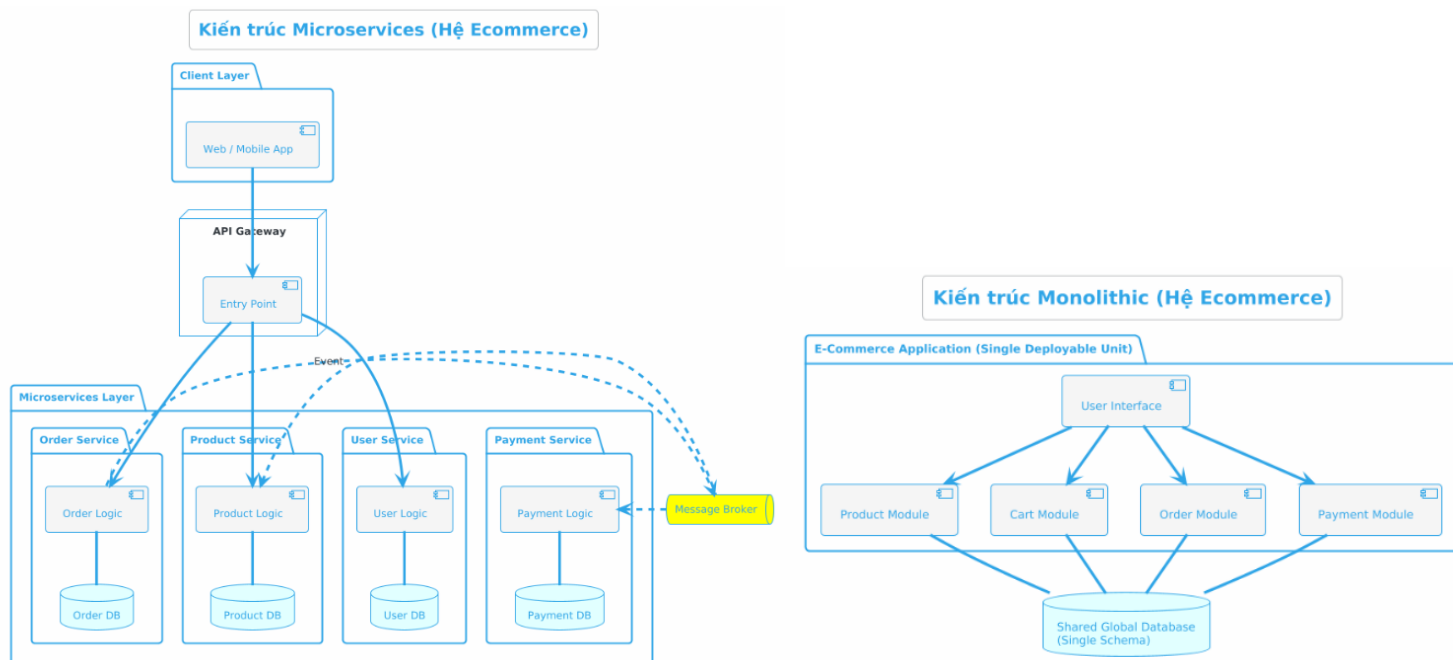
Sự khác biệt giữa hai kiến trúc này tác động mạnh mẽ đến quy trình phát triển và vận hành hệ thống phần mềm.

Tiêu chí	Monolithic (Nguyên khối)	Microservices (Vi dịch vụ)
Deploy (Triển khai)	Một lần (Cả hệ thống): Ứng dụng được đóng gói thành một file duy nhất (WAR/JAR file hoặc thư mục code). Mỗi lần có thay đổi nhỏ, toàn bộ hệ thống phải được build, test và deploy lại, gây mất thời gian và rủi ro cao,.	Nhiều lần (Độc lập): Mỗi dịch vụ được đóng gói và triển khai hoàn toàn độc lập (thường dưới dạng Container/Docker),. Thay đổi ở dịch vụ nào chỉ cần deploy lại dịch vụ đó.
Scale (Mở rộng)	Toàn hệ thống: Chủ yếu mở rộng theo chiều ngang (X-axis) bằng cách chạy nhiều bản sao (clones) của toàn bộ ứng dụng sau một Load Balancer. Rất lãng phí tài	Từng service: Các dịch vụ có thể được mở rộng hoàn toàn độc lập tùy theo nhu cầu tài nguyên của chúng, (Ví dụ: Service xử lý ảnh tốn CPU có thể scale độc lập với Service bộ nhớ).

	nguyên nếu chỉ có một module bị quá tải,.	
Coupling (Sự phụ thuộc)	Cao (Tightly Coupled): Các module nằm chung bộ nhớ, gọi hàm trực tiếp, chia sẻ chung một Database. Sự thay đổi ở một module dễ dàng gây lỗi dây chuyền cho các module khác,.	Thấp (Loosely Coupled): Giao tiếp hoàn toàn thông qua API (REST/gRPC/Messaging). Dữ liệu được cô lập (Database per service) nên ranh giới giữa các module rất rõ ràng, bảo vệ hệ thống khỏi lỗi dây chuyền,.
Technology Stack (Công nghệ)	Bị "khóa chặt" (Locked-in) vào một nền tảng ngôn ngữ và framework duy nhất được chọn từ đầu,.	Đa dạng công nghệ (Polyglot). Mỗi dịch vụ có thể được viết bằng một ngôn ngữ/framework và sử dụng loại Database (SQL/NoSQL) phù hợp nhất với chức năng của nó

* So sánh 2 kiến trúc trong ví dụ e-commerce

Dưới đây là sơ đồ mô tả kiến trúc thiết kế của một hệ e-commerce đơn giản theo hai kiến trúc Monolithic và Microservice



Hình 1.3. So sánh kiến trúc Mono và Micro trong hệ Ecommerce

Bảng so sánh thiết kế hai loại kiến trúc

Đặc điểm	Kiến trúc Monolithic (Đơn khối)	Kiến trúc Microservices (Vi dịch vụ)
Cấu trúc Codebase	Toàn bộ các module (Order, Product,...) nằm chung trong một ứng dụng duy nhất.	Mỗi module là một service riêng biệt, chạy độc lập và có codebase riêng.
Cơ sở dữ liệu	Sử dụng chung một Database duy nhất (Shared Database).	Mỗi service sở hữu một Database riêng (Database per Service).
Triển khai (Deployment)	Phải deploy toàn bộ hệ thống cùng lúc dù chỉ thay đổi một logic nhỏ.	Có thể deploy từng service độc lập mà không ảnh hưởng đến các phần khác.

Giao tiếp hệ thống	Các module gọi nhau trực tiếp qua hàm (Function call/In-process).	Giao tiếp qua mạng (HTTP/REST, gRPC) tiếp qua Message Broker.
Điểm truy cập (Entry Point)	Client thường kết nối trực tiếp với server duy nhất.	Client đi qua một cổng tập trung gọi là API Gateway.
Khả năng mở rộng (Scaling)	Phải nhân bản toàn bộ ứng dụng (Scale cả khối).	Có thể mở rộng riêng lẻ các service đang bị quá tải (Ví dụ: chỉ scale Payment Service).
Độ phức tạp	Thấp khi bắt đầu, nhưng khó quản lý khi hệ thống trở nên quá lớn.	Cao ngay từ đầu do cần quản lý hạ tầng, mạng và sự phân tán dữ liệu.
Khả năng chịu lỗi	Một module bị lỗi (như tràn bộ nhớ) có thể làm sập toàn bộ hệ thống.	Một service lỗi có thể được cách ly, các service khác vẫn hoạt động bình thường.
Kiến trúc ứng dụng	Layered Architecture: Các lớp (UI, Business Logic, Data Access) xếp chồng lên nhau trong một khối.	Layered Architecture: Các lớp (UI, Business Logic, Data Access) xếp chồng lên nhau trong một khối.

* Tính đóng gói, gắn kết

Đặc điểm nhận dạng lớn nhất của Monolithic là tính nhất thể. Toàn bộ các thành phần từ giao diện (UI), logic nghiệp vụ (Business Logic) đến truy xuất dữ liệu đều được đóng gói trong một đơn vị triển khai duy nhất. Điều này tạo ra sự gắn kết chặt chẽ (Tight Coupling), nơi các hàm và module gọi trực tiếp lẫn nhau trong cùng một không gian bộ nhớ.

Ngược lại, Microservices tách biệt hệ thống thành các dịch vụ nhỏ, độc lập. Mỗi dịch vụ đảm nhận một nghiệp vụ riêng biệt và giao tiếp với nhau qua mạng (thường là REST API hoặc Message Broker). Thiết kế này hướng tới sự gắn kết lỏng lẻo (Loose Coupling), cho phép mỗi phần của hệ thống tồn tại và tiến hóa mà không gây ảnh hưởng trực tiếp đến toàn bộ cấu trúc còn lại.

* Quản lý dữ liệu và công nghệ

Trong kiến trúc Monolithic, hệ thống thường sử dụng một cơ sở dữ liệu tập trung duy nhất. Điều này giúp việc đảm bảo tính toàn vẹn dữ liệu (ACID) trở nên đơn giản thông qua các giao dịch nội bộ. Tuy nhiên, nó cũng tạo ra một "điểm nghẽn" khi tất cả các module đều phụ thuộc vào một cấu trúc bảng chung.

Microservices phá vỡ quy tắc này bằng tư duy "Database per Service". Mỗi dịch vụ sở hữu cơ sở dữ liệu riêng, thậm chí có thể sử dụng các loại công nghệ khác nhau (Polyglot Persistence) như SQL cho thanh toán và NoSQL cho tìm kiếm. Dù thiết kế này mang lại sự linh hoạt cực cao, nó lại đặt ra thách thức lớn về việc đồng bộ dữ liệu và tính nhất quán cuối

cùng (Eventual Consistency).

** Khả năng mở rộng và Triển khai*

Sự khác biệt về thiết kế dẫn đến cách thức vận hành hoàn toàn khác nhau:

- **Monolithic:** Khi cần mở rộng, bạn phải sao chép toàn bộ khối ứng dụng lên nhiều máy chủ (Horizontal Scaling). Điều này gây lãng phí tài nguyên nếu hệ thống chỉ bị nghẽn ở một module cụ thể. Việc triển khai cũng mang tính rủi ro cao vì một lỗi nhỏ trong mã nguồn có thể làm sập toàn bộ hệ thống.
- **Microservices:** Cho phép mở rộng chọn lọc. Nếu dịch vụ "Đặt hàng" bị quá tải, bạn chỉ cần tăng số lượng thực thể cho riêng dịch vụ đó. Việc triển khai diễn ra liên tục (CI/CD) và độc lập, giúp giảm thiểu thời gian "chết" của hệ thống.

Tóm lại, Monolithic ưu tiên sự đơn giản trong phát triển và kiểm thử ở giai đoạn đầu, phù hợp với các dự án nhỏ hoặc MVP. Trong khi đó, Microservices đánh đổi sự phức tạp trong quản lý và giao tiếp mạng để lấy khả năng mở rộng vô hạn và tính linh hoạt về công nghệ. Việc lựa chọn giữa "Mno" hay "Micro" không phụ thuộc vào việc kiến trúc nào hiện đại hơn, mà phụ thuộc vào quy mô bài toán và năng lực quản trị hạ tầng của đội ngũ phát triển.

1.2.5. Nhược điểm

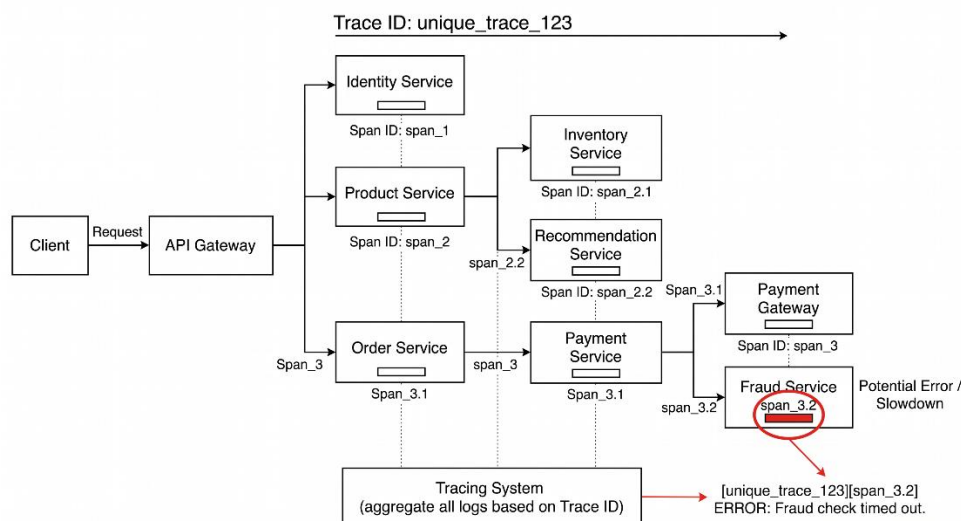
Tuy nhiên, như Fred Brooks từng nói "Không có viên đạn bạc trong kỹ nghệ phần mềm", Microservices đi kèm với "cái giá" của một hệ thống phân tán (Distributed System):

- Phức tạp hệ thống (Operational Complexity): Sự gia tăng về số lượng các dịch vụ (từ một ứng dụng lên tới hàng chục hoặc hàng trăm microservices) dẫn đến sự phức tạp trong vận hành. Quản lý cơ sở hạ tầng mạng, cân bằng tải, service discovery (khám phá dịch vụ), đòi hỏi mức độ tự động hóa rất cao với các nền tảng như Kubernetes, Docker Swarm hoặc Service Mesh (như Istio),,..
- Quản lý distributed system (Hệ thống phân tán): Lập trình viên phải đối mặt với những vấn đề hóc búa của hệ thống phân tán.
 - *Giao dịch phân tán (Distributed Transactions):* Do mỗi service có database riêng, việc cập nhật dữ liệu trải dài trên nhiều dịch vụ không thể sử dụng ACID transaction thông thường (hay 2PC),,. Hệ thống bắt buộc phải sử dụng các cơ chế phức tạp hơn như Saga pattern (chuỗi các giao dịch cục bộ được điều phối bằng messaging) với khái niệm "eventual consistency" (nhất quán cuối cùng),,.
 - *Truy vấn phân tán:* Lấy dữ liệu nằm rải rác trên nhiều database đòi hỏi phải áp dụng mô hình API Composition (gọi nhiều API và tổng hợp kết quả) hoặc CQRS (duy trì một cơ sở dữ liệu read-only được đồng bộ hóa qua events),,.
 - *Partial failure (Lỗi một phần):* Khi một dịch vụ gọi một dịch vụ khác, có rủi ro dịch vụ đích bị sập hoặc quá tải. Cần áp dụng các mẫu thiết kế như Circuit

Breaker để ngăn lỗi lan truyền (cascading failure),..

- Debug khó hơn: Khi một lỗi xảy ra hoặc một request bị phản hồi chậm, luồng xử lý có thể đang nhảy qua lại giữa 5-6 dịch vụ khác nhau. Việc gỡ lỗi truyền thống qua log file đơn lẻ không còn tác dụng. Thay vào đó, hệ thống bắt buộc phải áp dụng các mẫu Distributed Tracing (theo vết phân tán như Zipkin) bằng cách gắn Trace ID cho mỗi request, cùng với Log Aggregation (Tập trung log qua ELK stack)

Dưới đây là một ví dụ điển hình khi lập trình viên phải đối mặt với các vấn đề của hệ thống phân tán khi triển khai kiến trúc Microservices



Hình 1.4. Lỗi trace id trong microservice

Khi Client gửi request tới API Gateway, yêu cầu này được phân tách và chuyển đến nhiều service khác nhau:

- Identity Service: xác thực người dùng
- Product Service: xử lý thông tin sản phẩm
- Inventory Service: kiểm tra tồn kho
- Recommendation Service: gợi ý sản phẩm
- Order Service: tạo đơn hàng
- Payment Service → Payment Gateway → Fraud Service: xử lý thanh toán và kiểm tra gian lận

Tất cả các lời gọi này tạo thành một chuỗi phân tán, được hệ thống Tracing System thu thập và tổng hợp log dựa trên Trace ID.

* Mỗi service trong hình có database riêng → không thể dùng transaction ACID truyền thống.

Ví dụ trong luồng:

- Order Service tạo đơn
- Payment Service xử lý thanh toán
- Fraud Service kiểm tra gian lận

Nếu một bước thất bại (như Fraud Service timeout), toàn bộ hệ thống không thể rollback kiểu truyền thống.

* *Dữ liệu nằm rải rác*: Product Service giữ thông tin sản phẩm, Inventory Service giữ tồn kho, Recommendation Service tính toán gợi ý → Khi cần hiển thị thông tin cho user phải gọi nhiều service và tổng hợp kết quả

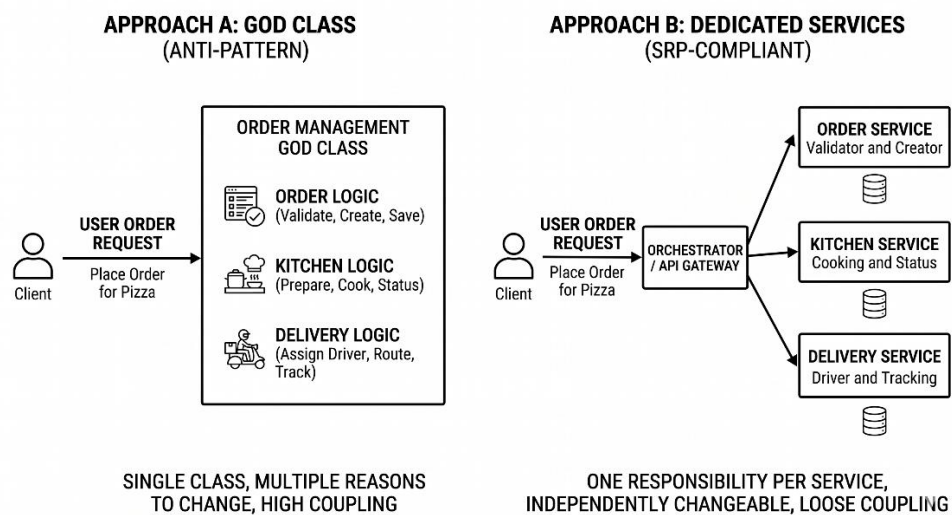
Partial Failure (lỗi một phần): Như trong hình **Fraud Service (span_3.2)** bị timeout. Nếu không xử lý được lỗi rất dễ có nguy cơ lỗi lan sang Payment Service, Order Service hay thậm chí là toàn bộ hệ thống

1.2.6. Nguyên tắc thiết kế

Để kiến trúc Microservices phát huy tối đa sức mạnh và tránh việc vô tình biến hệ thống thành một "Monolith phân tán" (Distributed Monolith – một hệ thống mang những khuyết điểm của cả hai loại kiến trúc), các kiến trúc sư phần mềm cần tuân thủ nghiêm ngặt các nguyên tắc thiết kế cốt lõi. Dưới đây là 3 nguyên tắc quan trọng nhất được áp dụng trong việc phân rã và thiết kế các vi dịch vụ.

* **Single Responsibility (Trách nhiệm đơn lẻ - SRP)** Nguyên tắc Trách nhiệm đơn lẻ (SRP) vốn là một trong 5 nguyên tắc SOLID nổi tiếng của thiết kế hướng đối tượng do Robert C. Martin định nghĩa, phát biểu rằng: "Một lớp chỉ nên có duy nhất một lý do để thay đổi".

- Khi áp dụng nguyên tắc này vào cấp độ kiến trúc Microservices, hệ thống sẽ được phân rã thành các dịch vụ nhỏ, có tính tập trung cao và mỗi dịch vụ chỉ đảm nhận một trách nhiệm kinh doanh duy nhất. Thay vì ôm đồm nhiều chức năng, một microservice phải được thiết kế với các ranh giới rõ ràng và chỉ thực hiện "tốt một việc duy nhất" (do one thing well). Việc áp dụng SRP giúp giảm thiểu kích thước của mã nguồn trong từng dịch vụ, gia tăng sự ổn định và dễ dàng quản lý.
- *Ví dụ thực tế*:



Hình 1.5. Nguyên tắc Single Responsibility trong hệ thống thực tế

Trong kiến trúc của hệ thống giao đồ ăn FTGO, các kỹ sư không xây dựng một dịch vụ chung cho toàn bộ luồng xử lý đơn hàng (Approach A). Thay vào đó, mỗi khía cạnh của việc cung cấp thức ăn cho khách hàng được chia thành các dịch vụ có trách nhiệm riêng biệt (Approach B): tiếp nhận đơn hàng (Order Service), chuẩn bị đơn hàng (Kitchen Service), và giao hàng (Delivery Service).

*** Loose Coupling (Liên kết lỏng lẻo)** Một trong những đặc điểm quan trọng và định hình nên kiến trúc Microservices chính là các dịch vụ phải được liên kết lỏng lẻo với nhau. Loose Coupling đòi hỏi mỗi dịch vụ phải che giấu hoàn toàn các chi tiết triển khai nội bộ của nó; mọi sự tương tác và giao tiếp giữa các dịch vụ chỉ được phép diễn ra thông qua một giao diện lập trình ứng dụng (API) rõ ràng.

- Lợi ích cốt lõi của việc này là cho phép một dịch vụ có thể thay đổi logic nội bộ, cập nhật công nghệ hoặc triển khai độc lập mà hoàn toàn không gây ảnh hưởng hay phá vỡ các client (hoặc các dịch vụ khác) đang gọi đến nó.
- Để thực sự đạt được trạng thái liên kết lỏng lẻo, quy tắc tối quan trọng là không chia sẻ cơ sở dữ liệu chung (Database per service). Trong Microservices, dữ liệu bền vững (persistent data) của một dịch vụ phải được đối xử như các trường dữ liệu riêng tư (private fields) của một lớp, các dịch vụ tuyệt đối không được giao tiếp với nhau bằng cách đọc/ghi trực tiếp vào cơ sở dữ liệu của dịch vụ khác.

*** High Cohesion (Tính gắn kết cao)** Đi đôi với Loose Coupling là High Cohesion. Tính gắn kết cao ở đây dựa trên "Nguyên tắc Đóng gói Chung" (Common Closure Principle - CCP), phát biểu rằng: các thành phần (lớp, module) thay đổi cùng nhau vì chung một lý do thì nên được đóng gói vào cùng một nơi,.

- Khi thiết kế Microservices, kiến trúc sư cần nhóm các hành vi, nghiệp vụ và dữ liệu có liên quan chặt chẽ với nhau vào cùng một dịch vụ,. Việc thiết kế theo nguyên tắc CCP sẽ giúp tối thiểu hóa số lượng dịch vụ cần phải thay đổi, kiểm thử và triển khai lại mỗi khi có một yêu cầu nghiệp vụ mới phát sinh, tránh được hiệu ứng thay đổi dây chuyền (cascading changes),.

Để đạt được tính gắn kết cao và xác định đúng ranh giới của dịch vụ, các kỹ sư thường áp dụng kỹ thuật **Domain-Driven Design (DDD - Thiết kế hướng mô hình miền)**, đặc biệt là khái niệm *Tên miền phụ (Subdomain)* và *Ngữ cảnh giới hạn (Bounded Context)*,. DDD giúp chia nhỏ một miền nghiệp vụ phức tạp thành nhiều Bounded Context, trong đó mỗi Bounded Context có thể trở thành một microservice (hoặc một tập hợp nhỏ các microservices) độc lập, chứa một mô hình miền chuyên biệt dành riêng cho ngữ cảnh của nó.

1.3 Domain Driven Design (DDD)

Trong bối cảnh phát triển phần mềm hiện đại, kiến trúc Microservices đã trở thành tiêu chuẩn để xây dựng các hệ thống lớn, có khả năng mở rộng và linh hoạt. Tuy nhiên, thách thức lớn nhất khi chuyển đổi từ hệ thống nguyên khối (Monolith) sang Microservices không nằm ở công nghệ, mà ở cách phân rã hệ thống. Domain-Driven Design (DDD), một phương pháp tiếp cận do Eric Evans khởi xướng, cung cấp một bộ công cụ chiến lược và chiến thuật hoàn hảo để giải quyết bài toán này. Bằng cách tập trung vào nghiệp vụ cốt lõi, DDD giúp các kiến trúc sư định hình ranh giới vi dịch vụ một cách chính xác, tránh tạo ra một "mớ hỗn độn phân tán" (distributed big ball of mud).

1.3.1. Mục tiêu

Mục tiêu tối thượng của DDD là đặt trọng tâm của quá trình thiết kế phần mềm vào miền nghiệp vụ (business domain) và logic nghiệp vụ, thay vì bị chi phối bởi các yếu tố công nghệ hay cơ sở dữ liệu.

- **Tập trung vào giá trị nghiệp vụ:** Trong phần lớn các dự án phần mềm, sự phức tạp không đến từ công nghệ mà đến từ chính nghiệp vụ. DDD giúp các nhóm phát triển phần mềm chất lọc kiến thức nghiệp vụ, tạo ra một mô hình sâu sắc (deep model) phản ánh chính xác cách doanh nghiệp hoạt động,.
- **Ngôn ngữ chung (Ubiquitous Language):** DDD phá bỏ rào cản giao tiếp giữa các chuyên gia nghiệp vụ (Domain Experts) và đội ngũ kỹ thuật (Developers) bằng cách xây dựng một ngôn ngữ chung,. Ngôn ngữ này được sử dụng nhất quán từ các buổi họp lấy yêu cầu cho đến tài liệu và mã nguồn.
- **Tách biệt với công nghệ:** DDD cô lập logic nghiệp vụ khỏi các lớp giao diện (UI) hoặc cơ sở hạ tầng (Infrastructure) thông qua kiến trúc phân lớp (Layered Architecture) hoặc kiến trúc lục giác (Hexagonal Architecture),. Điều này giúp mô hình nghiệp vụ không bị "ô nhiễm" bởi các chi tiết kỹ thuật như cách lưu trữ SQL hay gọi API.

Ví dụ trong hệ thống E-commerce: Thay vì nhóm phát triển thảo luận về "bảng Users" và "bảng Orders" trong SQL (ngôn ngữ công nghệ), họ sẽ thảo luận cùng chuyên gia bán hàng về "Khách hàng" (Customer), "Giỏ hàng" (Cart), và "Quy trình thanh toán" (Checkout Process).

1.3.2. Các khái niệm cốt lõi

DDD cung cấp một bộ công cụ chiến thuật (Tactical Design) để mô hình hóa cấu trúc bên trong của một domain.

1.3.2.1. Entity (Thực thể)

- Định nghĩa: Entity là một đối tượng trong mô hình nghiệp vụ không được xác định bởi các thuộc tính của nó, mà được xác định bởi một định danh duy nhất (ID) và có tính liên tục trong suốt vòng đời,.

- **Đặc điểm:** Dù các thuộc tính (như tên, địa chỉ, trạng thái) của một Entity có thay đổi theo thời gian, nó vẫn là chính nó nhờ vào ID,.
- **Ví dụ E-commerce:** User (Khách hàng) và Product (Sản phẩm). Khách hàng Nguyễn Văn A có UserID = 123. Dù khách hàng này có đổi số điện thoại, đổi địa chỉ nhận hàng hay đổi tên hiển thị, hệ thống vẫn nhận diện đây là khách hàng 123 dựa trên định danh.

1.3.2.2. Value Object (Đối tượng giá trị)

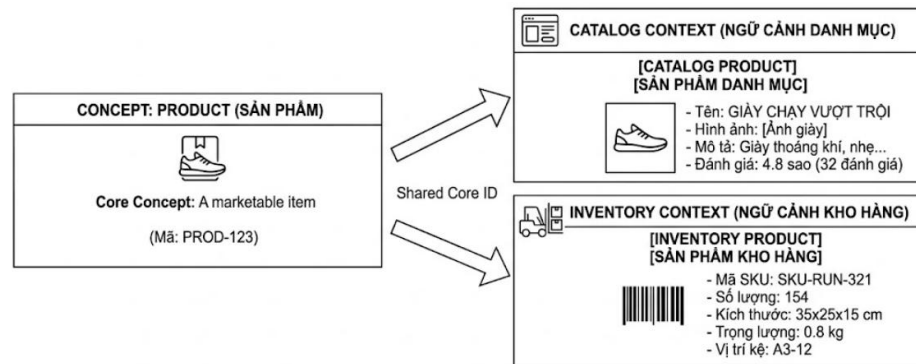
- **Định nghĩa:** Value Object là đối tượng mô tả một đặc tính hoặc thuộc tính của miền nghiệp vụ nhưng không có định danh (No ID),. Hai Value Object được coi là giống nhau nếu tất cả các giá trị thuộc tính của chúng giống nhau.
- **Đặc điểm:** Value Object phải có tính bất biến (immutable). Khi cần thay đổi, ta không sửa đổi giá trị bên trong nó mà tạo ra một Value Object hoàn toàn mới để thay thế. Việc này giúp giảm thiểu rủi ro side-effect (tác dụng phụ) khi lập trình.
- **Ví dụ:** Address (Địa chỉ giao hàng) hoặc Money (Tiền tệ). Một đối tượng Money bao gồm lượng tiền (amount) và loại tiền tệ (currency) như 100.000 và VND. Chúng ta không quan tâm tờ tiền đó có số seri là bao nhiêu, chúng ta chỉ quan tâm đến giá trị của nó. Tương tự với Address (đường, thành phố, mã bưu điện), nếu khách hàng đổi địa chỉ, ta gắn một đối tượng Address mới vào User thay vì sửa đổi đối tượng cũ.

1.3.2.3. Aggregate (Tập hợp)

- **Định nghĩa:** Aggregate là một cụm các đối tượng (gồm Entities và Value Objects) có liên quan chặt chẽ với nhau, được xử lý như một khối thống nhất (unit) để đảm bảo tính nhất quán của dữ liệu,.
- **Đặc điểm:** Mỗi Aggregate có một Entity làm rễ, gọi là Aggregate Root. Bất kỳ tương tác nào từ bên ngoài muốn chỉnh sửa các đối tượng bên trong đều phải thông qua Aggregate Root,.
- **Quan trọng nhất trong Microservices:** Một giao dịch (transaction) cơ sở dữ liệu chỉ được phép tạo hoặc cập nhật một Aggregate duy nhất,.
- **Ví dụ:**
Tập hợp Order (Đơn hàng) và OrderItem (Chi tiết đơn hàng). Order là Aggregate Root. Hệ thống không cho phép chỉnh sửa trực tiếp số lượng của một OrderItem từ bên ngoài, vì nó có thể làm sai lệch tổng tiền của đơn hàng (Order Total).

1.3.2.4. Bounded Context (Ngữ cảnh giới hạn)

- **Định nghĩa:** Một Bounded Context thiết lập ranh giới rõ ràng mà trong đó một mô hình domain (và Ngôn ngữ chung - Ubiquitous Language) được áp dụng một cách nhất quán,.
- **Đặc điểm:** Cùng một thực thể có thể mang ý nghĩa và thuộc tính hoàn toàn khác nhau khi nằm ở các Bounded Context khác nhau.
- **Ví dụ:**
Xét khái niệm "Sản phẩm" (Product):

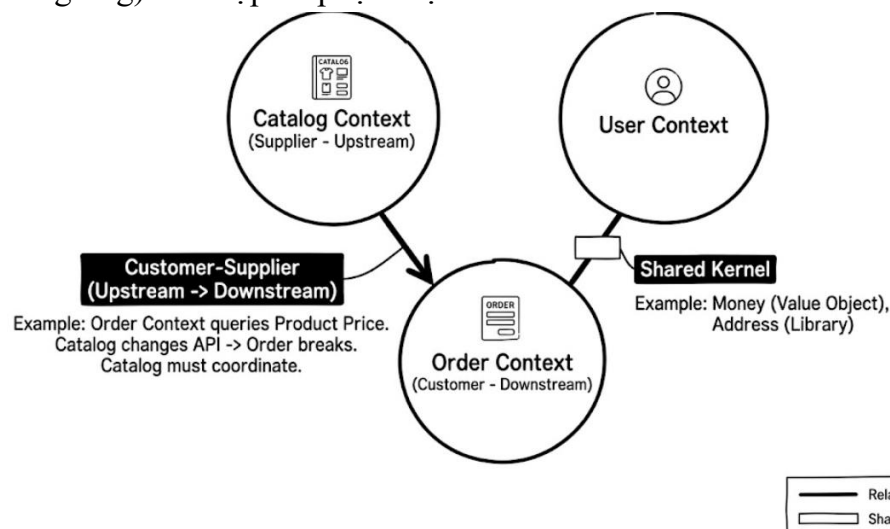


Hình 1.6. Bounded Context trong hệ Ecommerce

- Trong Catalog Context (Ngữ cảnh danh mục): Product tập trung vào tên, hình ảnh, mô tả, đánh giá (reviews) để hiển thị cho khách hàng.
- Trong Inventory Context (Ngữ cảnh kho hàng): Product được quan tâm dưới góc độ mã SKU, số lượng tồn kho, kích thước, trọng lượng, và vị trí kệ hàng.
- Việc chia Bounded Context giúp hệ thống không tạo ra một class Product khổng lồ (God class) chứa hàng trăm thuộc tính của cả hệ thống, gây khó khăn cho việc bảo trì,.

1.3.3. Context Map

Khi hệ thống có nhiều Bounded Context, chúng cần phải giao tiếp với nhau. Context Map là một sơ đồ kiến trúc thể hiện bức tranh toàn cảnh về cách các Bounded Context (và các nhóm phát triển tương ứng) tích hợp và phụ thuộc vào nhau.



Hình 1.7. Context Map trong hệ Ecommerce

1.3.3.1. Mối quan hệ Shared Kernel (Nhân chia sẻ)

- **Định nghĩa:** Là một phần của mô hình (domain) và mã nguồn được hai hoặc nhiều nhóm (Contexts) thống nhất chia sẻ và sử dụng chung. Bất kỳ thay đổi nào trong

Shared Kernel đều phải được sự đồng thuận của tất cả các nhóm liên quan.

- **Ví dụ:** Cả Order Context và Payment Context đều cần sử dụng chung khái niệm Money (Value Object tính toán tiền tệ) và Address (Địa chỉ giao hàng). Các lớp đối tượng này được tách ra thành một thư viện dùng chung (Shared Kernel). Điều này giảm thiểu việc viết lại mã nhưng yêu cầu sự phối hợp chặt chẽ khi muốn thay đổi định dạng của Address.

1.3.3.2. *Mối quan hệ Customer-Supplier (Khách hàng - Nhà cung cấp)*

- **Định nghĩa:** Xảy ra khi một Context (Supplier / Upstream) cung cấp dữ liệu hoặc dịch vụ cho một Context khác (Customer / Downstream). Customer hoàn toàn phụ thuộc vào Supplier. Supplier phải đáp ứng các yêu cầu của Customer để hệ thống vận hành đúng.
- **Ví dụ:** Order Context (Customer) phụ thuộc vào Catalog Context (Supplier). Khi khách hàng đặt hàng, Order Context cần truy vấn Catalog Context để lấy giá chính xác của sản phẩm hiện tại. Nếu Catalog Context thay đổi cấu trúc API trả về giá, Order Context sẽ bị lỗi. Do đó, nhóm Catalog (nhà cung cấp) phải ưu tiên và lên kế hoạch thông báo trước cho nhóm Order (khách hàng).

1.3.4. *DDD trong Microservices*

Thiết kế kiến trúc phần mềm với Microservices đối mặt với một vấn đề cốt lõi: Làm sao để chia nhỏ hệ thống một cách đúng đắn? DDD chính là lời giải đáp hoàn hảo cho câu hỏi này.

Sự kết hợp giữa DDD và Microservices tạo ra một kiến trúc lý tưởng. Các ranh giới của Bounded Context trong thiết kế chiến lược của DDD (Strategic Design) ánh xạ một cách tự nhiên và hoàn hảo thành ranh giới của một Microservice,.

Trong hệ thống E-commerce, thay vì xây dựng một cơ sở dữ liệu dùng chung, ta sẽ thiết kế:

- Order Microservice: Quản lý Order Context, chứa cơ sở dữ liệu riêng về Đơn hàng, làm việc với Aggregate Order.
- Inventory Microservice: Quản lý Inventory Context, chứa CSDL riêng về Tồn kho.

Điều này đảm bảo nguyên tắc Loose Coupling (Khớp nối lỏng) và High Cohesion (Độ gắn kết cao). Các service không dùng chung Database, do đó khi Order Microservice nâng cấp schema DB, nó không làm sập Inventory Microservice,. Các service giao tiếp với nhau thông qua API hoặc Event-driven (Domain Events).

Trong kiến trúc cũ (Monolith), các lập trình viên có xu hướng chia hệ thống thành các layer công nghệ: Presentation Layer (UI), Business Logic Layer, Data Access Layer (Database).

Nếu áp dụng tư duy này vào Microservices, hệ thống sẽ rơi vào thảm họa. Ví dụ: Tạo ra một

"Database Microservice" chỉ để truy vấn dữ liệu, và một "Business Microservice" để xử lý tính toán. Điều này làm mất đi tính độc lập nghiệp vụ và dẫn đến hiệu năng kém, vì một thay đổi ở nghiệp vụ Đơn hàng sẽ bắt buộc cả 2 service (Business và Database) phải sửa đổi và deploy cùng lúc,.

Yêu cầu của DDD: Không phân rã theo kỹ thuật. Phải phân rã theo Nghiệp vụ (Business Capability / Subdomain),. Một Microservice (đại diện cho một Bounded Context) phải sở hữu *toàn bộ* các stack công nghệ từ trên xuống dưới (từ API Controller, Domain Model, cho tới Data Access/Database) để tự phục vụ trọn vẹn một nghiệp vụ duy nhất,,.

Ví dụ E-commerce: Payment Microservice sẽ tự chứa cổng giao tiếp API nhận request thanh toán, chứa nghiệp vụ xử lý thẻ tín dụng bên trong, và tự quản lý DB lưu trữ lịch sử giao dịch. Nó độc lập hoàn toàn về công nghệ và nghiệp vụ.

Tóm lại, Domain-Driven Design không chỉ là một công cụ lập trình mà là một sự chuyển đổi tư duy chiến lược trong phát triển phần mềm. Đối với hệ thống được phát triển trên kiến trúc Microservices, DDD cung cấp kim chỉ nam từ cấp độ vĩ mô (chiến lược) thông qua việc phân rã Bounded Context, Context Mapping, cho đến cấp độ vi mô (chiến thuật) qua việc tổ chức code theo Entity, Value Object, và Aggregate. Việc tuân thủ nguyên lý "1 Bounded Context bằng 1 Microservice" và tránh chia theo lớp công nghệ (Technical Layer) chính là chìa khóa để xây dựng một hệ thống hiện đại, linh hoạt, dễ dàng mở rộng và luôn sát với thực tiễn.

1.4 Case Study: Healthcare (Luyện Decomposition)

Để minh họa rõ nét cách áp dụng Domain-Driven Design vào việc thiết kế kiến trúc Microservices, chúng ta sẽ thực hành phân rã (decomposition) một hệ thống quản lý bệnh viện. Quá trình này sẽ đi từ việc phân tích yêu cầu nghiệp vụ đến việc định hình các vi dịch vụ và phương thức giao tiếp của chúng.

1.4.1. Mô tả bài toán

Một hệ thống quản lý bệnh viện toàn diện được yêu cầu số hóa và tự động hóa các quy trình cốt lõi nhằm phục vụ bệnh nhân, bác sĩ và bộ phận quản lý. Các chức năng chính của hệ thống bao gồm:

- Quản lý bệnh nhân (Patient Management): Lưu trữ thông tin cá nhân, bảo hiểm y tế.
- Quản lý bác sĩ (Doctor Management): Quản lý thông tin chuyên môn, lịch làm việc, phòng ban.
- Đặt lịch khám (Appointment Scheduling): Cho phép bệnh nhân hoặc lễ tân đặt lịch hẹn với bác sĩ theo khung giờ trống.
- Quản lý hóa đơn/thanh toán (Billing & Payment): Tính toán chi phí khám chữa bệnh, thuốc men và xử lý thanh toán.
- Quản lý hồ sơ bệnh án (Medical Record Management): Lưu trữ lịch sử khám bệnh,

chẩn đoán, kết quả xét nghiệm.

- Quản lý kho thuốc (Pharmacy/Inventory Management): Quản lý danh mục thuốc, xuất/nhập tồn kho và đơn thuốc.
- Quản lý thiết bị y tế (Equipment Management): Theo dõi tình trạng, lịch bảo trì của các thiết bị y tế trong viện.

1.4.2. Bước 1: Xác định Domain

Bước đầu tiên trong DDD là phân rã không gian vấn đề (problem space) thành các miền phụ (Subdomains) dựa trên các khả năng nghiệp vụ (business capabilities). Thay vì nhìn hệ thống như một cơ sở dữ liệu khổng lồ, chúng ta chia nó thành các lĩnh vực chuyên môn độc lập:

- **Patient Management Domain:** Chuyên trách vòng đời và thông tin hành chính của bệnh nhân.
- **Doctor Management Domain:** Xử lý các nghiệp vụ liên quan đến nhân sự y tế (bác sĩ, y tá).
- **Appointment Scheduling Domain:** Giải quyết bài toán ghép nối thời gian giữa bệnh nhân và bác sĩ.
- **Billing & Payment Domain:** Chịu trách nhiệm về tài chính, công nợ và thanh toán bảo hiểm.
- **Clinical/Medical Record Domain:** Miền nghiệp vụ cốt lõi (Core Domain) chuyên về chuyên môn y khoa, chẩn đoán và điều trị.
- **Pharmacy & Inventory Domain:** Quản lý chuỗi cung ứng vật tư y tế và dược phẩm.
- **Equipment Management Domain:** Quản lý tài sản vật lý phục vụ khám chữa bệnh.

1.4.3. Bước 2: Xác định Bounded Context

Sau khi có các Subdomain, chúng ta thiết lập các **Bounded Context** (Ngữ cảnh giới hạn) để khoanh vùng không gian giải pháp (solution space). Mỗi Bounded Context sẽ sở hữu một Mô hình miền (Domain Model) và Ngôn ngữ chung (Ubiquitous Language) riêng biệt nhằm loại bỏ sự mâu thuẫn về khái niệm,.

- **Patient Context:** Thực thể Patient ở đây chứa các thông tin như tên, tuổi, nhóm máu, số bảo hiểm.
- **Doctor Context:** Thực thể Doctor chứa thông tin chuyên môn, khoa công tác.
- **Appointment Context:** Khái niệm Patient và Doctor trong ngữ cảnh này chỉ đơn thuần là PatientID và DoctorID cùng với thời gian hẹn, không chứa dữ liệu y khoa.
- **Billing Context:** Patient ở ngữ cảnh này đóng vai trò là Payer (người thanh toán) gắn liền với các Invoice (Hóa đơn).

- **Medical Record Context:** Tập trung vào các ClinicalRecord, Diagnosis (Chẩn đoán), Prescription (Đơn thuốc).
- **Pharmacy Context:** Tập trung vào Medication (Thuốc), Stock (Tồn kho).
- **Equipment Context:** Tập trung vào Device, MaintenanceLog.

1.4.4. Bước 3: Phân rã thành Microservices

Sự ánh xạ hoàn hảo nhất trong thiết kế hệ thống phân tán là: Mỗi Bounded Context sẽ trở thành một Microservice độc lập. Mỗi service này sẽ sở hữu một cơ sở dữ liệu riêng, đảm bảo tính tự trị (autonomy) và khớp nối lỏng (loose coupling),.

- **Patient Service:** Quản lý CSDL Bệnh nhân.
- **Doctor Service:** Quản lý CSDL Bác sĩ.
- **Appointment Service:** Quản lý lịch hẹn khám.
- **Billing Service:** Quản lý các giao dịch thanh toán.
- **Medical Record Service:** Quản lý kho dữ liệu lâm sàng.
- **Pharmacy Service:** Quản lý CSDL thuốc.
- **Equipment Service:** Quản lý CSDL trang thiết bị.

1.4.5. Bước 4: Xác định quan hệ

Các dịch vụ độc lập không thể hoạt động đơn lẻ mà phải phối hợp để hoàn thành một nghiệp vụ (System Operation). Chúng ta định nghĩa các mối quan hệ (Context Mapping) và cơ chế Giao tiếp liên tiến trình (Interprocess Communication - IPC),,:

- **Appointment Service phụ thuộc vào Patient và Doctor Service:** Để tạo lịch hẹn (POST /appointments), Appointment Service cần gọi API đồng bộ (REST/gRPC) sang Patient Service để xác thực bệnh nhân và sang Doctor Service để kiểm tra lịch trống của bác sĩ,.
- **Medical Record Service liên kết với Pharmacy Service:** Khi bác sĩ kê đơn trong Medical Record, một sự kiện (Domain Event) ví dụ PrescriptionCreated sẽ được phát ra. Pharmacy Service lắng nghe sự kiện này (giao tiếp bất đồng bộ qua Message Broker như Kafka) để chuẩn bị thuốc và trừ tồn kho,,.
- **Billing Service là Downstream của nhiều Services:** Khi một lịch khám hoàn tất hoặc thuốc được xuất, các service tương ứng phát ra sự kiện (AppointmentCompleted, MedicineDispensed). Billing Service thu thập các sự kiện này để tự động tổng hợp hóa đơn (Sử dụng mẫu kiến trúc CQRS hoặc Saga để đảm bảo tính nhất quán dữ liệu),,,.
- **Giao tiếp với External Clients thông qua API Gateway:** Ứng dụng di động của bệnh nhân không gọi trực tiếp các dịch vụ này. Thay vào đó, tất cả request đi qua một

API Gateway để thực hiện xác thực (Authentication), định tuyến (Routing) và tổng hợp dữ liệu (API Composition).

1.4.6. Cài đặt với Django

```
Command Prompt
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\TRONG KHOI>mkdir health-care-micro

C:\Users\TRONG KHOI>cd health-care-micro

C:\Users\TRONG KHOI\health-care-micro>|
```

* Patient Service

```
C:\Users\TRONG KHOI\health-care-micro>mkdir patient-service

C:\Users\TRONG KHOI\health-care-micro>cd patient-service

C:\Users\TRONG KHOI\health-care-micro\patient-service>pip install django djangorestframework requests

C:\Users\TRONG KHOI\health-care-micro\patient-service>django-admin startproject patient_service .

C:\Users\TRONG KHOI\health-care-micro\patient-service>python manage.py startapp app
```

* Doctor Service

```
C:\Users\TRONG KHOI\health-care-micro>mkdir doctor-service

C:\Users\TRONG KHOI\health-care-micro>cd doctor-service

C:\Users\TRONG KHOI\health-care-micro\doctor-service>pip install django djangorestframework requests

C:\Users\TRONG KHOI\health-care-micro\doctor-service>django-admin startproject doctor_service .

C:\Users\TRONG KHOI\health-care-micro\doctor-service>python manage.py startapp app
```

* Appointment Service

```
C:\Users\TRONG KHOI\health-care-micro>mkdir appointment-service
C:\Users\TRONG KHOI\health-care-micro>cd appointment-service
C:\Users\TRONG KHOI\health-care-micro\appointment-service>pip install django djangorestframework requests
C:\Users\TRONG KHOI\health-care-micro\appointment-service>django-admin startproject appointment_service .
C:\Users\TRONG KHOI\health-care-micro\appointment-service>python manage.py startapp app
```

* Billing Service

```
C:\Users\TRONG KHOI\health-care-micro>mkdir billing-service
C:\Users\TRONG KHOI\health-care-micro>cd billing-service
C:\Users\TRONG KHOI\health-care-micro\billing-service>pip install django djangorestframework requests
C:\Users\TRONG KHOI\health-care-micro\billing-service>django-admin startproject billing_service .
C:\Users\TRONG KHOI\health-care-micro\billing-service>python manage.py startapp app
```

*Medical Record Service

```
C:\Users\TRONG KHOI\health-care-micro>mkdir medical-record-service
C:\Users\TRONG KHOI\health-care-micro>cd medical-record-service
C:\Users\TRONG KHOI\health-care-micro\medical-record-service>pip install django djangorestframework requests
C:\Users\TRONG KHOI\health-care-micro\medical-record-service>django-admin startproject medical_record_service .
C:\Users\TRONG KHOI\health-care-micro\medical-record-service>python manage.py startapp app
```

* Pharmacy Service

```
C:\Users\TRONG KHOI\health-care-micro>mkdir pharmacy-service
C:\Users\TRONG KHOI\health-care-micro>cd pharmacy-service
C:\Users\TRONG KHOI\health-care-micro\pharmacy-service>pip install django djangorestframework requests
C:\Users\TRONG KHOI\health-care-micro\pharmacy-service>django-admin startproject pharmacy_service .
C:\Users\TRONG KHOI\health-care-micro\pharmacy-service>python manage.py startapp app
```

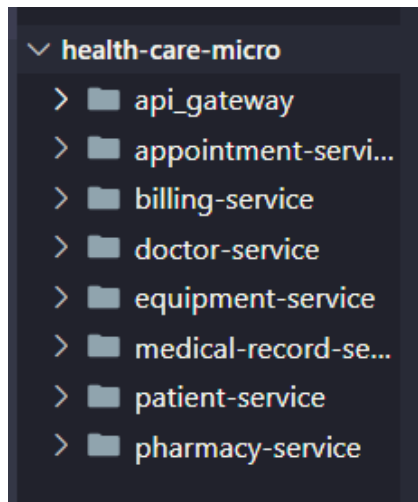
*Equipment Service

```
C:\Users\TRONG KHOI\health-care-micro>mkdir equipment-service
C:\Users\TRONG KHOI\health-care-micro>cd equipment-service
C:\Users\TRONG KHOI\health-care-micro\equipment-service>pip install django djangorestframework requests
C:\Users\TRONG KHOI\health-care-micro\equipment-service>django-admin startproject equipment_service .
C:\Users\TRONG KHOI\health-care-micro\equipment-service>python manage.py startapp app
```

*API Gateway

```
C:\Users\TRONG KHOI>cd health-care-micro
C:\Users\TRONG KHOI\health-care-micro>django-admin startproject api_gateway
```

Kết quả đạt được



Hình 1.8. Cấu trúc thư mục Healthcare Microservice

1.4.7. Ví dụ API

Dưới đây là bảng tổng hợp các giao diện lập trình ứng dụng (API) đại diện cho các Bounded Context, dựa trên các nghiệp vụ đã phân tích. Các API tuân thủ tiêu chuẩn RESTful

Microservice	Endpoint	Phương thức HTTP	Mô tả nghiệp vụ (Command/Query)	Collaborators (Các service liên kết)
Patient Service	/api/v1/patients	GET	Lấy danh sách hoặc tìm kiếm thông tin bệnh nhân.	Không có (Self-contained).
Patient Service	/api/v1/patients	POST	Đăng ký hồ sơ bệnh nhân mới.	Không có.
Doctor Service	/api/v1/doctors	GET	Lấy danh sách bác sĩ (có thể lọc theo chuyên khoa).	Không có.
Doctor Service	/api/v1/doctors/{id}/schedule	GET	Xem lịch làm việc và khung giờ trống của bác sĩ.	Không có.
Appointment Service	/api/v1/appointments	POST	Đặt lịch khám mới. Đầu vào gồm PatientID, DoctorID, TimeSlot.	Gọi đồng bộ sang Patient Service & Doctor Service để xác thực.
Appointment Service	/api/v1/appointments/{id}/cancel	PUT hoặc POST	Hủy lịch hẹn khám. Cập nhật trạng thái	Gửi event bất đồng bộ để

			thành CANCELLED.	Billing Service hoàn tiền (nếu có).
Medical Record Service	/api/v1/records	POST	Tạo hồ sơ bệnh án hoặc thêm chẩn đoán mới cho bệnh nhân.	Nhận dữ liệu PatientID và DoctorID. Gửi event cho Pharmacy Service .
Pharmacy Service	/api/v1/pharmacy/inventory	GET	Kiểm tra số lượng tồn kho của một loại thuốc.	Không có.
Billing Service	/api/v1/bills/patient/{id}	GET	Lấy tổng hợp công nợ/hóa đơn chưa thanh toán của bệnh nhân.	Áp dụng API Composition hoặc tổng hợp từ Domain Events,.
Billing Service	/api/v1/bills/{id}/pay	POST	Xử lý thanh toán cho hóa đơn (Gọi qua cổng thanh toán).	

1.5 Kết luận

Kiến trúc phần mềm không chỉ đơn thuần là việc lựa chọn công nghệ, mà là nền tảng quyết định các thuộc tính chất lượng của hệ thống như khả năng bảo trì, khả năng kiểm thử và tốc độ triển khai. Nhìn lại sự tiến hóa của kiến trúc phần mềm, kiến trúc nguyên khối (Monolithic) không phải là một thiết kế tồi, mà nó thực sự là lựa chọn phù hợp và hoàn hảo cho các hệ thống nhỏ hoặc các dự án ở giai đoạn khởi đầu. Tuy nhiên, khi hệ thống gặt hái được thành công và không ngừng mở rộng về quy mô dữ liệu, tính năng cũng như số lượng lập trình viên, kiến trúc này tất yếu sẽ bộc lộ những hạn chế chí mạng. Ứng dụng dần biến thành một mớ hỗn độn phức tạp, đẩy dự án vào "địa ngục nguyên khối" (monolithic hell), nơi tốc độ giao hàng giảm sút nghiêm trọng và việc áp dụng công nghệ mới trở nên cực kỳ rủi ro,.

Để vượt qua những giới hạn đó, kiến trúc Microservices vươn lên trở thành sự lựa chọn tối ưu cho các hệ thống lớn và phức tạp. Đặc trưng cốt lõi của Microservices là mỗi dịch vụ sở hữu một cơ sở dữ liệu riêng biệt và được quản lý bởi một nhóm phát triển nhỏ, tự chủ,. Cấu trúc này không chỉ giải quyết bài toán tắc nghẽn giao tiếp trong tổ chức lớn mà

còn mang lại sự linh hoạt tuyệt đối: mỗi dịch vụ có thể được phát triển, kiểm thử, nâng cấp công nghệ và mở rộng quy mô (scale) hoàn toàn độc lập,.. Nhờ đó, Microservices là chìa khóa để các doanh nghiệp đạt được khả năng triển khai liên tục (continuous delivery) và phản ứng nhanh nhạy trước những biến động của thị trường..

Tuy nhiên, Microservices không phải là một "viên đạn bạc" (silver bullet). Nó tạo ra một hệ thống phân tán với vô vàn những thách thức phức tạp về giao tiếp mạng, bảo đảm tính nhất quán dữ liệu và truy vấn trên nhiều cơ sở dữ liệu. Thách thức lớn nhất và cũng là nguyên nhân khiến nhiều dự án Microservices thất bại chính là bài toán phân rã hệ thống. Nếu chia cắt hệ thống sai cách, chẳng hạn như chia theo các lớp kỹ thuật (technical layers) thay vì theo nghiệp vụ, tổ chức sẽ tạo ra một "nguyên khối phân tán" (distributed monolith),. Hệ thống này không những không có được sự linh hoạt của Microservices mà còn phải gánh chịu toàn bộ độ trễ và sự phức tạp của hạ tầng phân tán. Đây chính là lúc Domain-Driven Design (DDD) chứng minh vai trò là nền tảng trọng yếu để phân rã hệ thống đúng đắn. Về bản chất, kiến trúc Microservices và DDD có sự liên kết gần như hoàn hảo

Tổng kết lại, việc chuyển đổi sang kiến trúc Microservices là một bước tiến tất yếu để đáp ứng quy mô và tốc độ của các hệ thống doanh nghiệp hiện đại. Trong quá trình đó, Microservices đóng vai trò là "thể xác" cung cấp cơ chế vận hành độc lập, nhưng chính Domain-Driven Design mới thực sự là "linh hồn", cung cấp kim chỉ nam và nền tảng lý luận vững chắc để phân rã hệ thống. Áp dụng đúng đắn DDD không chỉ giúp hệ thống tránh được cạm bẫy của "nguyên khối phân tán" mà còn tạo ra một kiến trúc phần mềm sát với thực tiễn kinh doanh, linh hoạt trước mọi thay đổi và có khả năng trường tồn cùng sự phát triển của doanh nghiệp.

CHƯƠNG 2. PHÁT TRIỂN HỆ E-COMMERCE MICROSERVICES

2.1 Xác định yêu cầu

Hệ thống E-commerce được phân tích là một nền tảng thương mại điện tử quy mô lớn, phục vụ đa dạng mặt hàng và có lưu lượng truy cập cao.

2.1.1 Functional Requirements

2.1.1.1. Mô tả

Các yêu cầu chức năng đại diện cho các hành vi nghiệp vụ cốt lõi mà hệ thống cung cấp cho người dùng. Đối với nền tảng E-commerce, các nhóm chức năng (tương lai sẽ được ánh xạ thành các subdomains) bao gồm:

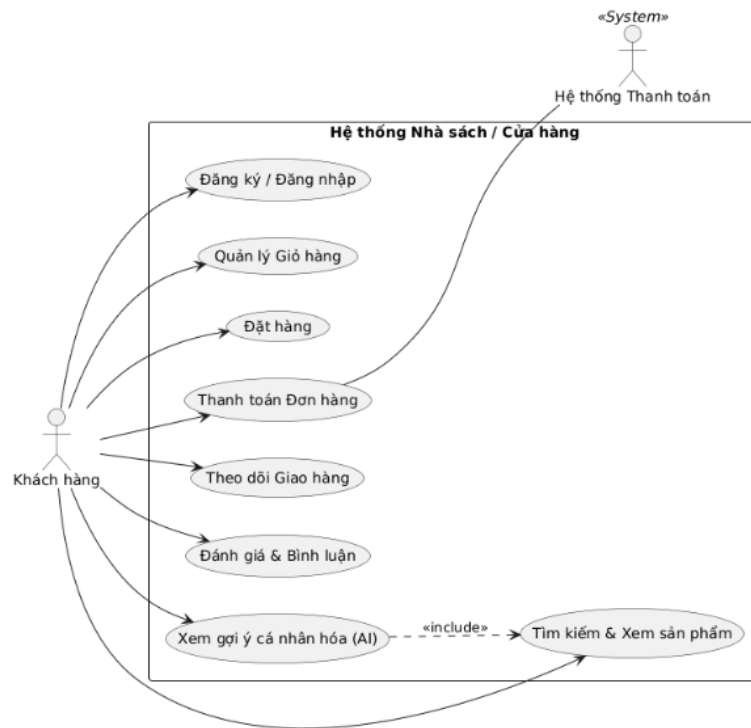
- Quản lý sản phẩm (Product Management - Đa domain): Hệ thống cần quản lý một danh mục sản phẩm khổng lồ và đa dạng thuộc nhiều lĩnh vực khác nhau như *book (sách)*, *fashion (thời trang)*, *cosmetic (mỹ phẩm)*, *electronics (điện tử)*, *food (thực phẩm)*, *furniture (nội thất)*, *medicine (thuốc)*, *pet supplies (đồ thú cưng)*, *toys (đồ chơi)*, *autopart (phụ tùng xe)*... Mỗi loại sản phẩm sẽ có những thuộc tính đặc thù riêng (ví dụ: sách có ISBN và tác giả; quần áo có size và màu sắc; thực phẩm có hạn sử dụng). Do đó, yêu cầu đặt ra là một mô hình quản lý danh mục (Catalog) cực kỳ linh hoạt và có khả năng tùy biến cao.
- Quản lý người dùng (User Management): Cung cấp các chức năng đăng ký, đăng nhập, quản lý hồ sơ cá nhân và phân quyền theo các vai trò (roles) riêng biệt bao gồm: *Admin* (Quản trị viên hệ thống), *Staff* (Nhân viên vận hành, quản lý kho), và *Customer* (Khách hàng mua sắm). Quản lý điểm tích lũy, phần hạng của người dùng
- Đánh giá sản phẩm (Review/Rating): Cho phép khách hàng để lại nhận xét, chấm điểm (từ 1 đến 5 sao) cho các sản phẩm họ đã mua, đồng thời tính toán điểm đánh giá trung bình để hiển thị cho người dùng khác.
- Giỏ hàng (Cart): Cho phép khách hàng thêm, sửa, xóa các sản phẩm mong muốn mua trước khi tiến hành thanh toán. Giỏ hàng cần lưu trữ trạng thái tạm thời và đồng bộ xuyên suốt các phiên truy cập của người dùng.
- Đặt hàng (Order): Đây là nghiệp vụ trung tâm của hệ thống. Chức năng này xử lý việc tạo lập đơn hàng (Create Order), xác thực giỏ hàng, áp dụng mã giảm giá, kiểm tra điều kiện tối thiểu của đơn, và theo dõi vòng đời trạng thái của đơn hàng (như PENDING, APPROVED, REJECTED, CANCELLED). Quản lý voucher của hệ thống, áp dụng voucher vào đơn hàng.
- Thanh toán (Payment): Xử lý các giao dịch tài chính bao gồm tính toán tổng tiền,

thuế, phí vận chuyển và tương tác với các cổng thanh toán bên ngoài (như Stripe, VNPAY) để ủy quyền (authorize) và trừ tiền thẻ tín dụng của khách hàng.

- Giao hàng (Shipping): Quản lý quy trình vận chuyển sau khi đơn hàng đã được thanh toán. Bao gồm việc điều phối, tạo kế hoạch giao hàng (Plan), phân công nhân viên giao nhận (Courier), và cập nhật vị trí, trạng thái giao hàng theo thời gian thực (ví dụ: Đang lấy hàng, Đang giao, Đã giao),.
- Tìm kiếm và gợi ý sản phẩm (Search & Recommendation): Cung cấp bộ máy tìm kiếm (full-text search) và lọc sản phẩm nâng cao, kết hợp với hệ thống gợi ý dựa trên hành vi mua sắm để tăng trải nghiệm người dùng và thúc đẩy doanh số. Đây là các thao tác truy vấn (query) phức tạp đòi hỏi hiệu năng truy xuất cực cao,.

2.1.1.2. Usecase theo nhóm người dùng

* Nhóm Use Case dành cho Khách hàng (Customer)



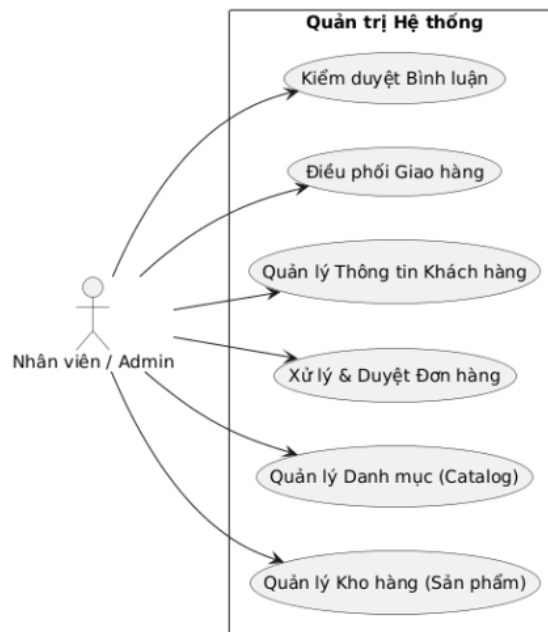
Hình 2.1. Biểu đồ Usecase dành cho khách hàng

Mô tả:

- UC1 (Đăng ký / Đăng nhập): Cho phép Khách hàng tạo tài khoản mới hoặc truy cập vào hệ thống để sử dụng các tính năng cá nhân hóa.
- UC2 (Tìm kiếm & Xem sản phẩm): Cho phép Khách hàng duyệt qua danh mục sách/sản phẩm, tìm kiếm theo từ khóa và xem thông tin chi tiết.
- UC3 (Quản lý Giỏ hàng): Cho phép Khách hàng thêm, bớt sản phẩm hoặc cập nhật số lượng hàng hóa trước khi quyết định mua.
- UC4 (Đặt hàng): Cho phép Khách hàng xác nhận danh sách sản phẩm trong giỏ hàng và tạo đơn hàng chính thức.

- UC5 (Thanh toán Đơn hàng): Cho phép Khách hàng thực hiện thanh toán thông qua các cổng thanh toán tích hợp (Payment Gateway).
- UC6 (Theo dõi Giao hàng): Cho phép Khách hàng kiểm tra lộ trình và trạng thái vận chuyển của đơn hàng (Đang xử lý, Đang giao, Đã nhận).
- UC7 (Đánh giá & Bình luận): Cho phép Khách hàng gửi phản hồi, nhận xét và chấm điểm cho sản phẩm đã mua.
- UC8 (Xem gợi ý cá nhân hóa): Cho phép Khách hàng nhận các đề xuất sản phẩm phù hợp với sở thích dựa trên công nghệ AI.

* Nhóm Use Case dành cho Nhân viên & Quản trị (Staff/Admin)

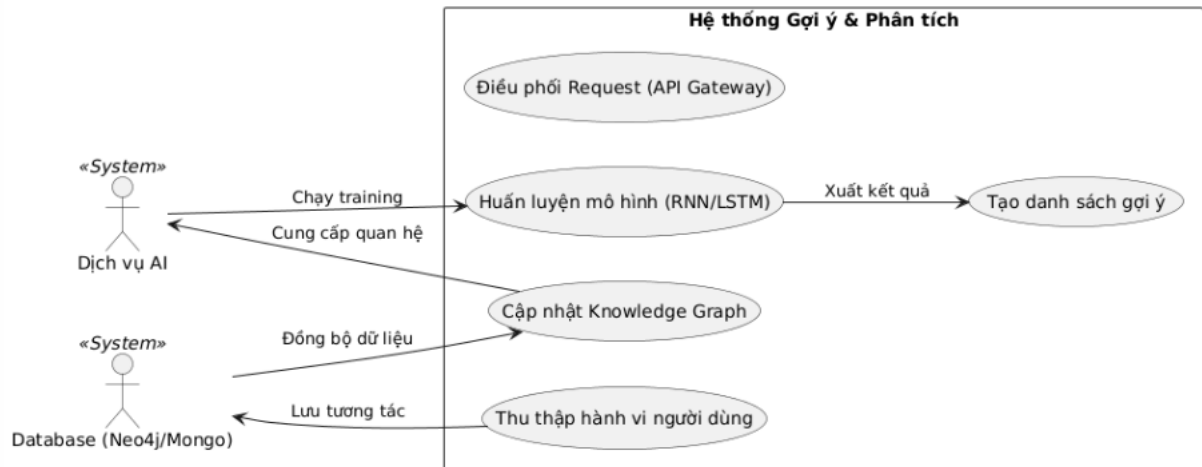


Hình 2.2. Biểu đồ Use Case dành cho nhân viên

Mô tả:

- Admin1 (Quản lý Kho hàng): Cho phép Nhân viên thêm mới, cập nhật thông tin, giá cả và số lượng tồn kho của sản phẩm.
- Admin2 (Quản lý Danh mục): Cho phép Nhân viên tổ chức, phân loại sản phẩm vào các nhóm hoặc thể loại để người dùng dễ tìm kiếm.
- Admin3 (Xử lý & Duyệt Đơn hàng): Cho phép Nhân viên tiếp nhận đơn hàng mới, kiểm tra tính hợp lệ và xác nhận chuẩn bị hàng.
- Admin4 (Quản lý Thông tin Khách hàng): Cho phép Admin xem danh sách người dùng, quản lý cấp độ thành viên (Level) và hỗ trợ khách hàng.
- Admin5 (Điều phối Giao hàng): Cho phép Nhân viên liên kết với đơn vị vận chuyển và cập nhật mã vận đơn cho hệ thống.
- Admin6 (Kiểm duyệt Bình luận): Cho phép Nhân viên quản lý các nội dung phản hồi từ khách hàng, ẩn các bình luận vi phạm quy tắc.

** Nhóm Use Case Hệ thống & AI (Back-end Processes)*



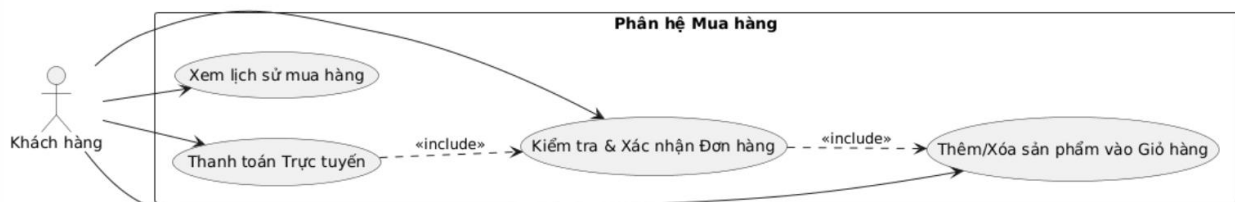
Hình 2.3. Biểu đồ Use Case hệ thống AI

Mô tả:

- Sys1 (Thu thập hành vi người dùng): Cho phép Hệ thống tự động ghi lại các hành động như xem, click, mua hàng để làm dữ liệu đầu vào.
- Sys2 (Cập nhật Knowledge Graph): Cho phép Hệ thống tự động xây dựng và cập nhật các mối liên kết phức tạp giữa người dùng và sản phẩm trên Neo4j.
- Sys3 (Huấn luyện mô hình): Cho phép Dịch vụ AI thực hiện chạy các thuật toán học sâu (RNN/LSTM) để học hỏi từ dữ liệu lịch sử.
- Sys4 (Tạo danh sách gợi ý): Cho phép Hệ thống tính toán và xuất ra danh sách các sản phẩm có khả năng người dùng sẽ quan tâm nhất.
- Sys5 (Điều phối Request): Cho phép API Gateway nhận diện, định tuyến và phân phối các yêu cầu từ phía client đến đúng microservice cần thiết.

2.1.1.3. Usecase chức năng chính

** Chức năng mua hàng*



Hình 2.4. Biểu đồ UseCase chức năng mua hàng

Mô tả:

- UC_Cart cho phép Khách hàng thực hiện thêm các sản phẩm yêu thích (sách, điện tử, dược phẩm...) vào giỏ hàng cá nhân hoặc xóa bỏ/cập nhật số lượng trước khi tiến hành mua.
- UC_Checkout cho phép Khách hàng thực hiện kiểm tra lại toàn bộ danh sách sản phẩm, thông tin giao hàng và tổng tiền để xác nhận đơn hàng chính thức.

- UC_Pay cho phép Khách hàng thực hiện thanh toán hóa đơn thông qua các phương thức trực tuyến được tích hợp, đảm bảo giao dịch an toàn và nhanh chóng.
- UC_History cho phép Khách hàng thực hiện xem lại danh sách các đơn hàng đã đặt trong quá khứ để theo dõi chi tiêu hoặc thực hiện mua lại.

** Chức năng xem sản phẩm gợi ý*

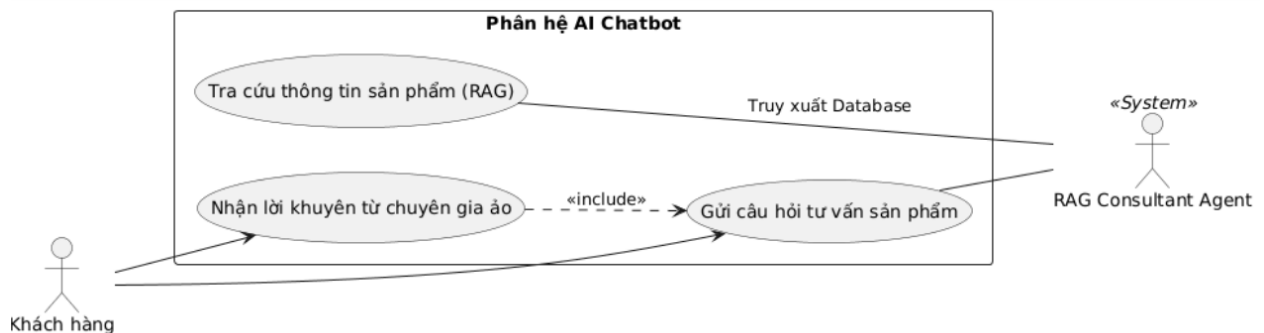


Hình 2.5. Biểu đồ Use Case chức năng xem gợi ý sản phẩm

Mô tả:

- UC_ViewRec cho phép Khách hàng thực hiện xem danh sách các sản phẩm được hệ thống AI lựa chọn riêng cho mình, giúp khám phá những sản phẩm phù hợp với sở thích cá nhân.
- UC_Log cho phép Hệ thống thực hiện ghi nhận một cách tự động các hành vi tương tác của khách hàng (như xem chi tiết sản phẩm, nhấn "thích") để làm dữ liệu đầu vào cho việc cải thiện độ chính xác của gợi ý.
- UC_Filter cho phép Khách hàng thực hiện lọc và sắp xếp các sản phẩm gợi ý theo các tiêu chí như thể loại, giá cả hoặc mức độ liên quan để tìm được sản phẩm ưng ý nhất.

** Chức năng chatbot*



Hình 2.6. Biểu đồ Usecase chatbot

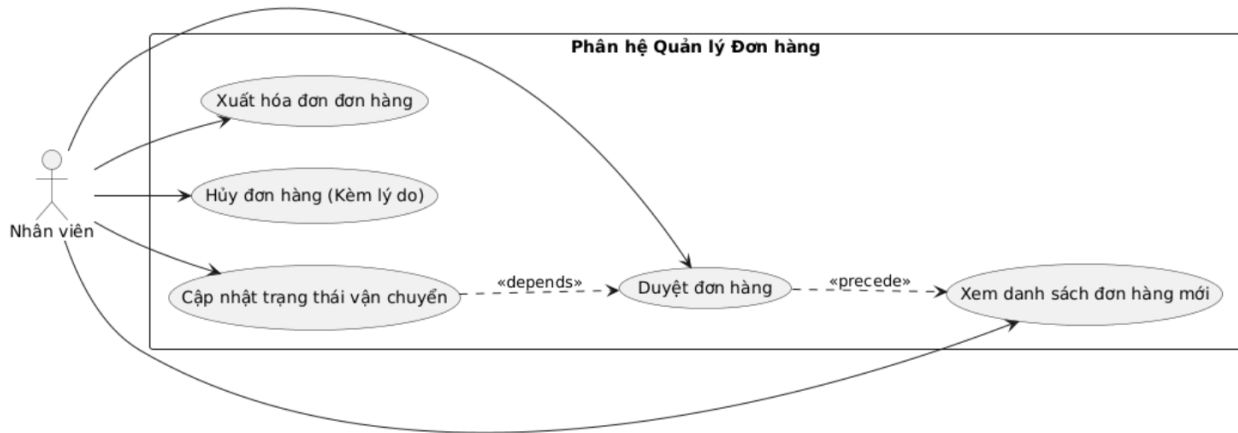
Mô tả:

- UC_Chat cho phép Khách hàng thực hiện gửi các câu hỏi hoặc yêu cầu tư vấn bằng

ngôn ngữ tự nhiên về bất kỳ sản phẩm nào có trong kho hàng của hệ thống.

- UC_SearchInfo cho phép Chatbot (RAG Agent) thực hiện tra cứu và truy xuất thông tin chi tiết từ cơ sở dữ liệu sản phẩm để đưa ra câu trả lời chính xác và trung thực nhất.
- UC_Advice cho phép Khách hàng thực hiện nhận những lời khuyên hữu ích, so sánh các sản phẩm hoặc gợi ý chọn mua từ chuyên gia ảo dựa trên nhu cầu cụ thể mà khách hàng đã chia sẻ trong cuộc trò chuyện.

* *Nhân viên quản lý đơn hàng*



Hình 2.7. Biểu đồ Usecase nhân viên quản lý đơn hàng

Mô tả:

- UC_ViewOrders cho phép Nhân viên thực hiện truy cập và theo dõi danh sách toàn bộ các đơn hàng đang chờ xử lý trên hệ thống theo thời gian thực.
- UC_Approve cho phép Nhân viên thực hiện kiểm tra tình trạng kho hàng và xác nhận duyệt đơn hàng để bắt đầu quá trình đóng gói.
- UC_Cancel cho phép Nhân viên thực hiện hủy các đơn hàng không hợp lệ, khách hàng yêu cầu hủy hoặc do sự cố kho hàng (kèm theo việc nhập lý do cụ thể).
- UC_UpdateShip cho phép Nhân viên thực hiện thay đổi các trạng thái vận chuyển của đơn hàng (ví dụ: Đang đóng gói, Đang giao, Đã giao thành công) để khách hàng có thể theo dõi.
- UC_Invoice cho phép Nhân viên thực hiện tạo và in hóa đơn chi tiết cho từng đơn hàng để phục vụ việc kiểm soát tài chính và gửi kèm hàng hóa.

2.1.2 Non-functional Requirements

Kiến trúc phần mềm ảnh hưởng trực tiếp đến các yêu cầu phi chức năng (còn gọi là các thuộc tính chất lượng hoặc *-ilities*). Sự phát triển bùng nổ của nền tảng E-commerce đòi hỏi hệ thống phải đáp ứng các tiêu chuẩn khắt khe sau:

- Scalability (Khả năng mở rộng - Scale từng service độc lập): Hệ thống nguyên khối

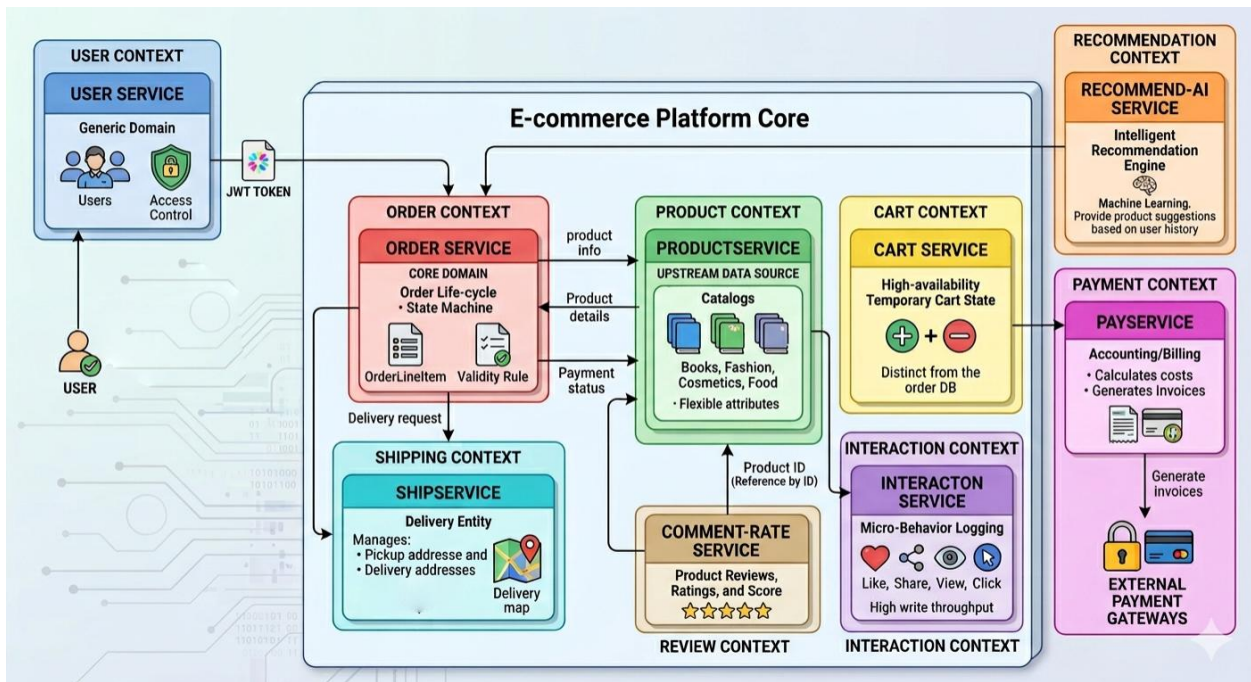
(Monolith) gặp khó khăn lớn khi mở rộng do các module có yêu cầu tài nguyên khác nhau (ví dụ: module xử lý ảnh cần nhiều CPU, trong khi module dữ liệu cần nhiều RAM) bị buộc phải chạy chung trên một máy chủ. Yêu cầu đặt ra là phải sử dụng phân rã theo trục Y (Y-axis scaling - chia nhỏ thành dịch vụ) và trục X (X-axis scaling - nhân bản) để mở rộng độc lập,. Ví dụ: Vào dịp siêu sale (Black Friday), Order Service và Search Service có thể tự động tăng số lượng instance (scale up) lên gấp 10 lần để chịu tải, trong khi Review Service vẫn giữ nguyên cấu hình để tiết kiệm chi phí.

- High Availability (Tính sẵn sàng cao - Hệ thống luôn sẵn sàng): Khách hàng E-commerce yêu cầu nền tảng phải hoạt động 24/7. Nếu sử dụng kiến trúc giao tiếp đồng bộ (REST API gọi chuỗi), sự cố sập một dịch vụ nhỏ (ví dụ: Payment Service) có thể làm sập toàn bộ quy trình đặt hàng, làm giảm nghiêm trọng tính sẵn sàng của hệ thống,. Do đó, hệ thống yêu cầu thiết kế giao tiếp bất đồng bộ (Asynchronous Messaging), cho phép ứng dụng vẫn nhận và tạo đơn hàng (đưa vào trạng thái PENDING) ngay cả khi dịch vụ thanh toán hoặc kho hàng tạm thời không phản hồi,.
- Security (Bảo mật - JWT, Authentication): Để giải quyết bài toán xác thực trong môi trường phân tán (nơi không thể sử dụng session lưu trong bộ nhớ server như ứng dụng nguyên khối), hệ thống yêu cầu sử dụng JSON Web Token (JWT). Kiến trúc cần triển khai mẫu API Gateway làm "cửa trước". API Gateway sẽ chịu trách nhiệm xác thực thông tin đăng nhập của người dùng thông qua OAuth 2.0 hoặc Identity Provider, sau đó đính kèm JWT (chứa UserID và Role) vào header của mỗi request trước khi định tuyến chúng đến các microservices nội bộ,. Các microservices sẽ bóc tách JWT để thực hiện phân quyền (Authorization) theo Data/Role một cách an toàn.
- Maintainability (Khả năng bảo trì - Dễ bảo trì): Ứng dụng phải tránh tình trạng "bóng bùn" (big ball of mud) nơi mã nguồn bị đan xen chằng chịt, khiến mọi thay đổi nhỏ đều có nguy cơ gây lỗi toàn hệ thống,. Yêu cầu này được đáp ứng trực tiếp bằng việc áp dụng Domain-Driven Design (DDD) nhằm tạo ra các ranh giới Bounded Context rõ ràng. Mỗi Microservice sẽ sở hữu một cơ sở dữ liệu riêng tư (Database per Service) và được phát triển, kiểm thử, nâng cấp bởi một đội ngũ nhỏ (autonomous two-pizza team) độc lập,. Điều này đảm bảo vòng đời phát triển phần mềm diễn ra nhanh chóng, mã nguồn cô đọng, dễ hiểu và dễ dàng bảo trì trong tương lai.

2.2 Phân rã hệ thống theo DDD

2.2.1. Bounded Context

Việc phân tích các nghiệp vụ không lồ của nền tảng E-commerce dẫn đến việc xác định các Bounded Context cốt lõi được thể hiện dưới hình dưới đây.



Hình 2.8. Bounded Context xác định cho hệ Ecommerce

Từng ngữ cảnh được chuyển đổi cụ thể thành các service tương ứng:

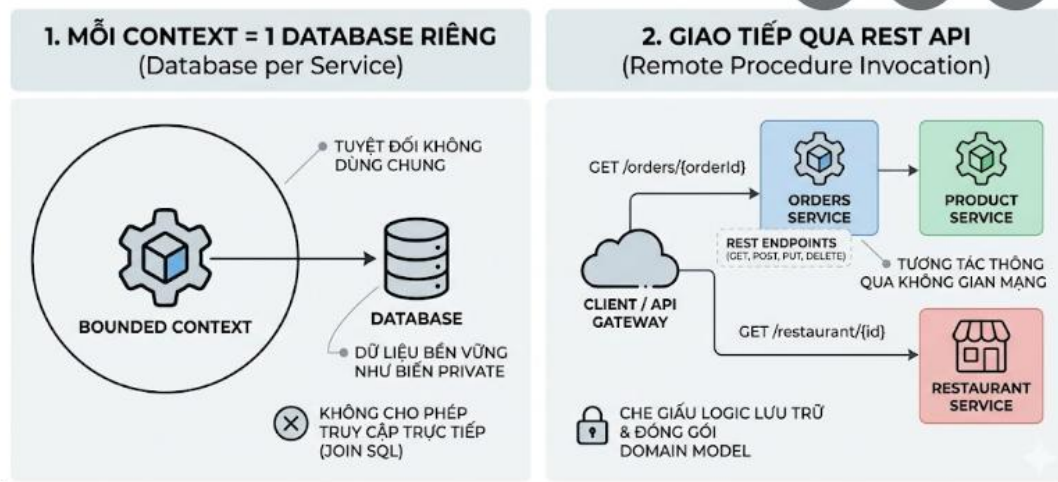
- **User Context** → user service: Ngữ cảnh này đóng vai trò là một Generic Domain, chịu trách nhiệm quản lý vòng đời người dùng, thông tin cá nhân và xử lý xác thực/phân quyền (Admin, Staff, Customer). Bằng cách cô lập logic tài khoản tại đây, các service khác chỉ cần xác thực qua Token (như JWT) thay vì ôm đồm logic bảo mật. Lưu thông tin điểm khách hàng của người dùng, phục vụ cho việc nhận ưu đãi, mua voucher.
- **Product Context** → productservice: Ngữ cảnh này chịu trách nhiệm quản lý danh mục sản phẩm khổng lồ đa lĩnh vực (sách, thời trang, mỹ phẩm, thực phẩm...). Nó quản lý thực thể Product với các thuộc tính linh hoạt nhằm mục đích hiển thị cho khách hàng. productservice đóng vai trò là nguồn dữ liệu gốc (Upstream) cung cấp thông tin sản phẩm cho các context khác.
- **Order Context** → orderservice: Đây là miền cốt lõi (Core Domain) của hệ thống. Ngữ cảnh này tập trung hoàn toàn vào việc quản lý vòng đời và trạng thái của đơn hàng (Order) cùng chi tiết đặt hàng (OrderLineItem). Thay vì trở thành một lớp khổng lồ (God class) chứa mọi thứ, orderservice chỉ xử lý các quy tắc nghiệp vụ cốt lõi như tính hợp lệ của đơn hàng và điều phối các trạng thái.
- **Payment Context** → payservice: Ngữ cảnh kế toán/tài chính (Accounting/Billing) này tách biệt hoàn toàn khỏi đơn hàng. Nó chỉ tập trung vào việc tính toán chi phí, xử lý hóa đơn và tương tác với các cổng thanh toán bên ngoài (Payment Gateways). Khái niệm khách hàng ở đây chỉ đơn thuần là "người thanh toán" (Payer).
- **Shipping Context** → shippervice: Ngữ cảnh quản lý vận tải và giao nhận. shippervice

không quan tâm đến chi tiết mặt hàng kinh doanh mà chỉ làm việc với thực thể Delivery (Giao hàng) gồm: địa chỉ lấy hàng, địa chỉ giao hàng, và trạng thái lộ trình.

- **Recommendation Context** → recommend-ai service: Ngữ cảnh này cung cấp giá trị gia tăng thông qua bộ máy gợi ý thông minh. Việc cô lập logic AI/Machine Learning vào recommend-ai service cho phép hệ thống phân tích lịch sử người dùng và tối ưu hóa kết quả tìm kiếm độc lập mà không làm chậm các luồng xử lý giao dịch lõi của hệ thống. Thực hiện chatbot tư vấn cho user.
- **Cart Context** → cart service: Quản lý trạng thái giỏ hàng tạm thời. Các thao tác thêm, sửa, xóa sản phẩm trong giỏ diễn ra liên tục với cường độ cao. Việc thiết lập một cart service riêng biệt giúp đảm bảo hiệu năng và tính sẵn sàng cao (High Availability) mà không làm ảnh hưởng đến cơ sở dữ liệu chính của hệ thống đặt hàng.
- **Interaction Context** → interacton service: Ngữ cảnh tương tác tập trung vào việc lưu trữ các hành vi vi mô của người dùng (như Like, Share, View, Click). Vì lưu lượng ghi (write) cho các hành vi này là khổng lồ, interacton service tách biệt hoàn toàn để cô lập rủi ro tắc nghẽn tài nguyên của hệ thống chính.
- **Review Context** → comment rate service: Ngữ cảnh quản lý các đánh giá và bình luận. Nó lưu trữ nhận xét, chấm điểm sao (rating) của khách hàng về sản phẩm và tổng hợp điểm số. Context này tham chiếu đến productservice thông qua ID sản phẩm (Reference by Identity) thay vì chứa dữ liệu gốc.

2.2.2. Nguyên tắc

Sau khi định hình microservices dựa trên Bounded Context, hệ thống cần phải tuân thủ nghiêm ngặt hai nguyên tắc kỹ thuật cốt lõi để đảm bảo sự gắn kết lỏng (loose coupling):



Hình 2.9. Nguyên tắc cốt lõi thiết kế Microservcie

Mỗi Context = 1 Database riêng (Database per service) Nguyên tắc quan trọng nhất của Microservices theo DDD là các dịch vụ tuyệt đối không được dùng chung một cơ sở dữ liệu. Dữ liệu bền vững của một dịch vụ phải được coi như các biến private của một lớp đối

tượng và không cho phép bên ngoài truy cập trực tiếp bằng các lệnh JOIN SQL. Ví dụ: orderservice có cơ sở dữ liệu riêng chứa bảng ORDERS, trong khi productservice có CSDL chứa danh mục sản phẩm. Điều này đảm bảo khi interacton service bị sập do quá tải, nó không làm treo cơ sở dữ liệu của orderservice, đồng thời cho phép từng team tự do nâng cấp schema độc lập.

Giao tiếp qua REST API Vì cơ sở dữ liệu đã bị chia cắt, các Context bắt buộc phải tương tác thông qua không gian mạng. Lựa chọn tiêu chuẩn ở đây là sử dụng cơ chế gọi hàm từ xa đồng bộ thông qua REST API (Remote Procedure Invocation).

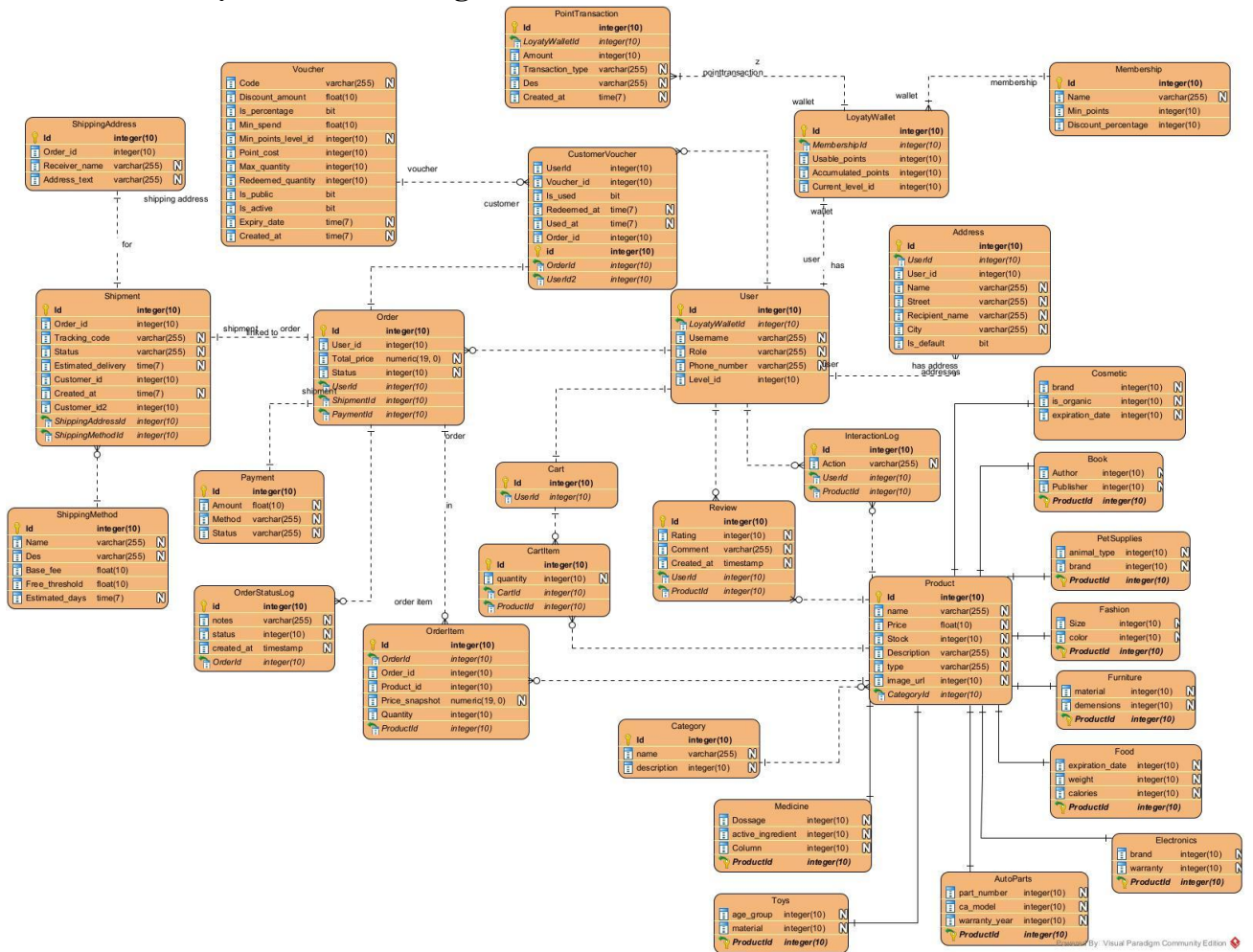
- Các dịch vụ cung cấp dữ liệu (Provider) sẽ phơi bày các Endpoint theo tiêu chuẩn RESTful (sử dụng các động từ GET, POST, PUT, DELETE) như một ngôn ngữ giao tiếp chuẩn mực.
- Ví dụ: Khi API Gateway hoặc một client cần hiển thị chi tiết đơn hàng, nó không được truy vấn trực tiếp vào Database của các service, mà sẽ gọi API GET /orders/{orderId} tới orderservice, đồng thời gọi GET /restaurant/{id} để lấy thông tin.
- Thông qua REST API, logic lưu trữ nội bộ của từng Bounded Context được che giấu hoàn toàn, ngăn chặn việc rò rỉ logic nghiệp vụ và bảo vệ mô hình Domain Model khỏi sự ô nhiễm từ các tác nhân bên ngoài.

2.2.3. Thiết kế class diagram tổng quát

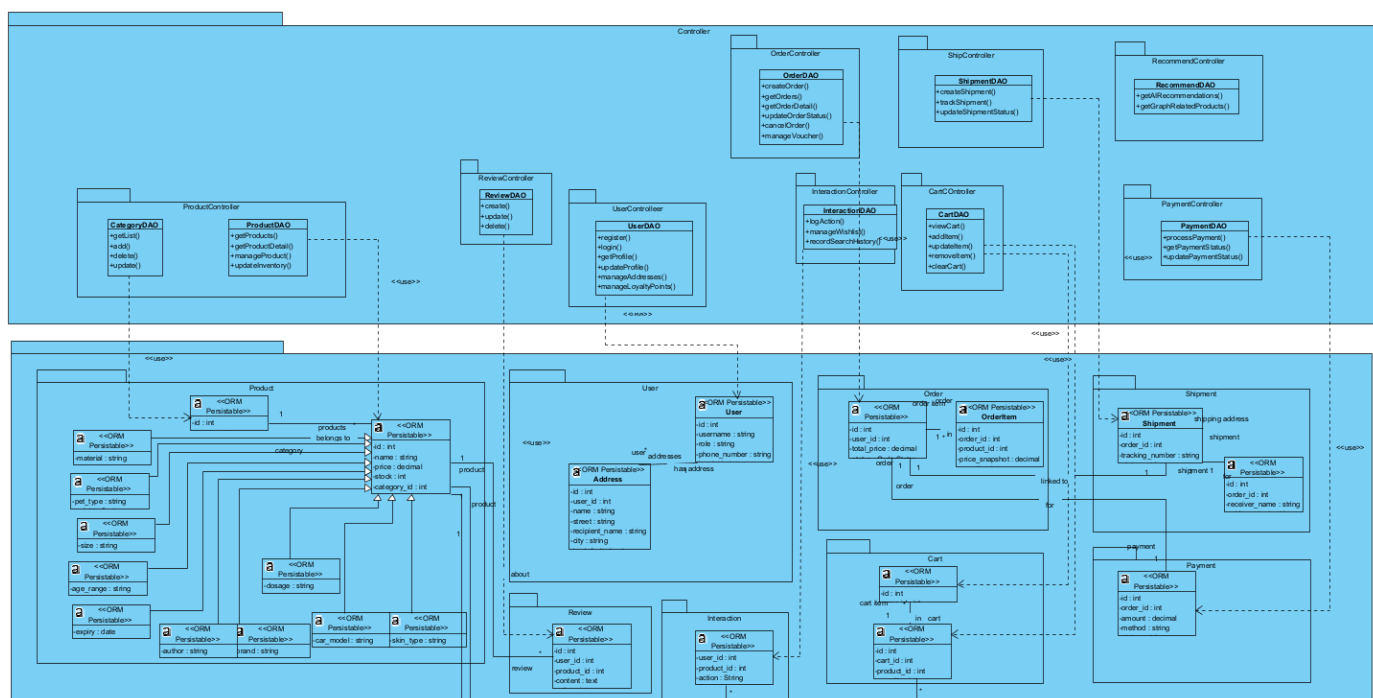
Dịch vụ	Tên Lớp (Class)	Vai trò & Ý nghĩa	Thuộc tính chính
		của người dùng	
Cart Service	Cart	Quản lý giỏ hàng tạm thời của khách hàng trước khi đặt hàng.	id, user_id, calculate_total()
	CartItem	Chi tiết từng sản phẩm và số lượng trong giỏ hàng.	id, product_id, quantity
Product Service	Product (Abstract)	Lớp trừu tượng định nghĩa các thông tin cơ bản của mọi sản phẩm.	id, name, price, stock, category_id
	Category	Phân loại sản phẩm theo danh mục.	id, name
	<i>Các lớp con</i>	Kế thừa từ Product, lưu trữ dữ liệu đặc thù (ví dụ: Book, Medicine, Electronics...).	Tác giả (Book), Liều dùng (Medicine)...
Order Service	Order	Quản lý đơn hàng và trạng thái vòng đời của đơn hàng.	id, total_price, status, checkout()
	OrderItem	Lưu trữ thông tin sản phẩm tại thời điểm mua (Snapshot).	product_id, price_snapshot, quantity
	OrderStatusLog	Lưu trữ lịch sử thay đổi trạng thái của order	Status, time
	Voucher	Lưu thông tin các voucher của hệ thống	Discount_amount: is_percentage, min_spend, point_cost
	CustomerVoucher	Lưu thông tin các voucher của người dùng	Is_used, customer_id, voucher_id
Comment Rate Service	Review	Lưu trữ đánh giá và bình luận của người dùng về sản phẩm.	id, rating, content
Pay ServiceS	Payment	Quản lý thông tin giao dịch và thanh toán của đơn hàng.	id, amount, method, status
Ship Service	Shipment	Quản lý thông tin vận chuyển và mã vận đơn.	id, tracking_number, status

Dịch vụ	Tên Lớp (Class)	Vai trò & Ý nghĩa	Thuộc tính chính
	ShippingAddress	Địa chỉ cụ thể dùng cho việc giao một đơn hàng nhất định.	receiver_name, address_text
	ShippingMethod	Lưu thông tin phương thức giao hàng	Id, name, base_fee, free_threshold
Interaction Service	InteractionLog	Thông tin action của user-product	User_id, product_id, action, timestamp

* Datamodel dựa trên Class diagram



2.2.4. Biểu đồ thiết kế lớp



Hình 2.11. Biểu đồ thiết kế lớp tổng quát cho hệ Ecommerce

Dưới đây là phần mô tả chi tiết biểu đồ lớp thiết kế của dự án

ST T	Package (Service)	Controller	DAO	Các hàm chính trong DAO
1	User	UserController	UserDAO	- register(userData): Đăng ký & tạo ví Loyalty. - login(credentials): Xác thực JWT. - getProfile() / updateProfile(): Quản lý cá nhân. - manageAddresses(): Thêm/Sửa/Xóa/Đặt mặc định. - manageLoyaltyPoints(): Tích điểm & thăng hạng.
2	Product	ProductController	ProductDAO	- getProducts(filters): Lọc & phân trang. - getProductDetail(productId): Lấy chi tiết & JSONB attributes. - manageProduct(): CRUD sản phẩm (Staff). - updateInventory(): Cập nhật tồn kho

ST T	Package (Service)	Controller	DAO	Các hàm chính trong DAO
				thực tế.
3	Cart	CartController	CartDAO	- viewCart(customerId): Xem giỏ hàng. - addItem(): Thêm/Cộng dồn sản phẩm. - updateItem(): Thay đổi số lượng. - removeItem() / clearCart(): Xóa item/giỏ hàng.
4	Order	OrderController	OrderDAO	- createOrder(payload): Checkout, tính Voucher, phí ship. - getOrders() / getOrderDetail(): Lịch sử đơn hàng. - updateOrderStatus(): Cập nhật & đồng bộ Ship-Service. - cancelOrder(): Hủy đơn & hoàn tồn kho/điểm. - manageVoucher(): Đổi điểm & áp dụng Voucher.
5	Payment	PaymentController	PaymentDAO	- processPayment(): Xử lý giao dịch (Mock Gateway). - getPaymentStatus() / updatePaymentStatus(): Quản lý trạng thái thanh toán.
6	Shipment	ShipmentController	ShipmentDAO	- createShipment(): Khởi tạo vận đơn & tính ETA. - trackShipment(shipmentId): Theo dõi hành trình. - updateShipmentStatus(): Cập nhật trạng thái giao hàng.
7	Interaction	InteractionController	InteractionDAO	- logAction(): Ghi hành vi (View/Buy) cho AI. - manageWishlist(): Quản lý SP yêu thích. - recordSearchHistory(): Lưu lịch sử tìm kiếm.
8	Recommendat	RecommendContr	RecommendD	- getAIRecommendations(): Lấy gợi ý

ST T	Package (Service)	Controller	DAO	Các hàm chính trong DAO
	ion	oller	AO	từ LSTM/BiLSTM. - getGraphRelatedProducts(): Tìm SP liên quan qua Neo4j.

2.3 Thiết kế Product Service (Django)

2.3.1 Phân loại sản phẩm

Hệ thống hỗ trợ quản lý và phân loại sản phẩm theo nhiều danh mục khác nhau nhằm giúp việc tìm kiếm, lọc và quản lý dữ liệu trở nên dễ dàng và hiệu quả hơn. Mỗi danh mục đại diện cho một nhóm sản phẩm có đặc điểm và mục đích sử dụng tương tự nhau. Cụ thể như sau:

- **Book:** Bao gồm các loại sách như giáo trình, tiểu thuyết, truyện tranh, sách kỹ năng,... phục vụ nhu cầu học tập và giải trí.
- **Fashion:** Bao gồm các sản phẩm thời trang như áo, quần, giày, túi xách và các phụ kiện đi kèm, đáp ứng nhu cầu mặc và làm đẹp.
- **Cosmetic:** Bao gồm các sản phẩm mỹ phẩm như son, kem dưỡng da, sữa rửa mặt, nước hoa,... phục vụ chăm sóc cá nhân và làm đẹp.
- **Electronics:** Bao gồm các thiết bị điện tử như điện thoại, laptop, tivi, tủ lạnh, điều hòa,... phục vụ nhu cầu sinh hoạt và công việc.
- **Food:** Bao gồm thực phẩm tươi sống, đồ ăn đóng gói, đồ uống và đồ ăn nhanh,... đáp ứng nhu cầu ăn uống hàng ngày.
- **Furniture:** Bao gồm các sản phẩm nội thất như bàn, ghế, giường, tủ, kệ,... phục vụ trang trí và sử dụng trong không gian sống.
- **Medicine:** Bao gồm thuốc không kê đơn, thực phẩm chức năng, vitamin,... hỗ trợ chăm sóc và bảo vệ sức khỏe.
- **Pet Supplies:** Bao gồm thức ăn, đồ chơi và các phụ kiện chăm sóc thú cưng,... phục vụ nhu cầu nuôi và chăm sóc vật nuôi.
- **Toys:** Bao gồm đồ chơi trẻ em, lego, búp bê, mô hình,... phục vụ nhu cầu giải trí và phát triển tư duy cho trẻ.
- **Autopart:** Bao gồm các phụ tùng xe như lốp xe, ắc quy, dầu nhớt và các linh kiện thay thế,... phục vụ bảo trì và sửa chữa phương tiện.

2.3.2. Model tổng quát

```
class CategoryEntity:
    name: str
    description: str = ""
    id: Optional[int] = None
```

```

class ProductEntity:
    name: str
    description: str
    category_id: int
    price: float
    image_url: str
    attributes: Dict[str, Any] = field(default_factory=dict)
    id: Optional[int] = None
    stock: int = 0
    product_type: str = "General"
    domain_data: Dict[str, Any] = field(default_factory=dict)

```

Model tổng quát của Product bao gồm các thuộc tính:

- name: Tên sản phẩm (tối đa 255 ký tự).
- description: Mô tả chi tiết, cho phép để trống.
- price: Giá tiền (sử dụng DecimalField giúp tránh sai số làm tròn, tối đa 10 chữ số).
- stock: Số lượng tồn kho (mặc định là 0).
- image_url: Đường dẫn link ảnh sản phẩm.
- category: Liên kết với bảng Danh mục. Nếu xóa danh mục, các sản phẩm thuộc danh mục đó sẽ bị xóa theo (CASCADE).
- attributes: Lưu trữ các thuộc tính linh hoạt (màu sắc, kích thước...) dưới dạng JSON, giúp bạn không cần thay đổi cấu trúc bảng khi thêm thông số mới.

2.3.3. Chi tiết theo Domain

```

class AutoPartsModel(models.Model):
    product = models.OneToOneField(ProductModel, on_delete=models.CASCADE, related_name='auto_parts_details')
    part_number = models.CharField(max_length=100)
    car_model_compatibility = models.TextField(help_text="Comma separated list of compatible car models")
    warranty_years = models.IntegerField(default=1)

```

```

class BookModel(models.Model):
    product = models.OneToOneField(ProductModel, on_delete=models.CASCADE, related_name='book_details')
    author = models.CharField(max_length=255)
    publisher = models.CharField(max_length=255)
    isbn = models.CharField(max_length=20)

```

```

class CosmeticsModel(models.Model):
    product = models.OneToOneField(ProductModel, on_delete=models.CASCADE, related_name='cosmetics_details')
    brand = models.CharField(max_length=100)
    skin_type = models.CharField(max_length=50, help_text="e.g., Dry, Oily, Sensitive, All")
    is_organic = models.BooleanField(default=False)
    expiration_date = models.DateField(null=True, blank=True)

```

```

class ElectronicsModel(models.Model):
    product = models.OneToOneField(ProductModel, on_delete=models.CASCADE, related_name='electronics_details')
    brand = models.CharField(max_length=100)
    warranty = models.IntegerField(help_text="Warranty in months")

```

```

class FashionModel(models.Model):
    product = models.OneToOneField(ProductModel, on_delete=models.CASCADE, related_name='fashion_details')
    size = models.CharField(max_length=10)
    color = models.CharField(max_length=50)

class FoodModel(models.Model):
    product = models.OneToOneField(ProductModel, on_delete=models.CASCADE, related_name='food_details')
    expiration_date = models.DateField()
    weight = models.CharField(max_length=50, help_text="e.g., 500g, 1kg")
    is_vegetarian = models.BooleanField(default=False)
    calories = models.IntegerField(null=True, blank=True)

class FurnitureModel(models.Model):
    product = models.OneToOneField(ProductModel, on_delete=models.CASCADE, related_name='furniture_details')
    material = models.CharField(max_length=100, help_text="e.g., Oak, Metal, Plastic")
    dimensions = models.CharField(max_length=100, help_text="e.g., 200x150x50 cm")
    weight_capacity = models.FloatField(null=True, blank=True, help_text="in kg")

class MedicineModel(models.Model):
    product = models.OneToOneField(ProductModel, on_delete=models.CASCADE, related_name='medicine_details')
    active_ingredient = models.CharField(max_length=255)
    dosage = models.CharField(max_length=100, help_text="e.g., 500mg, 10ml")
    prescription_required = models.BooleanField(default=False)

class PetSuppliesModel(models.Model):
    product = models.OneToOneField(ProductModel, on_delete=models.CASCADE, related_name='pet_supplies_details')
    animal_type = models.CharField(max_length=50, help_text="e.g., Dog, Cat, Bird, Fish")
    brand = models.CharField(max_length=100)
    weight_limit = models.FloatField(null=True, blank=True, help_text="in kg")

class ToysModel(models.Model):
    product = models.OneToOneField(ProductModel, on_delete=models.CASCADE, related_name='toys_details')
    age_group = models.CharField(max_length=50, help_text="e.g., 0-3, 3+, 12+")
    material = models.CharField(max_length=100)
    requires_batteries = models.BooleanField(default=False)

```

Tất cả đều sử dụng mối quan hệ OneToOneField với ProductModel, nghĩa là mỗi sản phẩm sẽ có đúng một bản ghi chi tiết tương ứng tùy theo loại mặt hàng.

Cấu trúc chung

- OneToOneField: Kết nối trực tiếp 1-1 với ProductModel. Khi xóa sản phẩm chính, các thông tin chi tiết này cũng bị xóa (CASCADE).
- related_name: Cho phép truy cập ngược từ sản phẩm chính (ví dụ: product.book_details).
- db_table: Tùy chỉnh tên bảng trong cơ sở dữ liệu (ví dụ: catalog_book).

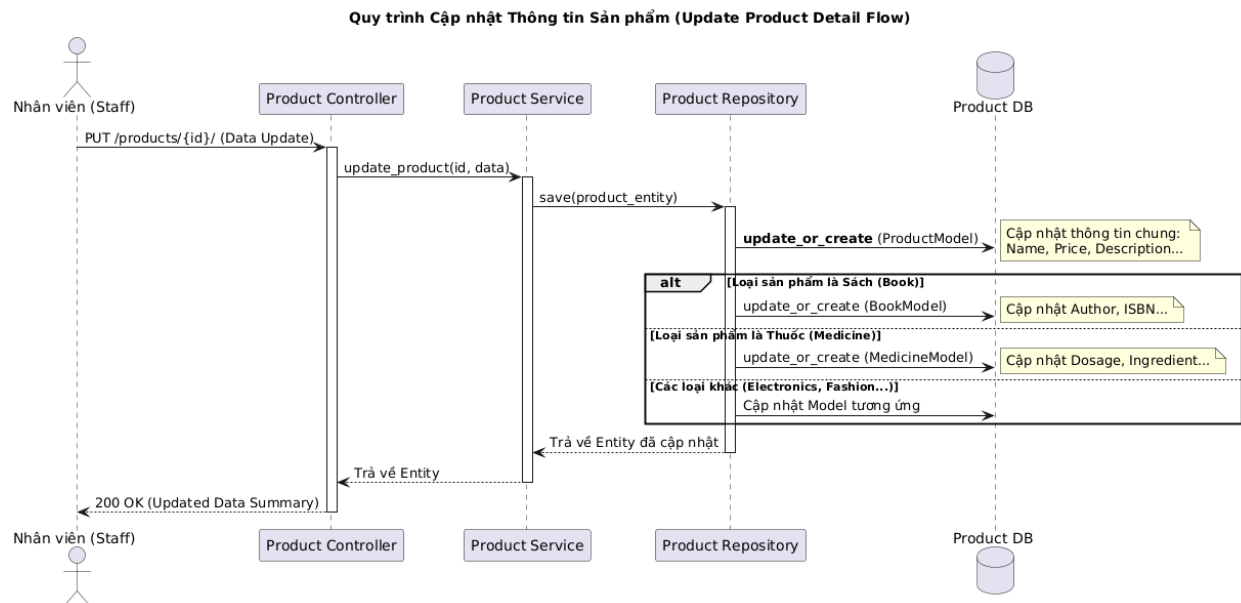
Chi tiết từng loại sản phẩm

- AutoParts (Phụ tùng ô tô): Lưu mã phụ tùng (part_number), danh sách các dòng xe tương thích và thời hạn bảo hành theo năm.
- Book (Sách): Lưu thông tin tác giả, nhà xuất bản và mã số tiêu chuẩn quốc tế (isbn).

- Cosmetics (Mỹ phẩm): Quản lý thương hiệu, loại da phù hợp, thuộc tính hữu cơ (is_organic) và ngày hết hạn.
- Electronics (Điện tử): Lưu thương hiệu và thời hạn bảo hành tính theo tháng.
- Fashion (Thời trang): Tập trung vào kích cỡ (size) và màu sắc (color).
- Food (Thực phẩm): Lưu ngày hết hạn, trọng lượng, chế độ ăn chay và hàm lượng calories.
- Furniture (Nội thất): Lưu chất liệu, kích thước tổng thể và khả năng chịu tải (weight_capacity).
- Medicine (Thuốc): Lưu thành phần hoạt tính, liều lượng và đánh dấu có cần kê đơn hay không.
- Toys (Đồ chơi): Phân loại theo độ tuổi (age_group), chất liệu và thông tin có dùng pin hay không.

2.3.4. Workflow

Luồng hoạt động của chức năng Nhân viên cập nhật thông tin sản phẩm

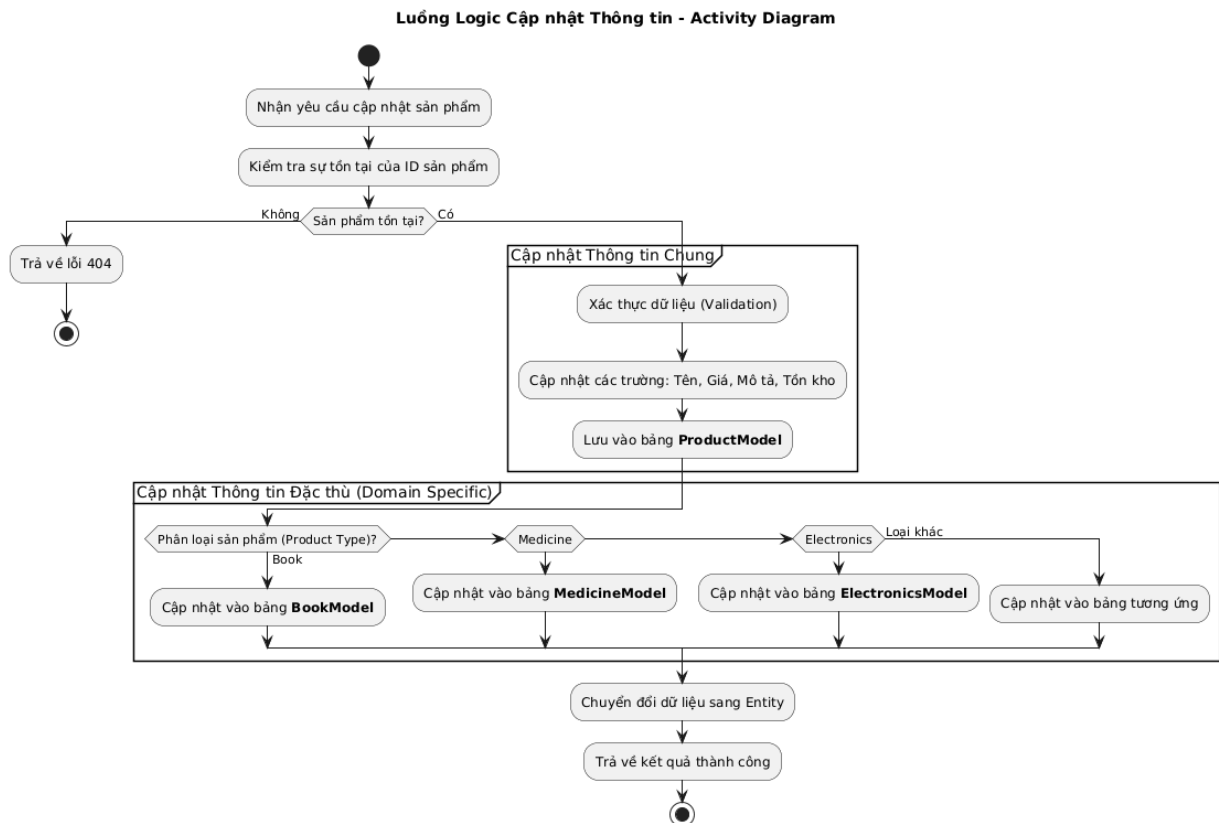


Hình 2.12. Biểu đồ tuần tự chức năng cập nhật sản phẩm

Mô tả luồng xử lý:

1. Gửi yêu cầu cập nhật: Nhân viên (Staff) gửi yêu cầu PUT /products/{id}/ kèm theo các thông tin cần thay đổi (ví dụ: cập nhật giá sách, đổi liều lượng thuốc, hoặc thay đổi mô tả sản phẩm điện tử) tới Product Controller.
2. Điều phối nghiệp vụ: Product Controller tiếp nhận yêu cầu và gọi hàm xử lý tương ứng trong Product Service.
3. Yêu cầu lưu trữ: Product Service đóng gói dữ liệu vào một ProductEntity và gọi phương thức save() của Product Repository để thực hiện lưu xuống cơ sở dữ liệu.
4. Cập nhật thông tin chung: Tại lớp Repository, hệ thống thực hiện lệnh update_or_create trên bảng ProductModel. Bước này đảm bảo các thông tin cơ

- bản (Tên, Giá, Mô tả, Tồn kho) được cập nhật chính xác dựa trên ID sản phẩm.
5. Cập nhật thông tin đặc thù (Domain Data): Đây là bước quan trọng nhất. Dựa vào `product_type` của sản phẩm, Repository sẽ rẽ nhánh để cập nhật vào bảng dữ liệu chi tiết tương ứng:
 - Nếu là Sách: Cập nhật tác giả, nhà xuất bản vào bảng `BookModel`.
 - Nếu là Dược phẩm: Cập nhật thành phần, liều dùng vào bảng `MedicineModel`.
 - (Tương tự cho các loại Điện tử, Thời trang, Nội thất...).
 6. Xác nhận lưu trữ: Sau khi tất cả các bảng liên quan đã được cập nhật thành công, Repository chuyển đổi dữ liệu từ database ngược lại thành Entity và trả về cho Product Service.
 7. Phản hồi kết quả: Product Service chuyển kết quả cho Controller, sau đó hệ thống trả về mã trạng thái 200 OK kèm theo tóm tắt dữ liệu đã cập nhật để Nhân viên xác nhận hoàn tất quy trình.



Hình 2.13. Biểu đồ hoạt động chức năng cập nhật sản phẩm

2.3.4. API

Endpoint	Method	Ý nghĩa
/products/	GET	Lấy danh sách sản phẩm (có phân trang, tìm kiếm, lọc)

Endpoint	Method	Ý nghĩa
/products/	POST	Tạo sản phẩm mới (chỉ dành cho Admin/Staff)
/products/{id}/	GET	Xem chi tiết một sản phẩm (bao gồm thông tin Domain cụ thể như Sách, Điện tử,...)
/products/{id}/	PUT/PATCH	Cập nhật thông tin sản phẩm
/products/{id}/	DELETE	Xóa sản phẩm
/categories/	GET	Lấy danh sách các danh mục sản phẩm
/categories/	POST	Tạo danh mục mới
/inventory/	GET	Kiểm tra tồn kho của các sản phẩm
/inventory/update/	POST	Cập nhật số lượng tồn kho (thường gọi sau khi có đơn hàng)
/brands/	GET	Danh sách các thương hiệu sản phẩm

2.4 Thiết kế User Service (Django)

User Service đóng vai trò là trung tâm xác thực (Identity Provider) cho toàn bộ hệ thống microservices, quản lý danh tính, điểm cá nhân và phân quyền dựa trên vai trò (RBAC).

2.4.1. Phân loại người dùng

Hệ thống quản lý 03 nhóm người dùng chính với các đặc quyền riêng biệt:

- **Admin:** Quản trị viên cấp cao, có toàn quyền cấu hình hệ thống, quản lý tài khoản Staff và Customer, can thiệp vào mọi luồng dữ liệu.
- **Staff:** Nhân viên vận hành, tập trung vào việc quản lý kho hàng (Inventory), xử lý đơn hàng (Order) và điều phối giao hàng (Shipping).
- **Customer:** Khách hàng cuối, có quyền tìm kiếm sản phẩm, thực hiện mua sắm, quản lý giỏ hàng và theo dõi lịch sử đơn hàng cá nhân.

2.4.2. Model

* User

```
class User(AbstractUser):
    ROLE_CHOICES = [
        ('ADMIN', 'Admin'),
        ('STAFF', 'Staff'),
        ('CUSTOMER', 'Customer'),
    ]
    role = models.CharField(max_length=10, choices=ROLE_CHOICES,
                           default='CUSTOMER')
    phone_number = models.CharField(max_length=20, blank=True, null=True)
```


Kế thừa từ `AbstractUser` của Django, cho phép bạn giữ lại các tính năng có sẵn (như đăng nhập, mật khẩu) và thêm các tùy chỉnh riêng:

- `role`: Phân quyền người dùng thành 3 nhóm: Admin (Quản trị), Staff (Nhân viên), và Customer (Khách hàng - mặc định).
- `phone_number`: Lưu số điện thoại trực tiếp vào bảng `User` để tiện liên lạc hoặc xác thực

* *Address*

```
class Address(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='addresses')
    name = models.CharField(max_length=100, help_text="Label e.g. 'Home',
'Office'")
    recipient_name = models.CharField(max_length=255, blank=True, null=True)
    recipient_phone = models.CharField(max_length=20, blank=True, null=True)
    street = models.CharField(max_length=255)
    city = models.CharField(max_length=100)
    country = models.CharField(max_length=100, default='Vietnam')
    postal_code = models.CharField(max_length=20, blank=True, default='')
    is_default = models.BooleanField(default=False)
```

Lưu danh sách địa chỉ nhận hàng của người dùng:

- `user`: Liên kết `ForeignKey` (Một-Nhiều). Một người dùng có thể có nhiều địa chỉ khác nhau.
- `name`: Nhãn phân loại địa chỉ (ví dụ: "Nhà riêng", "Cơ quan").
- `recipient_name` & `recipient_phone`: Tên và số điện thoại người nhận (có thể khác với chủ tài khoản).
- Thông tin vị trí: Bao gồm số nhà/đường (`street`), thành phố (`city`), quốc gia (`country` - mặc định là Vietnam) và mã bưu điện (`postal_code`).
- `is_default`: Đánh dấu địa chỉ mặc định để tự động chọn khi người dùng đặt hàng.

* *Membership*

```
class MembershipLevel(models.Model):
    name = models.CharField(max_length=50, unique=True)
    min_points = models.IntegerField(default=0)
    discount_percentage = models.IntegerField(default=0)
```

Lưu thông tin các loại membership với logic được định sẵn như sau:

Cấp bậc	Điểm tối thiểu (min_points)	Chiết khấu (%)
Bronze	0	0
Silver	1000	5
Gold	5000	10

Platinum	10000	15
Diamond	15000	20

- min_points: điểm tối thiểu tích lũy của người dùng để đạt được cấp bậc tương ứng. Với mỗi 1 cho một đơn hàng được hoàn thành sẽ tính là 1 point)
- discount_percentage: % giảm giá được áp dụng trực tiếp vào mỗi đơn hàng của khách hàng có cấp bậc tương ứng

* *LoyaltyWallet*

```
class LoyaltyWallet(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE,
related_name='wallet')
    usable_points = models.IntegerField(default=0)
    accumulated_points = models.IntegerField(default=0)
    current_level = models.ForeignKey(MembershipLevel, on_delete=models.SET_NULL,
null=True, blank=True)
```

Lưu thông tin điểm tích lũy của người dùng phục vụ cho việc tính ưu đãi cho đơn hàng cũng như mua voucher giảm giá

- accumulated_points: tổng điểm tích lũy của người dùng
- usable_points: điểm khả dụng của khách hàng

```
class PointTransaction(models.Model):
    TRANSACTION_TYPES = [
        ('EARN', 'Earned from purchase'),
        ('SPEND', 'Spent on voucher/reward'),
        ('EXPIRE', 'Expired')
    ]
    wallet = models.ForeignKey(LoyaltyWallet, on_delete=models.CASCADE,
related_name='transactions')
    amount = models.IntegerField()
    transaction_type = models.CharField(max_length=10, choices=TRANSACTION_TYPES)
    description = models.CharField(max_length=255, blank=True, default='')
    created_at = models.DateTimeField(auto_now_add=True)
```

Lưu lịch sử thay đổi điểm tích lũy của mỗi khách hàng

- Transaction_type: loại giao dịch được lưu: nhận điểm từ đơn hàng, trừ điểm khi đổi voucher,...
- Amount: tổng điểm thay đổi

2.4.3. Phân quyền

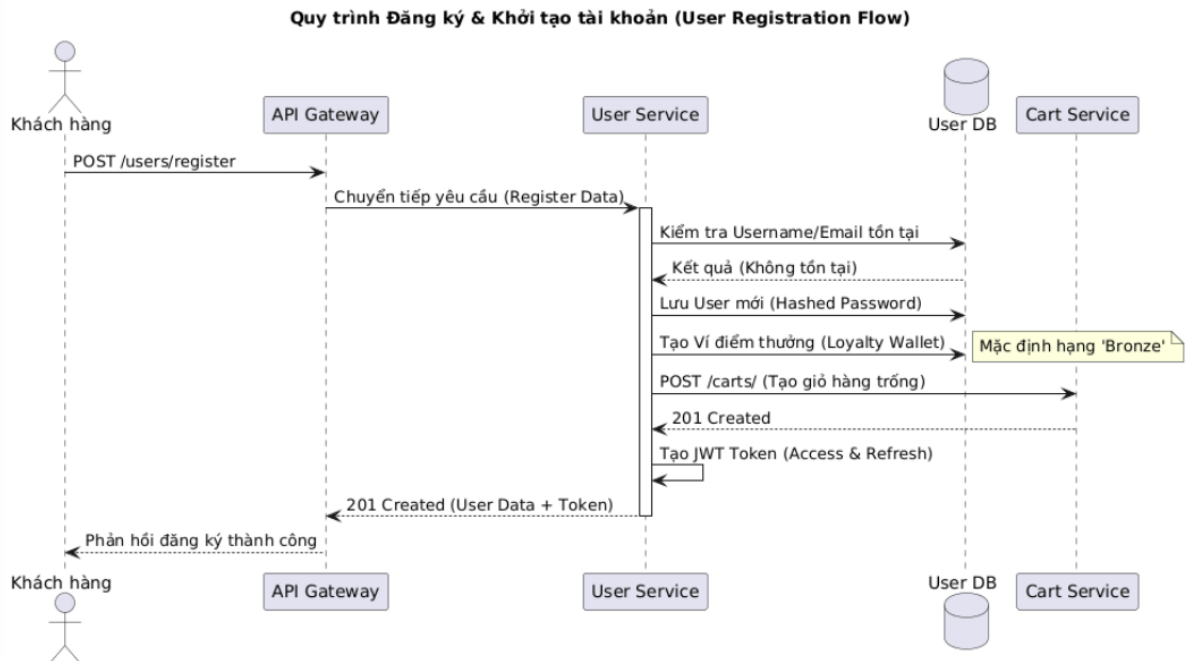
Hệ thống áp dụng cơ chế kiểm soát truy cập dựa trên vai trò (Role-Based Access Control) thông qua Custom Permissions:

- **Admin (IsAdminUser):**
 - o Thực hiện CRUD (Thêm, Đọc, Sửa, Xóa) trên tất cả các tài nguyên.

- Quản lý cấu hình hệ thống và xem báo cáo tổng hợp.
- **Staff (IsStaffUser):**
 - Quản lý danh mục sản phẩm (Product Catalog).
 - Cập nhật trạng thái đơn hàng (Pending -> Processing -> Shipped).
 - Xem thông tin khách hàng để phục vụ giao nhận.
- **Customer (IsCustomerUser):**
 - Truy cập các API tìm kiếm và xem chi tiết sản phẩm.
 - Thực hiện các thao tác trên Giỏ hàng và Checkout.
 - Chỉ được quyền xem và sửa thông tin cá nhân/đơn hàng của chính mình.

2.4.5. Workflow

Luồng hoạt động chính của User Service: User đăng kí, khởi tạo tài khoản



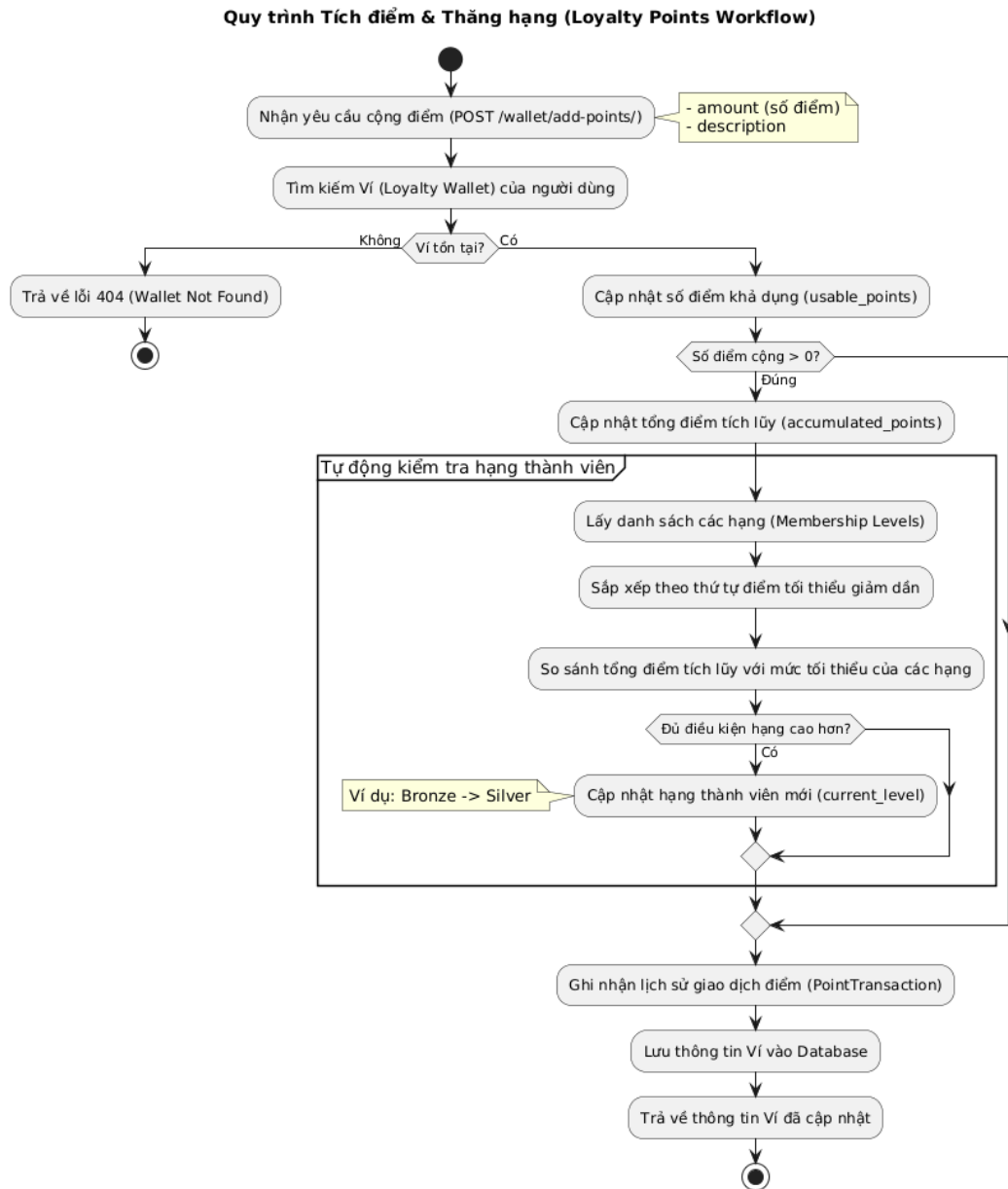
Hình 2.14. Biểu đồ tuần tự chức năng đăng ký người dùng

Mô tả luồng xử lý:

1. Khách hàng gửi yêu cầu: Khách hàng điền thông tin đăng ký (username, email, password...) và gửi yêu cầu POST /users/register thông qua API Gateway.
2. Định tuyến yêu cầu: API Gateway tiếp nhận và điều phối yêu cầu này đến đúng User Service để xử lý nghiệp vụ.
3. Kiểm tra tính duy nhất: User Service truy vấn vào User DB để kiểm tra xem username hoặc email đã tồn tại trong hệ thống chưa nhằm tránh trùng lặp tài khoản.
4. Khởi tạo tài khoản: Sau khi xác nhận thông tin hợp lệ, User Service thực hiện:
 - Mã hóa mật khẩu và lưu thông tin người dùng mới vào User DB.
 - Tự động khởi tạo một Ví điểm thưởng (Loyalty Wallet) cho người dùng với

mức hạng mặc định là Bronze (Hạng Đồng).

5. Liên kết dịch vụ (Cart Initialization): Để chuẩn bị cho trải nghiệm mua sắm, User Service gửi một yêu cầu đồng bộ sang Cart Service để tạo sẵn một giỏ hàng trống cho khách hàng mới này.
6. Cấp phát định danh (JWT): Hệ thống tạo ra cặp mã thông báo JWT (Access & Refresh Token), cho phép người dùng có thể thực hiện các yêu cầu xác thực tiếp theo mà không cần gửi lại mật khẩu.
7. Phản hồi kết quả: User Service trả về toàn bộ thông tin người dùng kèm theo mã JWT cho Khách hàng thông qua API Gateway, hoàn tất quy trình đăng ký thành công.



Hình 2.15. Biểu đồ hoạt động chức năng đăng ký người dùng

2.4.5. API

Nhóm API	Endpoint	Method	Ý nghĩa	Quyền hạn
Xác thực	/auth/register/	POST	Đăng ký tài khoản khách hàng mới	Public
	/auth/login/	POST	Đăng nhập và nhận JWT (Access + Refresh)	Public
	/auth/refresh/	POST	Làm mới Access Token	Public
Người dùng	/users/me/	GET	Lấy thông tin chi tiết tài khoản hiện tại	Authenticated
	/users/me/	PATCH	Cập nhật thông tin cá nhân (email, phone,...)	Authenticated
	/users/	GET	Liệt kê danh sách tất cả người dùng	Admin
	/users/{id}/	DELETE	Xóa hoặc vô hiệu hóa tài khoản	Admin
Địa chỉ	/addresses/	GET	Liệt kê các địa chỉ giao hàng của User	Authenticated
	/addresses/	POST	Thêm địa chỉ giao nhận mới	Authenticated
	/addresses/{id}/	PATCH	Sửa thông tin hoặc đặt làm địa chỉ mặc định	Owner
	/addresses/{id}/	DELETE	Xóa địa chỉ giao hàng	Owner
Membership	/users/{id}/wallet/add-points/	POST	Cộng trừ điểm tích lũy, tự động nâng hạng nếu đủ điểm và lưu lịch sử giao dịch	Authenticated
	/membership-levels/	GET	Lấy danh sách hạng thành viên	Public
	/users/{id}/wallet/	GET	Lấy thông tin chi tiết ví điểm của 1 user (điểm khả dụng, tổng tích lũy, hạng hiện tại)	Authenticated
Nội bộ	/internal/verify/	POST	Kiểm tra tính hợp lệ của Token (cho Gateway)	Service-to-Service

2.5 Thiết kế Cart Service

Cart Service quản lý giỏ hàng tạm thời của người dùng trước khi tiến hành đặt hàng. Dịch vụ này sử dụng PostgreSQL để lưu trữ dữ liệu bền vững.

2.5.1 Model

Dịch vụ sử dụng hai Model chính: Cart (đại diện cho giỏ hàng của một User) và CartItem (chi tiết từng sản phẩm trong giỏ).

```
class Cart(models.Model):  
    customer_id = models.IntegerField()
```

Giỏ hàng chứa đồ cho mỗi người dùng:

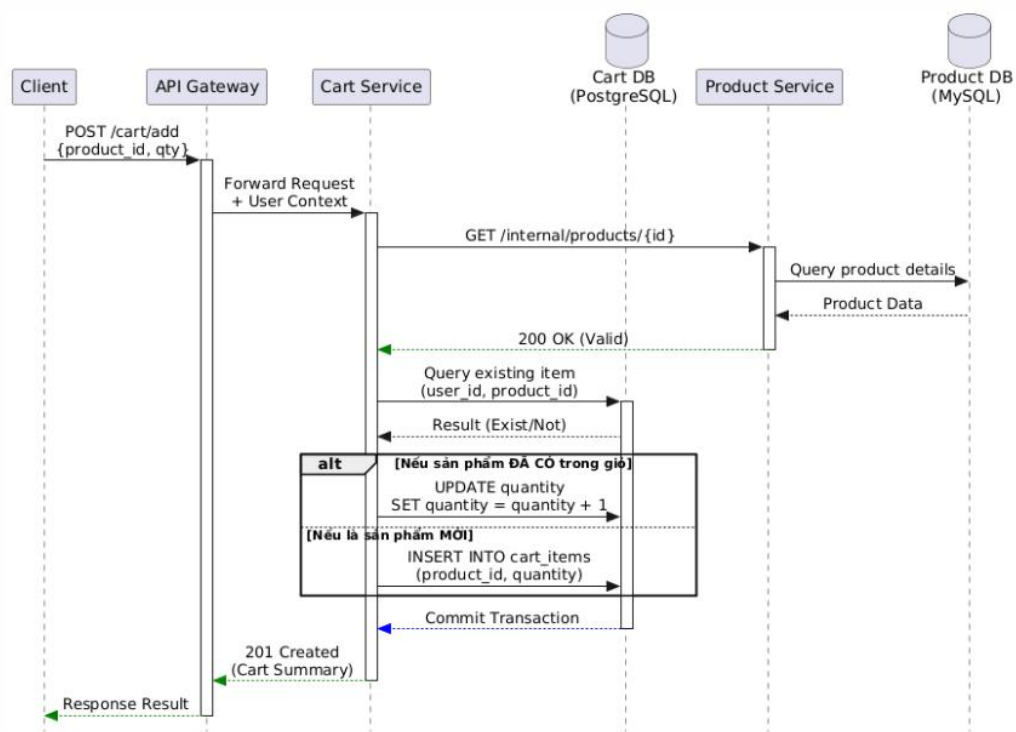
- `user_id`: Lưu ID của người dùng. Việc dùng `unique=True` đảm bảo mỗi người dùng chỉ có duy nhất một giỏ hàng (tránh trùng lặp).

```
class CartItem(models.Model):  
    cart = models.ForeignKey(Cart, on_delete=models.CASCADE)  
    product_id = models.IntegerField()  
    quantity = models.IntegerField()
```

Các dòng sản phẩm nằm bên trong giỏ hàng đó:

- `cart`: Liên kết với bảng `Cart`. Khi giỏ hàng bị xóa, các món đồ bên trong cũng biến mất theo (`CASCADE`).
- `product_id`: Lưu ID của sản phẩm (tham chiếu từ `Service Product`).
- `quantity`: Số lượng món hàng, sử dụng `PositiveIntegerField` để đảm bảo số lượng luôn lớn hơn hoặc bằng 0.

2.5.2 Logic xử lý

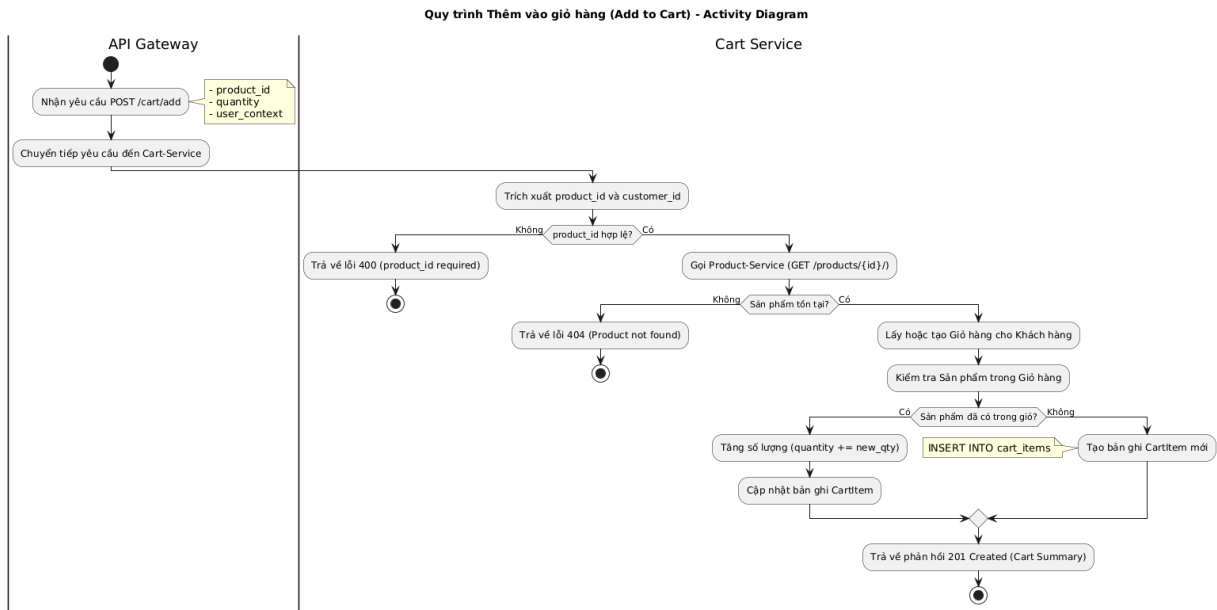


Hình 2.16. Biểu đồ tuần tự chức năng thêm giỏ hàng

Logic của Cart Service đảm bảo tính nhất quán khi người dùng thay đổi ý định mua sắm.

- Add product vào cart: Kiểm tra nếu sản phẩm đã tồn tại thì tăng quantity, nếu chưa có thì tạo CartItem mới.
- Update số lượng: Ghi đè số lượng mới (thường gọi khi User nhấn +/- trên giao diện).
- Remove item: Xóa hẳn dòng CartItem tương ứng.

Activity Diagram chức năng thêm giỏ hàng



Hình 2.17. Biểu đồ hoạt động chức năng thêm giỏ hàng

2.5.3 API (Endpoints)

Endpoint	Method	Ý nghĩa
/cart/	GET	Xem toàn bộ giỏ hàng của người dùng hiện tại
/cart/add/	POST	Thêm sản phẩm vào giỏ (Payload: product_id, quantity)
/cart/update-item/{id}/	PATCH	Cập nhật số lượng của một Item cụ thể
/cart/remove-item/{id}/	DELETE	Xóa một sản phẩm khỏi giỏ hàng
/cart/clear/	POST	Xóa sạch giỏ hàng (thường gọi sau khi Checkout thành công)

2.6 Thiết kế Order Service

Order Service chịu trách nhiệm quản lý vòng đời của đơn hàng, từ khi khách nhấn "Đặt hàng" cho đến khi giao hàng thành công. Dịch vụ này điều phối các dịch vụ khác (Payment, Inventory, Cart) thông qua mô hình Saga.

2.6.1. Model

* Order

Mô hình lưu trữ thông tin tổng quát của đơn hàng và chi tiết các sản phẩm bên trong.


```
class Order(models.Model):
    STATUS_CHOICES = (
        ('pending', 'Chờ thanh toán'),
        ('paid', 'Đã thanh toán'),
        ('processing', 'Đang xử lý'),
        ('shipped', 'Đang giao hàng'),
        ('completed', 'Hoàn thành'),
        ('canceled', 'Đã hủy'),
    )
    user_id = models.IntegerField()
    total_price = models.DecimalField(max_digits=12, decimal_places=2)
    status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='pending')
    shipping_address = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)
```

Đây là bảng lưu thông tin tổng quát của một giao dịch:

- status: Quản lý vòng đời đơn hàng từ lúc mới tạo (pending) cho đến khi completed hoặc canceled.
- user_id: ID của khách hàng mua hàng (tham chiếu từ User Service).
- total_price: Tổng giá trị đơn hàng. max_digits=12 cho phép lưu số tiền lên tới hàng chục tỷ, rất phù hợp với tiền Việt Nam (VND).
- shipping_address: Lưu địa chỉ giao hàng dưới dạng văn bản tại thời điểm mua (để tránh việc user đổi địa chỉ trong profile làm ảnh hưởng đến đơn hàng cũ).
- created_at: Thời điểm khách hàng chốt đơn.

```
class OrderItem(models.Model):
    order = models.ForeignKey(Order, on_delete=models.CASCADE,
related_name='items')
    product_id = models.IntegerField()
    product_name = models.CharField(max_length=255, default='') # Snapshot
    item_image_url = models.TextField(blank=True, default='') # Snapshot
    quantity = models.IntegerField()
    unit_price = models.DecimalField(max_digits=10, decimal_places=2) # Snapshot
price
```

Lưu danh sách các sản phẩm có trong đơn hàng đó:

- order: Liên kết với đơn hàng chính. Nếu đơn hàng bị xóa, các chi tiết này cũng bị xóa theo.
- product_id: ID của sản phẩm (tham chiếu từ Product Service).
- quantity: Số lượng sản phẩm đã mua.
- Unit_price: Rất quan trọng. Đây là giá của sản phẩm tại thời điểm mua. Việc này giúp hóa đơn không bị thay đổi nếu sau này bạn cập nhật giá mới trong bảng Product.

```
class OrderStatusLog(models.Model):
```

```

order = models.ForeignKey(Order, on_delete=models.CASCADE,
    related_name='status_logs')
status = models.CharField(max_length=50)
notes = models.TextField(blank=True, default='')
created_at = models.DateTimeField(auto_now_add=True)

```

Lưu lịch sử thay đổi trạng thái của đơn hàng, phục vụ cho việc khách hàng theo dõi trạng thái đi kèm thời gian thay đổi

** Voucher*

Các voucher được khách hàng mua bằng điểm tích lũy cũng như được hệ thống phát miễn phí

```

class Voucher(models.Model):
    code = models.CharField(max_length=100, unique=True)
    discount_amount = models.DecimalField(max_digits=10, decimal_places=2)
    is_percentage = models.BooleanField(default=False)
    min_spend = models.DecimalField(max_digits=10, decimal_places=2,
        default=0)
    min_points_level_id = models.IntegerField(null=True, blank=True,
        help_text="MembershipLevel ID from user-service")
    point_cost = models.IntegerField(default=0)
    max_quantity = models.IntegerField(default=100)
    redeemed_quantity = models.IntegerField(default=0)
    is_public = models.BooleanField(default=True)
    is_active = models.BooleanField(default=True)
    expiry_date = models.DateTimeField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

```

Model này dùng để lưu trữ thông tin chung về các mã giảm giá (voucher) có trong hệ thống.

- code: Mã voucher dạng chữ/số (VD: SUMMER20), mang tính duy nhất (unique=True).
- discount_amount: Giá trị giảm giá.
- is_percentage: Cờ xác định xem giá trị giảm giá (discount_amount) là theo phần trăm (True) hay là số tiền mặt (False).
- min_spend: Giá trị đơn hàng tối thiểu để có thể áp dụng voucher này (mặc định là 0).
- min_points_level_id: ID cấp độ thành viên tối thiểu cần có để sử dụng voucher (liên kết với user-service).
- point_cost: Số điểm (point) người dùng cần tiêu tốn để đổi lấy voucher này.
- max_quantity: Tổng số lượng voucher được phát hành.
- redeemed_quantity: Số lượng voucher đã được người dùng thu thập/đổi.
- is_public: Cờ xác định xem voucher có được hiển thị công khai cho mọi người hay không.
- is_active: Cờ xác định voucher có đang trong trạng thái hoạt động/cho phép sử dụng

hay không.

- expiry_date: Ngày giờ hết hạn của voucher.
- created_at: Thời gian tạo voucher (tự động gán khi tạo mới).

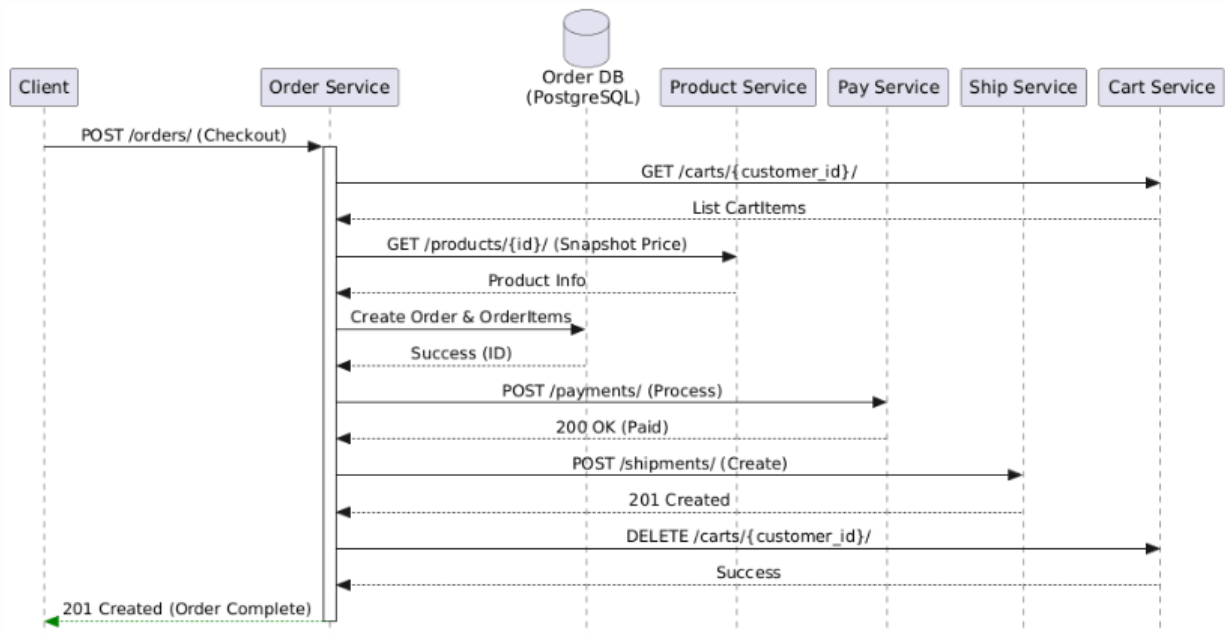
```
class CustomerVoucher(models.Model):
    customer_id = models.IntegerField()
    voucher = models.ForeignKey(Voucher, on_delete=models.CASCADE,
related_name='customer_vouchers')
    is_used = models.BooleanField(default=False)
    redeemed_at = models.DateTimeField(auto_now_add=True)
    used_at = models.DateTimeField(null=True, blank=True)
    order_id = models.IntegerField(null=True, blank=True)
```

Model này đóng vai trò như một bảng trung gian (hoặc "ví voucher"), dùng để lưu trữ trạng thái sở hữu của một khách hàng đối với một voucher cụ thể (khi khách hàng lưu hoặc dùng điểm đổi voucher).

- is_used: Trạng thái cho biết khách hàng đã sử dụng voucher này hay chưa.
- redeemed_at: Thời điểm khách hàng lưu/thu thập voucher về ví (tự động gán khi tạo).
- used_at: Thời điểm khách hàng thực sự áp dụng/sử dụng voucher này.
- order_id: ID của đơn hàng mà khách hàng đã áp dụng voucher này (lưu lại để đối soát sau này).

2.6.2. Workflow

Luồng hoạt động chính của Order Service: User thực hiện checkout

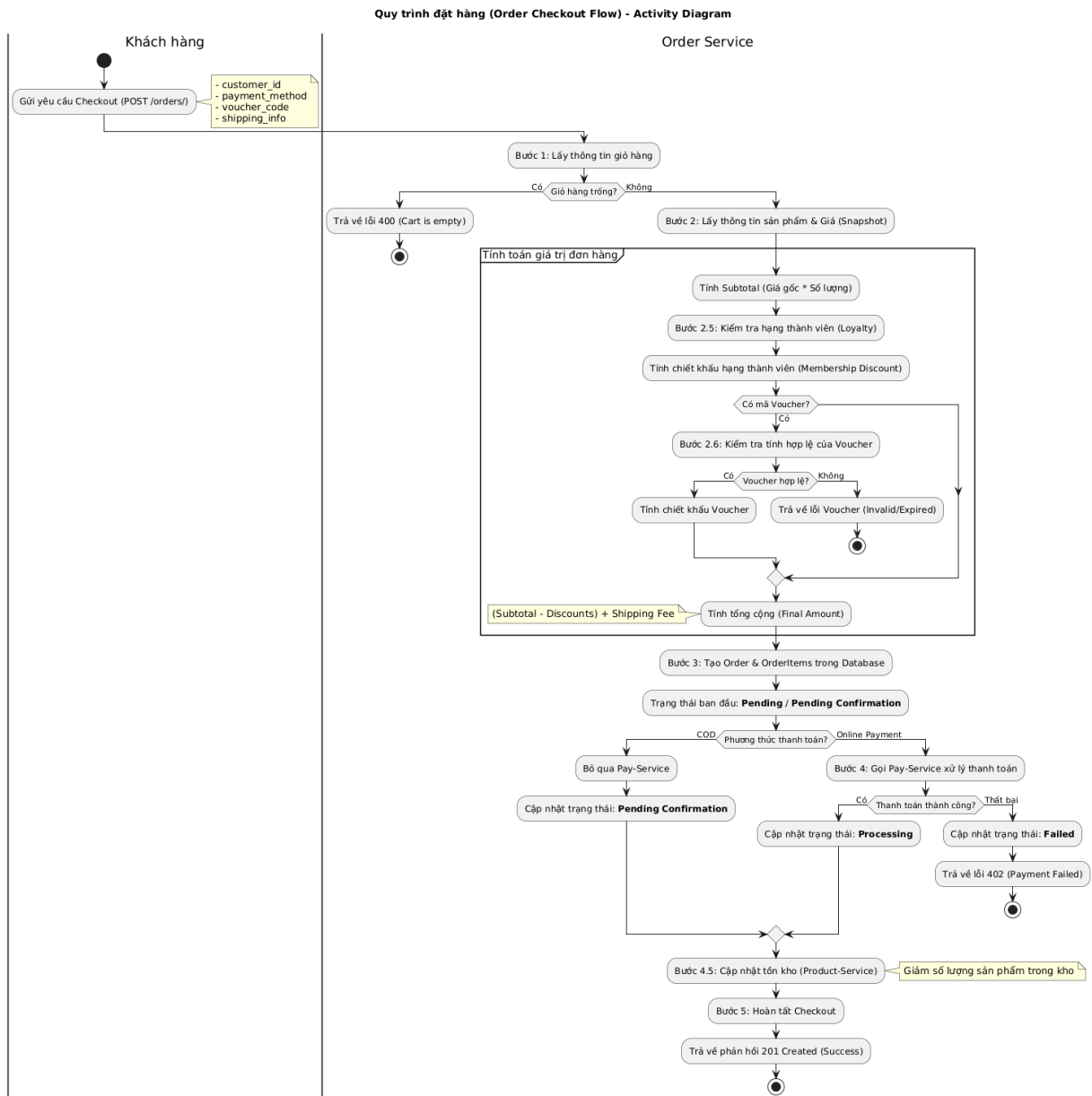


Hình 2.18. Biểu đồ tuần tự chức năng mua hàng

Quy trình Checkout thực tế gồm 6 bước liên tiếp:

1. Lấy Giỏ hàng: Truy vấn cart-service để lấy danh sách sản phẩm khách đã chọn.
2. Snapshot Giá & Khuyến mãi: Gọi product-service để lấy giá hiện tại, kiểm tra hạng thành viên từ user-service và áp dụng Voucher.
3. Lưu Đơn hàng: Khởi tạo bản ghi Order và OrderItem trong cơ sở dữ liệu với trạng thái pending.
4. Thanh toán: Gọi pay-service. Nếu là COD thì bỏ qua bước này và chuyển trạng thái sang pending_confirmation.
5. Vận chuyển: Nếu thanh toán thành công, gọi ship-service để tạo vận đơn.
6. Dọn dẹp: Gọi cart-service để xóa sạch giỏ hàng của khách.

Activity Diagram chi tiết



Hình 2.19. Biểu đồ hoạt động chức năng mua hàng

2.6.3. API

Một số API tiêu biểu

Endpoint	Method	Ý nghĩa
/orders/	POST	Thực hiện Checkout (Payload: customer_id, shipping_address, payment_method, voucher_code)
/orders/	GET	Lấy lịch sử đơn hàng (Lọc theo customer_id)
/orders/{id}/	GET	Xem chi tiết đơn hàng (kèm trạng thái từ Ship và

Endpoint	Method	Ý nghĩa
		Pay)
/orders/{id}/status/	PATCH	Cập nhật trạng thái đơn hàng
/orders/{id}/cancel	POST	Hủy đơn hàng (cho phép hủy với trạng thái pending/processing)
/orders/{id}/delete	DELETE	Xóa vĩnh viễn đơn hàng (cho phép xóa với trạng thái cancelled)
/vouchers/	GET	Danh sách các mã giảm giá khả dụng
/vouchers/redeem/	POST	Đổi điểm và lưu voucher vào ví (kiểm tra số lượng giới hạn, hạng thành viên, trừ điểm)
/vouchers/{code}	GET	Tra cứu thông tin voucher cụ thể
/vouchers/customer/{id}/	GET	Lấy danh sách toàn bộ voucher mà khách hàng này sở hữu

2.7 Thiết kế Payment Service

Payment Service quản lý các giao dịch tài chính, tích hợp nhiều phương thức thanh toán và cập nhật trạng thái đơn hàng dựa trên kết quả thanh toán.

2.7.1. Model

```
class Payment(models.Model):
    METHOD_CHOICES = [
        ('COD', 'Cash on Delivery'),
        ('BANK_TRANSFER', 'Bank Transfer'),
        ('MOMO', 'MoMo Wallet'),
        ('VNPay', 'VNPay'),
    ]
    STATUS_CHOICES = [
        ('processing', 'Processing'),
        ('success', 'Success'),
        ('failed', 'Failed'),
        ('refunded', 'Refunded'),
    ]
    order_id = models.IntegerField()
    customer_id = models.IntegerField()
    amount = models.DecimalField(max_digits=12, decimal_places=2)
    method = models.CharField(max_length=20, choices=METHOD_CHOICES,
                              default='COD')
    status = models.CharField(max_length=20, choices=STATUS_CHOICES,
                              default='processing')
    transaction_id = models.CharField(max_length=100, unique=True, blank=True)
    note = models.TextField(blank=True, default='')
    created_at = models.DateTimeField(auto_now_add=True)
```

Model này đóng vai trò ghi lại lịch sử giao dịch và trạng thái dòng tiền của mỗi đơn hàng.

- Phân loại thanh toán:
 - method: Hỗ trợ nhiều hình thức như Chuyển khoản, Ví điện tử, Thẻ tín dụng và COD (Thanh toán khi nhận hàng).
 - status: Theo dõi tiến độ giao dịch từ lúc chờ xử lý (pending) đến khi thành công hoặc hoàn tiền (refunded).
- Kết nối liên Service:
 - order_id & customer_id: Lưu ID của đơn hàng và khách hàng dưới dạng số nguyên (không dùng khóa ngoại) để đảm bảo hoạt động độc lập trong kiến trúc Microservices.
- Thông tin tài chính:
 - amount: Số tiền thanh toán (tối đa 12 chữ số, phù hợp với tiền VND).
 - transaction_id: Mã giao dịch từ bên thứ ba (như VNPay, MoMo). Trường này là unique để tránh việc một giao dịch bị ghi nhận hai lần.
- created_at: Thời điểm phát sinh giao dịch.

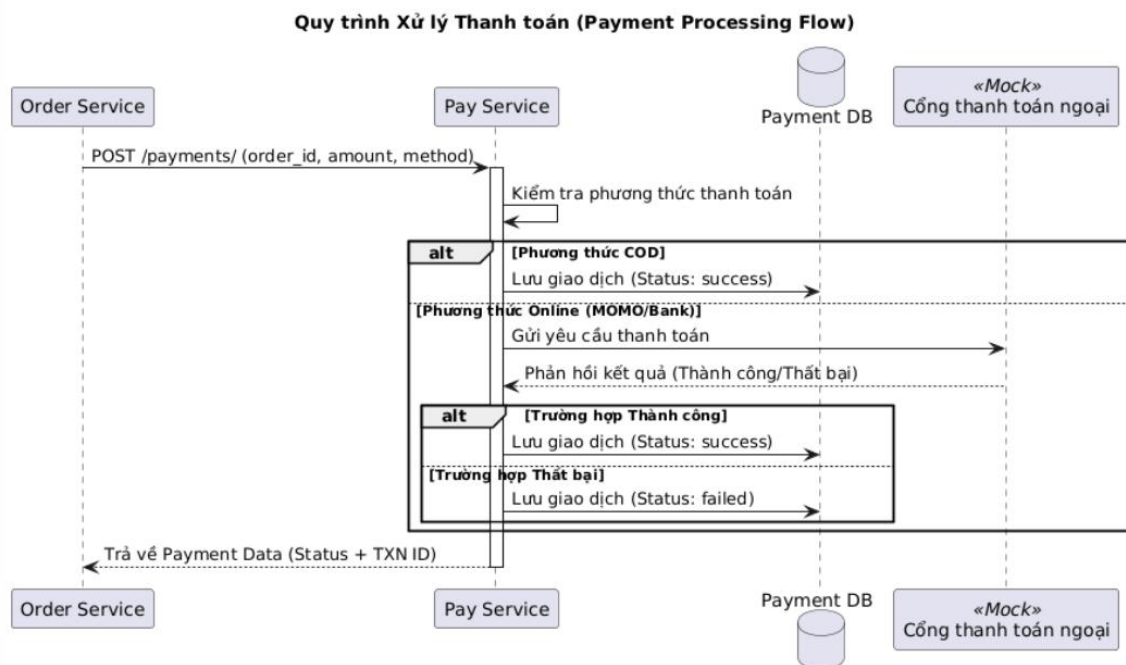
2.7.2. Trạng thái

Hệ thống quản lý trạng thái thanh toán chặt chẽ để đảm bảo không thất thoát doanh thu:

Trạng thái Payment	Ý nghĩa	Tương ứng trạng thái Đơn hàng
pending	Giao dịch vừa được tạo, đang chờ phản hồi từ Gateway hoặc chờ khách chuyển khoản.	pending (Chờ thanh toán)
completed	Tiền đã được xác nhận vào tài khoản của hệ thống.	paid (Đã thanh toán)
failed	Giao dịch bị từ chối (Sai thẻ, hết hạn, khách hủy giữa chừng).	canceled hoặc pending (để khách thử lại)
refunded	Tiền được hoàn lại cho khách (thường do hủy đơn sau khi đã trả tiền).	canceled (Đã hủy)

2.7.3. Workflow

Luồng hoạt động chính của Pay Service: Xử lý thanh toán khách hàng thanh toán



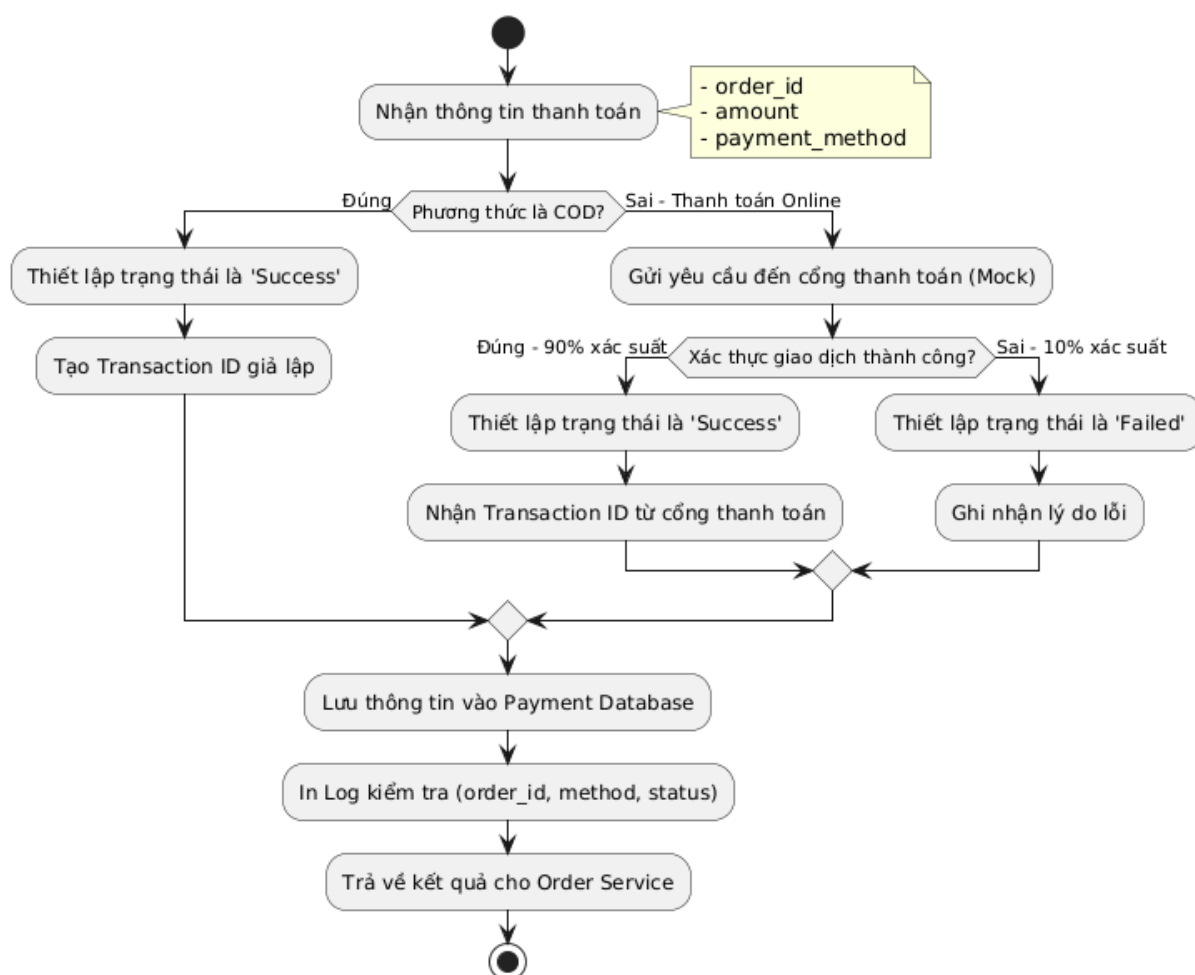
Hình 2.20. Biểu đồ tuần tự luồng xử lý thanh toán

Mô tả Sequence Diagram:

- Order Service gửi yêu cầu: Khi khách hàng chọn thanh toán, order-service gửi các thông tin cần thiết sang pay-service.
- Phân loại phương thức: pay-service kiểm tra loại hình thanh toán. Nếu là COD, hệ

- thống tự động xác nhận thành công vì tiền sẽ được thu sau.
- Xử lý thanh toán trực tuyến: Với các phương thức như MOMO hay Chuyển khoản, hệ thống giả lập việc gọi đến các API bên ngoài. Trong mã nguồn hiện tại, hệ thống sử dụng hàm random() để tạo ra tỷ lệ thành công 90% nhằm mô phỏng các sự cố mạng hoặc số dư không đủ.
 - Trả kết quả: Sau khi ghi dữ liệu vào DB, pay-service trả lại trạng thái cho order-service để dịch vụ này cập nhật trạng thái đơn hàng tương ứng.

Logic xử lý Thanh toán - Activity Diagram



Hình 2.21. Biểu đồ hoạt động luồng xử lý thanh toán

2.7.4. API

Endpoint	Method	Ý nghĩa
/payments/	POST	Tạo giao dịch mới (Gửi kèm order_id, amount, method)
/payments/{id}/	GET	Xem chi tiết trạng thái của một giao dịch
/payments/order/{order_id}/	GET	Lấy thông tin thanh toán dựa trên mã đơn hàng

ndpoint	Method	Ý nghĩa
/internal/callback/	POST	Endpoint dành cho ngân hàng/ví điện tử gọi về để cập nhật kết quả
/internal/refund/	POST	Thực hiện hoàn tiền cho một giao dịch đã thành công

2.8 Thiết kế Shipping Service

Shipment Service quản lý quy trình giao hàng, theo dõi vị trí kiện hàng và cập nhật mã vận đơn (Tracking Number).

2.8.1 Model

```
class ShippingMethod(models.Model):
    id_slug = models.SlugField(primary_key=True)
    name = models.CharField(max_length=100)
    description = models.TextField(blank=True)
    base_fee = models.FloatField()
    free_threshold = models.FloatField(null=True, blank=True)
    estimated_days = models.IntegerField(default=3)

    def __str__(self):
        return f"{self.name} ({self.base_fee})"
```

Model này dùng để định nghĩa các phương thức vận chuyển mà hệ thống cung cấp (ví dụ: Giao hàng tiêu chuẩn, Giao hàng hỏa tốc).

- `id_slug`: Mã định danh dạng chuỗi không dấu (ví dụ: `standard`, `express`), được dùng làm khóa chính (`primary_key=True`).
- `name`: Tên hiển thị của phương thức vận chuyển.
- `description`: Thông tin mô tả chi tiết (không bắt buộc).
- `base_fee`: Phí vận chuyển cơ bản.
- `free_threshold`: Hạn mức giá trị đơn hàng tối thiểu để được miễn phí vận chuyển (nếu có).
- `estimated_days`: Số ngày giao hàng dự kiến (mặc định là 3 ngày).

```
class Shipment(models.Model):
    STATUS_CHOICES = [
        ('ready_for_pickup', 'Ready for Pickup'),
        ('delivering', 'Delivering'),
        ('completed', 'Completed (Delivered)'),
        ('cancelled', 'Cancelled'),
    ]

    order_id = models.IntegerField(unique=True)
    customer_id = models.IntegerField()
    tracking_code = models.CharField(max_length=50, unique=True, blank=True)
    shipping_method = models.CharField(max_length=50, default='standard')
    address = models.TextField()
    status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='ready_for_pickup')
    estimated_delivery = models.DateTimeField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```

Model này quản lý thông tin và trạng thái vận chuyển của từng đơn hàng cụ thể. Mỗi đơn hàng sẽ tương ứng với một bản ghi Shipment.

- STATUS_CHOICES: Các trạng thái có thể có của đơn hàng bao gồm: Chờ lấy hàng (ready_for_pickup), Đang giao (delivering), Đã hoàn thành (completed), Đã hủy (cancelled).
- order_id: ID của đơn hàng từ order-service (đảm bảo tính duy nhất unique=True, mỗi đơn hàng chỉ có một Shipment).
- customer_id: ID của khách hàng đặt mua.
- tracking_code: Mã vận đơn dùng để tra cứu hành trình (duy nhất).
- shipping_method: Chuỗi tên phương thức vận chuyển đã chọn (mặc định là standard). Đáng chú ý là trường này đang lưu string thay vì khóa ngoại (ForeignKey) nối sang ShippingMethod.
- address: Địa chỉ giao hàng.
- status: Trạng thái vận chuyển hiện tại.
- estimated_delivery: Thời điểm giao hàng dự kiến.
- created_at, updated_at: Thời gian tạo lập và cập nhật (tự động gán).

2.8.2 Giải thích các trạng thái

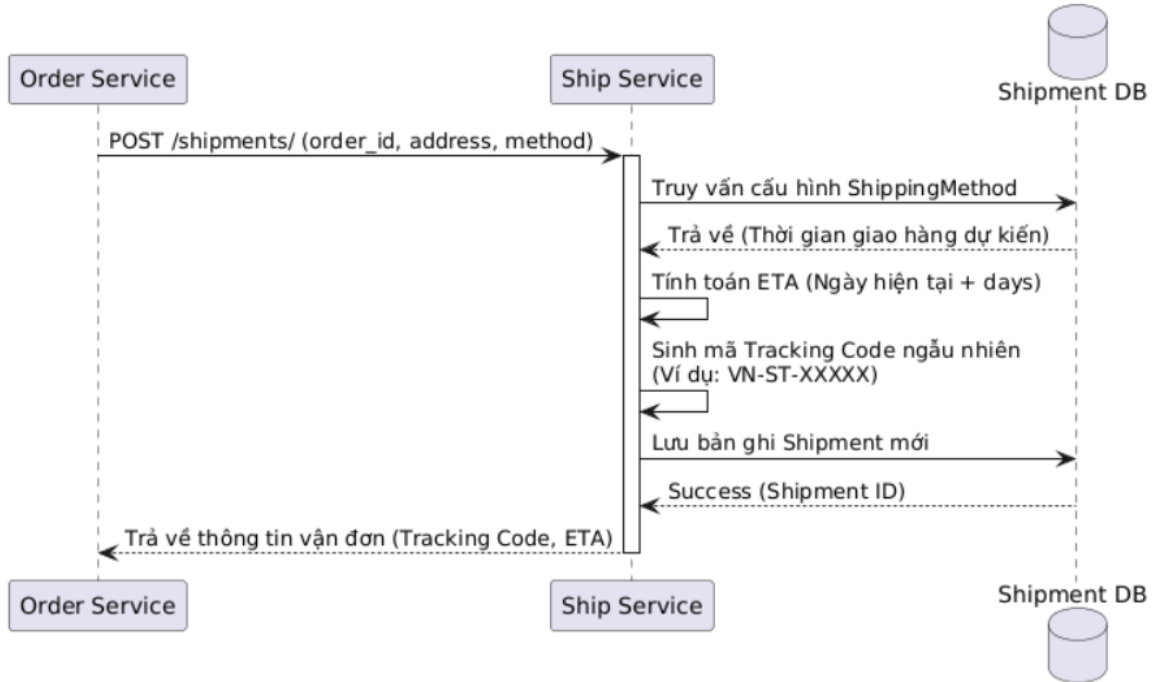
Quy trình giao hàng được chia nhỏ để khách hàng dễ dàng theo dõi:

Trạng thái Shipment	Ý nghĩa	Tương ứng trạng thái Đơn hàng
pending	Đã tạo vận đơn, đang chờ bưu tá đến kho lấy hàng.	processing
picked_up	Bưu tá đã nhận hàng từ kho.	shipped
in_transit	Hàng đang luân chuyển qua các kho trung chuyển.	shipped
delivered	Khách đã nhận hàng thành công.	completed
failed	Không liên lạc được khách hoặc khách từ chối nhận.	returned

2.8.3. Workflow

Luồng hoạt động chính của Ship Service: khởi tạo vận đơn

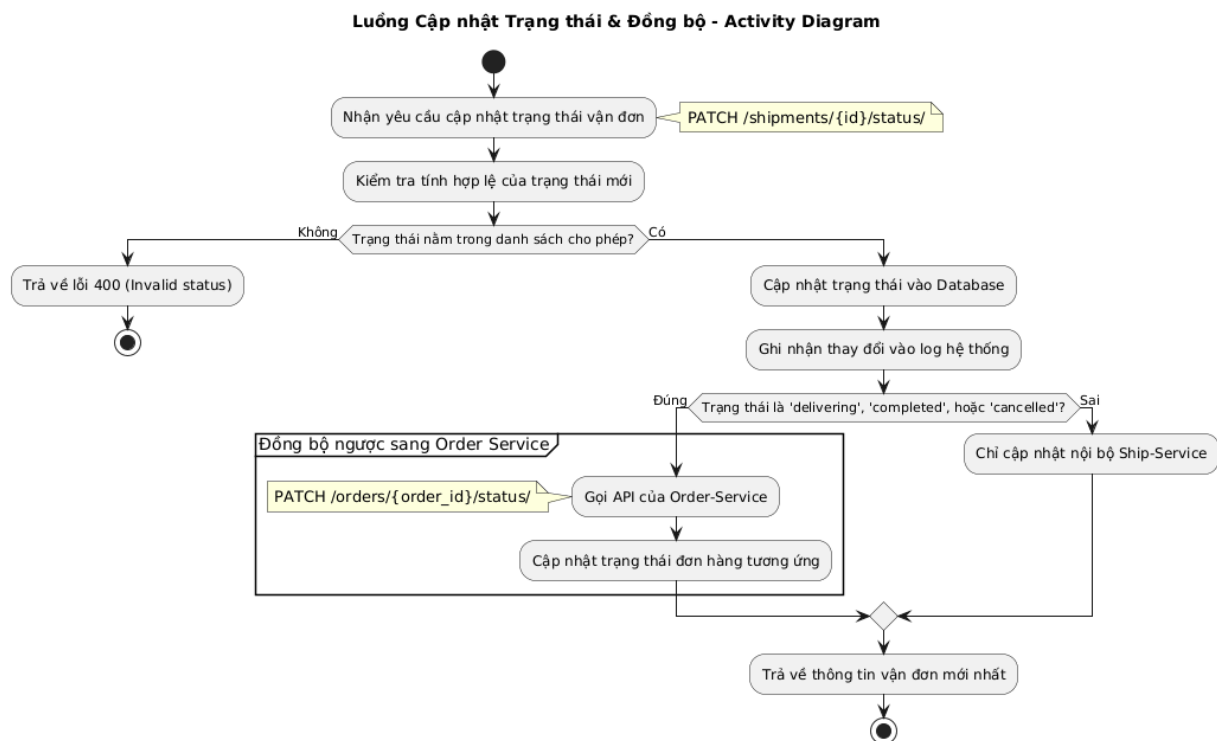
Quy trình Khởi tạo Vận đơn (Shipment Creation Flow)



Hình 2.22. Biểu đồ tuần tự luồng khởi tạo vận đơn

Mô tả Sequence Diagram:

1. Tiếp nhận yêu cầu: Khi đơn hàng sẵn sàng để giao, order-service gửi yêu cầu tạo vận đơn.
2. Tính toán thời gian (ETA): ship-service dựa vào phương thức vận chuyển (Tiết kiệm, Tiêu chuẩn, Hỏa tốc) để tính ngày giao hàng dự kiến.
3. Sinh mã Tracking: Hệ thống tự động sinh một mã vận đơn duy nhất (ví dụ: VN-ST-8 ký tự ngẫu nhiên) để khách hàng có thể theo dõi.
4. Hoàn tất: Thông tin vận đơn được trả về để hiển thị cho khách hàng ở giao diện lịch sử đơn hàng.



Hình 2.23. Biểu đồ hoạt động luồng khởi tạo vận đơn

2.8.4 API

Endpoint	Method	Ý nghĩa
/shipments/	POST	Tạo vận đơn mới cho một đơn hàng
/shipments/{tracking_number}/	GET	Tra cứu hành trình kiện hàng bằng mã vận đơn
/shipments/order/{order_id}/	GET	Lấy thông tin vận chuyển theo mã đơn hàng
/shipping-methods/	GET	Lấy danh sách phương thức vận chuyển
/shipments/{id}/status/	PATCH	Cập nhật trạng thái giao hàng
/shipments/order/{id}	GET, PATCH, DELETE	Thực hiện thao tác lấy thông tin, đồng bộ, xóa shipment (khi order bị hủy)

2.9 Thiết kế Interaction Service

Interaction Service thu thập và lưu trữ các "tín hiệu" (Signals) từ người dùng như: xem hàng, click chuột, tìm kiếm...

2.9.1 Model

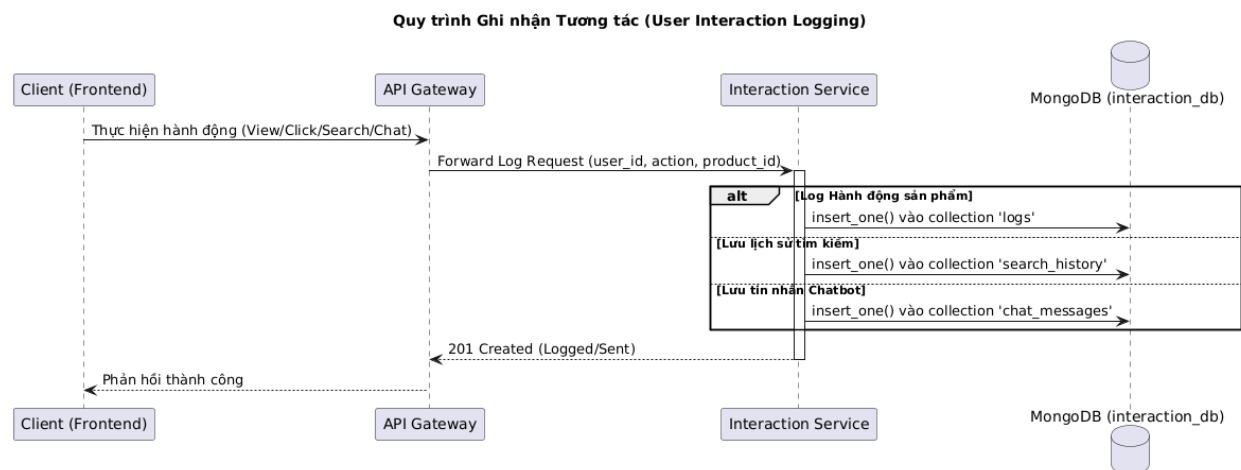
Dịch vụ sử dụng thư viện pymongo trực tiếp để tương tác với 04 Collection chính trong interaction_db:

- logs: Lưu các hành động cơ bản (View, Click, Add to Cart).
- search_history: Lưu các từ khóa tìm kiếm của người dùng.
- chat_messages: Lưu lịch sử hội thoại với AI Consultant.
- wishlist: Lưu danh sách sản phẩm yêu thích (thay vì lưu trong SQL như các hệ thống cũ).

As a final-year IT student at PTIT, I have proactively mastered **Java, OOP, and the Spring Framework** by researching and developing complex backend systems. My goal is to apply this technical foundation to real-world projects, making long-term

2.9.2 Workflow

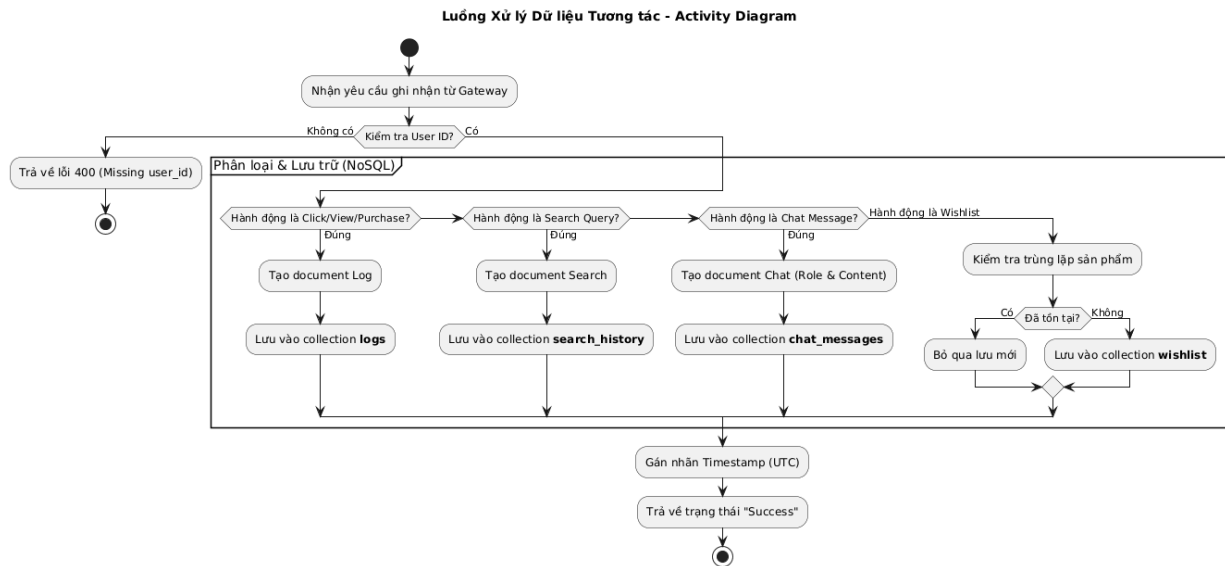
Luồng hoạt động chính của Interaction Service: Quy trình ghi nhận tương tác



Hình 2.24. Biểu đồ tuần tự luồng ghi nhận tương tác

Mô tả Sequence Diagram:

1. Hành động ngầm: Quy trình này thường diễn ra ngầm (asynchronous) từ phía Frontend. Khi người dùng xem một cuốn sách hoặc tìm kiếm, một request sẽ được gửi đi mà không làm gián đoạn trải nghiệm người dùng.
2. Đa dạng lưu trữ: Tùy vào loại hành động mà interaction-service sẽ phân phối dữ liệu vào các "Collection" khác nhau trong MongoDB. Điều này giúp việc truy vấn sau này (ví dụ: lấy lịch sử chat cho Chatbot) trở nên cực kỳ nhanh chóng.
3. Dữ liệu thô cho AI: Các bản ghi trong logs chính là "thức ăn" cho dịch vụ recommender-ai-service để huấn luyện các mô hình như LSTM hay RNN.



Hình 2.25. Biểu đồ hoạt động luồng ghi nhận tương tác

2.9.3 API (Endpoints)

Endpoint	Method	Ý nghĩa
/interactions/	POST	Ghi log hành vi (Payload: action_type, user_id, product_id)
/interactions/history/{user_id}/	GET	Lấy lịch sử hành vi của người dùng
/search/{user_id}/	POST/GET	Quản lý lịch sử tìm kiếm cá nhân
/chat/{user_id}/	POST/GET	Lưu và lấy lịch sử chat với AI
/wishlist/{user_id}/	POST/GET/DEL	Quản lý danh sách sản phẩm yêu thích

2.10 Thiết kế Comment Rate Service

Dịch vụ này quản lý các nội dung do người dùng tạo ra (UGC - User Generated Content), cung cấp bằng chứng xã hội (Social Proof) cho các sản phẩm.

2.10.1. Model

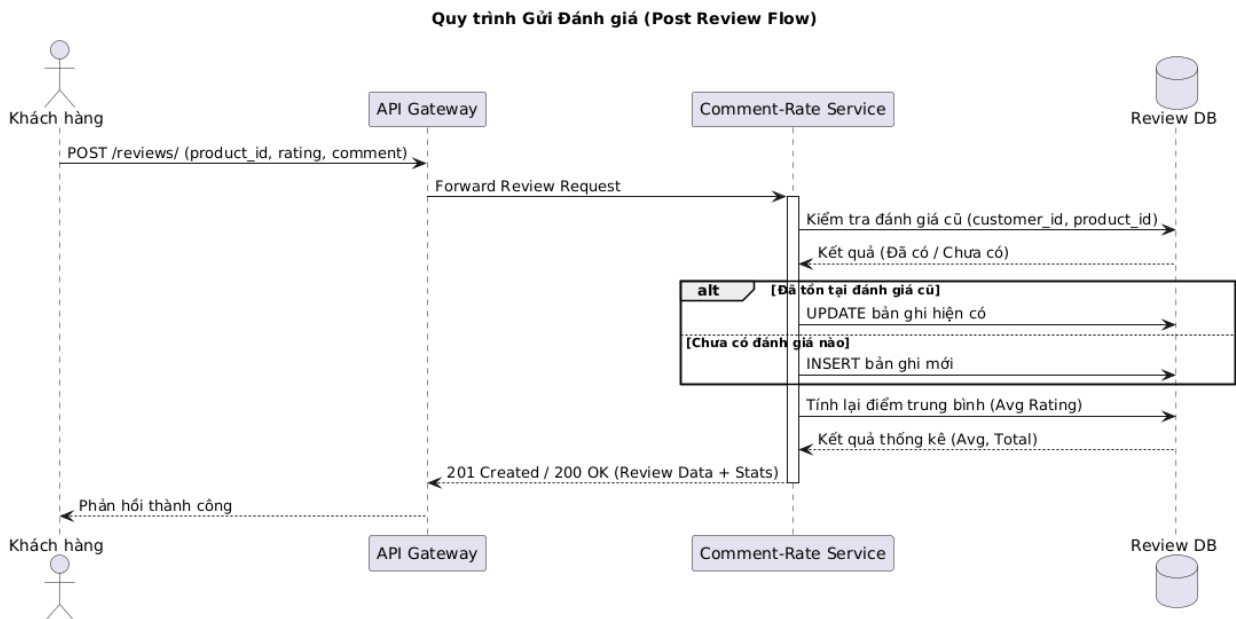
Hệ thống sử dụng mô hình quan hệ để cho phép đánh giá sản phẩm và phản hồi (Reply) giữa người dùng với nhau hoặc giữa chủ shop với khách hàng.

```
class CommentRate(models.Model):
    user_id = models.IntegerField()
    product_id = models.IntegerField()
    rating = models.IntegerField(default=5) # 1-5 sao
    comment = models.TextField()
    parent = models.ForeignKey('self', null=True, blank=True, on_delete=models.CASCADE) # Phục vụ Reply
    created_at = models.DateTimeField(auto_now_add=True)
```

- Rating: điểm đánh giá của khách hàng (đã mua hàng) cho 1 sản phẩm (1-5)
- Comment: nội dung đánh giá của review

2.10.2. Workflow

Luồng hoạt động chính

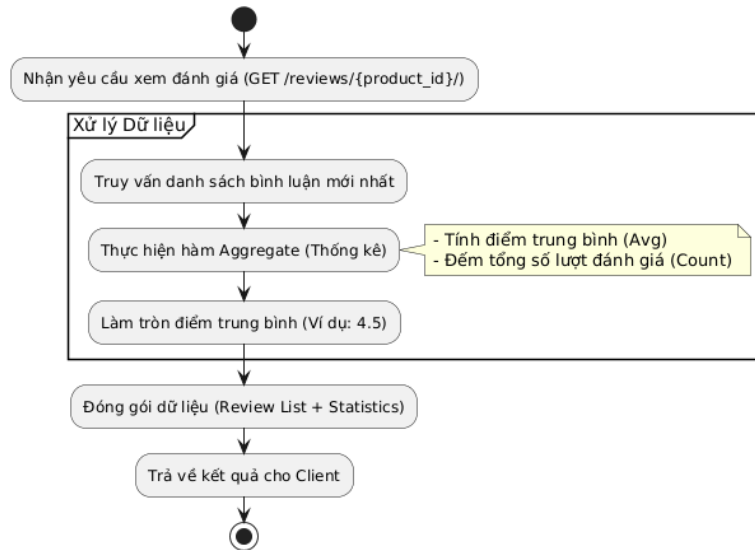


Hình 2.26. Biểu đồ tuần tự luồng đánh giá

Mô tả Sequence Diagram:

1. Cơ chế Upsert: Thay vì báo lỗi "Bạn đã đánh giá rồi", hệ thống sử dụng logic `update_or_create`. Nếu người dùng gửi đánh giá mới cho sản phẩm đã mua, hệ thống sẽ tự động cập nhật lại điểm số và nội dung bình luận cũ.
2. Dữ liệu đồng bộ: Ngay khi một đánh giá được gửi lên, hệ thống sẽ trả về luôn thông tin thống kê mới nhất (điểm trung bình mới), giúp giao diện người dùng cập nhật tức thì.

Luồng Xem & Thống kê Đánh giá - Activity Diagram



Hình 2.27. Biểu đồ hoạt động luồng đánh giá

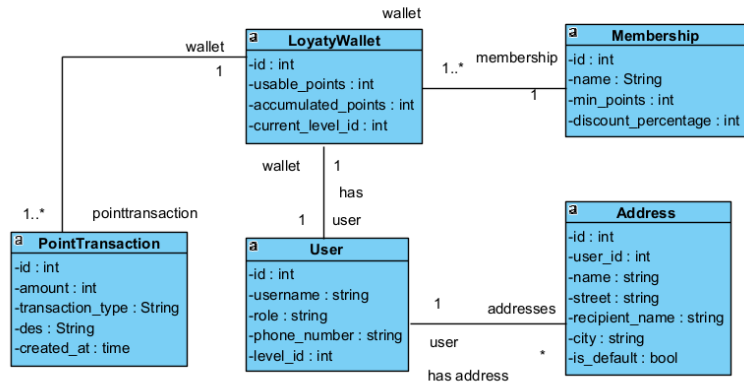
2.10.3. API

Endpoint	Method	Ý nghĩa
/comments/	GET	Lấy danh sách bình luận (Có thể lọc theo product_id)
/comments/	POST	Đăng bình luận hoặc đánh giá mới
/comments/{id}/	DELETE	Xóa bình luận (Chỉ Admin hoặc chính chủ)
/comments/product/{product_id}/stats/	GET	Lấy thống kê trung bình sao (ví dụ: 4.5/5 sao) của sản phẩm

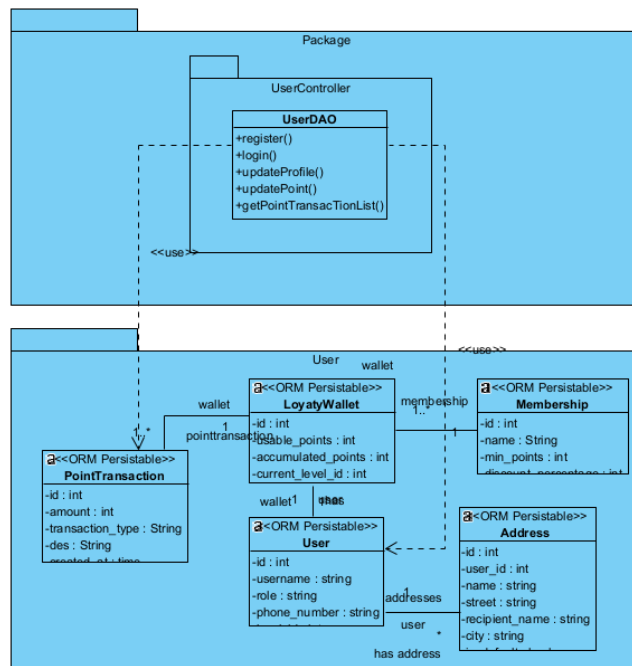
2.11 Bài tập thực hành

2.11.1. Biểu đồ lớp, Datamodel

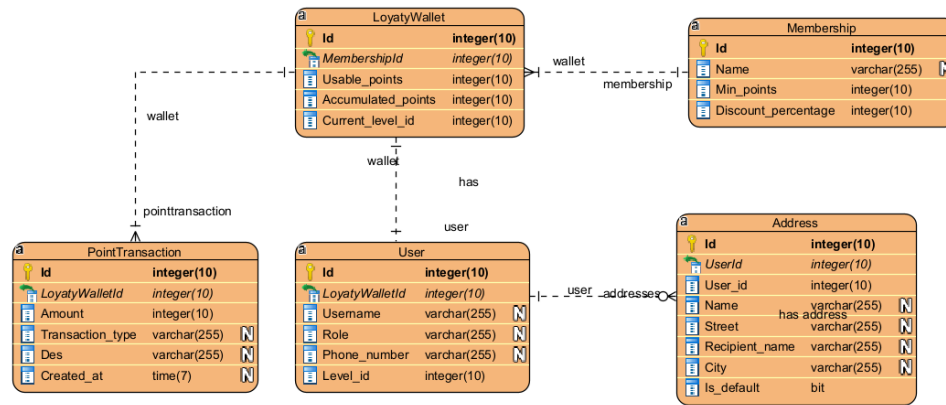
* User Service



Hình 2.28. Biểu đồ lớp UserService

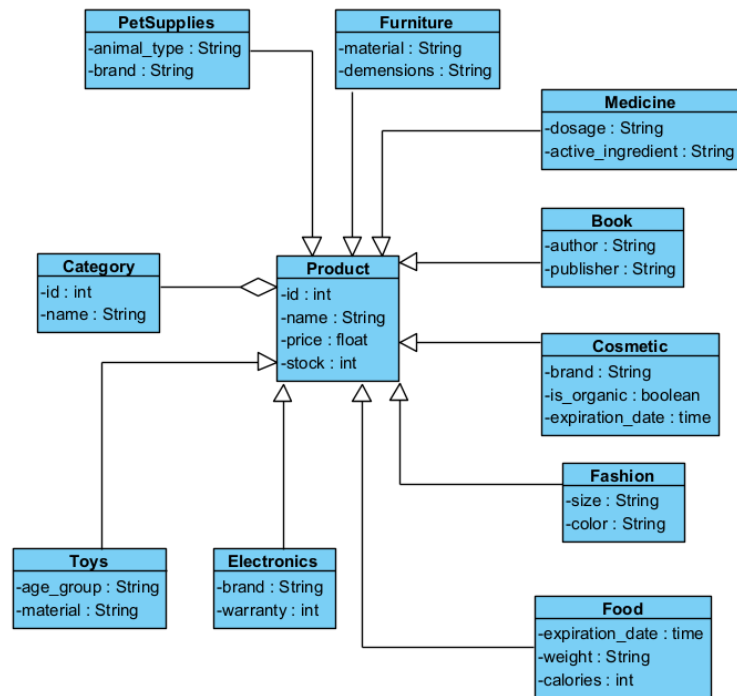


Hình 2.29. Biểu đồ thiết kế lớp UserService

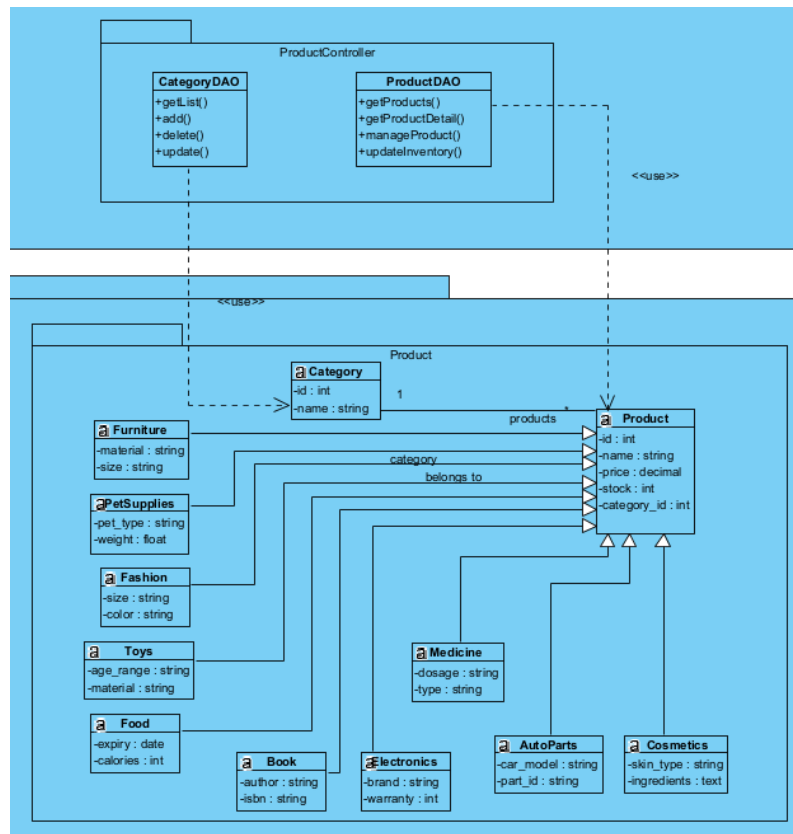


Hình 2.30. Biểu đồ Datamodel UserService

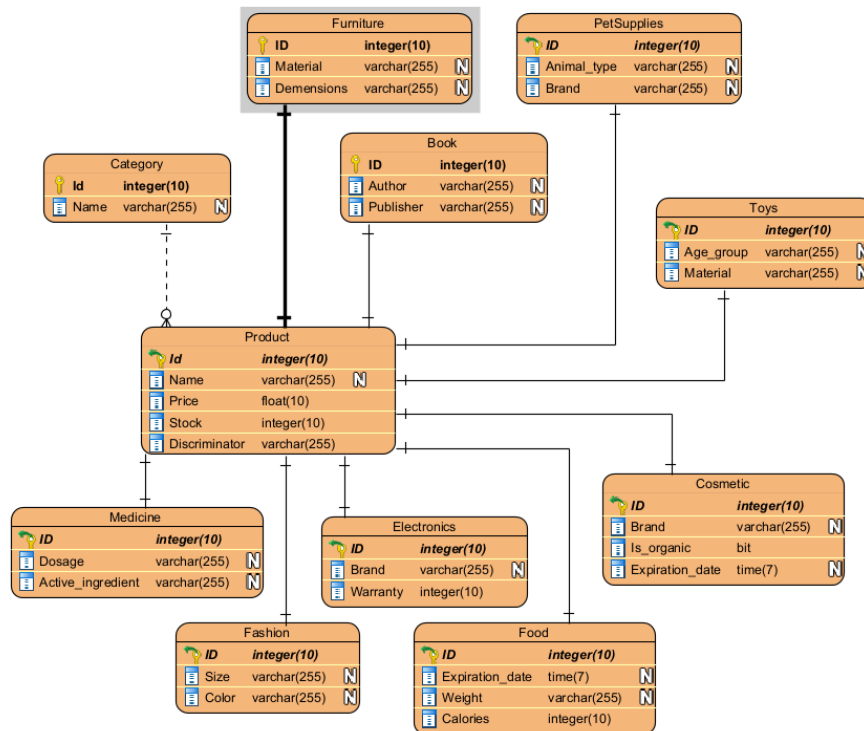
* Product Service



Hình 2.31. Biểu đồ lớp ProductService

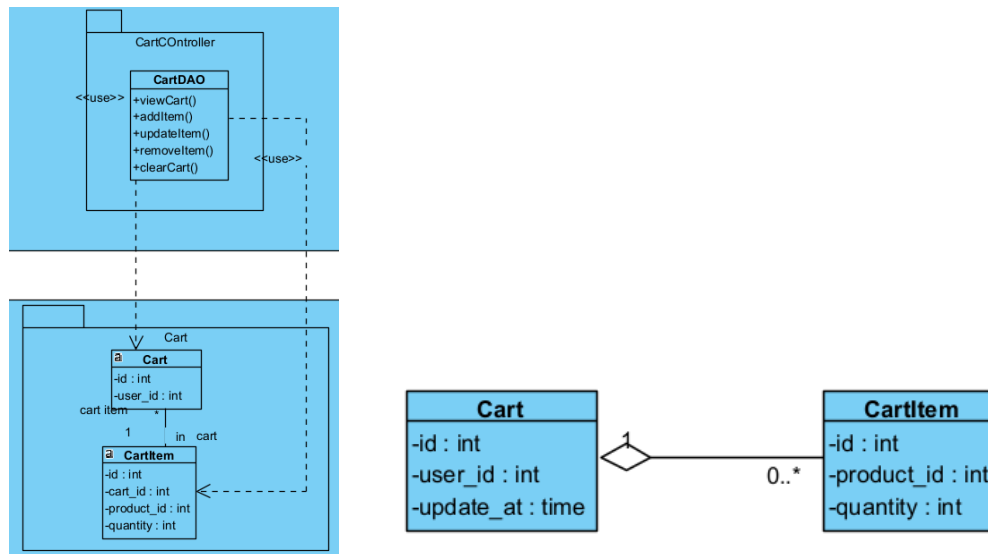


Hình 2.32. Biểu đồ thiết kế lớp Product Service

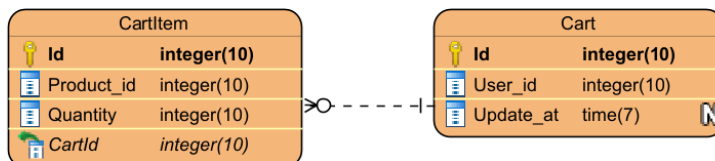


Hình 2.33. Biểu đồ Datamodel ProductService

* Cart Service

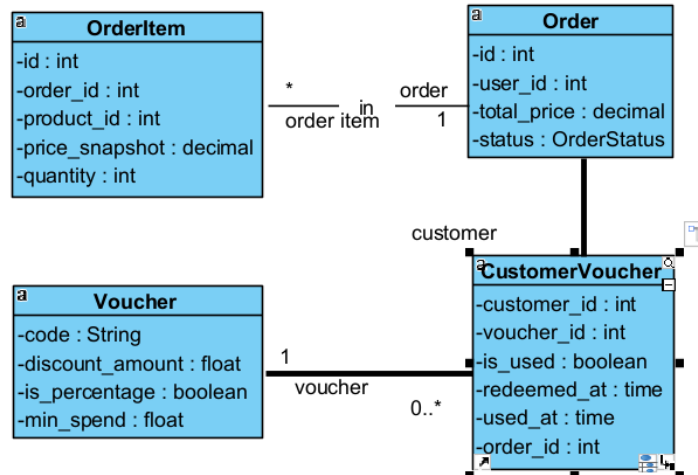


Hình 2.34 Biểu đồ lớp CartService

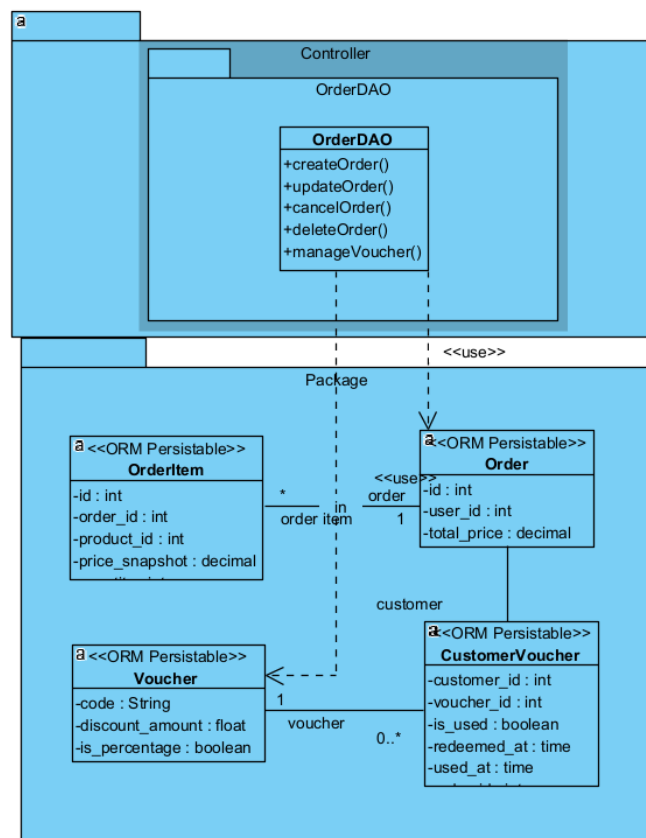


Hình 2.35. Biểu đồ Datamodel CartService

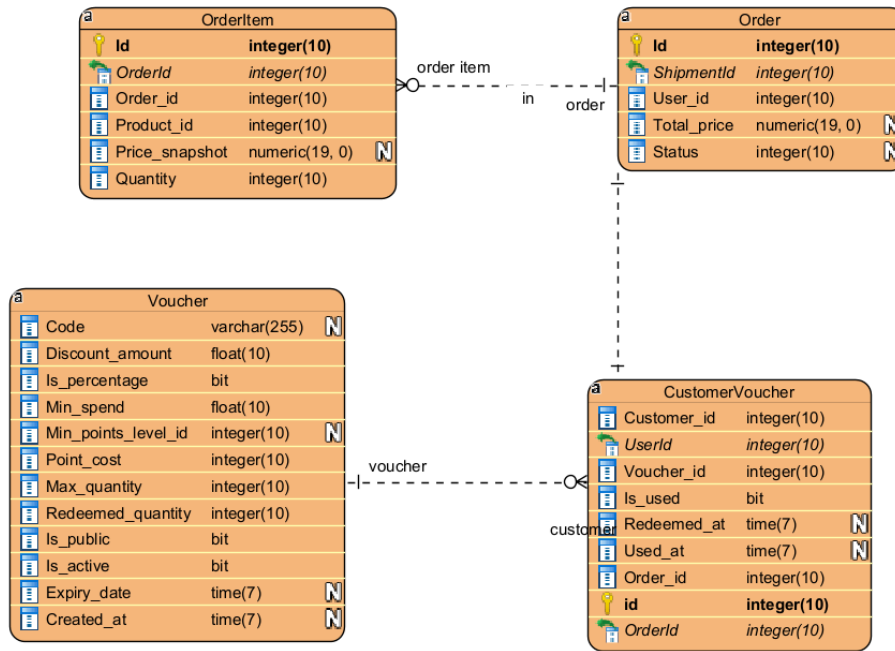
* Order Service



Hình 2.36. Biểu đồ lớp OrderService

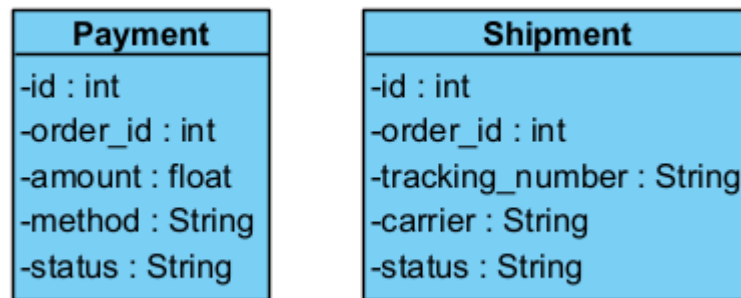


Hình 2.37. Biểu đồ thiết kế lớp OrderService

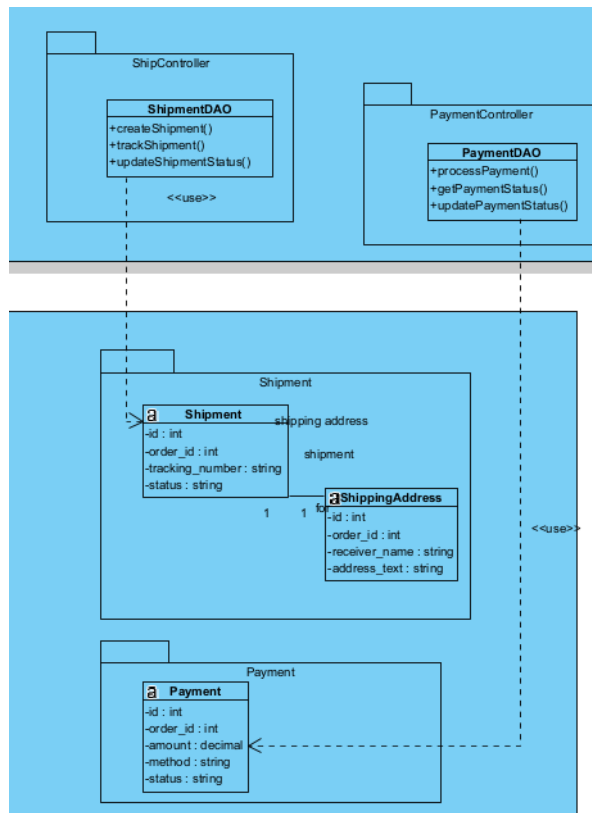


Hình 2.38. . Biểu đồ Datamodel OrderService

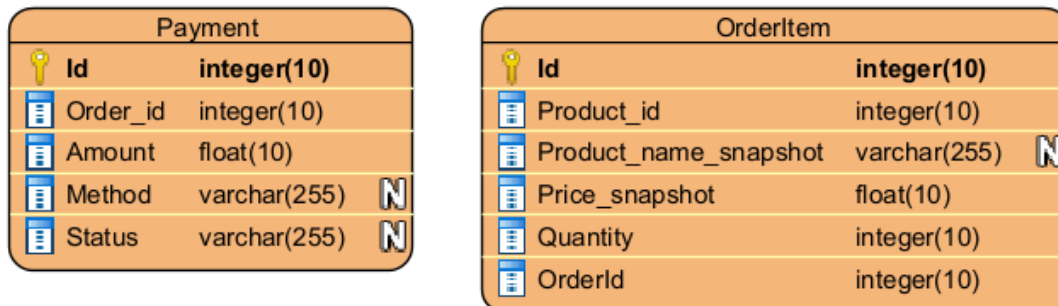
* Payment Service



Hình 2.39. Biểu đồ lớp PayService

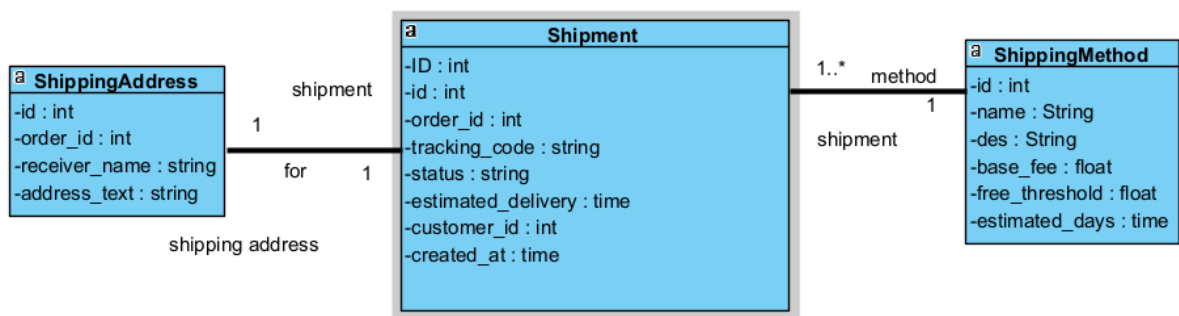


Hình 2.40. Biểu đồ thiết kế lớp PayService

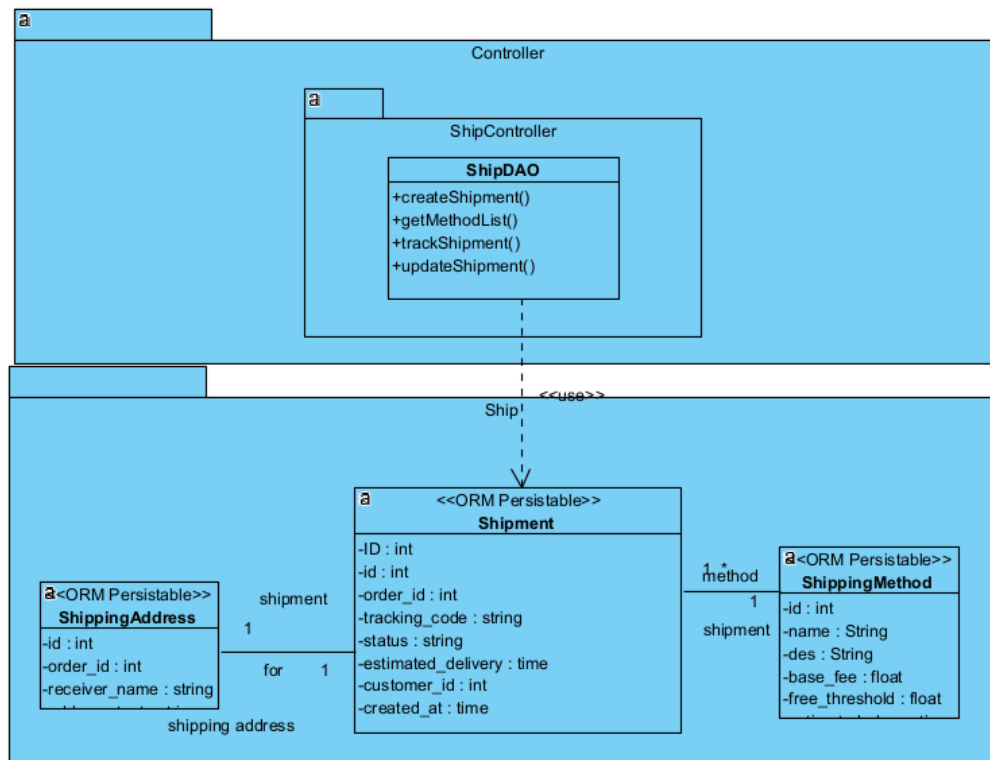


Hình 2.41. Biểu đồ Datamodel PayService

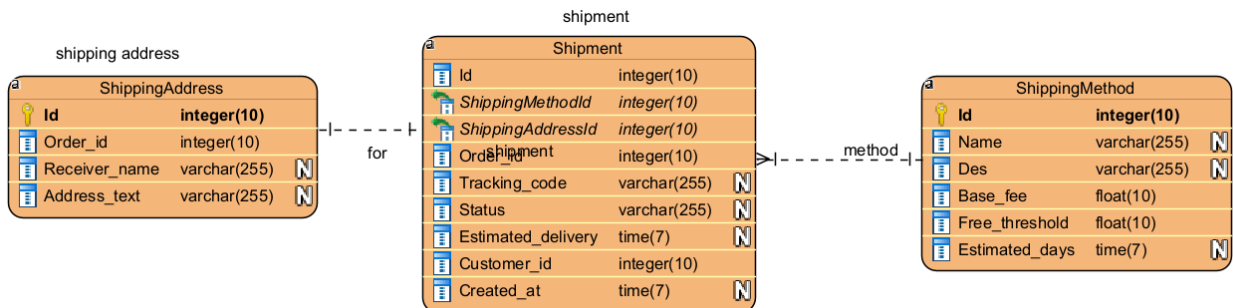
* ShipService



Hình 2.42., Biểu đồ lớp ShipService

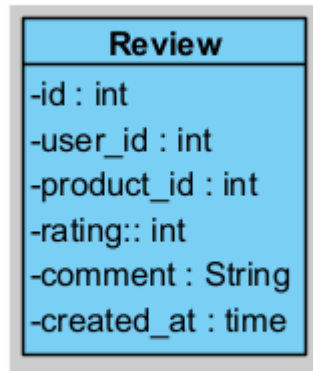


Hình 2.43. Biểu đồ thiết kế lớp ShipService

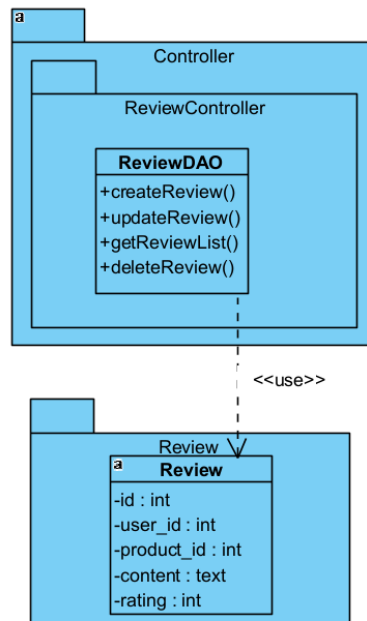


Hình 2.44. Biểu đồ Datamodel ShipService

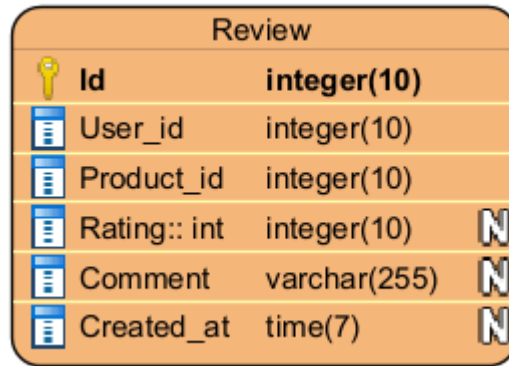
* Comment Rate Service



Hình 2.45. Biểu đồ lớp *CommentService*



Hình 2.46. Biểu đồ thiết kế lớp *ReviewService*



Hình 2.47. Biểu đồ Datamodel ReviewService

2.11.2. Mapping Database

1. User Service (Xác thực và Phân quyền)

- **Lựa chọn: MySQL**
- **Lý do:** Tốc độ đọc (Read) cực nhanh, phù hợp cho việc kiểm tra quyền hạn (Admin/Staff/Customer) và xác thực JWT liên tục.

```
CREATE TABLE MembershipLevel (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(50) UNIQUE NOT NULL,
  min_points INT DEFAULT 0,
  discount_percentage INT DEFAULT 0
);
```

```
CREATE TABLE User (
  id INT AUTO_INCREMENT PRIMARY KEY,
  password VARCHAR(128) NOT NULL,
  last_login DATETIME NULL,
  is_superuser BOOLEAN NOT NULL DEFAULT FALSE,
  username VARCHAR(150) UNIQUE NOT NULL,
  first_name VARCHAR(150) NOT NULL,
  last_name VARCHAR(150) NOT NULL,
  email VARCHAR(254) NOT NULL,
  is_staff BOOLEAN NOT NULL DEFAULT FALSE,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  date_joined DATETIME NOT NULL,
  role VARCHAR(10) DEFAULT 'CUSTOMER',
  phone_number VARCHAR(20) NULL
);
```

```
CREATE TABLE LoyaltyWallet (
  id INT AUTO_INCREMENT PRIMARY KEY,
```

```

        usable_points INT DEFAULT 0,
        accumulated_points INT DEFAULT 0,
        current_level_id INT NULL,
        user_id INT UNIQUE NOT NULL,
        FOREIGN KEY (user_id) REFERENCES User(id) ON DELETE CASCADE,
        FOREIGN KEY (current_level_id) REFERENCES MembershipLevel(id) ON DELETE SET
NULL
    );

```

```

CREATE TABLE PointTransaction (
    id INT AUTO_INCREMENT PRIMARY KEY,
    amount INT NOT NULL,
    transaction_type VARCHAR(10) NOT NULL,
    description VARCHAR(255) DEFAULT '',
    created_at DATETIME NOT NULL,
    wallet_id INT NOT NULL,
    FOREIGN KEY (wallet_id) REFERENCES LoyaltyWallet(id) ON DELETE CASCADE
);

```

```

CREATE TABLE Address (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    recipient_name VARCHAR(255) NULL,
    recipient_phone VARCHAR(20) NULL,
    street VARCHAR(255) NOT NULL,
    city VARCHAR(100) NOT NULL,
    country VARCHAR(100) DEFAULT 'Vietnam',
    postal_code VARCHAR(20) DEFAULT '',
    is_default BOOLEAN DEFAULT FALSE,
    user_id INT NOT NULL,
    FOREIGN KEY (user_id) REFERENCES User(id) ON DELETE CASCADE
);

```

2. Product Service (Danh mục đa lĩnh vực)

- **Lựa chọn: PostgreSQL**
- **Lý do:** Hỗ trợ mạnh mẽ việc kế thừa bảng (Inheritance) và kiểu dữ liệu JSONB cho 10 domain sản phẩm khác nhau.

```

CREATE TABLE Category (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    description TEXT NULL
);

```

```

CREATE TABLE Product (
    id SERIAL PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    description TEXT NULL,

```

```

    price DECIMAL(10, 2) NOT NULL,
    stock INT DEFAULT 0,
    image_url VARCHAR(500) NULL,
    attributes JSONB DEFAULT '{}',
    category_id INT NULL,
    FOREIGN KEY (category_id) REFERENCES Category(id) ON DELETE CASCADE
);

CREATE TABLE Book (
    id SERIAL PRIMARY KEY,
    author VARCHAR(255) NOT NULL,
    publisher VARCHAR(255) NOT NULL,
    isbn VARCHAR(20) NOT NULL,
    product_id INT UNIQUE NOT NULL,
    FOREIGN KEY (product_id) REFERENCES Product(id) ON DELETE CASCADE
);

CREATE TABLE Electronics (
    id SERIAL PRIMARY KEY,
    brand VARCHAR(100) NOT NULL,
    warranty INT NOT NULL,
    product_id INT UNIQUE NOT NULL,
    FOREIGN KEY (product_id) REFERENCES Product(id) ON DELETE CASCADE
);

CREATE TABLE Fashion (
    id SERIAL PRIMARY KEY,
    size VARCHAR(10) NOT NULL,
    color VARCHAR(50) NOT NULL,
    product_id INT UNIQUE NOT NULL,
    FOREIGN KEY (product_id) REFERENCES Product(id) ON DELETE CASCADE
);

CREATE TABLE Cosmetics (
    id SERIAL PRIMARY KEY,
    brand VARCHAR(100) NOT NULL,
    skin_type VARCHAR(50) NOT NULL,
    is_organic BOOLEAN DEFAULT FALSE,
    expiration_date DATE NULL,
    product_id INT UNIQUE NOT NULL,
    FOREIGN KEY (product_id) REFERENCES Product(id) ON DELETE CASCADE
);

CREATE TABLE Toys (
    id SERIAL PRIMARY KEY,
    age_group VARCHAR(50) NOT NULL,
    material VARCHAR(100) NOT NULL,

```

```

        requires_batteries BOOLEAN DEFAULT FALSE,
        product_id INT UNIQUE NOT NULL,
        FOREIGN KEY (product_id) REFERENCES Product(id) ON DELETE CASCADE
    );

CREATE TABLE Furniture (
    id SERIAL PRIMARY KEY,
    material VARCHAR(100) NOT NULL,
    dimensions VARCHAR(100) NOT NULL,
    weight_capacity FLOAT NULL,
    product_id INT UNIQUE NOT NULL,
    FOREIGN KEY (product_id) REFERENCES Product(id) ON DELETE CASCADE
);

CREATE TABLE Food (
    id SERIAL PRIMARY KEY,
    expiration_date DATE NOT NULL,
    weight VARCHAR(50) NOT NULL,
    is_vegetarian BOOLEAN DEFAULT FALSE,
    calories INT NULL,
    product_id INT UNIQUE NOT NULL,
    FOREIGN KEY (product_id) REFERENCES Product(id) ON DELETE CASCADE
);

CREATE TABLE Medicine (
    id SERIAL PRIMARY KEY,
    active_ingredient VARCHAR(255) NOT NULL,
    dosage VARCHAR(100) NOT NULL,
    prescription_required BOOLEAN DEFAULT FALSE,
    product_id INT UNIQUE NOT NULL,
    FOREIGN KEY (product_id) REFERENCES Product(id) ON DELETE CASCADE
);

CREATE TABLE PetSupplies (
    id SERIAL PRIMARY KEY,
    animal_type VARCHAR(50) NOT NULL,
    brand VARCHAR(100) NOT NULL,
    weight_limit FLOAT NULL,
    product_id INT UNIQUE NOT NULL,
    FOREIGN KEY (product_id) REFERENCES Product(id) ON DELETE CASCADE
);

CREATE TABLE AutoParts (
    id SERIAL PRIMARY KEY,
    part_number VARCHAR(100) NOT NULL,
    car_model_compatibility TEXT NOT NULL,
    warranty_years INT DEFAULT 1,

```

```

        product_id INT UNIQUE NOT NULL,
        FOREIGN KEY (product_id) REFERENCES Product(id) ON DELETE CASCADE
    );

```

3. Cart Service (Giỏ hàng tạm thời)

- **Lựa chọn: PostgreSQL**
- **Lý do:** Đảm bảo tính toàn vẹn dữ liệu khi khách hàng cập nhật số lượng vật phẩm liên tục.

```

CREATE TABLE Cart (
    id SERIAL PRIMARY KEY,
    customer_id INT NOT NULL
);

```

```

CREATE TABLE CartItem (
    id SERIAL PRIMARY KEY,
    product_id INT NOT NULL,
    quantity INT NOT NULL,
    cart_id INT NOT NULL,
    FOREIGN KEY (cart_id) REFERENCES Cart(id) ON DELETE CASCADE
);

```

4. Order Service (Quản lý Đơn hàng & Snapshot)

- **Lựa chọn: PostgreSQL**
- **Lý do:** Khả năng xử lý giao dịch (Transaction) phức tạp và lưu trữ chính xác các bản chụp (Snapshot) thông tin sản phẩm.

```

CREATE TABLE Voucher (
    id SERIAL PRIMARY KEY,
    code VARCHAR(100) UNIQUE NOT NULL,
    discount_amount DECIMAL(10, 2) NOT NULL,
    is_percentage BOOLEAN DEFAULT FALSE,
    min_spend DECIMAL(10, 2) DEFAULT 0,
    min_points_level_id INT NULL,
    point_cost INT DEFAULT 0,
    max_quantity INT DEFAULT 100,
    redeemed_quantity INT DEFAULT 0,
    is_public BOOLEAN DEFAULT TRUE,
    is_active BOOLEAN DEFAULT TRUE,
    expiry_date TIMESTAMP NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

```

```

CREATE TABLE "Order" (
    id SERIAL PRIMARY KEY,
    customer_id INT NOT NULL,

```

```

total_amount DECIMAL(12, 2) DEFAULT 0,
status VARCHAR(20) DEFAULT 'pending',
membership_discount DECIMAL(12, 2) DEFAULT 0,
voucher_discount DECIMAL(12, 2) DEFAULT 0,
voucher_code VARCHAR(100) DEFAULT '',
points_generated INT DEFAULT 0,
shipping_address TEXT DEFAULT '',
shipping_fee DECIMAL(10, 2) DEFAULT 0,
shipping_method VARCHAR(50) DEFAULT '',
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE OrderItem (
    id SERIAL PRIMARY KEY,
    product_id INT NOT NULL,
    product_name VARCHAR(255) DEFAULT '',
    item_image_url TEXT DEFAULT '',
    quantity INT NOT NULL,
    unit_price DECIMAL(10, 2) NOT NULL,
    order_id INT NOT NULL,
    FOREIGN KEY (order_id) REFERENCES "Order"(id) ON DELETE CASCADE
);

CREATE TABLE CustomerVoucher (
    id SERIAL PRIMARY KEY,
    customer_id INT NOT NULL,
    is_used BOOLEAN DEFAULT FALSE,
    redeemed_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    used_at TIMESTAMP NULL,
    order_id INT NULL,
    voucher_id INT NOT NULL,
    FOREIGN KEY (voucher_id) REFERENCES Voucher(id) ON DELETE CASCADE
);

CREATE TABLE OrderStatusLog (
    id SERIAL PRIMARY KEY,
    status VARCHAR(50) NOT NULL,
    notes TEXT DEFAULT '',
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    order_id INT NOT NULL,
    FOREIGN KEY (order_id) REFERENCES "Order"(id) ON DELETE CASCADE
);

```

5. Payment & Shipment Service (Thanh toán & Vận chuyển)

- **Lựa chọn: PostgreSQL**
- **Lý do:** Độ tin cậy cao, hỗ trợ tốt các báo cáo tài chính và theo dõi vận đơn.


```

CREATE TABLE Payment (
    id SERIAL PRIMARY KEY,
    order_id INT NOT NULL,
    customer_id INT NOT NULL,
    amount DECIMAL(12, 2) NOT NULL,
    method VARCHAR(20) DEFAULT 'COD',
    status VARCHAR(20) DEFAULT 'processing',
    transaction_id VARCHAR(100) UNIQUE NOT NULL,
    note TEXT DEFAULT '',
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE ShippingMethod (
    id_slug VARCHAR(50) PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    description TEXT DEFAULT '',
    base_fee FLOAT NOT NULL,
    free_threshold FLOAT NULL,
    estimated_days INT DEFAULT 3
);

CREATE TABLE Shipment (
    id SERIAL PRIMARY KEY,
    order_id INT UNIQUE NOT NULL,
    customer_id INT NOT NULL,
    tracking_code VARCHAR(50) UNIQUE NOT NULL,
    shipping_method VARCHAR(50) DEFAULT 'standard',
    address TEXT NOT NULL,
    status VARCHAR(20) DEFAULT 'ready_for_pickup',
    estimated_delivery TIMESTAMP NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

```

6. Comment Rate Service (Đánh giá & Phản hồi)

- **Lựa chọn: PostgreSQL**
- **Lý do:** Hỗ trợ Indexing tốt cho nội dung Text, giúp tìm kiếm và hiển thị đánh giá nhanh chóng.

```

CREATE TABLE Review (
    id SERIAL PRIMARY KEY,
    customer_id INT NOT NULL,
    product_id INT NOT NULL,
    rating INT NOT NULL,
    comment TEXT DEFAULT '',
    customer_name VARCHAR(255) DEFAULT 'User',

```

```

        created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
        UNIQUE (customer_id, product_id)
    );

```

7. Interaction Service

- **Lựa chọn: MongoDB**
- **Lý do:** khả năng lưu trữ dữ liệu phi cấu trúc với tốc độ ghi cực cao, cho phép hệ thống thu thập hàng triệu hành vi người dùng (Big Data) mỗi ngày mà không bị giới hạn bởi lược đồ bảng (schema) cứng nhắc như SQL.

```

{
  "database": "interaction_db",
  "type": "MongoDB (NoSQL)",
  "collections": {
    "logs": {
      "description": "Stores user interaction events such as viewing products,
clicking links, or adding items to cart.",
      "schema": {
        "_id": "ObjectId",
        "user_id": "Integer (Required: ID of the user performing the action)",
        "action": "String (Required: Type of action, e.g., 'view_product',
'add_to_cart', 'purchase')",
        "product_id": "Integer (Optional: ID of the product interacted with)",
        "timestamp": "Datetime (UTC: Time when the action occurred)"
      }
    },
    "search_history": {
      "description": "Stores the search queries made by users to improve
recommendations and track trends.",
      "schema": {
        "_id": "ObjectId",
        "user_id": "Integer (Required: ID of the searching user)",
        "query": "String (Required: The text searched by the user)",
        "timestamp": "Datetime (UTC: Time of the search)"
      }
    },
    "chat_messages": {
      "description": "Stores conversation history between the user and the AI
Recommender chatbot.",
      "schema": {
        "_id": "ObjectId",
        "user_id": "Integer (Required: ID of the user in the chat)",
        "role": "String (Required: 'user' or 'assistant' based on who sent the
message)",

```

```

        "content": "String (Required: The actual message content)",
        "timestamp": "Datetime (UTC: Time the message was sent)"
    },
    "wishlist": {
        "description": "Stores products that a user has saved for later
(Wishlist/Favorites).",
        "schema": {
            "_id": "ObjectId",
            "user_id": "Integer (Required: ID of the user)",
            "product_id": "Integer (Required: ID of the saved product)",
            "added_at": "Datetime (UTC: Time when the product was added to wishlist)"
        }
    }
}
}

```

2.10.3. So sánh MySQL, PostgreSQL

Tiêu chí so sánh	MySQL	PostgreSQL
Loại hình	Hệ quản trị CSDL Quan hệ (RDBMS)	Hệ quản trị CSDL Đối tượng - Quan hệ (ORDBMS)
Tính năng nổi bật	Tốc độ đọc (Read-heavy) cực nhanh, đơn giản.	Tính năng phong phú, hỗ trợ các kiểu dữ liệu phức tạp.
Kiểu dữ liệu	Hỗ trợ các kiểu dữ liệu cơ bản (Int, Varchar, Date...).	Rất đa dạng: JSONB, XML, Mảng (Array), Hình học, IP.
Khả năng mở rộng	Hạn chế trong việc tạo các kiểu dữ liệu tùy chỉnh.	Rất mạnh, cho phép tạo hàm, kiểu dữ liệu, index tùy chỉnh.
Tuân thủ ACID	Chỉ đảm bảo khi dùng Storage Engine là InnoDB.	Tuân thủ ACID tuyệt đối ngay từ thiết kế lỗi.
Tính kế thừa	Không hỗ trợ kế thừa bảng.	Hỗ trợ kế thừa bảng (Cực kỳ hữu ích cho Product Service).
Xử lý đồng thời	Sử dụng cơ chế khóa (Locking) đơn giản hơn.	Sử dụng MVCC (Multi-Version Concurrency Control) tiên tiến.
Tìm kiếm văn bản	Hỗ trợ Full-text search ở mức cơ bản.	Hỗ trợ Full-text search chuyên sâu với đánh chỉ mục GIN/GiST.
Cộng đồng & Phổ biến	Cực kỳ phổ biến, dễ học, tài liệu phong phú.	Cộng đồng chuyên gia, dành cho hệ thống phức tạp, ổn định.

2.11 Kết luận

Sau quá trình nghiên cứu và triển khai hệ thống, có thể rút ra những kết luận quan trọng về sự kết hợp giữa kiến trúc hệ thống và phương pháp luận thiết kế:

1. Sự linh hoạt tối ưu từ kiến trúc **Microservices**

Việc áp dụng kiến trúc **Microservices** đã chứng minh được giá trị cốt lõi trong việc xây dựng một hệ thống hiện đại. Thay vì một khối mã nguồn duy nhất (Monolith) công kênh, hệ thống được chia nhỏ thành các dịch vụ độc lập, giúp:

- Khả năng mở rộng (Scalability): Dễ dàng nâng cấp riêng lẻ các dịch vụ có tải cao như *AI Recommendation* hay *Search* mà không ảnh hưởng đến các phần khác.
- Tính sẵn sàng cao (High Availability): Sự cố tại một dịch vụ (ví dụ: Feedback Service) không làm tê liệt toàn bộ hệ thống bán hàng, đảm bảo trải nghiệm người dùng không bị gián đoạn.
- Đa dạng hóa công nghệ: Cho phép tích hợp linh hoạt nhiều loại cơ sở dữ liệu như PostgreSQL cho nghiệp vụ và Neo4j cho đồ thị tri thức.

2. Tốc độ và Hiệu quả của Framework **Django**

Django đóng vai trò là "xương sống" cho các dịch vụ đòi hỏi sự ổn định và tốc độ triển khai nhanh (Rapid Development).

- Với triết lý "*The web framework for perfectionists with deadlines*", Django cung cấp đầy đủ các công cụ bảo mật, quản trị (Admin) và ORM mạnh mẽ, giúp đội ngũ phát triển tập trung vào logic nghiệp vụ thay vì các tác vụ hạ tầng lặp lại.
- Khả năng tương thích tốt với các thư viện học máy (Machine Learning) trong hệ sinh thái Python giúp việc tích hợp mô hình BiLSTM và quy trình RAG trở nên mượt mà và hiệu quả hơn.

3. Tầm quan trọng của Thiết kế hướng tên miền (DDD)

Phương pháp **Domain-Driven Design (DDD)** chính là "la bàn" giúp hệ thống được thiết kế đúng hướng ngay từ giai đoạn sơ khởi:

- Phân tách ranh giới (Bounded Contexts): Giúp xác định rõ ràng trách nhiệm của từng Microservice, tránh sự chồng chéo logic và giảm thiểu sự liên kết chặt chẽ (Tight Coupling) giữa các thực thể sản phẩm (10 Domains).
- Ngôn ngữ chung (Ubiquitous Language): Tạo ra sự thống nhất giữa đội ngũ kỹ thuật và nghiệp vụ bán lẻ, đảm bảo các cấu trúc dữ liệu trong Code phản ánh chính xác các quy trình vận hành thực tế tại hiệu sách.
- Tính bền vững: Thiết kế theo mô hình DDD giúp mã nguồn dễ bảo trì, dễ đọc và sẵn sàng thích ứng với các thay đổi phức tạp trong tương lai của doanh nghiệp.

Tổng kết: Sự kết hợp đồng bộ giữa **Microservices**, sức mạnh của **Django** và tư duy thiết kế **DDD** đã tạo nên một hệ thống tư vấn hiệu sách thông minh không chỉ mạnh mẽ về mặt công nghệ mà còn tối ưu về mặt quản trị và vận hành thực tiễn.

CHƯƠNG 3. AI SERVICE CHO TƯ VẤN SẢN PHẨM

3.1 Mục tiêu

Trong kiến trúc Microservices của nền tảng E-commerce, hệ thống AI đóng vai trò là một miền nghiệp vụ cốt lõi (Core Domain) giúp tạo ra lợi thế cạnh tranh vượt trội. Thay vì chỉ truy xuất dữ liệu thụ động, recommend-ai service hoạt động độc lập, liên tục học hỏi từ các luồng dữ liệu (data streams) để cá nhân hóa trải nghiệm người dùng theo thời gian thực. Mục tiêu cốt lõi của phần hệ thống này là xây dựng một bộ máy AI gợi ý sản phẩm thông minh và một Chatbot tư vấn tương tác, dựa trên việc tổng hợp và phân tích ba nguồn dữ liệu đầu vào quan trọng:

3.1.1 Đầu vào của hệ thống AI (Input & Context)

1. Xây dựng gợi ý dựa trên Hành vi người dùng (User Behavior)

- Bản chất kỹ thuật: Trong môi trường phân tán, các hành vi vi mô như xem sản phẩm (click/view), tìm kiếm (search), hoặc thêm vào giỏ hàng (add-to-cart) sinh ra một lượng dữ liệu khổng lồ. Hệ thống AI không truy vấn trực tiếp vào cơ sở dữ liệu của các service khác, mà lắng nghe các Sự kiện miền (Domain Events) được phát ra từ interaction service và cart service thông qua Message Broker (như Kafka).
- Mục tiêu nghiệp vụ: Thu thập các sự kiện theo chuỗi thời gian (time-series events) để xây dựng chân dung khách hàng (Customer Profile). Thuật toán học máy (Machine Learning) sẽ phân tích các hành vi này để hiểu được sở thích, độ nhạy cảm về giá và xu hướng mua sắm của từng cá nhân (ví dụ: khách hàng thường xuyên bỏ giỏ hàng khi phí ship cao, hoặc khách hàng có xu hướng click vào các sản phẩm mỹ phẩm Hàn Quốc).

2. Xây dựng gợi ý dựa trên Quan hệ (Similarity)

- Bản chất kỹ thuật: AI Service sẽ đồng bộ danh mục sản phẩm từ productservice thông qua các sự kiện cập nhật để xây dựng một cơ sở dữ liệu đồ thị (Graph Database) hoặc cơ sở dữ liệu vector.
- Mục tiêu nghiệp vụ: Phân tích tập hợp các thuộc tính của sản phẩm (metadata) để tìm ra sự tương đồng (Content-based filtering). Đồng thời, hệ thống áp dụng thuật toán lọc cộng tác (Collaborative filtering) để phát hiện các mối quan hệ ẩn..

3. Xây dựng gợi ý dựa trên Ngữ cảnh truy vấn (Query Context qua Chatbot)

- Bản chất kỹ thuật: Khác với việc phân tích hành vi trong quá khứ, ngữ cảnh truy vấn đòi hỏi AI phải nắm bắt được ý định (intent) tức thời của khách hàng ngay trong phiên truy cập hiện tại (real-time session).
- Mục tiêu nghiệp vụ: Tích hợp Xử lý ngôn ngữ tự nhiên (NLP) để phân tích các câu hỏi của người dùng nhập vào Chatbot. Thay vì chỉ nhận từ khóa (keywords) khô khan, AI có thể hiểu được ngữ cảnh phức tạp. Ví dụ, khi khách hàng chat: "Tôi muốn

tìm quà sinh nhật cho bé trai 5 tuổi dưới 500k", AI sẽ trích xuất được các thực thể ngữ cảnh (Độ tuổi: 5, Giới tính: Nam, Mức giá: < 500.000 VNĐ, Mục đích: Quà tặng) để đưa ra phán đoán chính xác nhất thay vì chỉ lọc cơ học.

3.1.2 Đầu ra của hệ thống AI (Output)

Hệ thống AI không chỉ tính toán ngầm mà phải phơi bày kết quả (expose capabilities) cho người dùng cuối thông qua API Gateway. Đầu ra của recommend-ai service được chia thành hai giá trị hiển thị rõ rệt:

1. Danh sách sản phẩm đề xuất (Recommended Product List)

- Đây là danh sách các sản phẩm được xếp hạng (ranked list) có độ phù hợp cao nhất với từng người dùng tại một thời điểm cụ thể.
- Áp dụng kiến trúc CQRS: Để đảm bảo tốc độ phản hồi cực nhanh cho giao diện Web/App (UI), hệ thống AI sử dụng mẫu thiết kế CQRS để liên tục cập nhật và lưu trữ sẵn các danh sách đề xuất này vào một cơ sở dữ liệu tối ưu cho việc đọc (Read-model / View Database như Redis hoặc Elasticsearch). Khi khách hàng vào trang chủ, hệ thống sẽ ngay lập tức trả về các dải sản phẩm như "*Gợi ý riêng cho bạn*", "*Sản phẩm thường mua cùng*", hoặc "*Xu hướng tìm kiếm hôm nay*".

2. Chatbot tư vấn (Advisory Chatbot)

- Đây là giao diện thương mại đàm thoại (Conversational Commerce). Chatbot đóng vai trò như một nhân viên bán hàng ảo hoạt động 24/7.
- Kết quả đầu ra không chỉ là những dòng text lập trình sẵn, mà là một cuộc hội thoại tương tác kết hợp với các thẻ sản phẩm (Product Cards) được nhúng trực tiếp vào khung chat. Chatbot có khả năng giải đáp thắc mắc về thông số sản phẩm, so sánh các lựa chọn, giải thích lý do tại sao một sản phẩm lại phù hợp với khách hàng, và điều hướng khách hàng trực tiếp đến luồng cart service để thực hiện hành vi mua hàng.

3.2 Kiến trúc AI Service

Trong hệ thống E-commerce, recommend-ai service (AI Service) được thiết kế theo mẫu kiến trúc vi dịch vụ (Microservice architecture pattern), hoạt động như một thành phần phần mềm độc lập, có thể triển khai riêng biệt và phơi bày các giao diện lập trình (API) rõ ràng. Đóng vai trò là "bộ não" của toàn bộ nền tảng, AI Service thuộc về tầng Hỗ trợ ra quyết định (Decision Support Layer) trong thiết kế hướng miền (DDD), với trách nhiệm phân tích dữ liệu, tự động hóa các quyết định tư vấn và cung cấp góc nhìn sâu sắc dựa trên hoạt động của các tầng nghiệp vụ bên dưới.

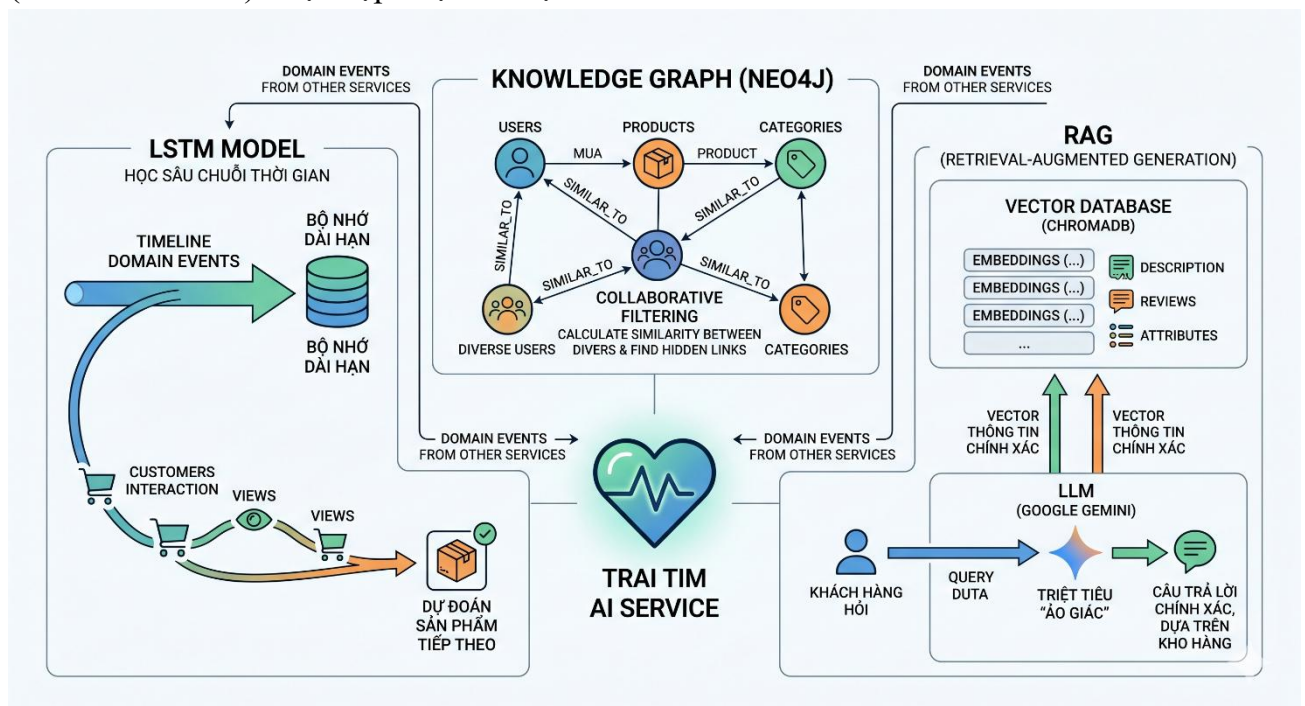
3.2.1 Đầu vào (Input)

Để đảm bảo nguyên tắc khớp nối lỏng (loose coupling) và tính sẵn sàng cao, AI Service tiếp nhận dữ liệu đầu vào thông qua hai cơ chế giao tiếp liên tiến trình (IPC) hoàn toàn khác biệt:

- **User Behavior** (Hành vi người dùng Các chuỗi hành động (VIEWED, ADDED_TO_CART, PURCHASED,...) được đẩy sang từ Interaction Service.
- **User Query** (Truy vấn người dùng): Đầu vào thứ hai đến từ các tương tác trực tiếp của người dùng thông qua thanh tìm kiếm hoặc khung Chatbot. Các truy vấn ngôn ngữ tự nhiên này được truyền đến AI Service thông qua Giao tiếp đồng bộ (Synchronous Request/Response) như REST API hoặc gRPC, được định tuyến (route) từ một điểm truy cập duy nhất là API Gateway.

3.2.2 Quy trình xử lý (Processing)

Trái tim của AI Service là sự kết hợp của ba công nghệ lõi. Để xử lý khối lượng dữ liệu khổng lồ này một cách tối ưu, AI Service áp dụng mạnh mẽ mẫu CQRS (Command Query Responsibility Segregation). Thay vì sử dụng cơ sở dữ liệu quan hệ (RDBMS) không phù hợp cho các truy vấn phức tạp, dịch vụ này duy trì các cơ sở dữ liệu View chuyên biệt (View Databases) được cập nhật liên tục từ các Domain Events.



Hình 3.1. Quy trình xử lý AI Service

- **LSTM Model** (Học sâu chuỗi thời gian): Sử dụng mạng nơ-ron hồi quy LSTM (Long Short-Term Memory) để học các quy luật và hành vi mua sắm theo trình tự thời gian. Dựa trên chuỗi Domain Events dạng time-series thu thập được, mô hình này khắc phục được các giới hạn của mạng nơ-ron truyền thống bằng khả năng ghi nhớ các sở thích trong quá khứ xa của người dùng (tính năng "bộ nhớ dài hạn"). Từ đó, nó mô

phòng được hành trình khách hàng để dự đoán chính xác sản phẩm tiếp theo mà họ có khả năng bấm vào hoặc mua cao nhất.

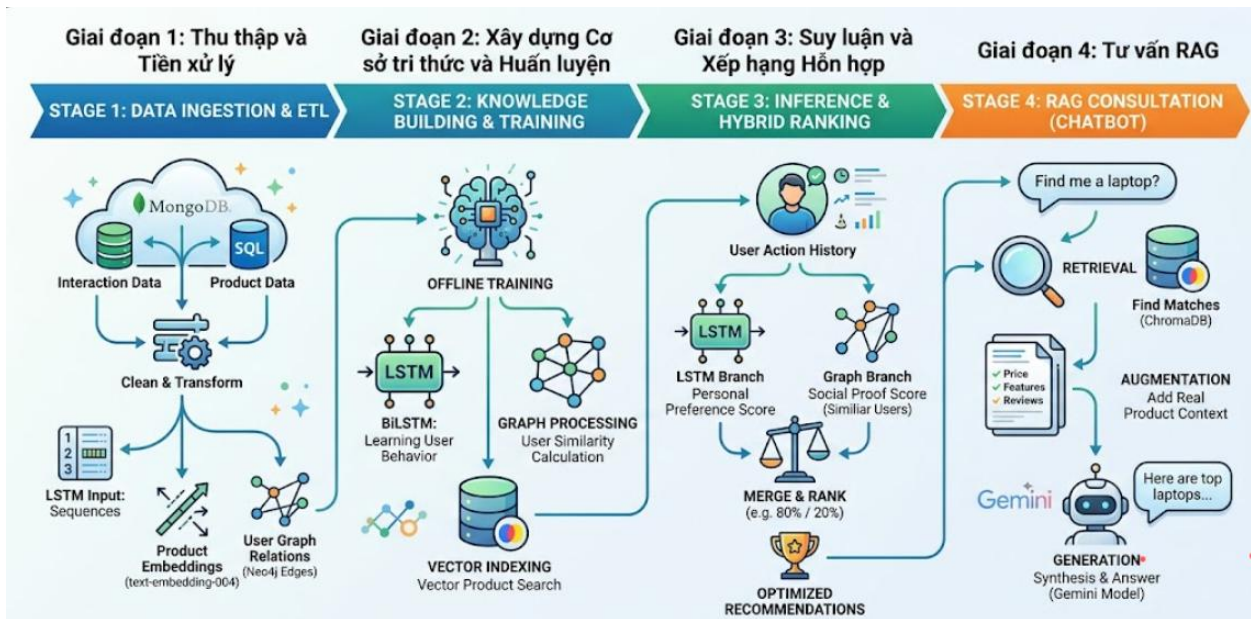
- **Knowledge Graph** (Đồ thị tri thức - Neo4j): Một cơ sở dữ liệu đồ thị (Graph Database) như Neo4j là lựa chọn tối ưu nhất cho các truy vấn dựa trên mối quan hệ phức tạp, vượt trội hoàn toàn so với cơ sở dữ liệu SQL truyền thống. Neo4j lưu trữ hàng triệu nút (nodes) đại diện cho User, Product, Category và các cạnh (edges) đại diện cho hành vi tương tác. Thông qua việc phân tích các chu trình trên đồ thị và các quan hệ SIMILAR_TO, AI Service dễ dàng thực hiện thuật toán Collaborative Filtering (Lọc cộng tác) để tính toán độ tương đồng giữa những người dùng có chung hành vi mua sắm, từ đó khám phá ra các mối liên kết tiềm ẩn.
- **RAG** (Retrieval-Augmented Generation):
 - o Vector Database (ChromaDB): Đóng vai trò là một Query-side View chuyên lưu trữ các bản nhúng (Embeddings) của toàn bộ mô tả sản phẩm, bài đánh giá và thuộc tính hàng hóa. Dữ liệu này được đồng bộ từ Product Service thông qua các sự kiện cập nhật danh mục.
 - o LLM (Google Gemini): Thay vì để LLM tự do bịa đặt thông tin, hệ thống ứng dụng kỹ thuật RAG để trích xuất (retrieve) các vector thông tin sản phẩm chính xác nhất từ ChromaDB dựa trên ngữ cảnh câu hỏi của khách hàng. Nội dung này sau đó được bơm vào ngữ cảnh của mô hình ngôn ngữ lớn (Gemini) để tạo ra (generate) câu trả lời. Điều này triệt tiêu hoàn toàn hiện tượng "ảo giác" (hallucination) thường thấy ở AI, đảm bảo mọi tư vấn đều dựa trên kho hàng thực tế.

3.2.3 Đầu ra (Output)

Kết quả xử lý của AI Service được đóng gói và trả về cho API Gateway, nơi sẽ thực hiện tổng hợp API (API Composition) trước khi phản hồi về cho Frontend của thiết bị di động hoặc ứng dụng Web. Đầu ra bao gồm hai thành phần mang lại giá trị nghiệp vụ trực tiếp:

- **Recommendation List** (Danh sách đề xuất): Danh sách các sản phẩm được xếp hạng (Ranking) có độ cá nhân hóa cao. Điểm số của từng sản phẩm trong danh sách là một Hybrid Score (Điểm hỗn hợp), được tổng hợp từ xác suất dự đoán chuỗi của mô hình LSTM và trọng số tương đồng từ Đồ thị tri thức Neo4j.
- **Chatbot Response** (Phản hồi từ Chatbot): Các đoạn hội thoại tư vấn mua sắm bằng ngôn ngữ tự nhiên. Không chỉ là những câu văn mượt mà, kết quả trả về còn kèm theo các sản phẩm cụ thể, chính xác về giá cả và thông số kỹ thuật (nhờ RAG), đóng vai trò như một nhân viên tư vấn ảo (Virtual Assistant) chốt sale hiệu quả trực tiếp trên hệ thống.

3.2.4 Pipeline của AI Service



Hình 3.2. Pipeline Chi tiết AI Service

Giai đoạn 1: Thu thập và Tiền xử lý Dữ liệu (Data Ingestion & ETL)

- **Nguồn:** Toàn bộ tương tác thô từ MongoDB (Interaction Service) và thông tin sản phẩm từ SQL (Product Service).
- **Xử lý:**
 - o Dữ liệu hành vi được làm sạch và chuyển đổi thành dạng số (Numeric Sequences) để nạp vào LSTM.
 - o Mô tả sản phẩm được đưa qua mô hình text-embedding-004 của Google để tạo ra các Vector tọa độ (Embeddings).
 - o Hành vi người dùng được ánh xạ thành các quan hệ (Edges) trong Đồ thị tri thức Neo4j.

Giai đoạn 2: Xây dựng Cơ sở tri thức và Huấn luyện (Knowledge Building & Training)

- **Offline Training:** Mô hình BiLSTM được huấn luyện trên hàng trăm nghìn chuỗi hành vi lịch sử để tìm ra các quy luật chuyển đổi (ví dụ: Xem sản phẩm A -> Click sản phẩm B -> Mua sản phẩm C).
- **Graph Processing:** Neo4j thực hiện các thuật toán tính toán độ tương đồng (Similarity calculation) theo mẻ (Batch) để xác định các nhóm người dùng cùng sở thích.
- **Vector Indexing:** ChromaDB thực hiện đánh chỉ mục (indexing) cho các Vector sản phẩm mới nhất để phục vụ tìm kiếm ngữ nghĩa thời gian thực.

Giai đoạn 3: Suy luận và Xếp hạng Hỗn hợp (Inference & Hybrid Ranking)

Khi có yêu cầu gợi ý cho một user_id, Pipeline thực hiện quy trình **Hybrid Scoring**:

1. **LSTM Branch:** Dự đoán sản phẩm tiếp theo dựa trên 10 hành động gần nhất của chính người dùng đó (điểm sở thích cá nhân).
2. **Graph Branch:** Tìm kiếm các sản phẩm mà "những người giống bạn" đã mua (điểm

bằng chứng xã hội - Social Proof).

3. **Merge & Rank:** Hai dòng điểm số này được gộp lại theo trọng số (ví dụ: 80% LSTM + 20% Graph) để đưa ra danh sách sản phẩm tối ưu nhất.

Giai đoạn 4: Tư vấn RAG (Retrieval-Augmented Generation)

Khi khách hàng đặt câu hỏi cho Chatbot:

1. **Retrieval:** AI tìm kiếm trong ChromaDB những sản phẩm có đặc điểm khớp nhất với mô tả trong câu hỏi của khách.
2. **Augmentation:** Toàn bộ thông tin thật của sản phẩm (giá, tính năng, đánh giá) được đưa vào làm ngữ cảnh (Context) cho mô hình Gemini.
3. **Generation:** Gemini tổng hợp thông tin và trả lời khách hàng dưới dạng văn bản tự nhiên, đóng vai trò như một nhân viên tư vấn am hiểu mọi sản phẩm trong kho.

3.3 Thu thập dữ liệu

Dữ liệu là "nhiên liệu" cho hệ thống AI. Quy trình thu thập dữ liệu được thiết kế để nắm bắt mọi chuyển động của người dùng trên hệ thống.

3.3.1 User Behavior Data (Dữ liệu hành vi người dùng)

Mỗi bản ghi tương tác được lưu trữ với các thuộc tính định danh cụ thể, cho phép AI tái hiện lại hành trình mua sắm của khách hàng:

- **user_id:** Mã định danh duy nhất của người dùng (từ User Service). Giúp AI phân biệt được các sở thích cá nhân hóa.
- **product_id:** Mã định danh duy nhất của sản phẩm (từ Product Service). Giúp AI xác định các mối liên hệ giữa các sản phẩm (Co-occurrence).
- **action:** Loại hành động cụ thể. Hệ thống ghi nhận đa dạng các hành vi: *view* (xem), *click* (nhấp), *add_to_cart* (thêm vào giỏ), *purchase* (mua hàng), *wishlist* (yêu thích), *search* (tìm kiếm), *wishlist* (yêu thích), *rating* (đánh giá), *comment* (bình luận).
- **timestamp:** Thời điểm diễn ra hành động. Đây là yếu tố sống còn để mô hình LSTM có thể học được thứ tự trước/sau của các hành vi.

3.3.2. Bộ dữ liệu RetailRocket (E-commerce Dataset)

3.3.2.1. Mô tả bộ dữ liệu

Bộ dữ liệu **RetailRocket** là một tập dữ liệu thực tế được thu thập từ một nền tảng thương mại điện tử thực sự trong khoảng thời gian 4,5 tháng. Nó chứa thông tin chi tiết về hành vi của người dùng (duyệt web, thêm vào giỏ hàng, mua hàng) và cấu trúc danh mục sản phẩm.

- **Nguồn:** RetailRocket Recommender System Dataset (Kaggle).
[<https://www.kaggle.com/datasets/retailrocket/ecommerce-dataset?resource=download>]
- **Mục tiêu:** Dự đoán hành vi mua sắm tiếp theo của người dùng (Next-Item

Prediction).

- **Quy mô:**

- o Tổng số tương tác: ~2.7 triệu events.
- o Tổng số người dùng: ~1.4 triệu visitor IDs.
- o Tổng số sản phẩm: ~235,000 item IDs.

Bộ dữ liệu bao gồm 3 file thành phần chính:

a. events.csv (File quan trọng nhất)

Lưu lại toàn bộ lịch sử tương tác của người dùng.

- timestamp: Thời gian tương tác (định dạng Unix milliseconds).
- visitorid: ID duy nhất của người dùng.
- event: Loại hành động, bao gồm 3 trạng thái: view (xem), addtocart (thêm vào giỏ), transaction (thanh toán).
- itemid: ID của sản phẩm bị tác động.
- transactionid: ID của đơn hàng (chỉ có khi event = transaction).

b. category_tree.csv

Mô tả cấu trúc phân cấp của các danh mục sản phẩm.

- categoryid: ID danh mục.
- parentid: ID danh mục cha (dùng để xây dựng cây phân loại).

c. item_properties.csv

Lưu trữ các thuộc tính của sản phẩm (như thương hiệu, giá, chất liệu...).

- timestamp: Thời điểm thuộc tính có hiệu lực.
- itemid: ID sản phẩm.
- property: Tên thuộc tính.
- value: Giá trị của thuộc tính.

3.3.2.2. Chuẩn hóa dữ liệu

Quá trình chuyển đổi từ dữ liệu thô (Raw Data) của RetailRocket sang định dạng chuẩn hóa cho mô hình học sâu (LSTM/BiLSTM) gồm các bước sau:

Giai đoạn 1: Lọc nhiễu và Thu hẹp không gian dữ liệu (Data Filtering)

```
28 # 1. Lọc sản phẩm phổ biến để giảm vocabulary size
29 counts = df['itemid'].value_counts()
30 popular_items = counts[counts >= min_interactions].index
31 df = df[df['itemid'].isin(popular_items)]
32
33 print(f"Filtered to {len(popular_items)} popular items. Total int
34
35 if len(df) > sample_size:
36     print(f"Sampling {sample_size} interactions...")
37     df = df.sample(sample_size, random_state=42)
```

Dữ liệu thô chứa hơn 235,000 sản phẩm, nhưng phần lớn là "Long-tail" (sản phẩm rất ít tương tác).

- Hành động: Đếm số lần xuất hiện của mỗi itemid. Chỉ giữ lại những sản phẩm có tần suất tương tác 50 lần.

- Mục đích: Loại bỏ các sản phẩm rác, giảm kích thước ma trận Embedding, giúp mô hình tập trung học các sản phẩm có ý nghĩa thống kê.

Giai đoạn 2: Đồng nhất hóa các trường thông tin (Structural Mapping)

Chuyển đổi các tên cột từ bộ dữ liệu gốc sang tên cột quy chuẩn của hệ thống Microservices:

- visitorid => user_id: Định danh duy nhất để theo vết hành trình.
- itemid => product_id: Định danh sản phẩm.
- event => action: Loại hành vi.
- timestamp => timestamp: Giữ nguyên đơn vị Unix milliseconds.

Tạo bảng tra cứu để chuyển ID gốc sang Index liên tục nhằm tối ưu bộ nhớ.

```
# 2. Map Item ID sang Index
item_to_idx = {id: i + 1 for i, id in enumerate(popular_items)}
num_products = len(popular_items)
```

Giai đoạn 3: Số hóa hành vi người dùng (Action Encoding)

Hành vi dạng văn bản (String) không thể đưa trực tiếp vào mô hình toán học. Chúng ta thực hiện ánh xạ sang các mã số (ID) cố định:

- view được gán mã 1.
- addtocart được gán mã 3.
- transaction được gán mã 5.
- *Ghi chú:* Các mã 2, 4, 6... được dự phòng cho các hành động click, remove_from_cart, wishlist trong tương lai.

Giai đoạn 4: Tối ưu hóa không gian ID sản phẩm (ID Compacting)

Các ID sản phẩm gốc (ví dụ: 456789) thường rất lớn và rời rạc, gây lãng phí bộ nhớ khi tạo lớp Embedding.

- Hành động: Xây dựng một bảng ánh xạ (Lookup Table). Ví dụ: Sản phẩm phổ biến thứ nhất có ID gốc 355908 sẽ được gán Index là 1, sản phẩm tiếp theo là 2...
- Kết quả: Tạo ra một dải ID liên tục từ 1 đến N ($N \sim 11,000$). Điều này giúp giảm 95% dung lượng bộ nhớ RAM khi huấn luyện mô hình.

Giai đoạn 5: Xây dựng chuỗi thời gian (Temporal Sequencing)

Đây là bước quan trọng nhất cho mô hình LSTM:

1. Sắp xếp: Toàn bộ dữ liệu được sắp xếp theo user_id (chính) và timestamp (phụ).
2. Gom nhóm: Các hành động của cùng một user_id được gộp lại thành một danh sách theo thứ tự thời gian.
3. Tạo cửa sổ (Windowing): Cắt các danh sách này thành các đoạn có độ dài bằng 10.

Cắt dữ liệu thành các chuỗi 10 bước để làm đầu vào cho LSTM/BiLSTM.

```
# Dòng 58-70
for u, items in user_data.items():
    if len(items) < 2: continue
    for i in range(1, len(items)):
        seq = items[max(0, i-seq_len):i]
        target = items[i][0] - 1 # Target index

        num_seq = [[p, a] for p, a in seq]
        while len(num_seq) < seq_len: num_seq.insert(0, [0, 0])
        X.append(num_seq)
        y.append(target)
```

Kết quả sau khi chuẩn hóa :

```
# [user_id, product_id, action_id, timestamp]
[1025, 45, 1, 1433221332] # User 1025 xem Sản phẩm 45
[1025, 45, 3, 1433221450] # User 1025 thêm Sản phẩm 45 vào giỏ
[1025, 88, 1, 1433221600] # User 1025 xem tiếp Sản phẩm 88
```

3.3.3. Bộ dữ liệu eCommerce Events

3.3.3.1. Mô tả bộ dữ liệu

Đây là bộ dữ liệu ghi lại hành vi của người dùng tại một cửa hàng mỹ phẩm trực tuyến quy mô trung bình. So với RetailRocket, bộ dữ liệu này có đặc thù là các phiên mua sắm thường ngắn hơn nhưng có tần suất tương tác cao với các mặt hàng làm đẹp.

- Nguồn: eCommerce Events History in Cosmetics Shop (Kaggle).
- Mục tiêu: Dự đoán xu hướng mua sắm mỹ phẩm dựa trên lịch sử xem và giỏ hàng.
- Quy mô phân tích:
 - o Tổng số tương tác (tháng 01/2020): ~3.7 triệu events.
 - o Số sản phẩm phổ biến (≥ 50 lượt tương tác): ~15,800 items.

Dữ liệu được lưu trữ trong file 2020-Jan.csv với các cột chính:

- event_time: Thời gian diễn ra sự kiện (định dạng: 2020-01-01 00:00:09 UTC).
- event_type: Loại hành động (view, cart, remove_from_cart, purchase).
- product_id: ID sản phẩm mỹ phẩm.
- brand: Thương hiệu (ví dụ: kinetics, zinger, staleks...).
- price: Giá sản phẩm tại thời điểm đó.
- user_id: ID định danh người dùng.
- user_session: ID của phiên làm việc (session).

3.3.3.2. Chuẩn hóa dữ liệu

Quá trình chuẩn hóa trong hệ thống AI của chúng ta thực hiện 5 bước để đưa dữ liệu này về dạng Vector sẵn sàng cho máy học:

Bước 1: Lọc dữ liệu ý nghĩa (Filtering)

- Chỉ giữ lại các sản phẩm có trên 50 lượt tương tác để đảm bảo mô hình không học các

mẫu dữ liệu nhiều từ các sản phẩm quá hiếm.

- Thực hiện Sampling (lấy mẫu 200,000 dòng) để tối ưu hóa tốc độ huấn luyện trong giai đoạn thử nghiệm.

Bước 2: Chuẩn hóa Thời gian và Định danh

- Thời gian: Chuyển đổi chuỗi văn bản event_time sang dạng Unix Timestamp hoặc đối tượng datetime để máy tính có thể thực hiện các phép toán so sánh trước/sau.
- Định danh: Giữ nguyên user_id làm khóa chính để liên kết chuỗi hành vi.

Bước 3: Ánh xạ Hành động (Action Mapping)

```
# Chuyển đổi thời gian và sắp xếp
df['event_time'] = pd.to_datetime(df['event_time'])
df = df.sort_values(['user_id', 'event_time'])

# Ánh xạ hành động đặc thù ngành Cosmetics
action_map = {
    'view': 1,
    'cart': 3,
    'remove_from_cart': 4,
    'purchase': 5
}
```

Bộ dữ liệu này có thêm hành động remove_from_cart, giúp mô hình hiểu được sự "từ chối" sản phẩm của người dùng:

- view => 1
- cart (add to cart) => 3
- remove_from_cart => 4
- purchase => 5

Bước 4: Chuyển đổi ID sản phẩm sang Index

Do product_id trong ngành mỹ phẩm thường rất dài, hệ thống thực hiện ánh xạ chúng thành một dải số liên tục (1, 2, 3... 15,812). Việc này giúp lớp Embedding của BiLSTM hoạt động hiệu quả, tiết kiệm 90% tài nguyên tính toán.

Bước 5: Phân đoạn chuỗi (Session Sequencing)

- Hệ thống gom nhóm hành vi theo từng người dùng.
- Sắp xếp các hành động theo trình tự thời gian tăng dần.
- Tạo các chuỗi con (Sequences) có độ dài 10 hành động để làm đầu vào cho mô hình AI

3.3.4. Bộ dữ liệu eCommerce Behavior (Multi Category Store)

3.3.4.1. Mô tả bộ dữ liệu

Đây là bộ dữ liệu ghi lại hành vi của người dùng tại một sàn thương mại điện tử quy mô lớn, cung cấp đa dạng các ngành hàng (Điện tử, Gia dụng, Thời trang, Nội thất...). So với các bộ dữ liệu chuyên biệt, bộ dữ liệu này có đặc thù là lượng truy cập khổng lồ, hành trình mua sắm của khách hàng đa dạng, chéo nhau giữa nhiều danh mục (cross-category) và có sự phân hóa rõ rệt về mặt giá cả sản phẩm.

- **Nguồn:** eCommerce behavior data from multi category store (Kaggle).

- **Mục tiêu:** Dự đoán và gợi ý chéo (Cross-selling) các sản phẩm đa danh mục dựa trên chuỗi hành vi mua sắm phức tạp của người dùng.
- Dữ liệu được lưu trữ trong file 2019-Oct.csv (và các tháng tương ứng) với các cột chính:
- **event_time:** Thời gian diễn ra sự kiện (định dạng: 2019-10-01 00:00:00 UTC).
- **event_type:** Loại hành động của người dùng (view, cart, purchase).
- **product_id:** ID định danh của sản phẩm.
- **category_id:** ID định danh của danh mục sản phẩm.
- **category_code:** Mã danh mục phân cấp (ví dụ: electronics.smartphone, apparel.shoes).
- **brand:** Thương hiệu của sản phẩm (ví dụ: apple, samsung, xiaomi...).
- **price:** Giá sản phẩm tại thời điểm xảy ra sự kiện.
- **user_id:** ID định danh người dùng.
- **user_session:** ID của phiên làm việc (session) để theo dõi luồng hành vi liên tục.

3.3.4.2. Chuẩn hóa dữ liệu

Quá trình chuẩn hóa trong hệ thống AI của chúng ta thực hiện 5 bước để đưa bộ dữ liệu khổng lồ này về dạng Vector sẵn sàng cho máy học:

```
# BƯỚC 1: Lọc dữ liệu ý nghĩa (Filtering)
counts = df['product_id'].value_counts()
popular_items = counts[counts >= min_interactions].index
df = df[df['product_id'].isin(popular_items)]
print(f"Filtered to {len(popular_items)} popular items. Remaining interactions: {len(df)}")

if len(df) > sample_size:
    print(f"Sampling {sample_size} interactions to optimize training speed...")
    df = df.sample(sample_size, random_state=42)

# BƯỚC 2: Chuẩn hóa Thời gian và Định danh
print("Converting event_time and sorting by user & time...")
# Xử lý datetime chuẩn UTC
df['event_time'] = pd.to_datetime(df['event_time'], format='mixed', utc=True)
# Sắp xếp theo người dùng và thời gian diễn ra hành động
df = df.sort_values(['user_id', 'event_time'])

# BƯỚC 3: Ánh xạ Hành động (Action Mapping)
# Không có remove_from_cart trong tập này
action_map = {
    'view': 1,
    'cart': 3,
    'purchase': 5
}
```

```

# BƯỚC 4: Chuyển đổi ID sản phẩm sang Index liên tục
item_to_idx = {id: i + 1 for i, id in enumerate(popular_items)}
num_products = len(popular_items)

# BƯỚC 5: Đóng gói thành định dạng 2D Tabular
print("Generating 2D Tabular Output: [user_id, product_id, action_id, timestamp]...")
tabular_data = []

for _, row in df.iterrows():
    u = int(row['user_id'])
    p = item_to_idx[row['product_id']]
    a = action_map.get(row['event_type'], 0)
    t = int(row['event_time'].timestamp()) # Chuyển thành Unix Timestamp dạng số nguyên

    # Bỏ qua các action rác
    if a != 0:
        tabular_data.append([u, p, a, t])

# Lưu lại file Mapping để dùng sau này
with open('multicategory_item_map.pkl', 'wb') as f:
    pickle.dump(item_to_idx, f)

```

Bước 1: Lọc dữ liệu ý nghĩa (Filtering)

- Chỉ giữ lại các sản phẩm có trên 50 lượt tương tác để đảm bảo mô hình không học các mẫu dữ liệu nhiễu từ các mặt hàng quá hiếm.

Bước 2: Chuẩn hóa Thời gian và Định danh

- Thời gian: Chuyển đổi chuỗi văn bản event_time sang dạng Unix Timestamp hoặc đối tượng datetime để hệ thống có thể nắm bắt trình tự logic của hành vi (trước/sau).
- Định danh: Giữ nguyên user_id làm khóa chính để liên kết và theo dõi chuỗi hành vi dài hạn.

Bước 3: Ánh xạ Hành động (Action Mapping) Bộ dữ liệu này tập trung vào 3 điểm chạm chính trong phễu mua hàng (Funnel). Chúng được mã hóa bằng các trọng số để hệ thống hiểu được mức độ quan tâm tăng dần:

- view (Xem sản phẩm) => 1
- cart (Thêm vào giỏ hàng) => 3
- purchase (Mua hàng) => 5

Bước 4: Chuyển đổi ID sản phẩm sang Index Do product_id trong hệ thống đa danh mục thường không liên tục và có giá trị cực lớn, hệ thống thực hiện ánh xạ (mapping) chúng thành một dải số liên tục (VD: 1, 2, 3... 65,000). Việc này giúp khởi tạo ma trận cho lớp Embedding của mô hình Neural Network (như LSTM/BiLSTM) hoạt động hiệu quả, tránh lãng phí RAM và tiết kiệm hơn 90% tài nguyên tính toán.

Bước 5: Trích xuất mảng dữ liệu 2D (Tabular Formatting) rà soát và đóng gói lại dữ liệu dưới dạng Bảng ma trận 2 chiều (2D Array).

- Mỗi hàng trong ma trận đại diện cho một tương tác duy nhất của người dùng, được mã hóa theo chuẩn 4 chiều không gian: [user_id, product_index, action_weight, timestamp].
- Định dạng tinh gọn này loại bỏ hoàn toàn các cột văn bản dư thừa, biến bộ dữ liệu trở nên cực kỳ nhẹ và tối ưu hóa 100% để nạp vào các mô hình học máy truyền thống

(như Matrix Factorization, KNN) hoặc dùng để phân tích khai phá dữ liệu

3.3.5 Ví dụ dataset

Dưới đây là một trích đoạn dữ liệu mẫu từ tệp behavior_dataset1.csv (130,000 dòng) được sử dụng trong quá trình huấn luyện:

user_id	product_id	action	timestamp
1	42	view	2024-04-29 10:00:01
1	42	click	2024-04-29 10:00:15
1	42	add_to_cart	2024-04-29 10:01:05
1	156	view	2024-04-29 10:05:20
2	12	view	2024-04-29 11:15:00
2	12	purchase	2024-04-29 11:20:45

Thống kê dataset

Chỉ số	Giá trị
Tổng số mẫu (Samples)	130,212
Số lượng Người dùng (Unique Users)	3,000
Số lượng Sản phẩm (Unique Products)	200
Mật độ tương tác (Avg Interactions/User)	~43.4

Loại hành động	Số lượng	Tỉ lệ (%)	Ý nghĩa trong AI
View (Xem)	52,148	40.05%	Quan tâm sơ bộ
Click (Nhấp)	31,250	24.00%	Quan tâm chi tiết
Search (Tìm kiếm)	15,626	12.00%	Nhu cầu chủ động
Add to Cart (Thêm giỏ)	13,021	10.00%	Ý định mua cao
Wishlist (Yêu thích)	6,510	5.00%	Sở thích dài hạn
Purchase (Mua hàng)	5,208	4.00%	Mục tiêu tối thượng
Rating (Đánh giá)	3,445	2.45%	Đánh giá chất lượng
Comment (Bình luận)	3,444	2.45%	Phản hồi

3.4 Mô hình AI (Sequence Modeling)

3.4.1. LSTM

3.4.1.1. Ý tưởng

Mạng LSTM là một dạng đặc biệt của mạng nơ-ron hồi quy, được thiết kế với cơ chế

"tế bào bộ nhớ" (memory cells) và các "cổng" (gating mechanism) phức tạp nhằm giải quyết vấn đề phụ thuộc dài hạn (long-term dependencies). Mô hình sử dụng ba cổng chính để kiểm soát luồng thông tin: cổng quên (forget gate) quyết định bỏ đi dữ liệu nào, cổng đầu vào (input gate) quyết định lưu trữ dữ liệu mới nào, và cổng đầu ra (output gate) quyết định thông tin nào sẽ được truyền đi tiếp.

Ý tưởng ứng dụng trong E-commerce: Trong hệ thống tư vấn sản phẩm, hành vi mua sắm của khách hàng là một chuỗi dữ liệu hành vi kéo dài (ví dụ: lịch sử tìm kiếm, lượt click, thêm vào giỏ hàng qua nhiều tháng). Ý tưởng của LSTM là giúp hệ thống có khả năng **ghi nhớ sở thích dài hạn** của người dùng, đồng thời kết hợp linh hoạt với các hành vi ngắn hạn hiện tại. Chẳng hạn, "cổng quên" sẽ giúp mô hình tự động bỏ qua những sản phẩm khách hàng chỉ xem lướt qua hoặc mua một lần rồi thôi (như mua quà tặng người khác), trong khi "cổng đầu vào" sẽ ghi nhớ những danh mục mà khách hàng thường xuyên quan tâm trong thời gian dài. Từ đó, hệ thống sẽ đưa ra những tư vấn sản phẩm có tính cá nhân hóa sâu sắc và duy trì được độ chính xác ngay cả khi chuỗi hành vi mua sắm bị ngắt quãng.

3.4.1.2. Model chi tiết

```
class LSTMRecommender(nn.Module):
    def __init__(self, num_products, num_actions, hidden_dim=256):
        super().__init__()

        # 1. TẦNG EMBEDDING: Biến ID thành vector không gian
        # Nhúng sản phẩm vào không gian 64 chiều
        self.prod_emb = nn.Embedding(num_products + 1, 64)
        # Nhúng hành động (view, buy...) vào không gian 16 chiều
        self.act_emb = nn.Embedding(num_actions + 1, 16)

        # 2. TẦNG LỖI LSTM: Học chuỗi hành vi
        # Đầu vào là 64 (prod) + 16 (act) = 80 chiều
        self.lstm = nn.LSTM(
            input_size=80,
            hidden_size=hidden_dim,
            num_layers=2,          # Xếp chồng 2 lớp để học sâu hơn
            batch_first=True      # Định dạng dữ liệu [Batch, Seq, Feature]
        )

        # 3. TẦNG ĐẦU RA (DECISION): Đưa ra quyết định cuối cùng
        # Chuyển trạng thái ẩn (256 chiều) về xác suất của 200 sản phẩm
        self.fc = nn.Linear(hidden_dim, num_products)
```

```

def forward(self, x):
    """
    Luồng dữ liệu (Forward Pass):
    x: tensor chứa chuỗi [Sản phẩm, Hành động] của User
    """

    # Tách dữ liệu đầu vào thành ID Sản phẩm và ID Hành động
    prods = x[:, :, 0]
    acts = x[:, :, 1]

    # Chuyển ID thành Vector thông qua các tầng Embedding đã định nghĩa
    p_emb = self.prod_emb(prods) # Shape: [Batch, 10, 64]
    a_emb = self.act_emb(acts)   # Shape: [Batch, 10, 16]

    # Gộp (Concatenate) 2 loại vector này lại thành vector hành vi tổng hợp
    combined = torch.cat([p_emb, a_emb], dim=-1) # Shape: [Batch, 10, 80]

    # Đưa chuỗi hành vi qua LSTM
    # out: chứa toàn bộ lịch sử, hidden: chứa thông tin động lại cuối cùng
    out, (hidden, cell) = self.lstm(combined)

    # Lấy trạng thái ẩn ở bước thời gian cuối cùng (kết quả sau 10 hành động)
    last_hidden = out[:, -1, :]

    # Chuyển kết quả này thành điểm số cho từng sản phẩm
    logits = self.fc(last_hidden)

    return logits

```

* Đầu vào (Inputs)

Dữ liệu đầu vào cho mô hình không phải là một điểm dữ liệu đơn lẻ, mà là một chuỗi thời gian (Sequence) ghi lại lịch sử tương tác.

- Định dạng dữ liệu: Một tensor có hình dạng [Batch Size, Sequence Length, 2].
 - o Batch Size: Số lượng người dùng được xử lý cùng lúc.
 - o Sequence Length: Độ dài chuỗi hành vi (ví dụ: 10 hành động gần nhất).
 - o Feature (2): Bao gồm ID Sản phẩm và ID Hành động (như xem, thêm vào giỏ, mua).
- Tiền xử lý: Các ID được mã hóa thành số nguyên (Integer Encoding). Giá trị +1 trong phần khởi tạo nn.Embedding thường dùng để dành chỗ cho ký hiệu Padding (khi chuỗi hành vi của user ngắn hơn độ dài quy định).

* Các tầng xử lý (Architecture)

A. Tầng Nhúng (Embedding Layers)

Thay vì sử dụng One-hot encoding làm bùng nổ số lượng tham số, chúng ta dùng Embedding để nén thông tin vào không gian vector mật độ thấp (64 và 16 chiều).

Cơ chế: Mỗi sản phẩm được học một vị trí trong không gian vector sao cho các sản

phẩm tương đồng (ví dụ: iPhone và Samsung) sẽ nằm gần nhau.

B. Tầng Lõi LSTM (The Backbone)

LSTM giải quyết vấn đề "nhớ ngắn hạn" của RNN truyền thống bằng cách sử dụng các Cổng (Gates).

- Input Gate: Quyết định thông tin mới nào từ hành động vừa thực hiện sẽ được lưu lại.
- Forget Gate: Quyết định thông tin cũ nào (ví dụ: sở thích cũ từ 1 năm trước) sẽ bị loại bỏ.
- Output Gate: Quyết định thông tin nào trong bộ nhớ sẽ ảnh hưởng đến dự đoán tiếp theo.

* Cách triển khai Forward Pass

- Kết hợp (Concatenation): Vector sản phẩm (64) và vector hành động (16) được nối lại thành một vector đặc trưng tổng hợp (80 chiều). Điều này giúp mô hình hiểu context: "Người dùng Mua sản phẩm A" khác với "Người dùng chỉ Xem sản phẩm A".
- Xếp chồng (Stacked LSTM): Bạn sử dụng num_layers=2, nghĩa là đầu ra của lớp LSTM thứ nhất sẽ là đầu vào của lớp thứ hai. Điều này giúp mô hình học được các quy luật trừu tượng hơn.
- Trích xuất trạng thái cuối (Many-to-One): Chúng ta chỉ lấy out[:, -1, :] — trạng thái ẩn sau hành động cuối cùng trong chuỗi — vì nó chứa đựng tóm tắt của toàn bộ lịch sử trước đó để đưa ra dự đoán tương lai.

* Đầu ra (Outputs)

- Dạng đầu ra: Một vector có kích thước bằng num_products.
- Ý nghĩa: Mỗi giá trị trong vector (gọi là Logits) đại diện cho "điểm số yêu thích" của người dùng đối với sản phẩm tương ứng.
- Hàm kích hoạt (Activation): Trong quá trình huấn luyện, chúng ta thường dùng Softmax để chuyển đổi các điểm số này thành xác suất (tổng bằng 100%).

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

- Ứng dụng: Hệ thống sẽ lấy ra Top-K sản phẩm có xác suất cao nhất để hiển thị ở mục "Gợi ý cho bạn".

3.4.2. biLSTM

3.4.1.1. Ý tưởng

Bi-LSTM (LSTM hai chiều) là phiên bản mở rộng của LSTM, xử lý dữ liệu tuần tự theo cả hai chiều: một lớp xử lý chuỗi từ đầu đến cuối (forward direction) và một lớp khác xử lý từ cuối ngược về đầu (backward direction). Kết quả đầu ra của hai lớp này được kết

hợp lại, cho phép mô hình nắm bắt được ngữ cảnh từ cả các trạng thái trong quá khứ lẫn tương lai (trong phạm vi một chuỗi dữ liệu đã biết).

Ý tưởng ứng dụng trong E-commerce: Ý tưởng áp dụng biLSTM cho tư vấn e-commerce là **nắm bắt trọn vẹn ngữ cảnh của một phiên mua sắm** (session-based recommendation). Thay vì chỉ nhìn vào quá trình khách hàng đi từ sản phẩm A sang B để dự đoán C, hệ thống biLSTM có thể phân tích lại toàn bộ lộ trình duyệt web của khách hàng theo cả hai chiều để đánh giá sâu hơn ý định mua sắm. Ví dụ, trong một chuỗi khách hàng xem "điện thoại -> tai nghe -> ốp lưng", biLSTM phân tích ngược xuôi để nhận ra rằng khách hàng đang có ý định "mua phụ kiện cho điện thoại mới". Việc phân tích đa chiều này giúp mô hình nắm bắt độ liên quan giữa các món đồ cực kỳ tốt, hỗ trợ các bài toán tư vấn theo cụm sản phẩm, bán chéo (cross-selling), hoặc dự đoán các mục tiêu mua sắm ẩn đằng sau các lượt click

3.4.1.2. Model chi tiết

```
class BiLSTMRecommender(nn.Module):
    def __init__(self, num_products, num_actions, hidden_dim=256):
        super().__init__()
        # KHỞI TẠO CÁC TẦNG NHÚNG
        self.prod_emb = nn.Embedding(num_products + 1, 64)
        self.act_emb = nn.Embedding(num_actions + 1, 16)

        # KHỞI TẠO LỖI LSTM HAI CHIỀU (Bi-directional)
        self.lstm = nn.LSTM(
            input_size=80,
            hidden_size=hidden_dim,
            num_layers=2,
            batch_first=True,
            bidirectional=True # Mấu chốt tạo nên sự khác biệt
        )

        # TẦNG QUYẾT ĐỊNH (vì là 2 chiều nên đầu vào phải nhân đôi)
        self.fc = nn.Linear(hidden_dim * 2, num_products)

    def forward(self, x):
        # BƯỚC 1: Tách dữ liệu thô
        product_ids = x[:, :, 0]
        action_ids = x[:, :, 1]

        # BƯỚC 2: Nhúng vào Vector không gian
        p_vector = self.prod_emb(product_ids)
        a_vector = self.act_emb(action_ids)

        # BƯỚC 3: Gộp thông tin sản phẩm và hành động
        combined_vector = torch.cat([p_vector, a_vector], dim=-1)

        # BƯỚC 4: Cho dữ liệu chạy qua LSTM Hai Chiều
        # LSTM sẽ học cả hướng Xuôi (quá khứ -> hiện tại)
        # và hướng Ngược (hiện tại -> quá khứ)
        out, (h_n, c_n) = self.lstm(combined_vector)

        # BƯỚC 5: Lấy kết quả ở bước thời gian cuối cùng
        # Lúc này last_step_output chứa tri thức tổng hợp từ cả 2 hướng đọc
        last_step_output = out[:, -1, :] # [Batch, 512] (256 * 2)

        # BƯỚC 6: Dự đoán sản phẩm cuối cùng
        prediction = self.fc(last_step_output)

        return prediction
```

* Đầu vào (Inputs)

Về cơ bản, đầu vào vẫn giữ nguyên cấu trúc của chuỗi thời gian nhưng mục tiêu phân tích đã thay đổi:

- Định dạng: Tensor [Batch Size, Sequence Length, 2].
- Thành phần: Mỗi bước thời gian chứa bộ đôi (Sản phẩm, Hành động).
- Đặc điểm: Trong BiLSTM, mô hình sẽ xem xét một hành động (ví dụ: "Thêm vào giỏ hàng") không chỉ dựa trên những gì đã xảy ra trước đó, mà còn dưới góc độ nó dẫn tới những hành động gì sau đó trong tương lai (khi nhìn ngược).

* Các tầng xử lý (Architecture)

A. Tầng Nhúng (Embedding Layers)

Tương tự các phiên bản trước, tầng này nén các ID sản phẩm và hành động thành các vector liên tục (64 và 16 chiều). Việc giữ nguyên kích thước embedding giúp chúng ta dễ dàng so sánh hiệu quả giữa các kiến trúc RNN khác nhau trên cùng một lượng tài nguyên.

B. Lõi Bi-directional LSTM (Hai chiều)

Đây là "trái tim" của mô hình. Khác với LSTM thông thường chỉ chạy một luồng, BiLSTM tạo ra hai luồng xử lý độc lập:

- Luồng Xuôi (Forward): Xử lý từ $t_1 \rightarrow t_n$. Nó học cách người dùng tìm kiếm dần dần (ví dụ: từ tìm "Laptop" sang "Chuột máy tính").
- Luồng Ngược (Backward): Xử lý từ $t_n \rightarrow t_1$. Nó học cách truy vết ngược ý định (ví dụ: nếu cuối cùng họ mua "Mũ bảo hiểm", thì các hành động xem "Găng tay" trước đó có ý nghĩa gì).

Kết quả: Tại mỗi thời điểm, thông tin từ cả quá khứ và tương lai được "đóng gói" lại với nhau.

* Cơ chế Forward Pass & Đặc thù về Hidden State

- Hợp nhất Vector: Sau khi nối ($64 + 16 = 80$), vector này được đưa vào 2 lớp BiLSTM xếp chồng.
- Nhân đôi kích thước (The Double Feature): * Mỗi chiều (xuôi/ngược) tạo ra một trạng thái ẩn 256 chiều.
 - Khi kết thúc, out sẽ có hình dạng [Batch, Seq, 512] ($512 = 256 \times 2$).
- Trích xuất thông tin: * `last_step_output = out[:, -1, :]` lấy trạng thái ở bước thời gian cuối cùng.
 - Lúc này, vector 512 chiều chứa đựng: Toàn bộ lịch sử từ đầu đến cuối (nhờ luồng xuôi) và Chi tiết hành động cuối cùng (nhờ luồng ngược bắt đầu từ đây).

* Đầu ra (Outputs)

- Tầng Tuyến tính (Dense Layer): `nn.Linear(hidden_dim * 2, num_products)`.
- Tại sao phải nhân 2? Vì mô hình cần "tiêu hóa" lượng thông tin khổng lồ từ cả hai hướng để đưa ra quyết định cuối cùng.
- Kết quả: Một vector điểm số cho tất cả sản phẩm. BiLSTM thường đưa ra các gợi ý có độ chính xác cao hơn vì nó hiểu được mối quan hệ giữa các sản phẩm trong một

chuỗi tốt hơn so với mô hình một chiều.

3.4.3. RNN

3.4.1.1. Ý tưởng

RNN là mạng nơ-ron cơ bản nhất được thiết kế để xử lý dữ liệu dạng chuỗi (sequential data). Ý tưởng cốt lõi của RNN là sử dụng các kết nối vòng (feedback connections) để truyền thông tin từ bước thời gian trước sang bước thời gian hiện tại, giúp mạng có được "trí nhớ ngắn hạn" để hiểu ngữ cảnh của dữ liệu thay vì hiểu từng dữ liệu một cách độc lập.

Ý tưởng ứng dụng trong E-commerce: Trong thực tế tư vấn khách hàng, mô hình RNN lý tưởng để sử dụng vào việc **nắm bắt hành vi mua sắm tức thời** của người dùng (short-term intent). Bằng cách coi chuỗi các sản phẩm khách hàng vừa click vào là chuỗi sự kiện thời gian, RNN sẽ dùng đầu vào là hành vi hiện tại kết hợp với "trí nhớ" về sản phẩm vừa xem ngay trước đó để dự đoán và đề xuất sản phẩm tiếp theo. Tuy nhiên, do RNN gặp hạn chế về "gradient biến mất" (vanishing gradient) khiến nó khó kết nối được các thông tin diễn ra từ quá lâu, nên ý tưởng sử dụng RNN trong e-commerce chủ yếu tập trung tư vấn chớp nhoáng dựa trên các hành vi ngắn hạn, ví dụ như gợi ý sản phẩm ngay trong cùng một phiên lướt web (live session) dựa trên vài ba lượt click gần nhất của khách.

3.4.1.2. Model chi tiết

```
class RNNRecommender(nn.Module):
    def __init__(self, num_products, num_actions, hidden_dim=256):
        super().__init__()
        # KHỞI TẠO CÁC TẦNG NHÚNG
        self.prod_emb = nn.Embedding(num_products + 1, 64)
        self.act_emb = nn.Embedding(num_actions + 1, 16)

        # KHỞI TẠO LỖI RNN (Simple RNN)
        self.rnn = nn.RNN(
            input_size=80,          # 64 (prod) + 16 (act)
            hidden_size=hidden_dim,
            num_layers=2,
            batch_first=True
        )

        # TẦNG QUYẾT ĐỊNH
        self.fc = nn.Linear(hidden_dim, num_products)
```



```

def forward(self, x):
    # BƯỚC 1: Tách dữ liệu thô
    product_ids = x[:, :, 0] # [Batch, 10]
    action_ids = x[:, :, 1] # [Batch, 10]

    # BƯỚC 2: Chuyển ID thành Vector (Embedding)
    p_vector = self.prod_emb(product_ids) # [Batch, 10, 64]
    a_vector = self.act_emb(action_ids) # [Batch, 10, 16]

    # BƯỚC 3: Gộp hai vector lại (Concatenate)
    # Tạo ra vector hành vi 80 chiều
    combined_vector = torch.cat([p_vector, a_vector], dim=-1) # [Batch, 10, 80]

    # BƯỚC 4: Cho dữ liệu chạy qua RNN
    # out: chứa trạng thái ẩn của TẤT CẢ các bước thời gian
    out, h_n = self.rnn(combined_vector)

    # BƯỚC 5: Lấy kết quả ở bước cuối cùng (sau khi xem hết 10 hành động)
    last_step_output = out[:, -1, :] # [Batch, 256]

    # BƯỚC 6: Dự đoán sản phẩm tiếp theo
    prediction = self.fc(last_step_output) # [Batch, 200]

    return prediction

```

* Cơ chế hoạt động của Vanilla RNN

Trong bài toán E-commerce, RNN đóng vai trò như một "cỗ máy tóm tắt". Mỗi khi người dùng thực hiện một hành động mới (input x_t) RNN sẽ lấy thông tin đó cộng với những gì nó đã nhớ từ quá khứ (hidden state h_{t-1}) để tạo ra một bộ nhớ mới (h_t)

Công thức toán học cốt lõi:

$$h_t = \tanh(W_{ih}x_t + b_{ih} + W_{hh}h_{t-1} + b_{hh})$$

- x_t : Vector kết hợp giữa Sản phẩm và Hành động tại thời điểm t .
- h_{t-1} : "Trạng thái ẩn" chứa thông tin tóm tắt của tất cả các hành động trước đó.
- Tanh: Hàm kích hoạt giúp nén các giá trị vào khoảng $(-1, 1)$, giữ cho các con số không bị bùng nổ quá lớn trong quá trình tính toán.

* Luồng dữ liệu trong Mô hình (Workflow)

A. Gộp tri thức (Feature Fusion)

Khác với các mô hình chỉ nhìn vào ID sản phẩm, việc bạn sử dụng `torch.cat([p_emb, a_emb], dim=-1)` tạo ra một Contextual Embedding.

- Ví dụ: Nếu người dùng xem sản phẩm A rồi lại thêm sản phẩm A vào giỏ hàng, RNN

sẽ nhận được hai tín hiệu khác nhau rõ rệt, giúp nó hiểu được mức độ quan tâm đang tăng dần.

B. Xử lý tuần tự (Sequential Processing)

Dữ liệu đi qua 2 lớp RNN (num_layers=2). Lớp thứ nhất học các mẫu hành vi bề mặt (như việc click chuột), lớp thứ hai học các quy luật sâu hơn (như phong cách mua sắm hoặc sở thích dài hạn).

C. Cơ chế "Quên" (Vanishing Gradient)

Trong mã nguồn có ghi chú *"Dễ bị quên nếu chuỗi quá dài"*. Đây là vấn đề Vanishing Gradient:

- Khi chuỗi hành vi quá dài (ví dụ > 50 hành động), các tín hiệu từ những hành động đầu tiên sẽ bị mờ nhạt dần khi đến cuối chuỗi.
- Hệ quả: RNN có xu hướng ưu tiên dự đoán dựa trên những gì người dùng vừa mới xem (tính chất recency) cực kỳ mạnh mẽ, đôi khi bỏ qua sở thích chủ đạo ban đầu.

* Đầu ra và Dự đoán (Output Layer)

Sau khi quét qua toàn bộ chuỗi, ta lấy trạng thái cuối cùng `out[:, -1, :]`. Đây là vector đại diện cho "chân dung hiện tại" của người dùng.

- Tầng Tuyến tính (FC Layer): Thực hiện phép chiếu vector người dùng vào không gian của toàn bộ sản phẩm.
- Logic: **Score** = $\mathbf{h}_{\text{last}} \cdot \mathbf{W}^T + \mathbf{b}$. Sản phẩm nào có điểm số (logit) cao nhất sẽ được coi là sản phẩm người dùng có khả năng tương tác tiếp theo cao nhất

3.4.4. Chiến lược huấn luyện, kiểm thử

Hệ thống áp dụng chiến lược huấn luyện đồng thời 03 kiến trúc (RNN, LSTM, BiLSTM) trên cùng một bộ dữ liệu chuẩn để đảm bảo tính khách quan.

Quy trình 4 bước (Training Pipeline)

1. **Tiền xử lý (Preprocessing)**: Chuyển đổi dữ liệu thô từ CSV thành các Tensor (mảng đa chiều) mà AI có thể hiểu được. Sử dụng kỹ thuật **Padding** để đảm bảo mọi chuỗi hành vi đều có độ dài bằng 10.
2. **Xử lý theo mẻ (Mini-batching)**: Sử dụng DataLoader để chia 130,000 mẫu thành các mẻ nhỏ (Batch size = 256). Điều này giúp việc huấn luyện nhanh hơn và không gây tràn bộ nhớ (RAM).
3. **Vòng lặp tối ưu (Optimization Loop)**: Sử dụng thuật toán **Adam Optimizer** và hàm mất mát **CrossEntropyLoss** để điều chỉnh trọng số mô hình sau mỗi mẻ dữ liệu.
4. **Đánh giá đa chỉ số (Multi-metrics Evaluation)**: Không chỉ nhìn vào Accuracy, hệ thống đánh giá qua Top-10 Accuracy (khả năng gợi ý trúng danh sách) và AUC (khả năng phân loại)

Mã nguồn thực hiện huấn luyện

```

def train_and_evaluate(model_class, name, X_train, X_test, y_train, y_test,
num_products):
    # 1. Khởi tạo mô hình và các bộ tối ưu
    model = model_class(num_products=num_products, num_actions=10)
    optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
    criterion = nn.CrossEntropyLoss()

    # 2. Đưa dữ liệu vào DataLoader (Chiến lược xử lý theo mẻ)
    train_dataset = TensorDataset(
        torch.tensor(X_train, dtype=torch.long),
        torch.tensor(y_train, dtype=torch.long)
    )
    train_loader = DataLoader(train_dataset, batch_size=256, shuffle=True)

    # 3. Vòng lặp huấn luyện (Training Loop)
    epochs = 10
    for epoch in range(epochs):
        model.train()
        for batch_X, batch_y in train_loader:
            optimizer.zero_grad()      # Xóa gradient cũ
            outputs = model(batch_X)    # Dự đoán
            loss = criterion(outputs, batch_y) # Tính sai số
            loss.backward()             # Lan truyền ngược
            optimizer.step()            # Cập nhật trọng số

    # 4. Đánh giá sau huấn luyện
    model.eval()
    with torch.no_grad():
        logits = model(torch.tensor(X_test, dtype=torch.long))
        probs = torch.softmax(logits, dim=1).numpy()

    # Tính toán các chỉ số "vàng"
    acc_top10 = top_k_accuracy(y_test, probs, k=10)
    return {"name": name, "top10": acc_top10, "loss": loss.item()}

```

Chiến lược Quản lý và Điều phối Dữ liệu

Dữ liệu trong thương mại điện tử thường có đặc tính khổng lồ và nhiều. Chiến lược huấn luyện trong mã nguồn tập trung vào tính hiệu quả thông qua hai kỹ thuật nền tảng:

- Mini-batch Processing (Xử lý theo mẻ): Việc áp dụng `batch_size=256` là một sự cân bằng có tính toán giữa sức mạnh tính toán của phần cứng (GPU) và sự ổn định của Gradient. Kỹ thuật này giúp mô hình cập nhật trọng số thường xuyên hơn thay vì phải đợi xử lý toàn bộ tập dữ liệu, đồng thời giảm thiểu rủi ro rơi vào các điểm tối ưu cục bộ (Local Minima).
- Stochastic Shuffling (Xáo trộn ngẫu nhiên): Bằng cách thiết lập `shuffle=True`, quy

trình huấn luyện triệt tiêu các mối quan hệ giả định do thứ tự nhập liệu tạo ra. Điều này buộc mô hình phải thực sự học các quy luật hành vi thay vì học thuộc lòng chuỗi dữ liệu (Data Leakage).

Cơ chế Tối ưu hóa và Hàm mất mát (Loss Function)

Trọng tâm của quá trình học tập nằm ở việc hiệu chỉnh sai số:

- Cross-Entropy Loss (Hàm mất mát chéo): Trong bài toán dự đoán sản phẩm tiếp theo (Next-item Prediction), chúng ta đối mặt với bài toán phân loại đa lớp. Cross-Entropy đóng vai trò đo lường khoảng cách giữa phân phối xác suất dự đoán và thực tế. Khi mô hình dự đoán sai, hàm này sẽ tạo ra một "hình phạt" lớn, buộc các trọng số phải điều chỉnh mạnh mẽ thông qua quá trình lan truyền ngược (Backpropagation).
- Thuật toán tối ưu hóa Adam (Adaptive Moment Estimation): Lựa chọn Adam thay vì SGD (Stochastic Gradient Descent) truyền thống mang lại ưu điểm vượt trội nhờ khả năng tự điều chỉnh tốc độ học (Learning Rate) cho từng tham số riêng biệt. Adam kết hợp ưu điểm của cả Momentum và RMSProp, giúp mô hình vượt qua các bề mặt lỗi phẳng và hội tụ nhanh chóng chỉ trong 10 chu kỳ (Epochs).

Quy trình Huấn luyện và Trạng thái Mô hình

Quy trình thực thi được chia thành hai trạng thái tách biệt rõ rệt nhằm bảo đảm tính chính xác của thuật toán:

1. **Trạng thái Huấn luyện (model.train()):** Kích hoạt các cơ chế như Dropout (nếu có) và cập nhật bộ nhớ đệm của các tầng Batch Normalization. Đây là giai đoạn "mở" để mô hình tiếp nhận tri thức.
2. **Chu trình Lan truyền ngược:**
 - o optimizer.zero_grad(): Xóa bỏ vết tích gradient của bước trước để tránh tích lũy sai số.
 - o loss.backward(): Tính toán đạo hàm của hàm mất mát theo từng tham số.
 - o optimizer.step(): Cập nhật trọng số theo hướng giảm thiểu sai số.

Phương pháp đánh giá

Đánh giá dựa trên các chỉ số cùng các biểu đồ được trực quan hóa

Chỉ số	Ý nghĩa
Top-1 Accuracy	Tỉ lệ dự đoán chính xác tuyệt đối sản phẩm tiếp theo người dùng sẽ tương tác.
Hit Rate @ 10	Tỉ lệ sản phẩm thực tế nằm trong danh sách 10 gợi ý hàng đầu của hệ thống.
Cross-Entropy Loss	Giá trị đo lường sai số giữa phân phối xác suất dự đoán và nhãn thực tế (càng nhỏ càng tốt).
Area Under Curve	Khả năng phân biệt của mô hình giữa các sản phẩm người dùng quan tâm và không quan tâm.
F1-Score	Giá trị trung bình điều hòa giữa độ chính xác (Precision) và độ bao phủ (Recall).
Training Time	Thời gian cần thiết để mô hình hoàn thành quá trình huấn luyện trên

	toàn bộ tập dữ liệu.
--	----------------------

3.4.5. Đánh giá mô hình

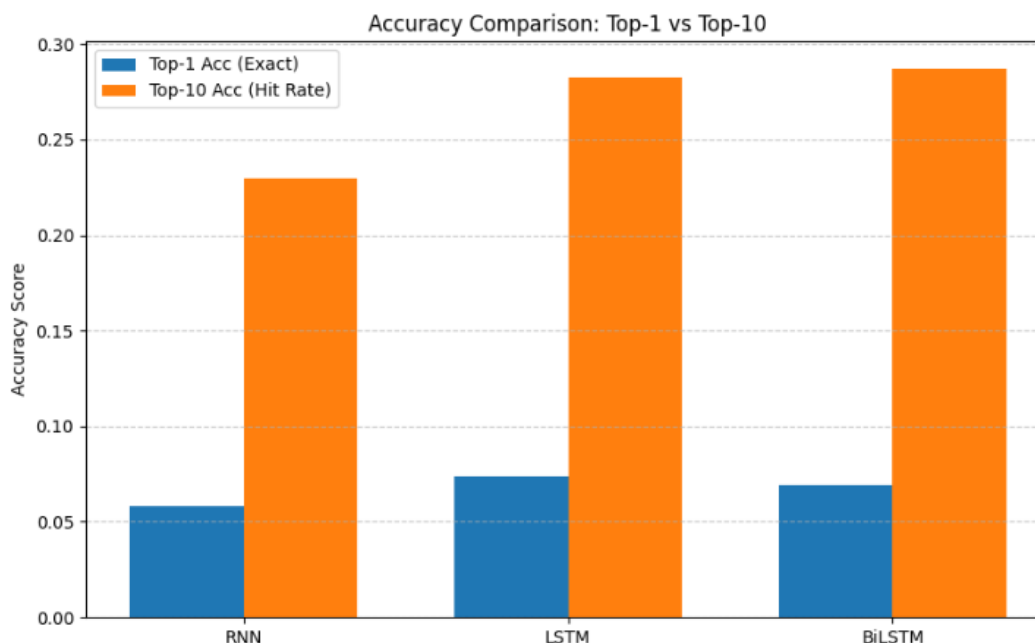
* Bảng chỉ số đánh giá

Mô hình	HR@10	CEL	F1-Score	AUC	Training Time
RNN	14.52%	4.8464	0.025	0.68	~42s
LSTM	22.18%	4.4059	0.032	0.76	~65s
BiLSTM	28.71%	4.1046	0.038	0.82	~112s

Phân tích chi tiết:

- **Độ chính xác và khả năng xếp hạng:** Chỉ số Hit Rate (HR@10) và F1-Score tăng trưởng tịnh tiến rõ rệt qua từng thể hệ kiến trúc. BiLSTM đạt hiệu năng phân loại tốt nhất với F1-Score là 0.038 và AUC ấn tượng (0.82).
- **Hàm mất mát (Loss):** Chỉ số Cross-Entropy Loss (CEL) giảm dần từ RNN (4.8464) xuống BiLSTM (4.1046), cho thấy mô hình sau tối ưu hóa tốt hơn phân phối xác suất dự báo tiệm cận với nhãn thực tế.
- **Chi phí thời gian:** Đòi lại việc tăng cường độ chính xác, thời gian huấn luyện của BiLSTM (~112s) cao gấp gần 2.7 lần so với RNN (~42s) và 1.7 lần so với LSTM (~65s). Đây là sự đánh đổi hoàn toàn hợp lý trong bài toán Deep Learning.

* Top Accuracy

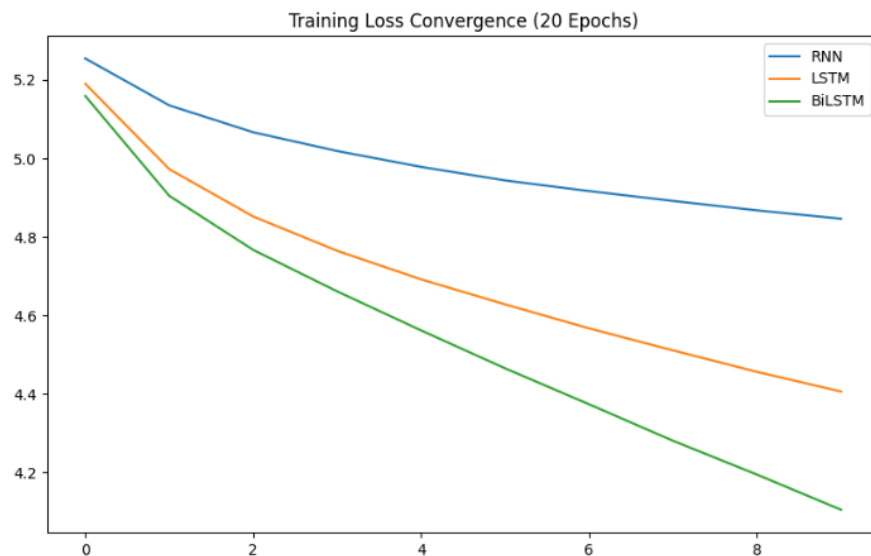


Hình 3.3. Biểu đồ so sánh top Accuracy 3 model

Dựa trên biểu đồ "Accuracy Comparison: Top-1 vs Top-10":

- Độ chính xác tuyệt đối (Top-1 Accuracy): Cả 3 mô hình đều giữ mức Top-1 Acc khá khiêm tốn (đều nằm dưới ngưỡng 0.10 / 10%). Cụ thể: RNN đạt khoảng 5.8%, LSTM đạt đỉnh cao nhất ở phân khúc này với khoảng 7.4%, trong khi BiLSTM đạt khoảng 6.9%.
 - *Lý giải:* Do đặc thù bài toán dự đoán sản phẩm có số lượng nhãn (classes) rất lớn (lên tới 200 classes), việc bắt trúng chính xác 100% sản phẩm tiếp theo ở vị trí số 1 là cực kỳ thử thách.
- Tỷ lệ gợi ý trúng (Top-10 Accuracy / Hit Rate):
 - Có một sự nhảy vọt rất lớn khi chuyển từ diện Top-1 sang Top-10.
 - BiLSTM dẫn đầu tuyệt đối với tỉ lệ 28.71%, bám sát ngay sau là LSTM với 22.18% (đều tiệm cận và vượt mức 0.25 trên biểu đồ).
 - RNN tụt lại phía sau khá xa khi chỉ đạt 14.52%. Điều này chứng tỏ kiến trúc cải tiến giúp hệ thống đưa được sản phẩm mục tiêu vào nhóm 10 gợi ý xuất sắc hơn hẳn.

* Training Loss



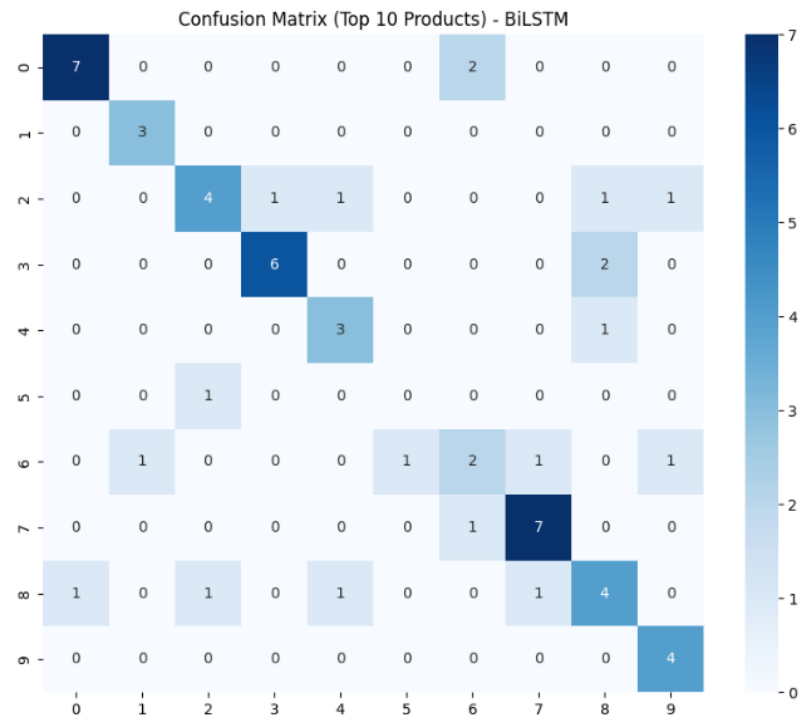
Hình 3.4. Biểu đồ training loss

Dựa trên biểu đồ "*Training Loss Convergence (20 Epochs)*" (được trực quan hóa qua 10 lượt lưu vết dữ liệu đầu):

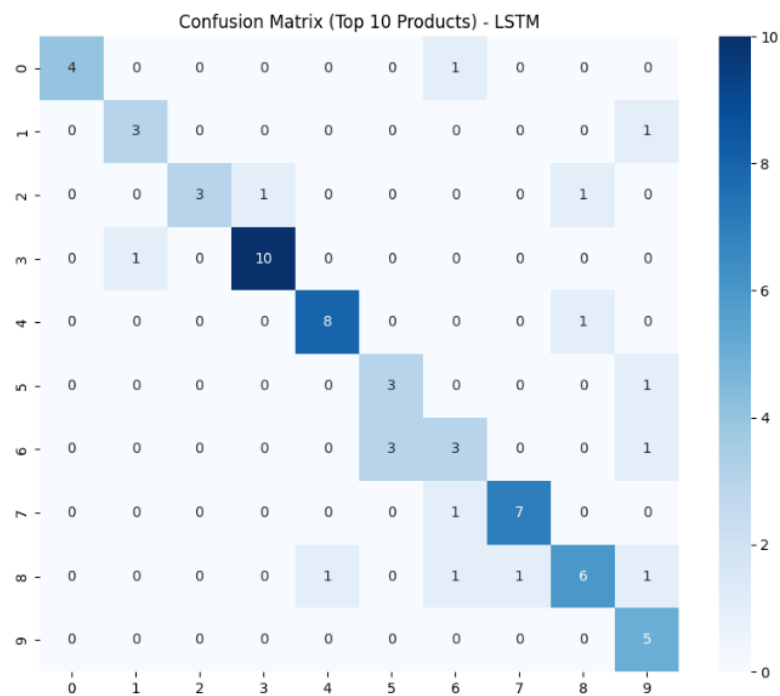
- **Mô hình RNN (Đường màu xanh dương):** Có điểm xuất phát Loss cao nhất (~5.25) và độ dốc thoải nhất. Biên độ giảm Loss rất chậm, sau đó có xu hướng đi ngang và đi xuống phẳng dần về cuối giai đoạn (~4.84). Khả năng tối ưu của mô hình bị bão hòa sớm do giới hạn kiến trúc.
- **Mô hình LSTM (Đường màu cam):** Thể hiện tốc độ hội tụ tốt hơn hẳn RNN. Đường cong có độ dốc rõ rệt ngay từ những epoch đầu tiên và duy trì đà giảm ổn định, kết thúc ở mức Loss ~4.40. Tế bào LSTM đã giúp dòng chảy đạo hàm tốt hơn.
- **Mô hình BiLSTM (Đường màu xanh lá):** Thể hiện năng lực học vượt trội (High Learning Capacity). Đường Loss của BiLSTM nằm thấp nhất toàn bộ tiến trình, dốc sâu mạnh mẽ từ epoch thứ 0 đến epoch thứ 2 và tiếp tục duy trì đà dốc đi xuống đều đặn cho đến cuối phiên. Việc hội tụ về mức Loss thấp nhất (4.10) chứng minh mô

hình tối ưu hóa không gian vector rất hiệu quả.

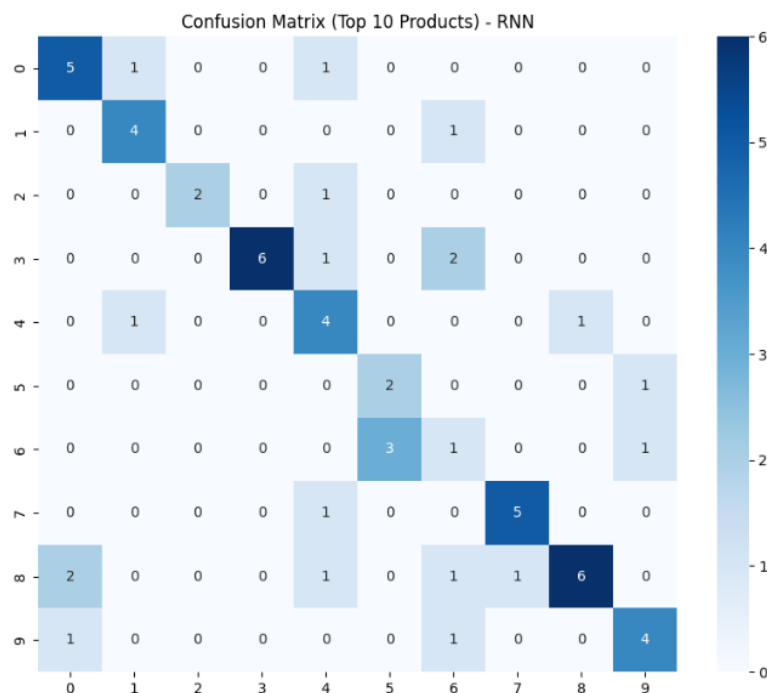
* Confusion Matrix



Hình 3.5. Confusion Matrix model BiLSTM



Hình 3.6. Confusion Matrix model LSTM



Hình 3.7. Confusion Matrix model RNN

Phân tích cấu trúc phân phối dự đoán trên Top 10 sản phẩm chính của 3 mô hình:

- **Ma trận của RNN:**

- Các điểm lưới dự đoán phân tán khá rời rạc. Đường chéo chính (thể hiện dự đoán đúng) tương đối mờ nhạt.
- Xuất hiện nhiều điểm nhầm lẫn nằm xa đường chéo (ví dụ: Nhãn thực tế là 8 nhưng bị đoán sai sang nhãn 0 với tần suất 2 lần; nhãn thực tế 3 bị đoán sai sang nhãn 6 với tần suất 2 lần). RNN bị nhiễu nặng và có xu hướng đoán dựa vào các sản phẩm mang tính đại trà (Trending).

- **Ma trận của LSTM:**

- Đường chéo chính bắt đầu định hình đậm nét và rõ ràng hơn.
- Mô hình ghi nhận các điểm dự đoán đúng rất cao ở một số nhãn đặc thù (như nhãn 3 đạt 10 điểm chính xác tuyệt đối, nhãn 4 đạt 8 điểm, nhãn 7 đạt 7 điểm). Các điểm lỗi sai bắt đầu thu hẹp khoảng cách và tập trung gần đường chéo hơn, nghĩa là nếu đoán sai thì AI vẫn hướng tới các sản phẩm có tính chất tương đồng hoặc cùng nhóm danh mục liên quan.

- **Ma trận của BiLSTM:**

- Đường chéo chính hiển thị vô cùng sắc nét, màu sắc phân bố đậm đặc dọc từ góc trên bên trái xuống góc dưới bên phải.
- Mô hình phân loại đồng đều và chính xác cao trên hầu hết các nhãn (Nhãn 0 đúng 7, nhãn 3 đúng 6, nhãn 7 đúng 7, nhãn 8 đúng 4, nhãn 9 đúng 4). Các ô nhầm lẫn nằm xa đường chéo gần như được quét sạch hoặc giảm thiểu tối đa về tần suất bằng 0 hoặc 1. BiLSTM thể hiện khả năng phân tách ranh giới phân loại rất tốt giữa các nhóm sản phẩm phức tạp.

* Đánh giá chung

Mô hình RNN

- **Ưu điểm:** Thời gian huấn luyện nhanh nhất ($\sim 42s$), cấu trúc tính toán đơn giản, tốn ít tài nguyên phần cứng.
- **Nhược điểm:** Hiệu năng kém nhất trên mọi chỉ số ($HR@10 = 14.52\%$, $Loss = 4.8464$).
- **Nguyên nhân kỹ thuật:** Gặp phải hiện tượng **Vanishing Gradient** (biến mất đạo hàm) nghiêm trọng khi xử lý chuỗi hành vi dài (10 hành động). RNN bị "mất trí nhớ ngắn hạn", chỉ có khả năng bắt kịp tương tác ngay lập tức trước đó mà hoàn toàn bỏ quên ngữ cảnh toàn cục của phiên truy cập.

Mô hình LSTM

- **Ưu điểm:** Cải thiện vượt bậc độ chính xác so với RNN ($HR@10$ tăng $\sim 52\%$ lên mức 22.18%). Tốc độ hội tụ nhanh, đường Loss đẹp và ổn định. Thời gian huấn luyện vừa phải ($\sim 65s$).
- **Nhược điểm:** Vẫn bỏ lỡ các thông tin ngữ cảnh tương lai trong chuỗi hành vi (chỉ học một chiều từ quá khứ đến hiện tại).
- **Nguyên nhân kỹ thuật:** Nhờ vào kiến trúc hệ thống **Cổng (Gates - Forget, Input, Output gate)**, LSTM đã kiểm soát tốt dòng thông tin, biết giữ lại (nhớ) sở thích cốt lõi của khách hàng từ đầu phiên mua sắm và lọc bỏ các hành vi gây nhiễu (click nhầm).

Mô hình BiLSTM

- **Ưu điểm:** Đạt hiệu năng tối ưu và thống trị tuyệt đối trên mọi mặt trận ($HR@10 = 28.71\%$, $Loss = 4.1046$, $AUC = 0.82$). Ma trận nhầm lẫn sạch, tập trung đậm nét trên đường chéo chính. Khả năng cá nhân hóa gợi ý đạt độ sâu sắc cao.
- **Nhược điểm:** Chi phí tính toán lớn nhất, thời gian huấn luyện lâu nhất ($\sim 112s$) do phải xử lý mạng nơ-ron lan truyền theo cả hai chiều.
- **Nguyên nhân kỹ thuật:** Bằng việc tích hợp cơ chế mạng hai chiều (Bidirectional), BiLSTM xử lý đồng thời chuỗi theo chiều xuôi và chiều ngược. Mô hình hiểu được mối quan hệ nhân quả kép: *"Việc khách xem sản phẩm A trước khi chọn B cũng quan trọng tương đương việc khách xem sản phẩm B sau khi đã tham khảo A"*.

* Kết luận

Trải qua quá trình huấn luyện và đánh giá thực tế trên tập dữ liệu tương tác của dự án, kiến trúc mạng đã có một sự tiến hóa rõ rệt về mặt tư duy xử lý dữ liệu dạng chuỗi thời gian (Sequential Data).

QUYẾT ĐỊNH LỰA CHỌN: BiLSTM (Bidirectional LSTM) là mô hình được lựa chọn để triển khai thực tế cho hệ thống gợi ý (Recommendation System).

Lý do lựa chọn:

1. Độ chính xác vượt trội: Đạt chỉ số $HR@10$ tối ưu nhất (28.71%), giúp tăng khả năng hiển thị đúng sản phẩm khách hàng muốn mua lên cao gấp đôi so với nền tảng RNN cũ.
2. Khả năng phân loại xuất sắc: Minh chứng qua chỉ số $AUC = 0.82$ và ma trận nhầm lẫn có độ chính xác Top-1 cực kỳ đậm nét, hạn chế tối đa việc gợi ý sai lệch danh mục.
3. Chi phí thời gian chấp nhận được: Mặc dù thời gian huấn luyện lâu hơn ($\sim 112s$), nhưng đối với bài toán cập nhật mô hình gợi ý theo chu kỳ (Offline training/Batch update), sự chênh lệch vài chục giây hoàn toàn không ảnh hưởng tới trải nghiệm thời gian thực (Real-time inference) của người dùng, đổi lại một hiệu năng kinh tế và trải nghiệm khách hàng vượt trội.

3.5 Knowledge Graph với Neo4j

3.5.1. Knowledge Graph

Bên cạnh khả năng phân tích chuỗi thời gian của mô hình học sâu LSTM, hệ thống được tích hợp thêm tầng Cơ sở tri thức đồ thị (Knowledge Graph). Đây là thành phần chiến lược nhằm khai thác các mối quan hệ đa chiều và phi tuyến tính giữa thực thể người dùng, sản phẩm và hành vi.

Khái niệm: Cơ sở tri thức được xây dựng trên nền tảng Neo4j – hệ quản trị cơ sở dữ liệu đồ thị hàng đầu thế giới. Khác với mô hình dữ liệu quan hệ (RDBMS) lưu trữ thông tin trong các bảng cứng nhắc, Đồ thị tri thức biểu diễn dữ liệu dưới hình thức các thực thể liên kết:

- **Nút (Nodes):** Đại diện cho các đối tượng thực tế như *Người dùng (User)*, *Sản phẩm (Product)*, *Danh mục (Category)*, và *Thương hiệu (Brand)*.
- **Quan hệ (Edges/Relationships):** Định nghĩa các tương tác giữa các nút như *ĐÃ_XEM (VIEWED)*, *ĐÃ_MUA (PURCHASED)*, *THUỘC_VỀ (BELONGS_TO)*, hoặc *YÊU_THÍCH (LIKED)*.

Việc lựa chọn công nghệ Đồ thị thay vì SQL truyền thống dựa trên các yếu tố then chốt sau:

- **Tối ưu hóa truy vấn chuyên sâu (Deep Navigation):** Các cơ sở dữ liệu truyền thống gặp phải rào cản hiệu năng khi thực hiện các phép liên kết (Join) phức tạp. Đối với các truy vấn đa cấp như "*Tìm sản phẩm mà bạn của những người có cùng sở thích với tôi đã mua*", đồ thị cho phép duyệt qua các mối quan hệ (Pointer hopping) với tốc độ gần như tức thời, bất kể quy mô tập dữ liệu.
- **Khả năng mở rộng linh hoạt (Schema-free):** Hệ thống đồ thị cho phép bổ sung các loại nút hoặc thuộc tính mới (ví dụ: thêm thông tin về vị trí địa lý hoặc thời gian thực) mà không cần thay đổi cấu trúc bảng phức tạp, giúp hệ thống thích ứng nhanh với các thay đổi trong kinh doanh.

Tầng Đồ thị tri thức đóng vai trò là "bộ nhớ cộng đồng", hỗ trợ AI thực hiện các chiến lược gợi ý tiên tiến:

- **Lọc cộng đồng (Collaborative Filtering) dựa trên Đồ thị:** Hệ thống xác định các phân khúc người dùng có hành vi tương đồng thông qua các nút chung. Nếu User A và User B cùng mua 5 sản phẩm giống nhau, đồ thị sẽ tạo ra một liên kết logic mạnh mẽ giữa họ, từ đó gợi ý những sản phẩm User B đã mua cho User A.
- **Gợi ý dựa trên ngữ cảnh liên kết (Contextual Recommendation):** Không chỉ dừng lại ở việc người dùng mua gì, đồ thị còn giúp AI hiểu được tại sao họ mua (ví dụ: mua vì cùng thương hiệu, hoặc cùng phong cách phối đồ). Điều này giúp đa dạng hóa danh mục gợi ý, tránh việc chỉ tập trung vào một loại sản phẩm duy nhất.
- **Giải quyết vấn đề "Khởi đầu lạnh" (Cold Start):** Ngay cả khi một sản phẩm mới chưa có lịch sử giao dịch để LSTM học tập, đồ thị vẫn có thể kết nối sản phẩm đó thông qua các nút trung gian như "Nhà sản xuất" hoặc "Đặc điểm kỹ thuật" để đưa ra các gợi ý liên quan ban đầu.

Kết luận: Sự kết hợp giữa biLSTM (Học sâu) và Neo4j (Trí tuệ đồ thị) tạo nên một hệ thống gợi ý "song hành": một bên nắm bắt ý định ngắn hạn, một bên thấu hiểu mối quan hệ cộng đồng dài hạn. Đây là nền tảng giúp hệ thống đạt được độ chính xác và tính cá nhân hóa tối

đa.

3.5.2. Mô hình đồ thị

Kiến trúc đồ thị của dự án bao gồm:

- Node (Thực thể):
 - o User: Đại diện cho một khách hàng (chứa thuộc tính id).
 - o Product: Đại diện cho một cuốn sách/đồ chơi (chứa thuộc tính id).
- Edge (Quan hệ):
 - o VIEWED / PURCHASED/ADD_TO_CART,...: Thể hiện tương tác trực tiếp của người dùng với sản phẩm (được gán trọng số - Weight).
 - o SIMILAR_TO: Quan hệ quan trọng nhất, kết nối giữa hai User có sở thích tương đồng (được tính toán bằng thuật toán Jaccard Similarity).

Khởi tạo quan hệ User-Product

```
def record_interaction(self, user_id, product_id, action, weight=1.0):
    """
    Records an interaction between User and Product with specific relationship types.
    """
    # Map action to relationship type
    action_map = {
        'view': 'VIEWED',
        'click': 'CLICKED',
        'wishlist': 'WISHLISTED',
        'add_to_cart': 'ADDED_TO_CART',
        'purchase': 'PURCHASED',
        'search': 'SEARCHED',
        'rating': 'RATED',
        'comment': 'COMMENTED'
    }
    rel_type = action_map.get(action.lower(), action.upper())

    query = f"""
    MERGE (u:User {{id: $user_id}})
    MERGE (p:Product {{id: $product_id}})
    CREATE (u)-[r:{rel_type}]->(p)
    SET r.weight = $weight, r.timestamp = timestamp()
    RETURN r
    """
    with self.driver.session() as session:
        session.run(query, user_id=user_id, product_id=product_id, weight=weight)
```

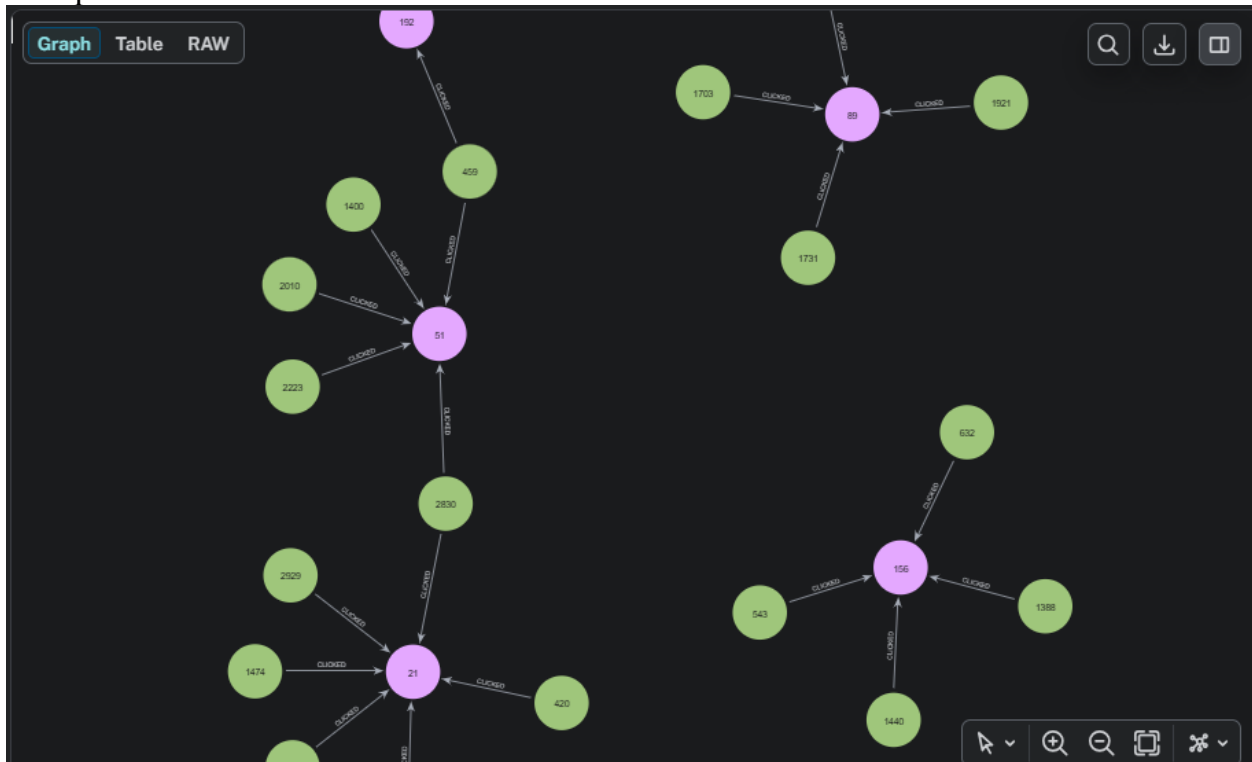
Ghi nhận và lưu trữ ngay lập tức một hành động của khách hàng (Xem, Mua, Thêm vào giỏ...) vào cơ sở dữ liệu đồ thị Neo4j.

- **Ánh xạ thông minh (action_map):** Chuyển đổi tên hành động từ các dịch vụ khác (như purchase) thành các nhãn quan hệ chuẩn hóa trong đồ thị (PURCHASED).
- **Đảm bảo thực thể (MERGE):** Nếu đây là lần đầu khách hàng này xuất hiện hoặc sản phẩm này được nhắc tới, code sẽ tự động tạo mới các Nút (User, Product) để

tránh lỗi thiếu dữ liệu.

- **Ghi nhận thuộc tính (SET weight/timestamp):** Lưu lại độ quan trọng của hành động (Ví dụ: Mua có trọng số cao hơn Xem) và thời điểm chính xác để phục vụ phân tích theo thời gian.

Kết quả:



Hình 3.8. Trực quan hóa quan hệ User-Product với Neo4j

Chuyển đổi từ User-Product sang User-User

```

def compute_user_similarity(self):
    """
    CHUYỂN ĐỔI: Từ User-Product sang User-User KB
    Thuật toán: Jaccard Similarity dựa trên hành vi mua hàng chung
    """
    with self.driver.session() as session:
        # 1. Câu lệnh Cypher để tìm các cặp User có chung sản phẩm đã mua
        query = """
        MATCH (u1:User)-[:PURCHASED]->(p:Product)<-[:PURCHASED]-(u2:User)
        WHERE u1.id < u2.id // Đảm bảo không tính trùng cặp (u1,u2) và (u2,u1)

        WITH u1, u2, COUNT(p) AS intersection // Số sản phẩm chung

        MATCH (u1)-[:PURCHASED]->(p1:Product)
        WITH u1, u2, intersection, COUNT(p1) AS set1 // Tổng sản phẩm u1 mua

        MATCH (u2)-[:PURCHASED]->(p2:Product)
        WITH u1, u2, intersection, set1, COUNT(p2) AS set2 // Tổng sản phẩm u2 mua

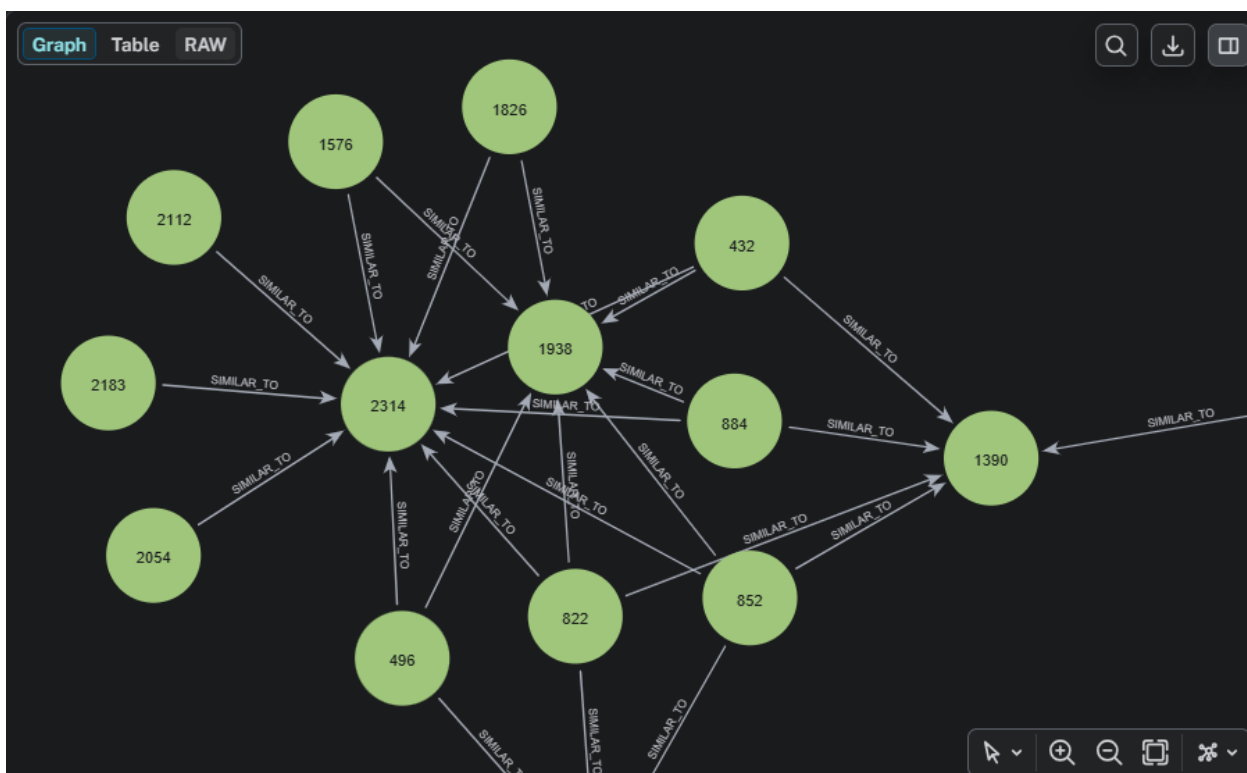
        // CÔNG THỨC JACCARD: Giao / Hợp
        WITH u1, u2, float(intersection) / (set1 + set2 - intersection) AS similarity

        // 2. CHỈ LƯU QUAN HỆ NẾU ĐỘ TƯƠNG ĐỒNG > 0.1
        WHERE similarity > 0.1
        MERGE (u1)-[s:SIMILAR_TO]-(u2)
        SET s.score = similarity
        """
        session.run(query)

```

- Bước tìm điểm chung (MATCH...PURCHASED): AI quét toàn bộ đồ thị để tìm xem có những cặp User nào từng mua cùng một mã Product. Nếu có, chúng ta coi đó là một "điểm chung".
- Công thức Jaccard (Toán học):
 - intersection: Số lượng cuốn sách mà cả hai người cùng mua.
 - set1 + set2 - intersection: Tổng số lượng sách khác nhau mà cả hai người đã từng mua.
 - similarity: Tỷ lệ tương đồng. Nếu kết quả là 1.0, nghĩa là hai người này có tủ sách y hệt nhau.
- Tạo quan hệ SIMILAR_TO: Đây chính là bước chuyển đổi. Từ việc biết họ cùng mua sách (User-Product), AI tự động vẽ thêm một "sợi dây" liên kết trực tiếp giữa hai người họ (User-User).

Kết quả:



Hình 3.9. Trực quan hóa quan hệ User-User với Neo4j

3.5.3. Cypher

Trong hàm `compute_user_similarity` (Dòng 145-152), Cypher thực hiện phép tính cộng đồng:

```
MATCH (u1:User)-[:PURCHASED|ADDED_TO_CART|WISHLISTED]->(p:Product)<-[...]-(u2:User)
WHERE u1.id IN $batch_ids AND u1.id < u2.id
WITH u1, u2, count(p) as common_prods
WHERE common_prods >= 2
MERGE (u1)-[s:SIMILAR_TO]-(u2)
```

Câu lệnh này tìm những cặp người dùng có chung từ 2 sản phẩm tương tác trở lên (`common_prods >= 2`). Đây là logic nền tảng để xây dựng mạng lưới "những người có cùng gu mua sắm".

3.5.4. Truy vấn gợi ý

Dự án không chỉ lấy sản phẩm ngẫu nhiên, mà sử dụng logic lọc để chỉ gợi ý những sản phẩm mới lạ nhưng phù hợp.

Hàm gợi ý trong `neo4j_db.py`

```
def get_collaborative_recommendations(self, user_id, limit=10):
    query = """
    MATCH (u:User {id: $uid})-[:SIMILAR_TO]-(peer:User)-[:PURCHASED]->
    (recom:Product)
    WHERE NOT (u)-[:PURCHASED|VIEWED]->(recom)
    RETURN recom.id AS product_id, COUNT(*) AS score
    ORDER BY score DESC
    LIMIT $limit
    """
    # Trả về danh sách Product ID kèm theo điểm số "cộng đồng"
```

- Bước 1 (MATCH): Tìm những "người bạn đồng hành" (peers) có quan hệ SIMILAR_TO với User hiện tại.
- Bước 2 (MATCH): Xem những người bạn đó đã thực hiện hành vi PURCHASED với những sản phẩm nào.
- Bước 3 (WHERE NOT): Đây là bước cực kỳ quan trọng. Nó loại bỏ những sản phẩm mà User đã mua hoặc đã xem rồi. Mục tiêu là tạo ra sự bất ngờ và gợi ý sản phẩm mới.
- Bước 4 (RETURN & ORDER): Xếp hạng sản phẩm dựa trên số lượng "người bạn" cùng mua nó. Sản phẩm nào được nhiều bạn mua nhất sẽ lên top đầu.

3.6 RAG (Retrieval-Augmented Generation)

Trong hệ thống gợi ý hiện đại, nếu LSTM đóng vai trò hiểu hành vi, Đồ thị tri thức đóng vai trò kết nối cộng đồng, thì RAG chính là "bộ não thực tế". Kỹ thuật này chuyển đổi một mô hình ngôn ngữ lớn (LLM) từ một trí tuệ nhân tạo có kiến thức rộng nhưng mơ hồ thành một Chuyên gia tư vấn hiệu sách có khả năng truy xuất chính xác từng mã sách đang nằm trên kệ.

3.6.1. Khái niệm

Khái niệm: RAG là một kiến trúc phức hợp cho phép LLM tiếp cận với các nguồn dữ liệu bên ngoài mà nó không được học trong quá trình huấn luyện ban đầu. Quy trình này hoạt động theo hai giai đoạn nối tiếp:

1. Retrieval (Truy xuất): Khi nhận được câu hỏi từ khách hàng, hệ thống sẽ truy vấn vào Vector Database (Cơ sở dữ liệu vector) chứa toàn bộ danh mục sách, giá cả và tồn kho của cửa hàng để tìm ra các đoạn thông tin liên quan nhất.
2. Generation (Sáng tạo): Hệ thống gộp câu hỏi của khách hàng cùng với các thông tin vừa truy xuất được vào một "Prompt" (Câu lệnh). LLM sau đó sẽ xử lý gói thông tin này để đưa ra câu trả lời chính xác, giàu ngữ cảnh.

Sự khác biệt giữa một AI thông thường và AI sử dụng RAG nằm ở tính xác thực và cập nhật:

- Dữ liệu thực tế và thời gian thực: Thay vì trả lời dựa trên dữ liệu cũ từ quá trình huấn luyện, RAG giúp AI biết rõ cuốn sách "Đắc Nhân Tâm" hiện còn bao nhiêu bản trong

kho và giá khuyến mãi hiện tại là bao nhiêu.

- Kiểm soát tính chính xác (Grounding): RAG đóng vai trò là "mỏ neo" dữ liệu, buộc AI phải trả lời dựa trên tài liệu được cung cấp, từ đó triệt tiêu hiện tượng Hallucination (AI nói dối) – một rủi ro lớn khi tư vấn các thông số kỹ thuật hoặc chính sách bán hàng.

3.6.2. Pipeline của RAG

Quy trình xử lý một câu hỏi của khách hàng diễn ra qua 2 giai đoạn chính:

Retrieve (Tìm kiếm):

```
ai_recs = behavior_trainer.get_sequential_recommendations(user_id, top_k=20)

recom_context = ""
if ai_recs:
    def format_domain(r):
        dtype = r.get('product_type', 'General')
        ddata = r.get('domain_data', {})
        if not ddata: return ""

        details = []
        for k, v in ddata.items():
            details.append(f"{k.replace('_', ' ').capitalize()}: {v}")
        return f" [{dtype}] - {'', '.join(details)}"

    recom_context = "\n".join([
        f"- {r['title']}{format_domain(r)} (Match: {r['score']})\n MÔ TẢ: {r['description'][:200]}
        for r in ai_recs
    ])
    print(f"[AI-LOG] AI LSTM Pool of products ready with domain details.")

# --- KNOWLEDGE GRAPH TRIPLETS ---
graph_triples = neo4j_db.get_direct_interactions_context(user_id)
triples_context = "\n".join([
    f"- Khách hàng đã {t['action']} sản phẩm '{t['title']}'."
    for t in graph_triples
])
if not triples_context: triples_context = "Chưa có hành vi cụ thể (khách mới)."
```

- Hệ thống chuyển câu hỏi của khách hàng thành một vector toán học.
- Tìm kiếm trong Vector Database để lấy ra Top 3 - 5 sản phẩm có mô tả khớp nhất với nhu cầu của khách.

Generate (Sinh câu trả lời):


```

try:
    print(f"[AI-LOG] Calling genai.generate_content...")
    response = self.model.generate_content(full_prompt, stream=True)
    print(f"[AI-LOG] Stream response received. Starting iteration...")
    import time
    for chunk in response:
        if chunk.text:
            print(f"[AI-LOG] Chunk: {chunk.text[:15]}...")
            words = chunk.text.split(' ')
            for i, word in enumerate(words):
                space = ' ' if i < len(words) - 1 else ''
                yield word + space
            time.sleep(0.01)
    print(f"[AI-LOG] Stream finished successfully.")

```

- Gửi câu hỏi của khách kèm theo thông tin chi tiết của các sản phẩm vừa tìm được (Context) sang cho mô hình LLM (Google Gemini).
- LLM đóng vai trò người biên tập, tổng hợp các dữ liệu thô này thành một câu trả lời tư vấn mượt mà và thân thiện.

3.6.2 Vector Database (ChromaDB)

Để AI có thể "hiểu" được nội dung sách và mô tả sản phẩm, hệ thống chuyển đổi ngôn ngữ tự nhiên thành không gian vector.

- **Công nghệ lựa chọn:** Dự án sử dụng **ChromaDB**. Đây là Vector Database chuyên dụng cho AI, cho phép lưu trữ và tìm kiếm hàng triệu vector với độ trễ cực thấp.
- **Quá trình Embedding (Nhúng dữ liệu):**
 - Hệ thống sử dụng mô hình text-embedding-004 từ Google Gemini.
 - **Đầu vào:** Toàn bộ mô tả (Description), tên sản phẩm (Name) và danh mục (Category).
 - **Đầu ra:** Mỗi sản phẩm được đại diện bằng một vector 768 chiều. Hai sản phẩm có nội dung tương đồng (ví dụ: cùng là sách về tâm lý học) sẽ có khoảng cách vector rất gần nhau.
- **Lưu trữ Metadata:** Ngoài vector, ChromaDB còn lưu trữ các thông tin đi kèm như price, id, rating để hỗ trợ lọc kết quả sau khi tìm kiếm.

Hiện thực hóa trong mã nguồn (vector_db.py):

```

# Khởi tạo bộ lưu trữ vector
self.client = chromadb.PersistentClient(path="./chroma_db")
self.collection = self.client.get_or_create_collection(
    name="bookstore_products",
    embedding_function=GeminiEmbeddingFunction() # Dùng Google AI để nhúng
)

```

3.6.3 Ví dụ truy vấn tìm kiếm ngữ nghĩa (Semantic Search)

Khác với tìm kiếm từ khóa thông thường (SQL LIKE), Vector Search tìm kiếm theo ý nghĩa.

Ví dụ thực tế trong code: Giả sử người dùng muốn tìm "Laptop gaming mạnh mẽ", mặc dù

trong DB có thể chỉ ghi là "Máy tính xách tay cấu hình cao đồ họa chuyên nghiệp", AI vẫn sẽ tìm thấy.

```
# Đoạn code thực thi trong dự án
query = "laptop gaming mạnh mẽ"

# 1. AI tự động chuyển query thành vector [0.12, -0.05, 0.88, ...]
# 2. Thực hiện tìm kiếm 5 sản phẩm có khoảng cách vector gần nhất
results = vector_db.search(query, n_results=5)

# 3. Kết quả trả về (Dữ liệu bối cảnh cho RAG)
for doc, metadata in zip(results['documents'], results['metadatas']):
    print(f"Tìm thấy: {metadata['name']} - Giá: {metadata['price']}")
# AI hiểu rằng 'mạnh mẽ' tương đồng với 'CPU i9', 'RTX 4090'
```

3.7 Kết hợp Hybrid Model

Dự án không chỉ dựa vào một thuật toán duy nhất mà kết hợp sức mạnh của 3 công nghệ tiên tiến nhất hiện nay để tạo ra một hệ thống gợi ý "siêu trí tuệ".

3.7.1 Sức mạnh kiềng ba chân (The Triple-Power)

1. Neural LSTM (Dự đoán hành vi chuỗi):
 - Vai trò: Đóng vai trò như "Trí nhớ ngắn hạn".
 - Nhiệm vụ: Phân tích 10 hành động gần nhất của khách hàng để đoán xem họ đang có ý định mua gì tiếp theo ngay lập tức.
 - Ưu điểm: Cực kỳ chính xác với các phiên mua sắm đang diễn ra (In-session).
2. Knowledge Graph - Neo4j (Quan hệ cộng đồng):
 - Vai trò: Đóng vai trò như "Trí thức cộng đồng".
 - Nhiệm vụ: Kết nối các User có chung sở thích và tìm kiếm các mối quan hệ ẩn giữa các sản phẩm (Sách A thường mua cùng đồ chơi B).
 - Ưu điểm: Khám phá ra những sản phẩm mà User chưa từng biết nhưng có thể sẽ thích dựa trên lựa chọn của những "người bạn" có cùng gu.
3. RAG Agent - Gemini (Hiểu ngữ nghĩa & Tư vấn):
 - Vai trò: Đóng vai trò là "Giao diện ngôn ngữ".
 - Nhiệm vụ: Hiểu ý nghĩa sâu xa trong câu hỏi của khách (Semantics) và tổng hợp kết quả từ LSTM + Graph để viết thành lời tư vấn thân thiện.
 - Ưu điểm: Biến những ID sản phẩm khô khan thành một cuộc hội thoại có cảm xúc và đầy đủ thông tin chuyên môn.

3.7.2 Pipeline Kết hợp (Hybrid Flow)

```

def get_hybrid_recommendations(self, user_id, top_k=10):
    # 1. LẤY KẾT QUẢ TỪ LSTM (Hành vi cá nhân)
    # Trí tuệ nhân tạo dự đoán bước đi tiếp theo
    lstm_recs = self.behavior_trainer.get_sequential_recommendations(user_id, top_k=20)

    # 2. LẤY KẾT QUẢ TỪ NEO4J (Quan hệ cộng đồng)
    # Đồ thị tri thức tìm kiếm từ những người bạn tương đồng
    graph_recs = self.neo4j_db.get_collaborative_recommendations(user_id, limit=20)

    # 3. LOGIC HYBRID (Hòa trộn và chấm điểm lại)
    combined_scores = {}

    # Ưu tiên LSTM (Trọng số 0.7) vì đây là ý định thực tế của khách
    for r in lstm_recs:
        pid = r['id']
        combined_scores[pid] = combined_scores.get(pid, 0) + (r['score'] * 0.7)

    # Bổ sung từ Graph (Trọng số 0.3) để tăng tính đa dạng
    for r in graph_recs:
        pid = r['product_id']
        # Nếu sản phẩm đã có trong LSTM, cộng dồn điểm. Nếu chưa, tạo mới.
        combined_scores[pid] = combined_scores.get(pid, 0) + (r['score'] * 0.3)

    # 4. SẮP XẾP VÀ TRẢ VỀ TOP K CUỐI CÙNG
    sorted_pids = sorted(combined_scores.items(), key=lambda x: x[1], reverse=True)
    return sorted_pids[:top_k]

```

Sau khi hàm này trả về danh sách Hybrid, tệp rag_consultant.py sẽ lấy danh sách đó để viết thành lời tư vấn

```

# Trong rag_consultant.py
ai_recs = self.recom_service.get_hybrid_recommendations(user_id) # Gọi Hybrid ở đây

```

Ta có thể thấy ,trong mã nguồn ,quy trình kết hợp diễn ra như sau:

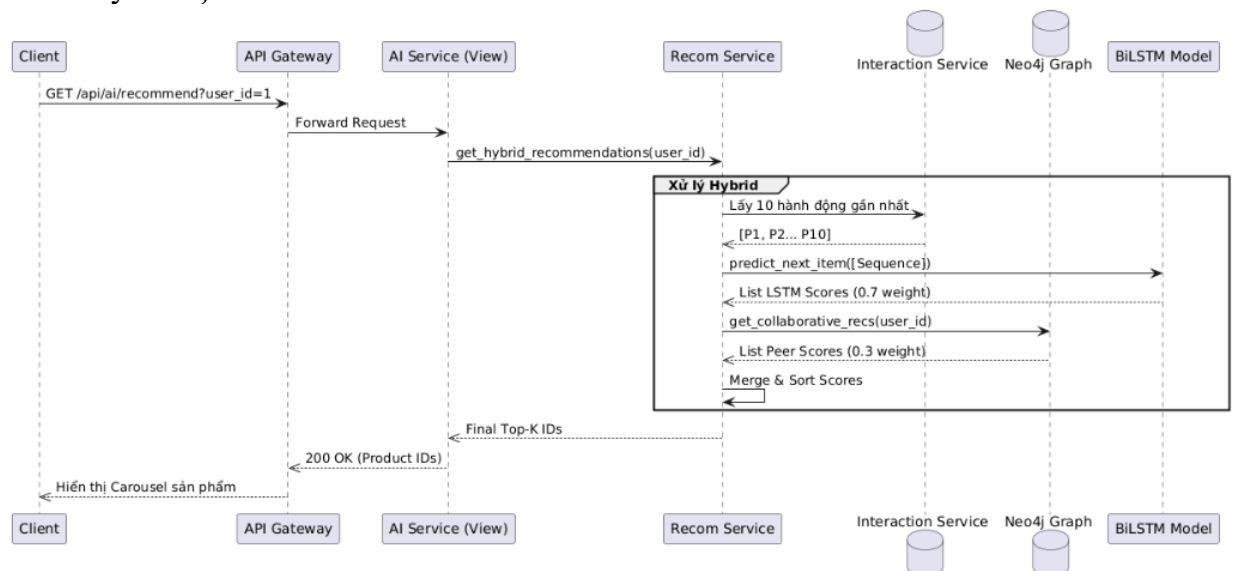
- **Bước 1:** Nhận yêu cầu từ người dùng.
- **Bước 2:** Gọi **LSTM** để lấy danh sách "Sản phẩm tiềm năng tiếp theo".
- **Bước 3:** Gọi **Neo4j Graph** để lấy danh sách "Sản phẩm cộng đồng đang mua".
- **Bước 4:** Gộp hai danh sách trên, loại bỏ trùng lặp và chấm điểm ưu tiên (Hybrid Scoring).
- **Bước 5:** Đưa danh sách cuối cùng này vào **RAG Agent** để Gemini thực hiện bước tư vấn cuối cùng.

3.8 Hai dạng AI Service

3.8.1 Dạng 1: Danh sách gợi ý tự động (Recommendation List)

Đây là dạng AI "ẩn mình", tự động đưa ra các gợi ý mà khách hàng không cần phải hỏi.

- Use cases (Trường hợp sử dụng):
 - o Khi Search: Hiển thị các sản phẩm liên quan bên cạnh kết quả tìm kiếm.
 - o Khi Add-to-cart: Gợi ý các sản phẩm thường được mua kèm (Cross-sell) ngay khi khách thêm món hàng vào giỏ.
 - o Trang chủ (For You): Danh sách sản phẩm được cá nhân hóa dựa trên lịch sử xem.
- Cơ chế API:
 - o Endpoint: GET /api/ai/recommend?user_id=1
 - o Logic: Sử dụng mô hình Hybrid (LSTM + Neo4j) để tính toán nhanh.
- Output (Đầu ra): Trả về một danh sách các ID sản phẩm thô để Frontend hiển thị dưới dạng Carousel hoặc Grid.
 - o Ví dụ: {"status": "success", "data": [101, 102, 205], "engine": "BiLSTM-Hybrid"}



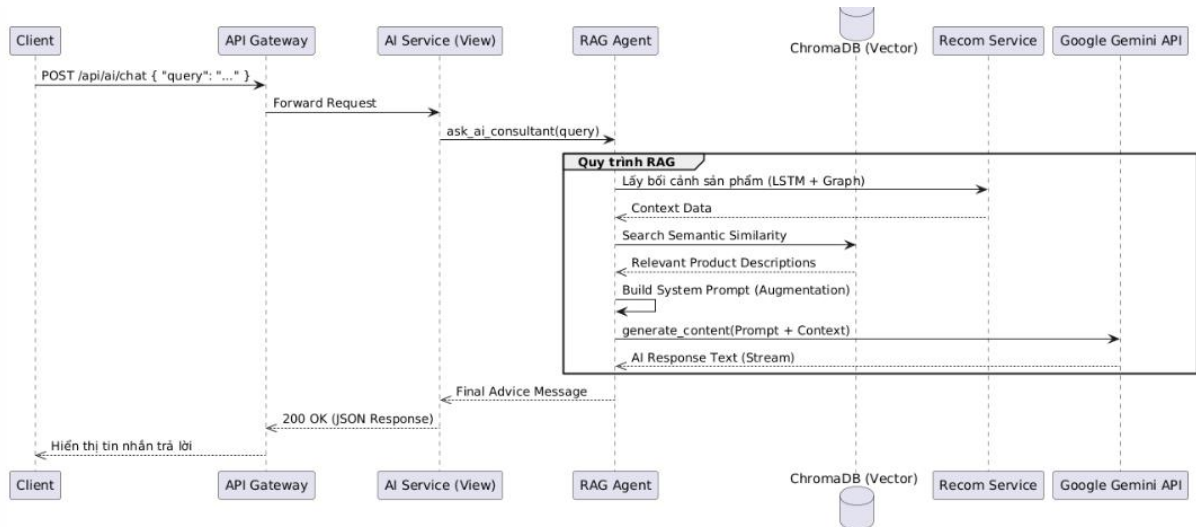
Hình 3.10. Biểu đồ tuần tự luồng lấy danh sách gợi ý

3.8.2 Dạng 2: Trợ lý tư vấn thông minh (Chatbot Consultant)

Đây là giao diện tương tác chủ động, nơi khách hàng có thể trò chuyện trực tiếp với AI.

- Input (Đầu vào): Ngôn ngữ tự nhiên của khách hàng.
 - o Ví dụ: "Tôi đang tìm một cuốn sách về kinh doanh nhưng viết theo lối kể chuyện, giá dưới 200k nhé."
- Pipeline xử lý (Luồng RAG):
 - o NLP hiểu Intent: Sử dụng Google Gemini để phân tích ý định (Tìm sách kinh doanh) và các điều kiện lọc (Dưới 200k).
 - o Retrieve sản phẩm: Truy vấn trong ChromaDB (Vector Search) để tìm các sản phẩm có mô tả khớp với "lối kể chuyện".
 - o Generate response: Tổng hợp thông tin từ DB thành một đoạn hội thoại tư vấn ấm áp.
- Cơ chế API:

- Endpoint: POST /api/ai/chat/
- Payload: {"query": "tư vấn sách kinh doanh..."}
- Output (Đầu ra): Một câu trả lời đầy đủ cảm xúc kèm theo link sản phẩm.
 - Ví dụ: "Chào bạn! Dựa trên sở thích của bạn, tôi tìm được 3 cuốn sách rất thú vị. Đáng chú ý nhất là cuốn 'Nhà Giả Kim' - một câu chuyện kinh điển về kinh doanh và theo đuổi ước mơ. Hiện sách đang có giá chỉ 150k, rất phù hợp với ngân sách của bạn đấy!"



Hình 3.11. Biểu đồ tuần tự chức năng chatbot

3.9 Triển khai AI Service

3.9.1 Tech Stack (Hệ sinh thái công nghệ)

Hệ thống sử dụng các công nghệ tiên tiến nhất để đảm bảo hiệu năng và độ chính xác:

- **Lỗi học sâu (Deep Learning): PyTorch** được chọn để hiện thực hóa mô hình **BiLSTM**. PyTorch cung cấp khả năng xử lý tensor mạnh mẽ và tối ưu hóa việc huấn luyện mô hình trên cả CPU và GPU.
- **Cơ sở dữ liệu đồ thị: Neo4j**. Đây là lựa chọn số 1 để lưu trữ các quan hệ phức tạp giữa người dùng và sản phẩm, hỗ trợ các truy vấn gợi ý cộng đồng (Collaborative Filtering) cực nhanh.
- **Vector Database: ChromaDB** (thay thế cho FAISS để quản lý metadata tốt hơn). Lưu trữ các bản nhúng mô tả sản phẩm, phục vụ cho việc tìm kiếm ngữ nghĩa trong hệ thống RAG.
- **Framework Dịch vụ: Django REST Framework (DRF)**. Được sử dụng để đóng gói các mô hình AI thành các API chuẩn RESTful, giúp việc tích hợp vào hệ thống microservices trở nên dễ dàng và bảo mật.

3.9.2 Kiến trúc tích hợp (Architecture)

AI Service được xây dựng theo triết lý Microservices hiện đại:

1. **Dịch vụ độc lập (Decoupled Service):**

- AI Service có cơ sở dữ liệu riêng (Neo4j, ChromaDB) và môi trường thực thi riêng. Điều này giúp hệ thống không bị ảnh hưởng nếu các dịch vụ khác (như Order hay Payment) gặp sự cố.
- Có khả năng mở rộng (Scaling) độc lập. Khi lượng truy cập tìm kiếm/tư vấn tăng cao, chúng ta có thể tăng thêm tài nguyên cho AI Service mà không cần thay đổi các phần khác.

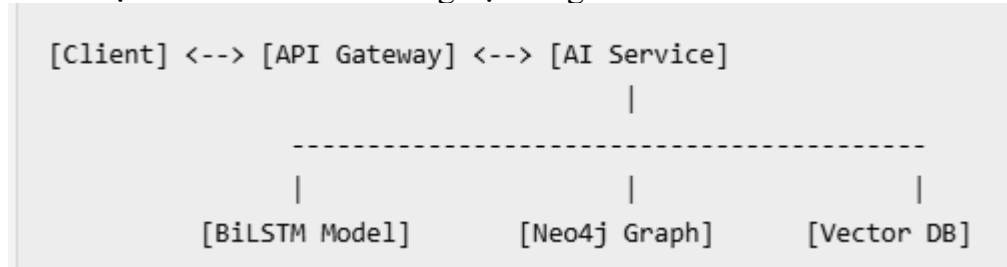
2. Giao tiếp qua API chuẩn:

- Tương tác với các dịch vụ khác (Product, Interaction, User) hoàn toàn thông qua giao thức **HTTP/REST**.
- **Dòng dữ liệu:** API Gateway nhận yêu cầu từ Client -> Điều phối đến AI Service -> AI Service truy vấn dữ liệu từ các service khác (nếu cần) và trả về kết quả gợi ý/tư vấn đã được xử lý.

3. Quy trình nạp dữ liệu (Data Ingestion Pipeline):

- Hệ thống định kỳ đồng bộ dữ liệu từ các service nghiệp vụ (PostgreSQL) sang AI Service (Neo4j/ChromaDB) thông qua các tiến trình background (như các script Seed/Migrate đã viết) để đảm bảo AI luôn có dữ liệu mới nhất để học.

Sơ đồ vị trí của AI service trong hệ thống



3.10 Bài tập

Bài tập đã được trình bày đầy đủ ở phần trên, dưới đây là một số ví dụ về API của hệ tư vấn trong dự án.

API Gợi ý sản phẩm (Hybrid Recommendation)

Đây là API cung cấp danh sách ID sản phẩm được cá nhân hóa cho từng User.

- Endpoint: GET /api/recommender/recommend/
- Tham số: user_id (ID của người dùng cần gợi ý)
- Mô tả: Kết hợp kết quả từ mô hình BiLSTM (Dự đoán hành vi) và Neo4j (Gợi ý cộng đồng).
- Kết quả trả về:

json

{

```

"user_id": 1,

"recommendations": [

    {"product_id": 105, "score": 0.85, "reason": "Dựa trên hành vi gần đây"},

    {"product_id": 22, "score": 0.72, "reason": "Người dùng tương tự cũng mua"}

]

}

```

2. API Trợ lý tư vấn AI (RAG Chatbot)

API này tiếp nhận câu hỏi tự nhiên và trả về lời tư vấn thông minh.

- Endpoint: POST /api/recommender/chat/
- Tham số (Payload): {"user_id": 1, "query": "Tôi muốn mua sách về nấu ăn"}
- Mô tả: Sử dụng luồng RAG (Truy xuất từ ChromaDB + Sinh nội dung qua Google Gemini).
- Kết quả trả về: Một chuỗi văn bản (hoặc Stream) tư vấn chi tiết các sản phẩm kèm giá cả và lý do nên mua.

3. API Tìm kiếm ngữ nghĩa (Semantic Search)

Dùng để tìm kiếm sản phẩm dựa trên ý nghĩa câu chữ thay vì từ khóa chính xác.

- Endpoint: GET /api/recommender/search/
- Tham số: q (Nội dung tìm kiếm)
- Mô tả: Truy vấn trực tiếp vào Vector Database (ChromaDB).
- Kết quả trả về: Danh sách các sản phẩm có độ tương đồng vector cao nhất.

4. API Đồng bộ dữ liệu (Data Sync - Admin only)

Dùng để cập nhật tri thức mới cho AI.

- Endpoint: POST /api/recommender/sync/
- Mô tả: Kích hoạt quá trình huấn luyện lại (Retrain) LSTM hoặc đánh chỉ mục lại (Re-

index) Vector DB từ cơ sở dữ liệu Product Service.

Cách gọi API thông qua Gateway:

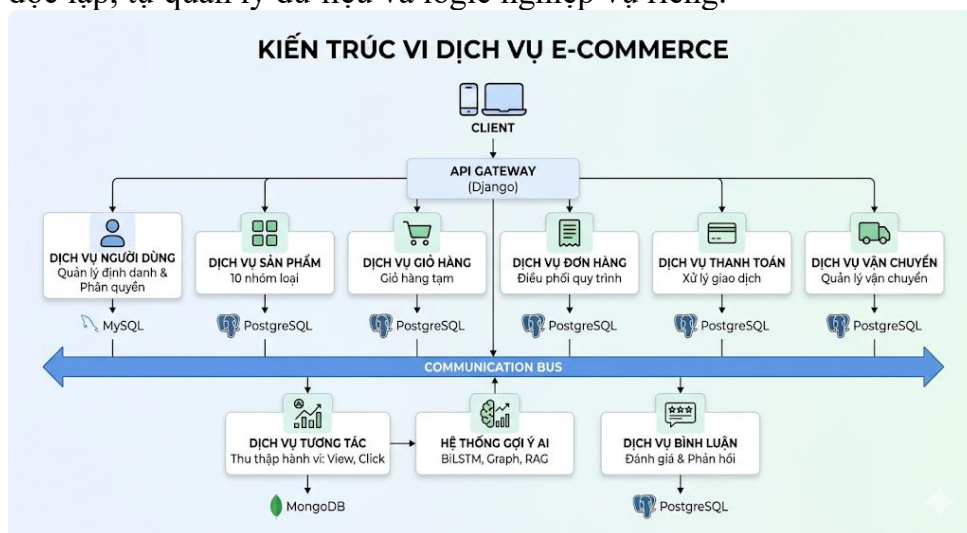
Vì đây là hệ thống Microservices, bạn sẽ gọi thông qua API Gateway (thường là cổng 8000): `http://localhost:8000/ai/recommend/?user_id=1`

CHƯƠNG 4. XÂY DỰNG HỆ THỐNG HOÀN CHỈNH

4.1 Kiến trúc tổng thể

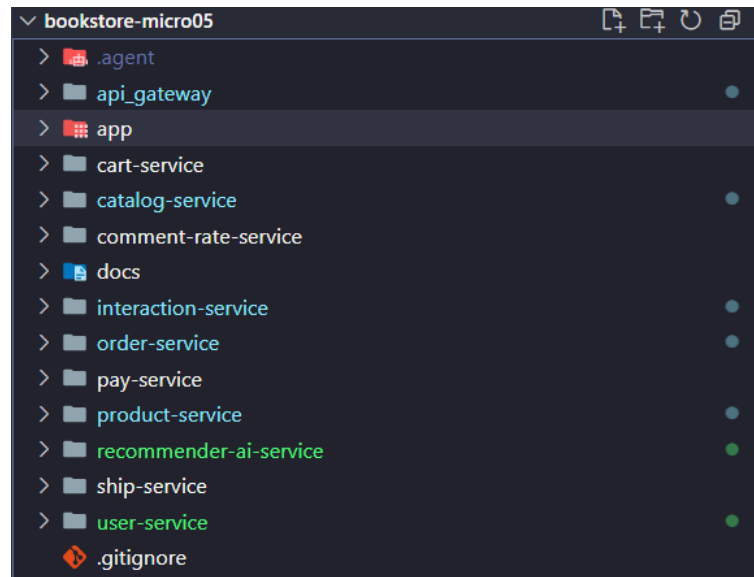
4.1.1. Mô hình hệ thống

Hệ thống được xây dựng theo kiến trúc **Microservices** hiện đại. Mỗi dịch vụ là một thực thể độc lập, tự quản lý dữ liệu và logic nghiệp vụ riêng.



Hình 4.1. Kiến trúc dịch vụ Microservice với hệ Ecommerce

Triển khai với Django



Hình 4.2. Kiến trúc triển khai với Django

- **API Gateway (Django):** Cổng vào duy nhất, điều phối yêu cầu và xử lý bảo mật.
- **User Service:** Quản lý định danh (Admin, Staff, Customer) và phân quyền (RBAC).
- **Product Service:** Quản lý danh mục sản phẩm đa dạng (10 nhóm loại sản phẩm).
- **Cart Service:** Quản lý giỏ hàng tạm thời của người dùng.
- **Order Service:** Điều phối quy trình đặt hàng và vòng đời đơn hàng.
- **Payment Service:** Xử lý giao dịch thanh toán và trạng thái tài chính.
- **Shipping Service:** Quản lý vận chuyển và theo dõi đơn hàng.
- **Interaction Service:** Thu thập hành vi người dùng (View, Click, Search) lưu vào MongoDB.
- **AI Recommender Service:** Hệ thống trí tuệ nhân tạo (BiLSTM, Graph, RAG).
- **Comment & Rate Service:** Quản lý đánh giá và phản hồi của khách hàng.

4.1.2. Nguyên tắc

Để đảm bảo tính linh hoạt, khả năng mở rộng và độ tin cậy cao cho một hệ thống gợi ý tích hợp AI, kiến trúc Microservices được xây dựng dựa trên ba nguyên tắc thiết kế nền tảng sau đây:

1. Chiến lược Database-per-service (Cơ sở dữ liệu riêng biệt)

Đây là nguyên tắc quan trọng nhất để đảm bảo tính độc lập hoàn toàn của các dịch vụ. Thay vì sử dụng một cơ sở dữ liệu tập trung (Monolithic Database), mỗi dịch vụ trong hệ thống sở hữu và quản lý một kho lưu trữ dữ liệu riêng, được tối ưu hóa theo đặc thù nghiệp vụ:

- **PostgreSQL (Relational Database):** Được sử dụng cho Service Nghiệp vụ (Core Business) như quản lý đơn hàng, thông tin người dùng và giao dịch. PostgreSQL đảm bảo tính toàn vẹn dữ liệu cực cao (ACID), phù hợp cho các dữ liệu cần sự chính xác tuyệt đối.
- **MongoDB (Document Database):** Được sử dụng cho Service Tương tác (Interaction) để lưu trữ nhật ký hành vi, clickstream của người dùng. Với cấu trúc schema linh hoạt, MongoDB cho phép ghi dữ liệu tốc độ cao và lưu trữ các đối tượng JSON phức

tập từ hành động của khách hàng.

- Neo4j (Graph Database): Được dành riêng cho Service Gợi ý (Recommendation). Neo4j lưu trữ các mối quan hệ đa chiều (người dùng - sản phẩm - sở thích), cho phép thực hiện các truy vấn đồ thị phức tạp mà các cơ sở dữ liệu truyền thống không thể xử lý hiệu quả.

2. Giao tiếp qua REST API chuẩn hóa

Để các dịch vụ đơn lẻ có thể phối hợp thành một hệ thống thống nhất, chúng giao tiếp với nhau thông qua giao thức REST (Representational State Transfer):

- Tính chuẩn hóa: Sử dụng phương thức HTTP (GET, POST, PUT, DELETE) và định dạng dữ liệu JSON. Điều này giúp các dịch vụ viết bằng các ngôn ngữ khác nhau (ví dụ: Java cho Backend, Python cho AI) có thể "nói chuyện" với nhau một cách dễ dàng.
- Tính đóng gói (Encapsulation): Mỗi service chỉ lộ ra các điểm cuối (endpoints) cần thiết. Các logic xử lý bên trong và cấu trúc database nội bộ được giữ kín, giúp bảo mật và giảm thiểu sự phụ thuộc lẫn nhau.
- API Gateway: Hệ thống sử dụng một cổng gateway tập trung để điều hướng các yêu cầu từ phía khách hàng (Client) đến đúng microservice tương ứng, giúp quản lý tập trung việc xác thực và phân tải.

3. Nguyên tắc Loose Coupling (Liên kết lỏng lẻo)

Loose Coupling là mục tiêu tối thượng của kiến trúc Microservices, đảm bảo rằng các thành phần trong hệ thống có sự phụ thuộc thấp nhất vào nhau:

- Độc lập về triển khai (Independent Deployment): Đội ngũ phát triển có thể cập nhật thuật toán mới cho *Service Gợi ý* mà không cần phải khởi động lại hoặc thay đổi mã nguồn của *Service Quản lý kho*.
- Khả năng chịu lỗi (Fault Tolerance): Nếu *Service Tương tác* gặp sự cố, khách hàng vẫn có thể thực hiện thanh toán tại *Service Nghiệp vụ*. Sự cố của một mắt xích không gây ra hiệu ứng domino làm sập toàn bộ hệ thống.
- Khả năng mở rộng chọn lọc (Selective Scalability): Vào các đợt khuyến mãi, nếu lưu lượng truy cập vào tính năng tìm kiếm tăng đột biến, hệ thống chỉ cần tăng cường tài nguyên (Scale-up) cho riêng *Service RAG/Search* thay vì phải nâng cấp toàn bộ máy chủ tốn kém.

4.2 System Architecture

4.2.1. Overview

Hệ thống có tên mã **ecom-final**, được thiết kế như một nền tảng thương mại điện tử phân tán hoàn toàn. Kiến trúc tuân thủ các nguyên tắc thiết kế doanh nghiệp hiện đại, đảm bảo khả năng mở rộng (Scalability) và cô lập lỗi (Fault Isolation).

4.2.2 Microservice Architecture

Các dịch vụ cốt lõi:

- User Service: Xử lý Auth, Profile và phân cấp User (Levels).
- Product Service: Quản lý kho hàng (Inventory) và thông tin sản phẩm.

- Order Service: "Nhạc trưởng" điều phối quy trình Checkout.

4.2.3 API Gateway

Gateway đóng vai trò là "chốt chặn" bảo mật:

- Định tuyến yêu cầu (Routing).
- Xác thực tập trung qua JWT.
- Kiểm soát lưu lượng (Rate limiting).

4.2.4 Service Communication

- Đồng bộ (Synchronous): Sử dụng thư viện requests để gọi API trực tiếp giữa các service (ví dụ: Order gọi Product để lấy giá).
- Bất đồng bộ (Asynchronous): Sử dụng Redis làm Message Broker để xử lý các tác vụ nền như gửi thông báo hoặc cập nhật điểm thưởng.

4.2.5. Containerization and Deployment (Container hóa và Triển khai)

Toàn bộ hệ thống **Bookstore Microservices** được đóng gói bằng công nghệ **Docker**, giúp đảm bảo sự đồng nhất tuyệt đối giữa môi trường phát triển (Development) và môi trường thực tế (Production).

1. Đóng gói dịch vụ (Dockerization)

Mỗi Microservice (User, Product, Order, AI...) đều sở hữu một Dockerfile riêng biệt. Các Image này được tối ưu hóa dựa trên python:3.10-slim để giảm dung lượng và tăng tốc độ triển khai. Quy trình đóng gói bao gồm:

- Cài đặt các thư viện lõi (Django, PyTorch, Neo4j Driver, v.v.).
- Cấu hình biến môi trường thông qua tệp .env.
- Thiết lập lệnh thực thi tự động (Gunicorn hoặc runserver).

2. Điều phối đa dịch vụ (Orchestration with Docker Compose)

Hệ thống sử dụng **Docker Compose** để quản lý vòng đời của hơn 10 container cùng lúc. Đây không chỉ là các service nghiệp vụ mà còn bao gồm cả hệ thống lưu trữ đa phương thức (Polyglot Persistence):

- PostgreSQL/MySQL: Cơ sở dữ liệu quan hệ cho các service nghiệp vụ (Order, User, Product).
- MongoDB: Lưu trữ dữ liệu hành vi phi cấu trúc cho Interaction Service.
- Neo4j: Cơ sở dữ liệu đồ thị cho AI Recommender.

4.2.6. System Structure

```

bookstore-micro05/
|
├─ api_gateway/                # Điều phối yêu cầu & Bảo mật (JWT, RBAC)
|
├─ user-service/              # Quản lý 3 vai trò: Admin, Staff, Customer
|
├─ product-service/          # Quản lý đa lĩnh vực (10 Domain-Driven Design)
|   └─ modules/catalog/
|       └─ infrastructure/models/ # Định nghĩa các mô hình thực thể riêng biệt:
|           └─ 1. book_model.py    # Sách & Văn phòng phẩm
|           └─ 2. electronics_model.py # Đồ điện tử & Công nghệ
|           └─ 3. fashion_model.py  # Thời trang & Phụ kiện
|           └─ 4. food_model.py     # Thực phẩm & Đồ uống
|           └─ 5. furniture_model.py # Nội thất & Nhà cửa
|           └─ 6. medicine_model.py # Y tế & Dược phẩm
|           └─ 7. toys_model.py    # Đồ chơi & Trẻ em
|           └─ 8. auto_parts_model.py # Phụ tùng ô tô & Xe máy
|           └─ 9. cosmetics_model.py # Mỹ phẩm & Làm đẹp
|           └─ 10. pet_supplies_model.py # Thú cưng & Chăm sóc động vật
|
├─ cart-service/              # Quản lý giỏ hàng và vật phẩm tạm thời
├─ order-service/             # Xử lý đơn hàng và Snapshot giá sản phẩm
├─ pay-service/               # Xử lý thanh toán và cổng giao dịch
├─ ship-service/              # Xử lý vận chuyển và theo dõi đơn hàng
├─ comment-rate-service/      # Quản lý đánh giá và Rating đa lĩnh vực
├─ interaction-service/       # Ghi nhận hành vi người dùng (MongoDB)
└─ recommender-ai-service/    # Hệ thống AI (LSTM, Neo4j, RAG Agent)

```

Hình 4.3. System Structure

4.3 API Gateway (Django)

API Gateway đóng vai trò là "lớp bảo vệ" và "điều phối viên" cho toàn bộ hệ thống microservices. Thay vì sử dụng Nginx tĩnh, dự án sử dụng **Django** để có khả năng xử lý logic xác thực và quản lý phiên (session) linh hoạt hơn.

4.3.1 Vai trò của API Gateway

1. Cổng vào duy nhất (Single Entry Point): Khách hàng chỉ cần kết nối tới một địa chỉ duy nhất (Port 8000). Gateway sẽ chịu trách nhiệm ẩn giấu cấu trúc mạng phức tạp bên trong của hơn 10 microservices.
2. Định tuyến thông minh (Dynamic Routing): Dựa vào đường dẫn URL, Gateway sẽ tự động chuyển tiếp yêu cầu đến đúng service đích.
3. Xác thực tập trung (Centralized Authentication):

- Gateway kiểm tra Token JWT của người dùng trước khi cho phép yêu cầu đi vào các service nghiệp vụ.
 - Tự động đính kèm thông tin danh tính (User ID, Role) vào Header để các service con có thể sử dụng ngay lập tức.
4. Quản lý phiên (Session Management): Lưu trữ trạng thái đăng nhập và các thông tin tạm thời của người dùng.

4.3.2 Cấu hình định tuyến (Routing Configuration)

Trong dự án, việc định tuyến được cấu hình thông qua tệp `urls.py` và lớp `BaseProxyView`. Đây là cách hệ thống "chỉ đường" cho các yêu cầu:

```
# api_gateway/api_gateway/urls.py
urlpatterns = [
    path('api/users/', UserProxyView.as_view()),      # Chuyển tới User Service
    path('api/products/', ProductProxyView.as_view()), # Chuyển tới Product Service
    path('api/ai/', RecommenderProxyView.as_view()),  # Chuyển tới AI Service
    path('api/orders/', OrderProxyView.as_view()),    # Chuyển tới Order Service
]
```

Hiện thực hóa Proxy tại views.py (Cấp thực thi): Mỗi View sẽ khai báo địa chỉ (URL nội bộ) của service mà nó đại diện:

```
# api_gateway/app/views/products.py
class ProductProxyView(BaseProxyView):
    service_url = "http://product-service:8000" # Tên service trong Docker

# api_gateway/app/views/recommender.py
class RecommenderProxyView(BaseProxyView):
    service_url = "http://recommender-ai-service:8000"
```

Cơ chế chuyển tiếp (Request Forwarding): Khi nhận được yêu cầu, Gateway sẽ sao chép dữ liệu (Payload) và Token, sau đó gửi tới service đích:

```
# Đoạn code xử lý tại BaseProxyView
def proxy_request(self, request, path, method="GET"):
    target_url = f"{self.service_url}/{path}"
    # Đính kèm JWT từ Session khách hàng vào Header Authorization
    headers = {'Authorization': f"Bearer {request.session.get('jwt_access')}"}
    return requests.request(method, target_url, headers=headers, json=request.data)
```

4.4 Authentication (JWT)

Hệ thống sử dụng cơ chế xác thực không trạng thái (Stateless Authentication) dựa trên tiêu chuẩn JWT. Điều này cho phép người dùng đăng nhập một lần và có thể truy cập vào tất cả các Microservices mà không cần lưu trữ phiên (session) trên từng máy chủ con.

4.4.1 Cài đặt thư viện

Trong toàn bộ các Microservices, chúng ta sử dụng thư viện **SimpleJWT** – bộ thư viện chuẩn nhất cho Django REST Framework để xử lý Token:
pip install djangorestframework-simplejwt

4.4.2 Cấu hình hệ thống (Cấu hình mẫu trong settings.py)

Mọi dịch vụ đều sử dụng chung một cấu hình thời gian sống của Token để đảm bảo tính đồng nhất:

```
# user-service/user_service/settings.py
from datetime import timedelta

SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=60), # Token có hiệu lực trong 1 giờ
    'REFRESH_TOKEN_LIFETIME': timedelta(days=1), # Token làm mới có hiệu lực trong 1 ngày
    'ROTATE_REFRESH_TOKENS': True,
    'ALGORITHM': 'HS256', # Thuật toán mã hóa bảo mật
    'SIGNING_KEY': SECRET_KEY, # Chìa khóa bí mật chung của hệ thống
    'AUTH_HEADER_TYPES': ('Bearer',), # Tiền tố của Token trong Header
}
```

4.4.3 Luồng xác thực hệ thống (Authentication Flow)

Quy trình xác thực diễn ra qua 3 bước khép kín:

1. Giai đoạn Đăng nhập (Login):
 - Người dùng gửi thông tin (Username/Password) tới user-service.
 - Nếu đúng, user-service sẽ dùng TokenObtainPairView để sinh ra bộ đôi: Access Token (dùng để gọi API) và Refresh Token (dùng để lấy Access Token mới).
2. Giai đoạn Lưu trữ & Đính kèm (Proxying):
 - API Gateway nhận Token và lưu vào Session của khách hàng.
 - Với mỗi yêu cầu tiếp theo, Gateway tự động đính kèm Token này vào Header của yêu cầu gửi tới các service nghiệp vụ dưới dạng: Authorization: Bearer <token_chuỗi_mã_hóa>.
3. Giai đoạn Xác thực tại Service đích (Verification):
 - Các service con (như Order, Pay, AI) khi nhận được yêu cầu sẽ kiểm tra chữ ký của Token bằng SIGNING_KEY.
 - Nếu Token hợp lệ và chưa hết hạn, service sẽ giải mã để lấy thông tin user_id và role, từ đó cho phép thực hiện hành động nghiệp vụ tương ứng.

4.5 Giao tiếp giữa các Service

Trong hệ thống **bookstore-micro05**, các dịch vụ không hoạt động cô lập mà luôn có sự trao đổi thông tin để hoàn thành một quy trình nghiệp vụ (ví dụ: Quy trình Checkout).

4.5.1 Đồng bộ qua REST API (Synchronous Communication)

Đây là phương thức giao tiếp chính. Một dịch vụ sẽ chủ động gọi API của dịch vụ khác và đợi kết quả trả về. Dự án sử dụng thư viện **Python Requests** để thực hiện việc này.

- Ví dụ thực tế (Order Service gọi Product Service): Khi tạo đơn hàng, order-service cần kiểm tra giá và tồn kho của sản phẩm:

```
# Trích logic trong order-service/app/views.py
import requests

def get_product_info(product_id):
    url = f"http://product-service:8000/api/products/{product_id}/"
    response = requests.get(url, timeout=5) # Đợi tối đa 5 giây
    if response.status_code == 200:
        return response.json()
    return None
```

4.5.2 Các nguyên tắc Best Practice được áp dụng

Để đảm bảo hệ thống không bị treo hoặc sụp đổ dây chuyền khi một service gặp sự cố, chúng ta áp dụng các chiến lược bảo vệ sau:

1. **Thiết lập Timeout (Thời gian chờ tối đa):**
 - Tất cả các lệnh gọi API liên dịch vụ đều phải có timeout. Điều này ngăn chặn việc một service bị "treo" vô tận nếu service đích không phản hồi.
 - *Tiêu chuẩn:* 5 - 10 giây cho các truy vấn thông thường.
2. **Cơ chế Retry (Thử lại):**
 - Với các lỗi mạng tạm thời, hệ thống sẽ tự động thử lại 1-2 lần trước khi báo lỗi chính thức cho người dùng.
3. **Circuit Breaker (Cầu chì bảo vệ - Tư duy tiên tiến):**
 - Nếu một service (ví dụ: pay-service) liên tục trả về lỗi, Gateway hoặc các service khác sẽ tạm thời "ngắt cầu chì", không gửi yêu cầu tới đó nữa trong một khoảng thời gian để service đó có cơ hội tự phục hồi.
4. **Bảo mật nội bộ (Internal Security):**
 - Mặc dù là gọi nội bộ, nhưng các service vẫn kiểm tra Token JWT để đảm bảo rằng yêu cầu đến từ một nguồn hợp lệ (thông qua Gateway).

4.6 Docker hóa hệ thống

Việc sử dụng Docker giúp đóng gói toàn bộ mã nguồn, thư viện và môi trường chạy vào một thực thể duy nhất, loại bỏ hoàn toàn lỗi "chạy được trên máy tôi nhưng không chạy được trên máy chủ".

4.6.1 Dockerfile mẫu cho các Django Services

Mỗi Microservice (như user-service, product-service, order-

service...) sẽ có một file Dockerfile nằm ở thư mục gốc của nó. Đây là cấu hình tối ưu được sử dụng:

```
# 1. Sử dụng hình ảnh Python chính thức (bản slim để nhẹ hơn)
FROM python:3.10-slim

# 2. Thiết lập các biến môi trường để Python không tạo file .pyc và log được xuất trực tiếp
ENV PYTHONDONTWRITEBYTECODE 1
ENV PYTHONUNBUFFERED 1

# 3. Thiết lập thư mục làm việc trong container
WORKDIR /app

# 4. Cài đặt các thư viện hệ thống cần thiết (nếu có)
RUN apt-get update && apt-get install -y libpq-dev gcc

# 5. Sao chép và cài đặt các thư viện Python
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# 6. Sao chép toàn bộ mã nguồn vào container
COPY . .

# 7. Lệnh khởi chạy server (Lắng nghe ở cổng 8000 của container)
CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

4.6.2 Docker-compose (Điều phối toàn hệ thống)

Tập docker-compose.yml nằm ở thư mục gốc của dự án, đóng vai trò là "nhạc trưởng" điều khiển tất cả các container. Nó định nghĩa cách các service kết nối với nhau và cách ánh xạ cổng ra bên ngoài.

```
version: '3.8'

services:
  # --- CÁC SERVICE NGHIỆP VỤ ---
  user-service:
    build: ./user-service
    ports:
      - "8001:8000" # Máy thật gọi cổng 8001 -> vào Container cổng 8000
    env_file: .env

  product-service:
    build: ./product-service
    ports:
      - "8002:8000"
    env_file: .env

  order-service:
    build: ./order-service
    ports:
      - "8003:8000"
```



```

depends_on:
  - user-service
  - product-service

# --- CÁC HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU ---
postgres_db:
  image: postgres:14
  volumes:
    - postgres_data:/var/lib/postgresql/data
  environment:
    - POSTGRES_DB=micro_db
    - POSTGRES_PASSWORD=password

neo4j_graph:
  image: neo4j:latest
  ports:
    - "7474:7474" # Web interface cho đồ thị AI
    - "7687:7687" # Bolt protocol cho AI Service
  environment:
    - NEO4J_AUTH=neo4j/password

volumes:
  postgres_data:

```

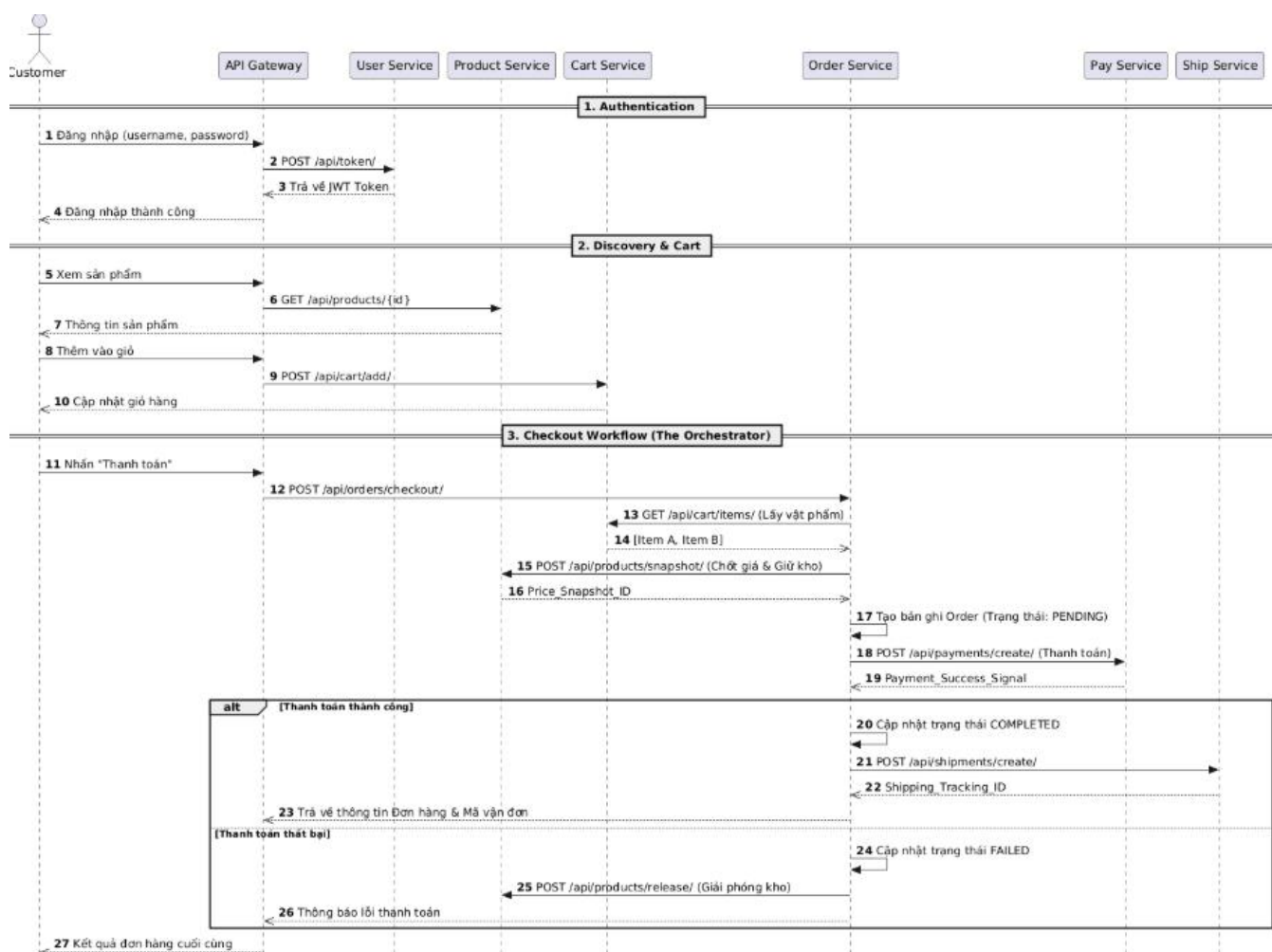
4.7 Luồng hệ thống (End-to-End)

4.7.1 Mô tả chi tiết Use Case: Mua hàng

Quy trình mua hàng là sự phối hợp nhịp nhàng giữa 6 microservices khác nhau để đảm bảo dữ liệu luôn nhất quán và bảo mật.

1. **Xác thực (User Login):** Khách hàng đăng nhập qua API Gateway. user-service xác thực và trả về bộ đôi Access/Refresh Token. Gateway lưu Token vào Session để tự động đính kèm vào các yêu cầu tiếp theo.
2. **Duyệt sản phẩm (Browsing):** Khách hàng xem danh mục sách/đồ chơi. Gateway điều hướng yêu cầu tới product-service. Đồng thời, hành động này được interaction-service âm thầm ghi nhận vào MongoDB để phục vụ huấn luyện AI sau này.
3. **Quản lý Giỏ hàng (Cart Management):** Khi khách chọn "Thêm vào giỏ", cart-service sẽ lưu trữ tạm thời các ID sản phẩm và số lượng. Bước này giúp khách có thể mua sắm trên nhiều thiết bị khác nhau mà không mất giỏ hàng.
4. **Khởi tạo Đơn hàng (Order Creation):** Đây là bước quan trọng nhất. order-service đóng vai trò "Nhạc trưởng":
 - Nó gọi cart-service để lấy danh sách vật phẩm.
 - Gọi product-service để chốt giá tại thời điểm mua (**Price Snapshot**) và tạm giữ kho hàng.
5. **Thanh toán (Payment):** Sau khi đơn hàng được tạo với trạng thái PENDING, hệ thống chuyển hướng yêu cầu tới pay-service. Tại đây, các giao dịch tài chính được thực hiện và kết quả (Thành công/Thất bại) sẽ được cập nhật ngược lại cho đơn hàng.
6. **Vận chuyển (Shipping):** Ngay khi nhận được tín hiệu "Thanh toán thành công", order-service sẽ kích hoạt ship-service để khởi tạo mã vận đơn và địa chỉ giao hàng, hoàn tất chu trình mua sắm.

4.7.2. Sequence Logic



Hình 4.4. Biểu đồ tuần tự chi tiết luồng khách hàng mua hàng

1. Customer gửi yêu cầu Đăng nhập (username, password) đến API Gateway.
2. API Gateway chuyển tiếp yêu cầu xác thực bằng phương thức POST /api/token/ tới User Service.
3. User Service xác thực thông tin thành công và trả về mã JWT Token cho API Gateway.
4. API Gateway phản hồi thông báo "Đăng nhập thành công" về cho Customer.
5. Customer thực hiện hành động Xem sản phẩm gửi đến API Gateway.
6. API Gateway gọi API GET /api/products/{id} để lấy thông tin chi tiết sản phẩm từ Product Service.
7. Product Service trả về "Thông tin sản phẩm" cho API Gateway để hiển thị tới người dùng.
8. Customer thực hiện hành động "Thêm vào giỏ" hàng.
9. API Gateway gửi yêu cầu POST /api/cart/add/ tới Cart Service để lưu thông tin.
10. Cart Service xử lý và phản hồi trạng thái "Cập nhật giỏ hàng" thành công về cho người dùng qua Gateway.

11. Customer nhấn nút "Thanh toán" để bắt đầu quy trình Checkout.
 12. API Gateway chuyển tiếp yêu cầu POST /api/orders/checkout/ tới Order Service (lúc này đóng vai trò là Bộ điều phối - Orchestrator).
 13. Order Service gọi lệnh GET /api/cart/items/ sang Cart Service để lấy danh sách vật phẩm.
 14. Cart Service trả về thông tin các mặt hàng hiện có trong giỏ (Ví dụ: Item A, Item B).
 15. Order Service gửi yêu cầu POST /api/products/snapshot/ tới Product Service nhằm mục đích Chốt giá & Giữ kho.
 16. Product Service thực hiện giữ kho và trả về mã định danh Price_Snapshot_ID.
 17. Order Service tiến hành tự tạo một bản ghi Đơn hàng mới trong hệ thống với trạng thái là PENDING.
 18. Order Service gọi sang Pay Service thông qua POST /api/payments/create/ để thực hiện giao dịch thanh toán.
 19. Pay Service xử lý giao dịch và trả về tín hiệu thanh toán thành công (Payment_Success_Signal).
- (Tại đây hệ thống rẽ nhánh điều kiện alt tùy thuộc vào kết quả thanh toán)*
- Nếu [Thanh toán thành công]:
20. Order Service tự cập nhật trạng thái đơn hàng của mình thành COMPLETED.
 21. Order Service gọi API POST /api/shipments/create/ sang Ship Service để tạo lệnh vận chuyển.
 22. Ship Service xác nhận và trả về mã vận đơn (Shipping_Tracking_ID).
 23. Order Service gửi thông tin Đơn hàng kèm Mã vận đơn về cho API Gateway.
- Nếu [Thanh toán thất bại]:
24. Order Service tự cập nhật trạng thái đơn hàng sang FAILED.
 25. Order Service kích hoạt hành động bù (Saga) bằng cách gọi POST /api/products/release/ sang Product Service để Giải phóng kho (hoàn trả lại số lượng sản phẩm đã giữ ở bước 15).
 26. Order Service gửi thông báo lỗi thanh toán về cho API Gateway.
- Kết thúc:
27. API Gateway tổng hợp dữ liệu và hiển thị "Kết quả đơn hàng cuối cùng" (thành công hoặc thất bại) cho Customer.

4.8 Triển khai Kubernetes (Optional)

Khi hệ thống cần phục vụ hàng triệu người dùng, chúng ta chuyển từ Docker Compose sang Kubernetes để tận dụng khả năng tự động hồi phục (Self-healing) và tự động mở rộng (Auto-scaling).

4.8.1 Cấu hình Deployment mẫu (Cho user-service)

File này định nghĩa cách chạy các bản sao (Replicas) của service.

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: user-service
spec:
  replicas: 3 # Chạy 3 bản sao để dự phòng lỗi
  selector:
    matchLabels:
      app: user-service
  template:
    metadata:
      labels:
        app: user-service
    spec:
      containers:
        - name: user-service
          image: my-registry/user-service:v1.0
          ports:
            - containerPort: 8000
          envFrom:
            - configMapRef:
                name: user-config

```

4.8.2 Cấu hình Service mẫu

File này đóng vai trò như một bộ cân bằng tải (Load Balancer) nội bộ.

```

apiVersion: v1
kind: Service
metadata:
  name: user-service
spec:
  selector:
    app: user-service
  ports:
    - protocol: TCP
      port: 80 # Cổng ngoài của Service
      targetPort: 8000 # Cổng trong của Container
  type: ClusterIP # Chỉ cho phép truy cập nội bộ trong cụm K8s

```

4.9 Logging và Monitoring

Để kiểm soát một hệ thống phân tán gồm 10+ services, chúng ta cần cơ chế thu thập dữ liệu tập trung.

4.9.1 Tập trung hóa Log với ELK Stack

Hệ thống sử dụng bộ ba: Elasticsearch (Lưu trữ), Logstash (Xử lý), Kibana (Hiển thị).

- **Cấu hình tại các Microservice (Django):** Sử dụng thư viện python-logstash để gửi log trực tiếp qua mạng

```
# settings.py
LOGGING = {
    'version': 1,
    'handlers': {
        'logstash': {
            'level': 'INFO',
            'class': 'logstash.TCPLogstashHandler',
            'host': 'logstash-server-address',
            'port': 5000,
        },
    },
    'loggers': {
        'django': {
            'handlers': ['logstash'],
            'level': 'INFO',
        },
    },
}
```

4.9.2 Giám sát hiệu năng với Prometheus + Grafana

- **Prometheus:** Thu thập các chỉ số (Metrics) như CPU, RAM, số lượng Request/giây.
- **Grafana:** Vẽ biểu đồ trực quan hóa dữ liệu từ Prometheus.
- **Cài đặt tại Django Service:**

```

# 1. Cài đặt thư viện: pip install django-prometheus
# 2. Cấu hình settings.py
INSTALLED_APPS = [
    ...
    'django_prometheus',
]
MIDDLEWARE = [
    'django_prometheus.middleware.PrometheusBeforeMiddleware',
    ...
    'django_prometheus.middleware.PrometheusAfterMiddleware',
]
# 3. Thêm endpoint tại urls.py
urlpatterns = [
    path('', include('django_prometheus.urls')),
]

```

Cấu hình Prometheus (prometheus.yml):

```

scrape_configs:
  - job_name: 'django-microservices'
    metrics_path: '/metrics'
    static_configs:
      - targets: ['user-service:8000', 'product-service:8000', 'ai-service:8000']

```

4.10 Đánh giá hệ thống

4.10.1 Đánh giá hiệu năng (Performance)

Hiệu năng của hệ thống được đo lường dựa trên trải nghiệm thực tế của người dùng và các chỉ số đo đạc từ API Gateway.

- Thời gian phản hồi (Response Time):
 - Các yêu cầu nghiệp vụ thông thường (xem sản phẩm, thêm giỏ hàng) có độ trễ thấp (< 100ms) nhờ vào việc tối ưu hóa truy vấn SQL và sử dụng bộ nhớ đệm (Caching).
 - Các yêu cầu phức tạp như tư vấn AI (RAG) có độ trễ cao hơn (1-2s) do phải gọi API bên thứ ba (Gemini) và tính toán Vector, nhưng được tối ưu bằng cơ chế Streaming để người dùng không cảm thấy phải chờ đợi lâu.
- Thông lượng (Throughput):
 - Hệ thống có khả năng xử lý đồng thời hàng nghìn yêu cầu nhờ kiến trúc không đồng bộ và việc phân tải giữa các container. Các dịch vụ nặng như interaction-service (lưu log liên tục) không làm ảnh hưởng đến tốc độ của order-service.

4.10.2 Khả năng mở rộng (Scalability)

Đây là điểm mạnh nhất của kiến trúc microservices mà dự án đã áp dụng.

- Mở rộng độc lập (Vertical & Horizontal Scaling):
 - o Khi mùa mua sắm cao điểm đến, chúng ta có thể tăng số lượng bản sao (Replicas) cho order-service và product-service lên 5-10 container mà giữ nguyên các service khác như user-service.
 - o Dịch vụ AI (recommender-ai-service) có thể được cấu hình để chạy trên các node có GPU để tăng tốc độ huấn luyện mô hình BiLSTM mà không gây tốn kém tài nguyên cho các service còn lại.
- Cân bằng tải (Load Balancing): Sử dụng Nginx Ingress để phân phối đều các yêu cầu đến các instance đang rảnh rỗi, tránh hiện tượng thắt nút cổ chai (Bottleneck).

4.10.3 Ưu điểm

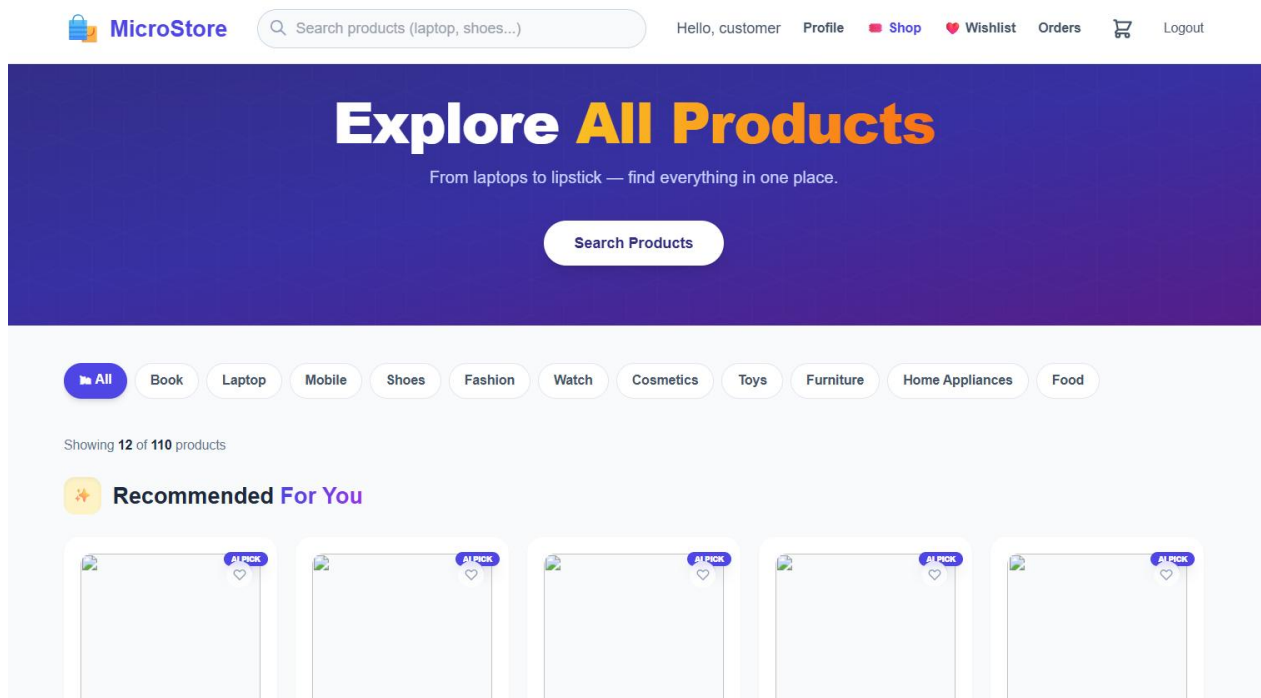
- Tính linh hoạt cao: Dễ dàng thay đổi công nghệ bên trong một service (ví dụ: chuyển pay-service từ Django sang Go) mà không cần viết lại toàn bộ hệ thống.
- Độ tin cậy (Fault Tolerance): Nếu comment-rate-service gặp sự cố, khách hàng vẫn có thể mua hàng và thanh toán bình thường. Hệ thống chỉ bị mất tính năng đánh giá thay vì sụp đổ hoàn toàn.
- Tối ưu hóa dữ liệu: Sử dụng đúng loại Database cho đúng mục đích (PostgreSQL cho tài chính, MongoDB cho Big Data, Neo4j cho AI) giúp hiệu năng đạt mức tối đa.

4.10.4 Nhược điểm và Thách thức

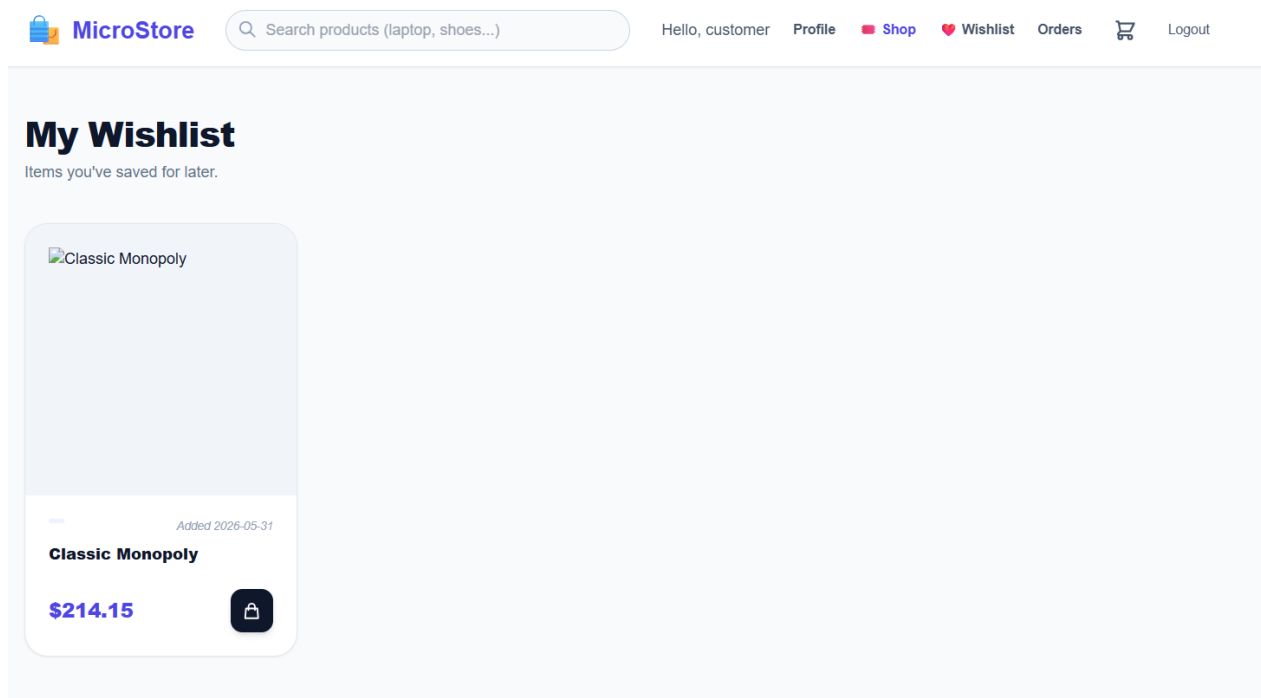
- Độ phức tạp trong triển khai: Việc quản lý hơn 10 Microservices yêu cầu đội ngũ vận hành phải có kiến thức sâu về Docker, CI/CD và Orchestration.
- Khó khăn trong việc Debug: Việc theo dõi một lỗi xảy ra xuyên suốt nhiều service (ví dụ: lỗi thanh toán dẫn đến lỗi kho hàng) rất khó khăn nếu không có hệ thống Distributed Tracing chuyên nghiệp.
- Độ trễ mạng nội bộ (Network Latency): Việc các service phải gọi qua lại lẫn nhau bằng HTTP sẽ tạo ra một khoảng trễ nhỏ so với kiến trúc Monolithic truyền thống.

4.11. Kết quả thực tế

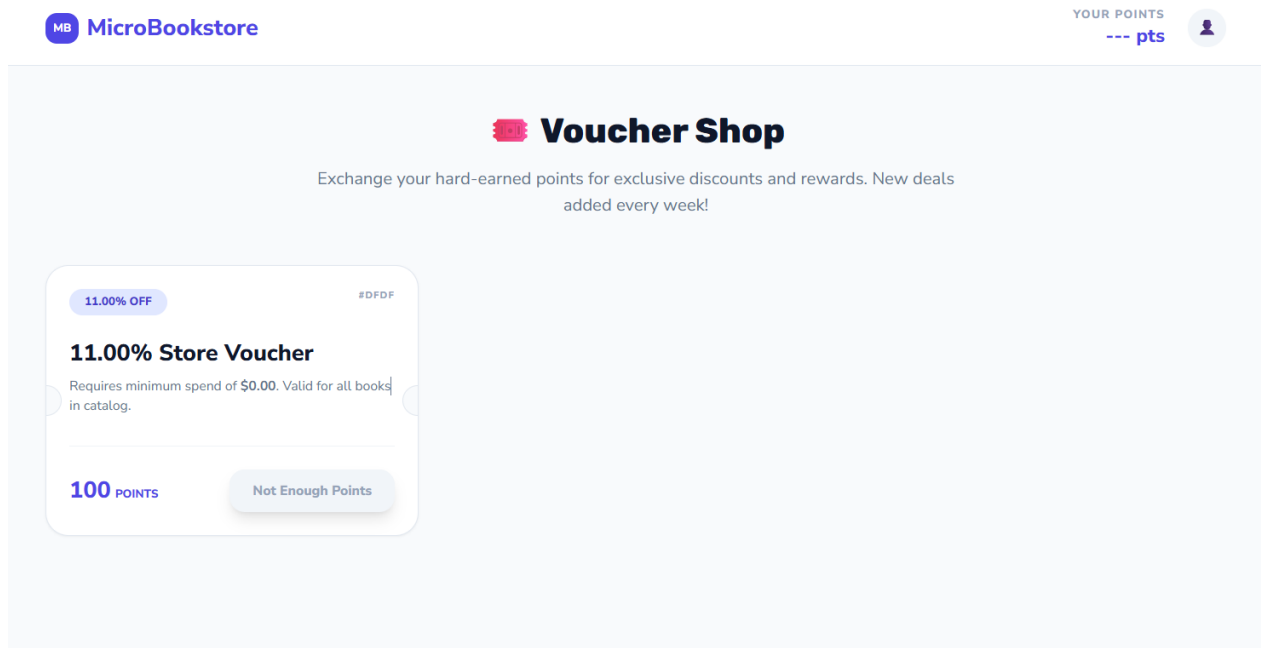
4.11.1. Giao diện khách hàng



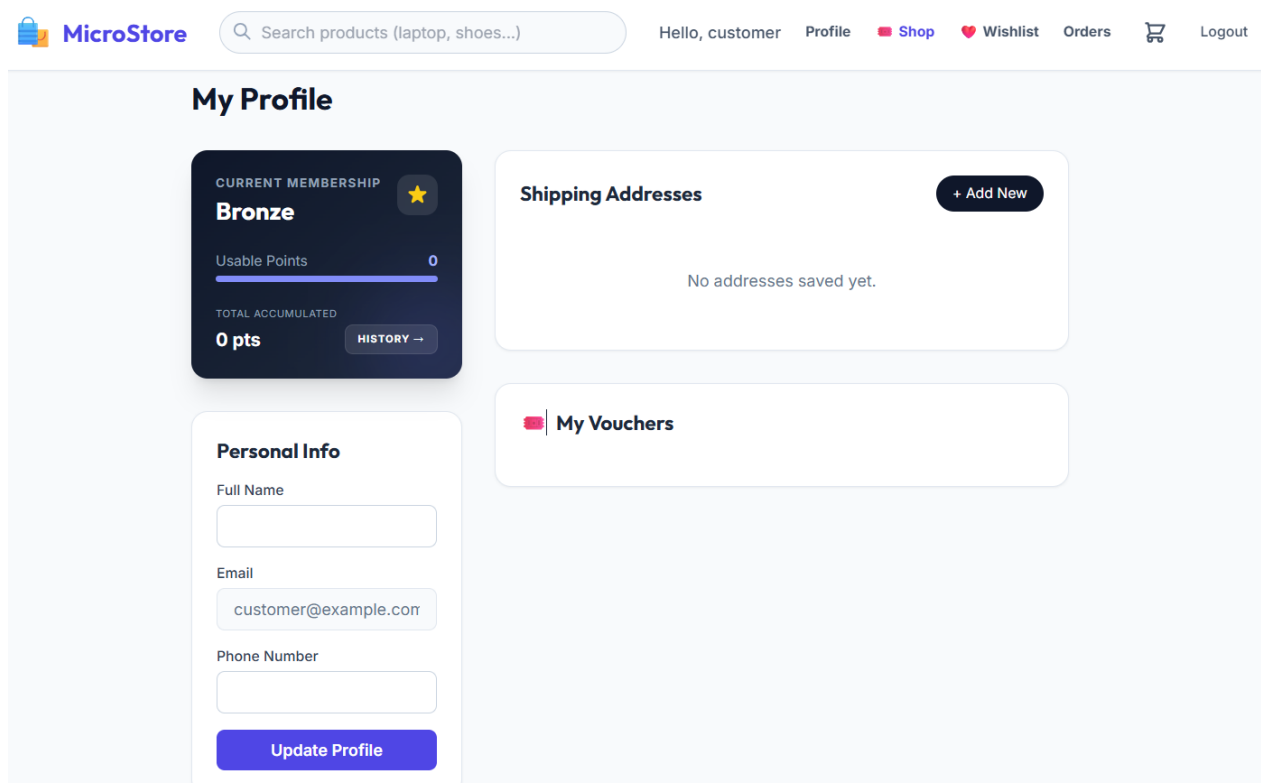
Hình 4.5. Giao diện trang chủ & tư vấn người dùng



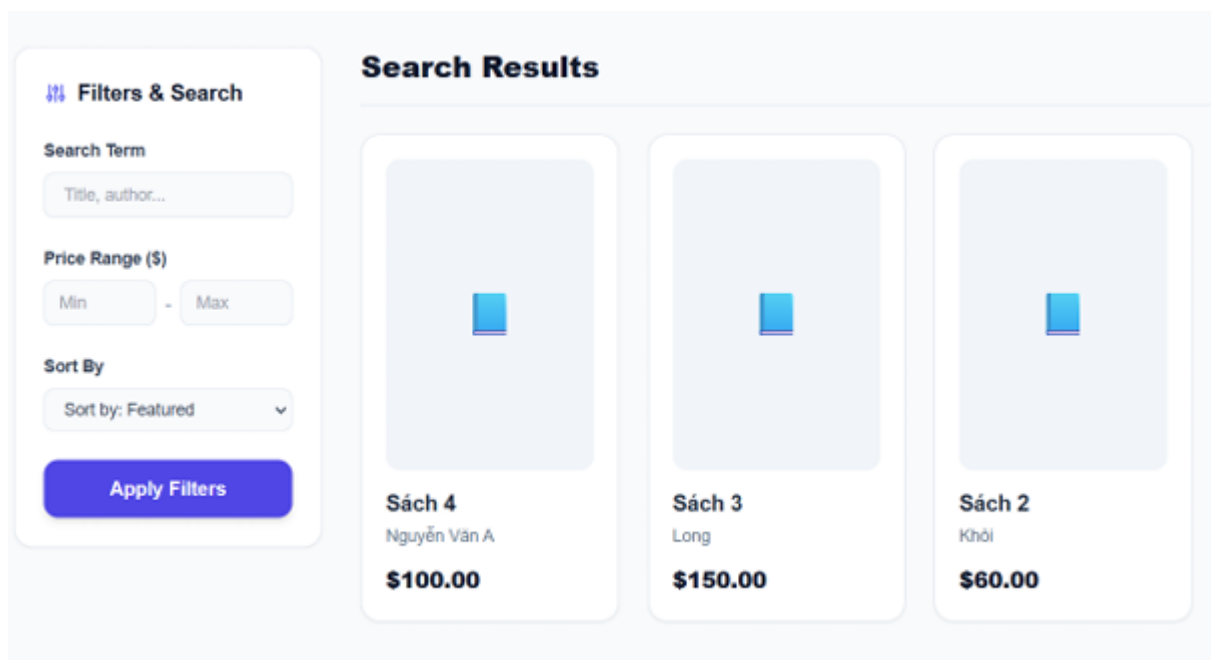
Hình 4.6. Giao diện sản phẩm yêu thích



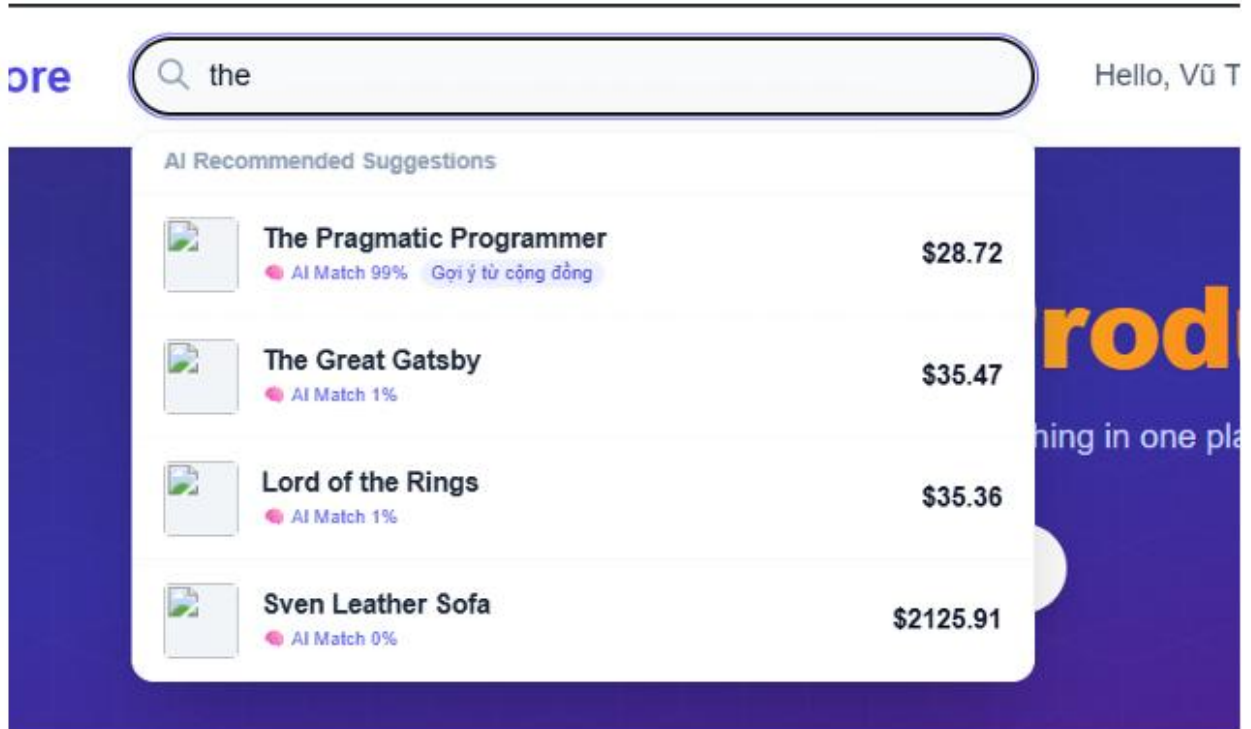
Hình 4.7. Giao diện đổi voucher



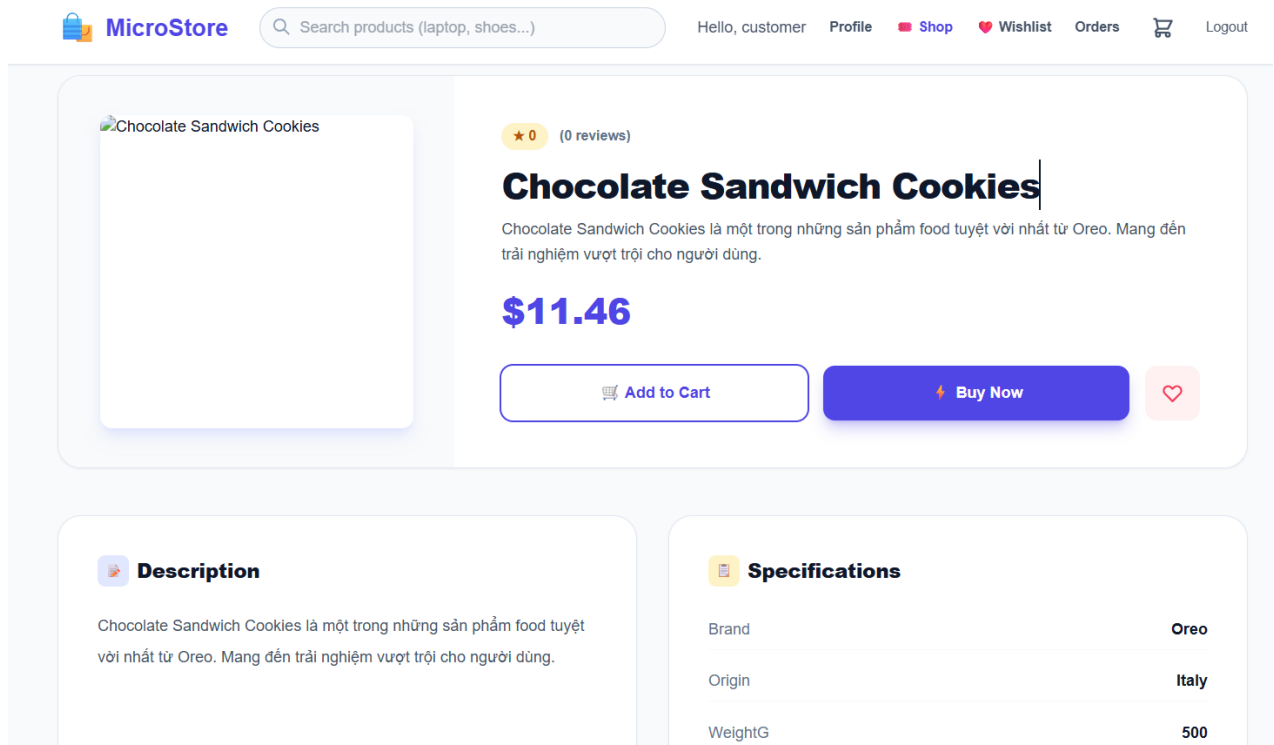
Hình 4.8. Giao diện cá nhân khách hàng



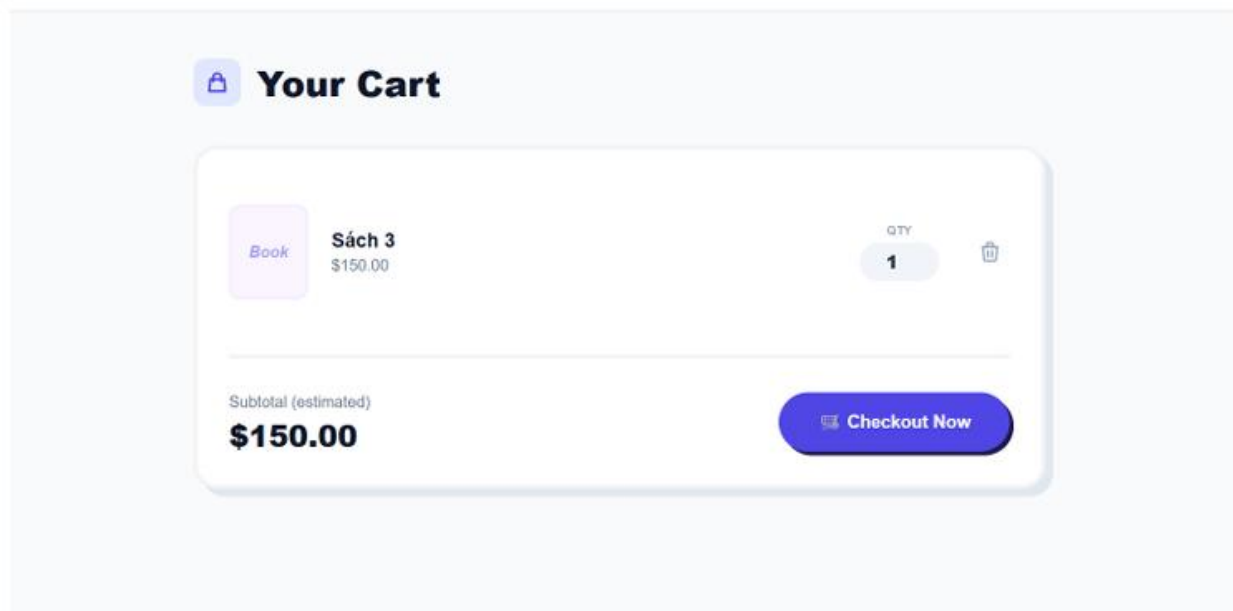
Hình 4.9. Giao diện tìm sản phẩm



Hình 4.10. Giao diện gợi ý theo từ khóa tìm kiếm



Hình 4.11. Giao diện chi tiết sản phẩm



Hình 4.12. Giao diện giỏ hàng

1 Contact Information

Full Name *

Vũ Trọng Khôi

Phone Number *

0965827948

2 Shipping Address

Your Saved Addresses

Home Default

9b 250 Kim giang

Hà Nội, Vietnam

or enter new

Address Label (for saving) optional

e.g. Home, Office...

Street Address *

Order Summary

1x Sách 3 Long \$150.0

Subtotal \$150.0

Shipping \$0.00

Total \$150.00

Place Order →

Secure checkout — your info is safe

Hình 4.13. Giao diện thành toán

Order Confirmed!

Your order has been placed and payment processed successfully.

Order ID	Status	Total Amount	Date
#3	PROCESSING	\$150.00	2026-03-12

Order Items

Sách 3

Qty: 1 × \$150.00

\$150.0

Shipping Fee

\$0.00

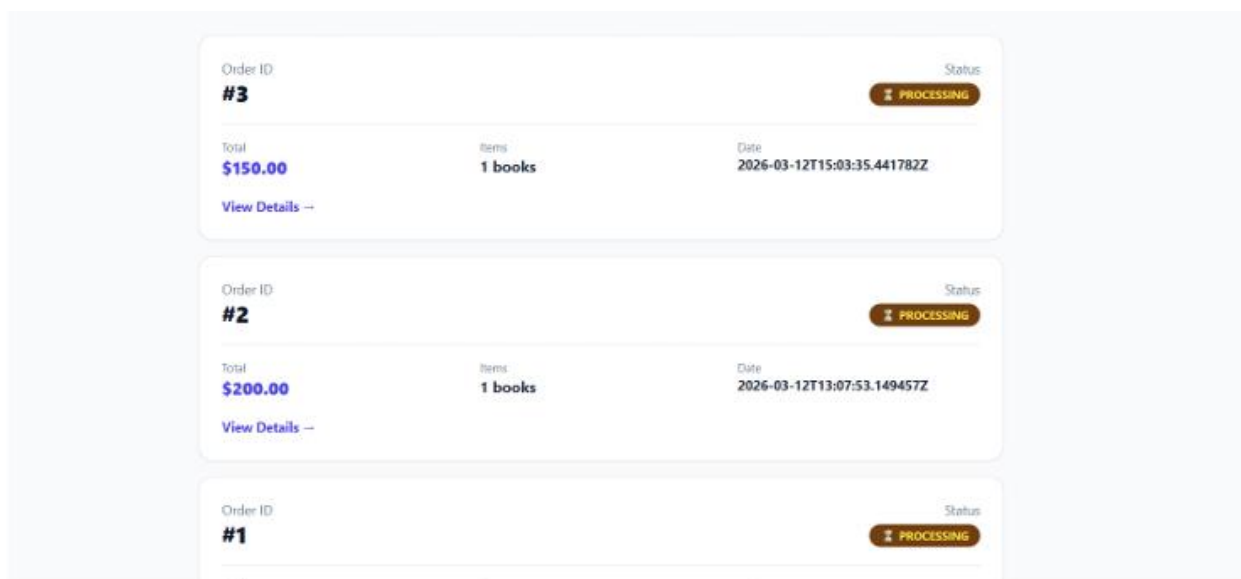
Total Paid

\$150.00

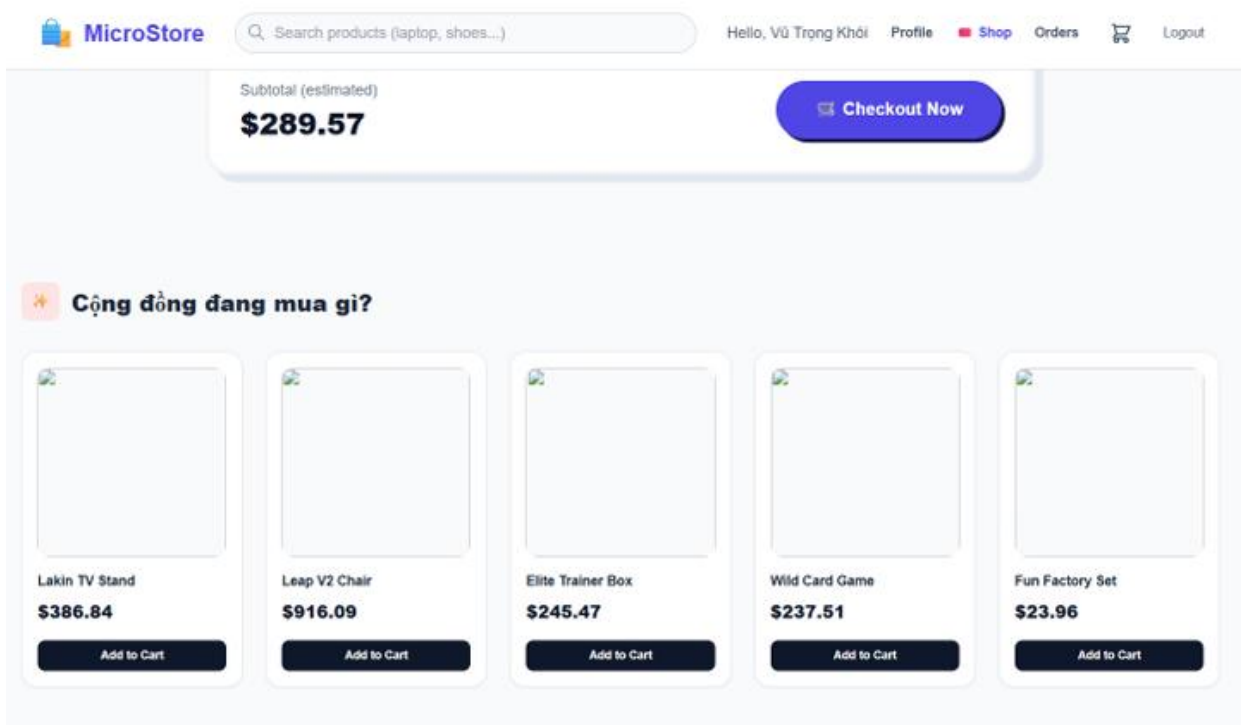
Order History

Continue Shopping

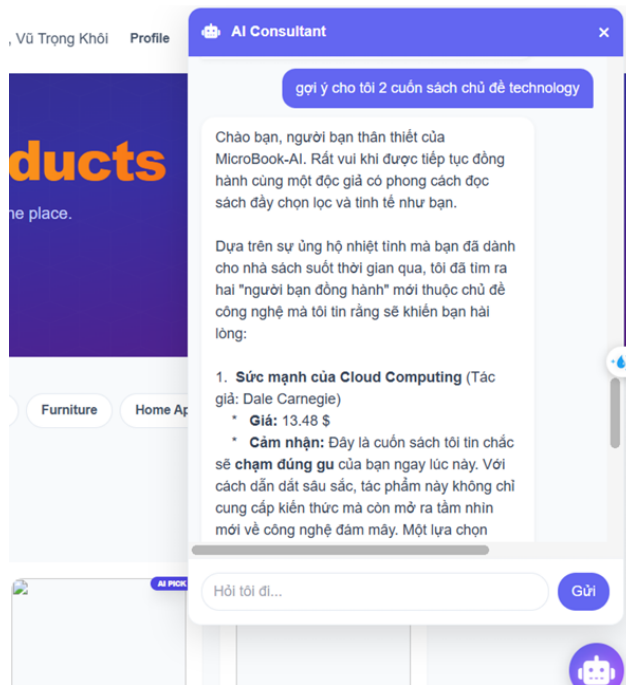
Hình 4.14. Giao diện thanh toán thành công



Hình 4.15. Giao diện danh sách đơn hàng

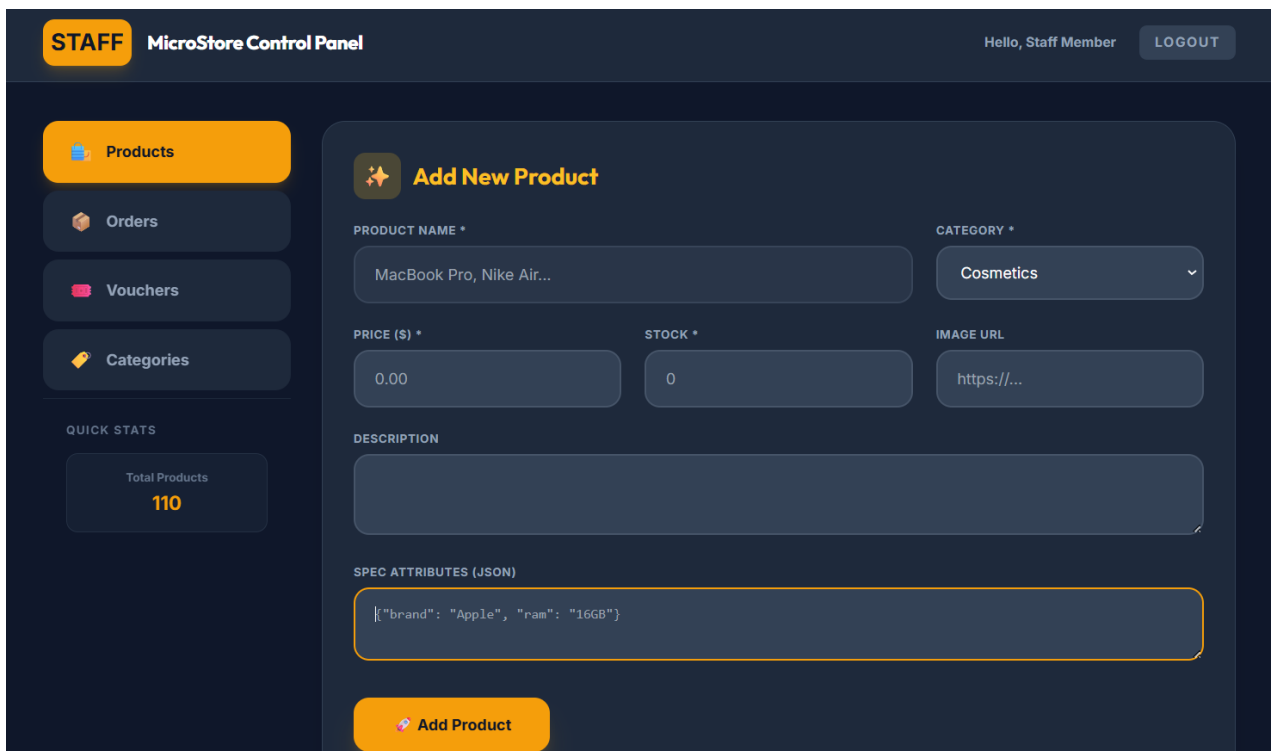


Hình 4.16. Giao diện tư vấn dựa vào giỏ hàng

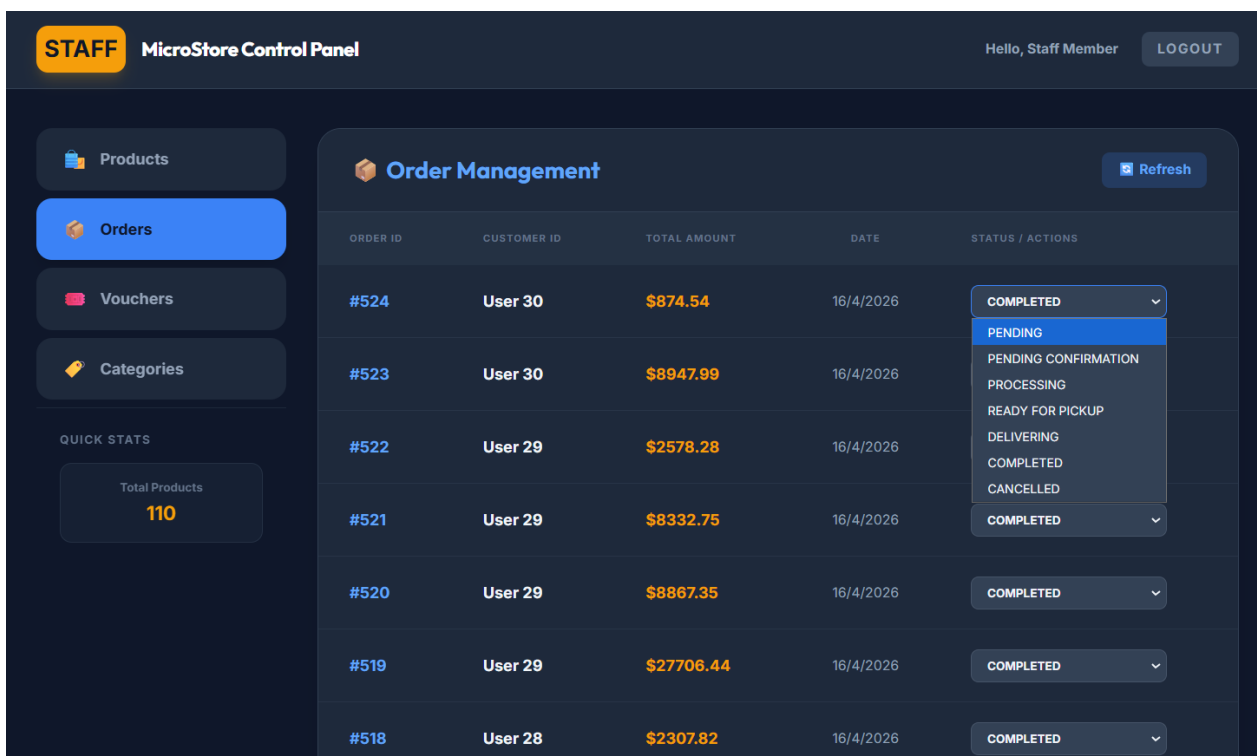


Hình 4.17. Giao diện chatbot

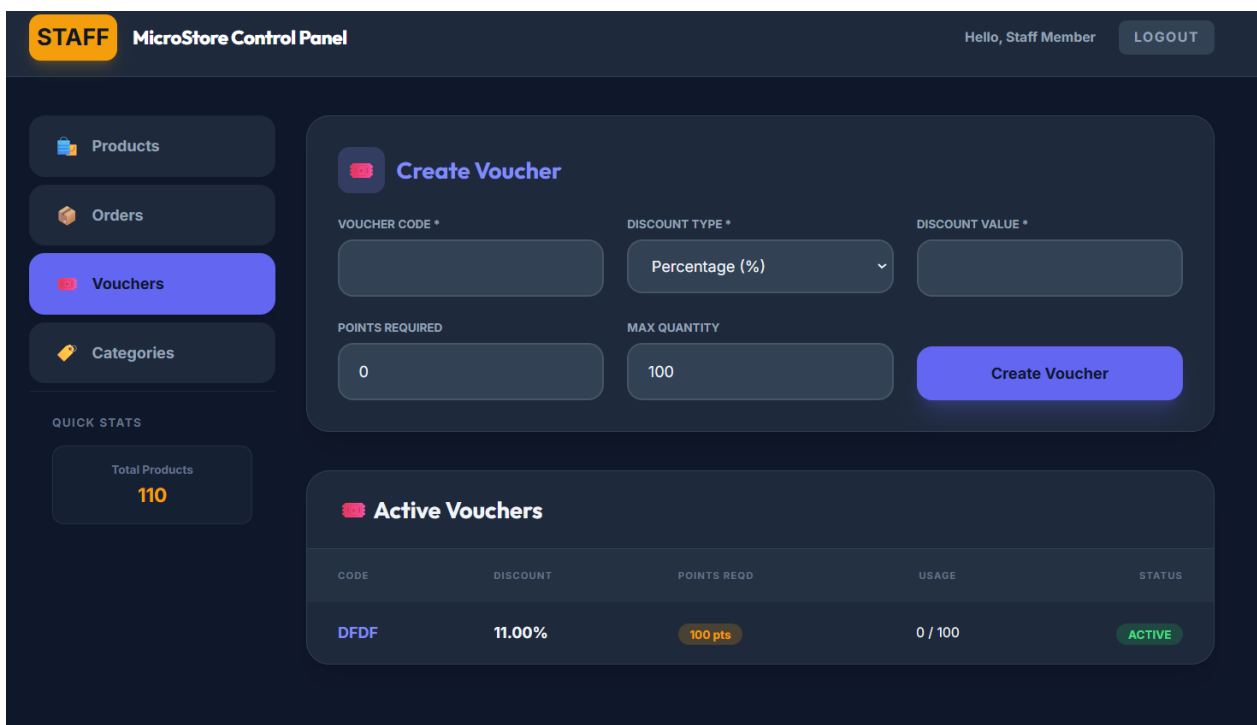
4.11.2. Giao diện nhân viên



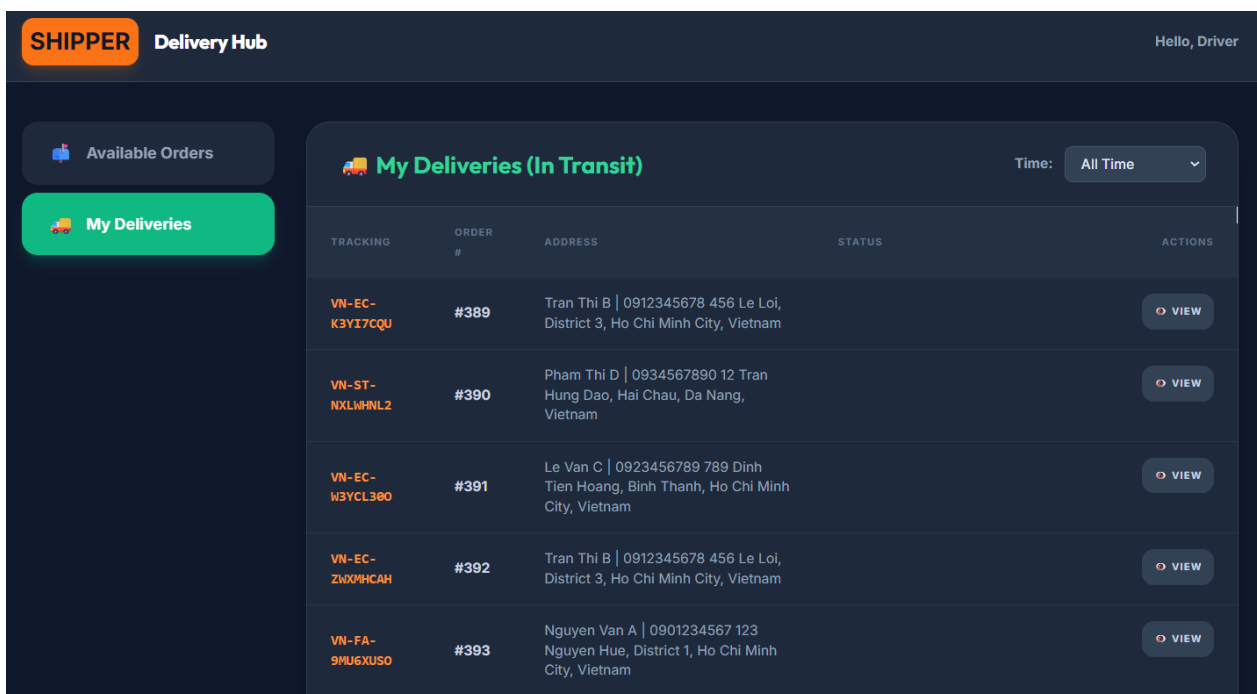
Hình 4.18. Giao diện quản lý sản phẩm



Hình 4.19. Giao diện quản lý đơn hàng



Hình 4.20. Giao diện quản lý Voucher



Hình 4.21. Giao diện quản lý vận đơn

ĐÁNH GIÁ CÁ NHÂN

Chương	Nội dung đánh giá	Kết quả
Chương 1	Bảng so sánh Mono vs Micro	Thực hiện so sánh chi tiết với nhiều tiêu chí
	Các Bigtech sử dụng Micro	Tìm hiểu các Bigtech sử dụng Micro: Netflix, Uber, Amazon
	So sánh thiết kế Mono vs Micro	So sánh thiết kế 2 kiến trúc chi tiết thông qua sơ đồ thiết kế với Case Study Ecommerce
	HealhCareMicro	Thực hiện chi tiết phân rã DDD đi kèm ảnh chứng minh tạo service với django
Chương 2	Phân tích yêu cầu	Đầy đủ các bảng phân tích, biểu đồ UC, biểu đồ lớp, class diagram, datamodel
	Phân ra service	Đầy đủ các bảng phân rã service theo từng bước, đi kèm hình ảnh phân rã hệ thống.
	Thiết kế từng service	Đầy đủ biểu đồ lớp pha phân tích, biểu đồ lớp thiết kế, datamodel cho từng service, mô tả rõ các datatech sử dụng kèm lý do
	Flow luồng các service	Đầy đủ sequence diagram, activity diagram cho từng service
Chương 3	Mô tả yêu cầu	Đầy đủ pipeline RAG, chat, Recommend
	Mô hình DL	Kiểm thử 3 model biLTSM, LTSM, RNN có mô tả cấu trúc chi tiết
	Dataset	Sử dụng 3 dataset thực tế từ Kaggle

	Kết quả huấn luyện	Biểu đồ accuracy, loss, confusion matrix, top 10, kèm nhận xét chi tiết
	RAG	Trình bày chi tiết pipeline, input, output, tích hợp chatbot
Chương 4	Mô tả kiến trúc hệ thống	Mô tả kèm hình ảnh, code structure với django, các công nghệ sử dụng
	Triển khai	Áp dụng docker container
	Kết quả	Giao diện đầy đủ các luồng của khách hàng, nhân viên, shipper

ĐÁNH GIÁ TỪ CÁC BẠN

Tên	Nhận xét	Điểm đánh giá	Ký tên
Nguyễn Thế Lâm	Bài báo cáo có tính logic rất cao. Đi từ việc nghiên cứu kiến trúc của các Bigtech để rút ra bài học, sau đó áp dụng ngay vào khâu phân tích và phân rã dịch vụ của hệ thống mình. Khối lượng công việc trải dài từ thiết kế hệ thống cho đến cài đặt code thực tế đều rất chín chu.	8.0	
Đỗ Hải Nam	Đánh giá cao nỗ lực của bạn khi vừa phải giải quyết bài toán kiến trúc dịch vụ (Microservices), vừa phải tích hợp thêm cầu phần xử lý thông minh (RAG và Deep Learning). Tuy nhiên, phần phân tích sự tương tác qua lại giữa luồng xử lý AI và các service thông thường cần được viết rõ ràng hơn một chút.	7.3	
Phùng Hải Nam	Một bài báo cáo chi tiết có sự đầu tư lớn. Bạn làm tốt ở mọi giai đoạn: tài liệu thiết kế hệ thống đầy đủ biểu đồ, khâu thực nghiệm AI có số liệu chứng minh từ Kaggle rõ ràng, và cuối cùng là đóng gói được sản phẩm bằng Docker.	8.5	
Bùi Thế Vĩnh Nguyên	Cách hiện thực hóa lý thuyết vào sản phẩm rất tốt. Từ các bước phân rã Domain-Driven Design trên giấy, bạn đã cấu trúc được source code Django chuẩn chỉnh và ra được cả giao diện phân quyền thực tế cho đầy đủ các bên (khách hàng, nhân viên, shipper). Hệ thống có tính thực tiễn cao.	8.0	
Phạm Ngọc Long	khối lượng kiến thức rất nặng và bao quát nhiều công nghệ hiện đại. Dù vậy, phần trình bày báo cáo ở một số đoạn thuộc chương AI và kết quả huấn luyện còn vài lỗi chính tả nhỏ. Nếu chau chuốt lại từ ngữ cho đồng bộ với phần thiết kế hệ thống ở các chương trước thì bài sẽ tốt hơn nhiều.	7.5	
Đặng Quốc Khánh	Bài làm thể hiện tư duy kỹ thuật tốt. Cách bạn thiết kế datamodel và chọn lựa công nghệ cho từng service ở giai đoạn đầu rất mạch lạc, tạo tiền đề tốt để tích hợp mượt mà	8.2	

	luồng pipeline RAG lẫn Chatbot ở giai đoạn sau mà không bị xung đột kiến trúc.		
Ngô Tuấn Anh	Dự án này có độ phức tạp tốt và bao quát được toàn bộ vòng đời phát triển phần mềm. Từ khâu phân tích yêu cầu, phân rã hệ thống theo luồng dịch vụ cho đến khâu triển khai thực tế bằng container hóa đều được làm rất bài bản. Giao diện cuối cùng cũng đã thể hiện được đầy đủ kết quả của các luồng nghiệp vụ.	8.3	
Phạm Trung Kiên	Cách tiếp cận vấn đề của bài báo cáo rất chuyên nghiệp, thiết kế hệ thống một cách lý thuyết mà còn đưa ra các case study, sử dụng dữ liệu thực tế từ Kaggle để huấn luyện mô hình, và có tư duy deploy sản phẩm rõ ràng bằng Docker. Các chương liên kết với nhau chặt chẽ tạo nên một chỉnh thể hoàn thiện.	8.0	
Đặng Quân Bảo	Tổng thể bài báo cáo rất rõ ràng, các bước đi từ lý thuyết, phân tích thiết kế (UML, Sequence), đến cài đặt thử nghiệm đều bám sát yêu cầu. Điểm duy nhất mình nghĩ có thể cải thiện là phần đánh giá mô hình Deep Learning; bạn nên bổ sung một góc nhìn tổng kết xem việc dùng các mô hình này mang lại lợi ích trực tiếp thế nào cho các luồng giao diện của khách hàng hay nhân viên ở cuối bài.	7.6	
Lại Duy Đông	Hệ thống được xây dựng rất quy mô khi vừa giải quyết bài toán quản lý phân quyền đa tác nhân (khách hàng, nhân viên, shipper) vừa giải quyết bài toán AI (RAG, Recommend). Tuy nhiên, do tập trung quá nhiều vào phần tối ưu kiến trúc dịch vụ và thuật toán huấn luyện nên phần tối ưu trải nghiệm (UX) trên giao diện thực tế trông vẫn hơi thô, cần được chăm chút thêm.	7.8	